MDN JavaScript Documentation

Volume 8 of 9 — Built-in objects: Temporal

Unofficial print compilation. This volume contains 242 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Very wide tables** (eight or more columns) are printed turned a quarter turn on a page of their own; turn the book to read them.
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

Temporal 5
- Temporal.Duration 23
  - Temporal.Duration() constructor 28
  - Temporal.Duration.compare() 31
  - Temporal.Duration.from() 34
  - Temporal.Duration.prototype.abs() 36
  - Temporal.Duration.prototype.add() 37
  - Temporal.Duration.prototype.negated() 39
  - Temporal.Duration.prototype.round() 40
  - Temporal.Duration.prototype.subtract() 44
  - Temporal.Duration.prototype.toJSON() 45
  - Temporal.Duration.prototype.toLocaleString() 47
  - Temporal.Duration.prototype.toString() 49
  - Temporal.Duration.prototype.total() 51
  - Temporal.Duration.prototype.valueOf() 53
  - Temporal.Duration.prototype.with() 54
  - Temporal.Duration.prototype.blank 56
  - Temporal.Duration.prototype.days 57
  - Temporal.Duration.prototype.hours 58
  - Temporal.Duration.prototype.microseconds 60
  - Temporal.Duration.prototype.milliseconds 61
  - Temporal.Duration.prototype.minutes 63
  - Temporal.Duration.prototype.months 64
  - Temporal.Duration.prototype.nanoseconds 65
  - Temporal.Duration.prototype.seconds 67
  - Temporal.Duration.prototype.sign 68
  - Temporal.Duration.prototype.weeks 69
  - Temporal.Duration.prototype.years 70
- Temporal.Instant 73
  - Temporal.Instant() constructor 77
  - Temporal.Instant.compare() 78
  - Temporal.Instant.from() 80
  - Temporal.Instant.fromEpochMilliseconds() 81
  - Temporal.Instant.fromEpochNanoseconds() 83
  - Temporal.Instant.prototype.add() 84
  - Temporal.Instant.prototype.equals() 86
  - Temporal.Instant.prototype.round() 88
  - Temporal.Instant.prototype.since() 89
  - Temporal.Instant.prototype.subtract() 91
  - Temporal.Instant.prototype.toJSON() 93
  - Temporal.Instant.prototype.toLocaleString() 94
  - Temporal.Instant.prototype.toString() 96
  - Temporal.Instant.prototype.toZonedDateTimeISO() 98
  - Temporal.Instant.prototype.until() 100
  - Temporal.Instant.prototype.valueOf() 101
  - Temporal.Instant.prototype.epochMilliseconds 103
  - Temporal.Instant.prototype.epochNanoseconds 104
- Temporal.Now 106
  - Temporal.Now.instant() 108
  - Temporal.Now.plainDateISO() 109
  - Temporal.Now.plainDateTimeISO() 110
  - Temporal.Now.plainTimeISO() 112
  - Temporal.Now.timeZoneId() 113
  - Temporal.Now.zonedDateTimeISO() 114
- Temporal.PlainDate 116
  - Temporal.PlainDate() constructor 122
  - Temporal.PlainDate.compare() 124
  - Temporal.PlainDate.from() 126
  - Temporal.PlainDate.prototype.add() 130
  - Temporal.PlainDate.prototype.equals() 133
  - Temporal.PlainDate.prototype.since() 135
  - Temporal.PlainDate.prototype.subtract() 137
  - Temporal.PlainDate.prototype.toJSON() 139
  - Temporal.PlainDate.prototype.toLocaleString() 140
  - Temporal.PlainDate.prototype.toPlainDateTime() 143
  - Temporal.PlainDate.prototype.toPlainMonthDay() 144
  - Temporal.PlainDate.prototype.toPlainYearMonth() 146
  - Temporal.PlainDate.prototype.toString() 147
  - Temporal.PlainDate.prototype.toZonedDateTime() 149
  - Temporal.PlainDate.prototype.until() 151
  - Temporal.PlainDate.prototype.valueOf() 153
  - Temporal.PlainDate.prototype.with() 154
  - Temporal.PlainDate.prototype.withCalendar() 156
  - Temporal.PlainDate.prototype.calendarId 158
  - Temporal.PlainDate.prototype.day 159
  - Temporal.PlainDate.prototype.dayOfWeek 161
  - Temporal.PlainDate.prototype.dayOfYear 163
  - Temporal.PlainDate.prototype.daysInMonth 165
  - Temporal.PlainDate.prototype.daysInWeek 166
  - Temporal.PlainDate.prototype.daysInYear 168
  - Temporal.PlainDate.prototype.era 169
  - Temporal.PlainDate.prototype.eraYear 171
  - Temporal.PlainDate.prototype.inLeapYear 173
  - Temporal.PlainDate.prototype.month 174
  - Temporal.PlainDate.prototype.monthCode 176
  - Temporal.PlainDate.prototype.monthsInYear 179
  - Temporal.PlainDate.prototype.weekOfYear 180
  - Temporal.PlainDate.prototype.year 182
  - Temporal.PlainDate.prototype.yearOfWeek 184
- Temporal.PlainDateTime 186
  - Temporal.PlainDateTime() constructor 192
  - Temporal.PlainDateTime.compare() 194
  - Temporal.PlainDateTime.from() 196
  - Temporal.PlainDateTime.prototype.add() 199
  - Temporal.PlainDateTime.prototype.equals() 201
  - Temporal.PlainDateTime.prototype.round() 202
  - Temporal.PlainDateTime.prototype.since() 204
  - Temporal.PlainDateTime.prototype.subtract() 206
  - Temporal.PlainDateTime.prototype.toJSON() 208
  - Temporal.PlainDateTime.prototype.toLocaleString() 209
  - Temporal.PlainDateTime.prototype.toPlainDate() 212
  - Temporal.PlainDateTime.prototype.toPlainTime() 213
  - Temporal.PlainDateTime.prototype.toString() 215
  - Temporal.PlainDateTime.prototype.toZonedDateTime() 217
  - Temporal.PlainDateTime.prototype.until() 219
  - Temporal.PlainDateTime.prototype.valueOf() 221
  - Temporal.PlainDateTime.prototype.with() 223
  - Temporal.PlainDateTime.prototype.withCalendar() 225
  - Temporal.PlainDateTime.prototype.withPlainTime() 226

Temporal.PlainDateTime.prototype.calendarId 228
Temporal.PlainDateTime.prototype.day 229
Temporal.PlainDateTime.prototype.dayOfWeek 230
Temporal.PlainDateTime.prototype.dayOfYear 231
Temporal.PlainDateTime.prototype.daysInMonth 232
Temporal.PlainDateTime.prototype.daysInWeek 233
Temporal.PlainDateTime.prototype.daysInYear 234
Temporal.PlainDateTime.prototype.era 235
Temporal.PlainDateTime.prototype.eraYear 236
Temporal.PlainDateTime.prototype.hour 238
Temporal.PlainDateTime.prototype.inLeapYear 239
Temporal.PlainDateTime.prototype.microsecond 240
Temporal.PlainDateTime.prototype.millisecond 241
Temporal.PlainDateTime.prototype.minute 242
Temporal.PlainDateTime.prototype.month 243
Temporal.PlainDateTime.prototype.monthCode 244
Temporal.PlainDateTime.prototype.monthsInYear 245
Temporal.PlainDateTime.prototype.nanosecond 246
Temporal.PlainDateTime.prototype.second 247
Temporal.PlainDateTime.prototype.weekOfYear 248
Temporal.PlainDateTime.prototype.year 249
Temporal.PlainDateTime.prototype.yearOfWeek 251
Temporal.PlainMonthDay 252
Temporal.PlainMonthDay() constructor 255
Temporal.PlainMonthDay.from() 258
Temporal.PlainMonthDay.prototype.equals() 262
Temporal.PlainMonthDay.prototype.toJSON() 263
Temporal.PlainMonthDay.prototype.toLocaleString() 265
Temporal.PlainMonthDay.prototype.toPlainDate() 267
Temporal.PlainMonthDay.prototype.toString() 269
Temporal.PlainMonthDay.prototype.valueOf() 271
Temporal.PlainMonthDay.prototype.with() 273
Temporal.PlainMonthDay.prototype.calendarId 275
Temporal.PlainMonthDay.prototype.day 276
Temporal.PlainMonthDay.prototype.monthCode 278
Temporal.PlainTime 280
Temporal.PlainTime() constructor 283
Temporal.PlainTime.compare() 285
Temporal.PlainTime.from() 287
Temporal.PlainTime.prototype.add() 290
Temporal.PlainTime.prototype.equals() 291
Temporal.PlainTime.prototype.round() 292
Temporal.PlainTime.prototype.since() 294
Temporal.PlainTime.prototype.subtract() 296
Temporal.PlainTime.prototype.toJSON() 297
Temporal.PlainTime.prototype.toLocaleString() 299
Temporal.PlainTime.prototype.toString() 301
Temporal.PlainTime.prototype.until() 303
Temporal.PlainTime.prototype.valueOf() 305
Temporal.PlainTime.prototype.with() 306
Temporal.PlainTime.prototype.hour 308
Temporal.PlainTime.prototype.microsecond 309
Temporal.PlainTime.prototype.millisecond 310
Temporal.PlainTime.prototype.minute 312
Temporal.PlainTime.prototype.nanosecond 313
Temporal.PlainTime.prototype.second 314
Temporal.PlainYearMonth 316
Temporal.PlainYearMonth() constructor 320
Temporal.PlainYearMonth.compare() 322
Temporal.PlainYearMonth.from() 325
Temporal.PlainYearMonth.prototype.add() 328
Temporal.PlainYearMonth.prototype.equals() 331
Temporal.PlainYearMonth.prototype.since() 333
Temporal.PlainYearMonth.prototype.subtract() 335
Temporal.PlainYearMonth.prototype.toJSON() 337
Temporal.PlainYearMonth.prototype.toLocaleString() 338
Temporal.PlainYearMonth.prototype.toPlainDate() 341
Temporal.PlainYearMonth.prototype.toString() 342
Temporal.PlainYearMonth.prototype.until() 345
Temporal.PlainYearMonth.prototype.valueOf() 346
Temporal.PlainYearMonth.prototype.with() 348
Temporal.PlainYearMonth.prototype.calendarId 350
Temporal.PlainYearMonth.prototype.daysInMonth 351
Temporal.PlainYearMonth.prototype.daysInYear 352
Temporal.PlainYearMonth.prototype.era 353
Temporal.PlainYearMonth.prototype.eraYear 354
Temporal.PlainYearMonth.prototype.inLeapYear 356
Temporal.PlainYearMonth.prototype.month 357
Temporal.PlainYearMonth.prototype.monthCode 358
Temporal.PlainYearMonth.prototype.monthsInYear 359
Temporal.PlainYearMonth.prototype.year 360
Temporal.ZonedDateTime 362
Temporal.ZonedDateTime() constructor 374
Temporal.ZonedDateTime.compare() 376
Temporal.ZonedDateTime.from() 378
Temporal.ZonedDateTime.prototype.add() 382
Temporal.ZonedDateTime.prototype.equals() 385
Temporal.ZonedDateTime.prototype.getTimeZoneTransition() 387
Temporal.ZonedDateTime.prototype.round() 389
Temporal.ZonedDateTime.prototype.since() 391
Temporal.ZonedDateTime.prototype.startOfDay() 394
Temporal.ZonedDateTime.prototype.subtract() 395
Temporal.ZonedDateTime.prototype.toInstant() 397
Temporal.ZonedDateTime.prototype.toJSON() 398
Temporal.ZonedDateTime.prototype.toLocaleString() 400

Temporal.ZonedDateTime.prototype.toPlainDate() 403
Temporal.ZonedDateTime.prototype.toPlainDateTime() 404
Temporal.ZonedDateTime.prototype.toPlainTime() 406
Temporal.ZonedDateTime.prototype.toString() 407
Temporal.ZonedDateTime.prototype.until() 410
Temporal.ZonedDateTime.prototype.valueOf() 412
Temporal.ZonedDateTime.prototype.with() 413
Temporal.ZonedDateTime.prototype.withCalendar() 417
Temporal.ZonedDateTime.prototype.withPlainTime() 418
Temporal.ZonedDateTime.prototype.withTimeZone() 420
Temporal.ZonedDateTime.prototype.calendarId 422
Temporal.ZonedDateTime.prototype.day 423
Temporal.ZonedDateTime.prototype.dayOfWeek 425
Temporal.ZonedDateTime.prototype.dayOfYear 426
Temporal.ZonedDateTime.prototype.daysInMonth 427
Temporal.ZonedDateTime.prototype.daysInWeek 428
Temporal.ZonedDateTime.prototype.daysInYear 429
Temporal.ZonedDateTime.prototype.epochMilliseconds 430
Temporal.ZonedDateTime.prototype.epochNanoseconds 432
Temporal.ZonedDateTime.prototype.era 433
Temporal.ZonedDateTime.prototype.eraYear 434
Temporal.ZonedDateTime.prototype.hour 436
Temporal.ZonedDateTime.prototype.hoursInDay 437
Temporal.ZonedDateTime.prototype.inLeapYear 439
Temporal.ZonedDateTime.prototype.microsecond 440
Temporal.ZonedDateTime.prototype.millisecond 441
Temporal.ZonedDateTime.prototype.minute 442
Temporal.ZonedDateTime.prototype.month 444
Temporal.ZonedDateTime.prototype.monthCode 445
Temporal.ZonedDateTime.prototype.monthsInYear 446
Temporal.ZonedDateTime.prototype.nanosecond 447
Temporal.ZonedDateTime.prototype.offset 448
Temporal.ZonedDateTime.prototype.offsetNanoseconds 450
Temporal.ZonedDateTime.prototype.second 451
Temporal.ZonedDateTime.prototype.timeZoneId 453
Temporal.ZonedDateTime.prototype.weekOfYear 454
Temporal.ZonedDateTime.prototype.year 455
Temporal.ZonedDateTime.prototype.yearOfWeek 457

JavaScript › Reference › Standard built-in objects

Temporal

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal`** object enables date and time management in various scenarios, including built-in time zone and calendar representation, wall-clock time conversions, arithmetics, formatting, and more. It is designed as a full replacement for the `Date` object.

Description

Unlike most global objects, `Temporal` is not a constructor. You cannot use it with the `new` operator or invoke the `Temporal` object as a function. All properties and methods of `Temporal` are static (just like the `Math` object).

`Temporal` has an intricate and powerful API. It exposes over 200 utility methods via several classes, so it could appear very complex. We will provide a high-level overview of how these APIs are related to each other.

Background and concepts

JavaScript has had the `Date` object for handling date and time since its first days. However, the `Date` API is based on the poorly designed `java.util.Date` class from Java, which was replaced in the early 2010s; but, because of JavaScript's goal of backward compatibility, `Date` sticks around in the language.

The important lesson to preface the whole introduction is that **date handling is complex**. Most of the problems of `Date` are fixable by adding more methods, but a fundamental design flaw remains: it exposes so many methods on the same object that developers are often confused about what to use, leading to unexpected pitfalls. A well-designed API not only needs to do more, but also should do *less* with each level of abstraction, because preventing misuse is as important as enabling use cases.

`Date` objects wear two hats simultaneously:

- As a timestamp: the number of milliseconds or nanoseconds elapsed since a fixed point in time (known as the *epoch*).
- As a combination of components: year, month, day, hour, minute, second, millisecond, and nanosecond. The year, month, and day identifiers only make sense with reference to a *calendar system*. The whole combination maps to a unique instant in history when associated with a time zone. `Date` objects provide methods for reading and modifying these components.

Time zones underlie a significant number of date-related bugs. When interacting with a `Date` via the "combination of components" model, the time can only be in two time zones: UTC and local (device), and there's no way to specify an arbitrary time zone. Also lacking is the concept of "no time zone": this is known as a *calendar date* (for dates) or *wall-clock time* (for times), which is a time you "read off a calendar or clock". For example, if you are setting a daily wake up alarm, you will want to set it to "8:00AM" regardless of whether it is daylight saving time or not, whether you have traveled to a different time zone, etc.

A second feature lacking from `Date` is a calendar system. Most people may be familiar with the Gregorian calendar, where there are two eras, BC and AD; there are 12 months; each month has a different number of days; there's a leap year every 4 years; and so on. However, some of these concepts may not apply when you are working with another calendar system, such as the Hebrew calendar, the

Chinese calendar, the Japanese calendar, etc. With `Date`, you can only work with the Gregorian calendar model.

There are many other undesirable legacies about `Date`, such as all setters being mutating (which often causes unwanted side effects), the date time string format being impossible to parse in a consistent way, etc. In the end, the best solution is to build a new API from scratch, which is what `Temporal` is.

API overview

`Temporal` is a namespace, like `Intl`. It contains several classes and namespaces, each of which is designed to handle a specific aspect of date and time management. The classes can be grouped as such:

- Representing a time duration (a difference between two time points): `Temporal.Duration`
- Representing a time point:
  - Representing a unique instant in history:
    - As a timestamp: `Temporal.Instant`
    - As a date-time component combination paired with a time zone: `Temporal.ZonedDateTime`
  - Representing a time-zone-unaware date/time (which are all prefixed with "Plain"):
    - Date (year, month, day) + time (hour, minute, second, millisecond, microsecond, nanosecond): `Temporal.PlainDateTime` (Note: `ZonedDateTime` is equivalent to `PlainDateTime` plus a time zone)
      - Date (year, month, day): `Temporal.PlainDate`
        - Year, month: `Temporal.PlainYearMonth`
        - Month, day: `Temporal.PlainMonthDay`
      - Time (hour, minute, second, millisecond, microsecond, nanosecond): `Temporal.PlainTime`

Furthermore, there's also another utility namespace, `Temporal.Now`, which provides methods for getting the current time in various formats.

Shared class interface

There are many classes in the `Temporal` namespace, but they share many similar methods. The following table lists all methods of each class (except conversion methods):

	Instant	ZonedDateTime	PlainDateTime	PlainDate	PlainTime	PlainYearMonth	PlainMonthDay
Construction	Instant() Instant.from() Instant.fromEpochMilliseconds() Instant.fromEpochNanoseconds()	ZonedDateTime() ZonedDateTime.from()	PlainDateTime() PlainDateTime.from()	PlainDate() PlainDate.from()	PlainTime() PlainTime.from()	PlainYearMonth() PlainYearMonth.from()	PlainMonthDay() PlainMonthDay.from()
Updater	N/A	with() withCalendar() withTimeZone() withPlainTime()	with() withCalendar() withPlainTime()	with() withCalendar()	with()	with()	with()
Arithmetic	add() subtract() since() until()	add() subtract() since() until()	add() subtract() since() until()	add() subtract() since() until()	add() subtract() since() until()	add() subtract() since() until()	N/A
Rounding	round()	round()	round()	N/A	round()	N/A	N/A
Comparison	equals() Instant.compare()	equals() ZonedDateTime.compare()	equals() PlainDateTime.compare()	equals() PlainDate.compare()	equals() PlainTime.compare()	equals() PlainYearMonth.compare()	equals()
Serialization	toJSON() toLocaleString() toString() valueOf()	toJSON() toLocaleString() toString() valueOf()	toJSON() toLocaleString() toString() valueOf()	toJSON() toLocaleString() toString() valueOf()	toJSON() toLocaleString() toString() valueOf()	toJSON() toLocaleString() toString() valueOf()	toJSON() toLocaleString() toString() valueOf()

The following table summarizes which properties are available on each class, giving you a sense of what information each class can represent.

	Instant	ZonedDateTime	PlainDateTime	PlainDate	PlainTime	PlainYearMonth	PlainMonthDay
Calendar	N/A	calendarId	calendarId	calendarId	N/A	calendarId	calendarId
Year-related	N/A	era eraYear year inLeapYear monthsInYear daysInYear	era eraYear year inLeapYear monthsInYear daysInYear	era eraYear year inLeapYear monthsInYear daysInYear	N/A	era eraYear year inLeapYear monthsInYear daysInYear	N/A
Month-related	N/A	month monthCode daysInMonth	month monthCode daysInMonth	month monthCode daysInMonth	N/A	month monthCode daysInMonth	monthCode
Week-related	N/A	weekOfYear yearOfWeek daysInWeek	weekOfYear yearOfWeek daysInWeek	weekOfYear yearOfWeek daysInWeek	N/A	N/A	N/A
Day-related	N/A	day dayOfWeek dayOfYear	day dayOfWeek dayOfYear	day dayOfWeek dayOfYear	N/A	N/A	day
Time components	N/A	hour minute second millisecond microsecond nanosecond	hour minute second millisecond microsecond nanosecond	N/A	hour minute second millisecond microsecond nanosecond	N/A	N/A
Time zone	N/A	timeZoneId offset offsetNanoseconds hoursInDay getTimeZoneTransition() startOfDay()	N/A	N/A	N/A	N/A	N/A
Epoch time	epochMilliseconds epochNanoseconds	epochMilliseconds epochNanoseconds	N/A	N/A	N/A	N/A	N/A

Conversion between classes

The table below summarizes all conversion methods that exist on each class.

How to convert from…							
to…	**Instant**	**ZonedDateTime**	**PlainDateTime**	**PlainDate**	**PlainTime**	**PlainYearMonth**	**PlainMonthDay**
Instant	/	`toInstant()`	Convert to `ZonedDateTime` first				
ZonedDateTime	`toZonedDateTimeISO()`	/	`toZonedDateTime()`	`toZonedDateTime()`	`PlainDate#toZonedDateTime()` (pass as argument)	Convert to `PlainDate` first	
PlainDateTime	Convert to `ZonedDateTime` first	`toPlainDateTime()`	/	`toPlainDateTime()`	`PlainDate#toPlainDateTime()` (pass as argument)		
PlainDate		`toPlainDate()`	`toPlainDate()`	/	No overlap in information	`toPlainDate()`	`toPlainDate()`
PlainTime		`toPlainTime()`	`toPlainTime()`	No overlap in information	/	No overlap in information	
PlainYearMonth	Convert to `PlainDate` first			`toPlainYearMonth()`	No overlap in information	/	Convert to `PlainDate` first
PlainMonthDay				`toPlainMonthDay()`		Convert to `PlainDate` first	/

With these tables, you should have a basic idea of how to navigate the `Temporal` API.

Calendars

A calendar is a way to organize days, typically into periods of weeks, months, years, and eras. Most of the world uses the Gregorian calendar, but there are many other calendars in use, especially in religious and cultural contexts. By default, all calendar-aware `Temporal` objects use the ISO 8601 calendar system, which is based on the Gregorian calendar and defines additional week-numbering rules. `Intl.supportedValuesOf()` lists most of the calendars likely to be supported by browsers. Here we provide a brief overview of how calendar systems are formed to help you internalize what factors may vary between calendars.

There are three prominent periodic events on Earth: its rotation around the sun (365.242 days for one revolution), the moon's rotation around the Earth (29.53 days from new moon to new moon), and its rotation around its axis (24 hours from sunrise to sunrise). Every culture has the same measure of a "day", which is 24 hours. Occasional changes such as daylight saving time are not part of the calendar, but are part of the time zone's information.

- Some calendars primarily define one year as 365.242 days on average, by defining years to have 365 days, and adding an extra day, the *leap day*, about every 4 years. Then, the year may be further divided into parts called months. These calendars are called *solar calendars*. The Gregorian calendar and the Solar Hijri calendar are solar calendars.
- Some calendars primarily define one month as 29.5 days on average, by defining months to alternate between 29 and 30 days. Then, 12 months may be grouped into a year of 354 days. These calendars are called *lunar calendars*. The Islamic calendar is a lunar calendar. Because a lunar year is artificial and does not correlate with the season cycle, lunar calendars are generally rarer.
- Some calendars also primarily define months based on lunar cycles, like lunar calendars. Then, to compensate for the 11-day discrepancy with the solar year, an extra month, the *leap month*, is added about every 3 years. These calendars are called *lunisolar calendars*. The Hebrew calendar and the Chinese calendar are lunisolar calendars.

In `Temporal`, every date under one calendar system is uniquely identified by three components: `year`, `month`, and `day`. While `year` is typically a positive integer, it may also be zero or negative, and increases monotonically with time. The year `1` (or `0`, if it exists) is known as the calendar epoch, and is arbitrary for each calendar. `month` is a positive integer that increments by 1 every time, starting at `1` and ending at `date.monthsInYear`, then resetting back to `1` as the year advances. `day` is also a positive integer, but it may not start at 1 or increment by 1 every time, because political changes may cause days to be skipped or repeated. But in general, `day` monotonically increases and resets as the month advances.

In addition to `year`, a year can also be uniquely identified by the combination of `era` and `eraYear`, for calendars that use eras. For example, the Gregorian calendar uses the era "CE" (Common Era) and "BCE" (Before Common Era), and the year `-1` is the same as `{ era: "bce", eraYear: 2 }` (note that year `0` always exists for all calendars; for the Gregorian calendar, it corresponds to 1 BCE due to astronomical year numbering). `era` is a lowercase string, and `eraYear` is an arbitrary integer that may be zero or negative, or even decrease with time (usually for the oldest era).

> **Note:** Always use `era` and `eraYear` as a pair; don't use one property without also using the other. In addition, to avoid conflicts, don't combine `year` and `era`/`eraYear` when designating a year. Pick one year representation and use it consistently.
>
> Be careful of the following incorrect assumptions about years:
>
> - Don't assume that `era` and `eraYear` are always present; they may be `undefined`.
> - Don't assume that `era` is a user-friendly string; use `toLocaleString()` to format your date instead.
> - Don't assume that two `year` values from different calendars are comparable; use the `compare()` static method instead.
> - Don't assume that years have 365/366 days and 12 months; use `daysInYear` and `monthsInYear` instead.
> - Don't assume that leap years (`inLeapYear` is `true`) have one extra day; they may have an extra month.

In addition to `month`, a month in a year can also be uniquely identified by the `monthCode`. `monthCode` usually maps to the month's name, but `month` does not. For example, in the case of lunisolar calendars, two months with the same `monthCode`, where one belongs to a leap year and the other one does not, will have different `month` values if they come after the leap month, due to the insertion of an extra month.

> **Note:** To avoid conflicts, don't combine `month` and `monthCode` when designating a month. Pick one month representation and use it consistently. `month` is more useful if you need the order of months in a year (e.g., when looping through the months), while `monthCode` is more useful if you need the name of the month (e.g., when storing birthdays).
>
> Be careful of the following incorrect assumptions about months:
>
> - Don't assume that `monthCode` and `month` always correspond.
> - Don't assume the number of days in a month; use `daysInMonth` instead.
> - Don't assume that `monthCode` is a user-friendly string; use `toLocaleString()` to format your date instead.
> - Generally, don't cache the name of months in an array or object. Even though `monthCode` usually maps to the month's name within one calendar, we recommend always computing the month's name using, for example, `date.toLocaleString("en-US", { calendar: date.calendarId, month: "long" })`.

In addition to `day` (which is a month-based index), a day in a year can also be uniquely identified by the `dayOfYear`. `dayOfYear` is a positive integer that increments by 1 every time, starting at `1` and ending at `date.daysInYear`.

The concept of a "week" is not connected with any astronomical event, but is a cultural construct. While the most common length is `7` days, weeks can also have 4, 5, 6, 8, or more days — or even lack a fixed number of days altogether. To get the specific number of days of the week of a date, use the date's `daysInWeek`. `Temporal` identifies weeks by the combination of `weekOfYear` and `yearOfWeek`. `weekOfYear` is a positive integer that increments by 1 every time, starting at `1`, then resetting back to `1` as the year advances. `yearOfWeek` is generally the same as `year`, but may be different at the start or end of each year, because one week may cross two years, and `yearOfWeek` picks one of the two years based on the calendar's rules.

> **Note:** Always use `weekOfYear` and `yearOfWeek` as a pair; don't use `weekOfYear` and `year`.
>
> Be careful of the following incorrect assumptions about weeks:
>
> - Don't assume that `weekOfYear` and `yearOfWeek` are always present; they may be `undefined`.
> - Don't assume that weeks are always 7 days long; use `daysInWeek` instead.
> - Note that the current `Temporal` API does not support year-week dates, so you can't construct dates using these properties or serialize dates to year-week representations. They are only informational properties.

RFC 9557 format

All `Temporal` classes can be serialized and deserialized using the format specified in RFC 9557, which is based on ISO 8601 / RFC 3339. The format, in its full form, is as follows (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss Z/±HH:mm [time_zone_id] [u-ca=calendar_id]
```

Different classes have different requirements for the presence of each component, so you will find a section titled "RFC 9557 format" in each class's documentation, which specifies the format recognized by that class.

This is very similar to the date time string format used by `Date`, which is also based on ISO 8601. The main addition is the ability to specify micro- and nanosecond components, and the ability to specify the time zone and calendar system.

Representable dates

All `Temporal` objects that represent a specific calendar date impose a similar limit on the range of representable dates, which is $\pm 10^8$ days (inclusive) from the Unix epoch, or the range of instants from `-271821-04-20T00:00:00` to `+275760-09-13T00:00:00`. This is the same range as valid dates. More specifically:

- `Temporal.Instant` and `Temporal.ZonedDateTime` apply this limit directly on its `epochNanoseconds` value.
- `Temporal.PlainDateTime` interprets the date-time in the UTC time zone and requires it to be $\pm(10^8 + 1)$ days (exclusive) from the Unix epoch, so its valid range is `-271821-04-19T00:00:00` to `+275760-09-14T00:00:00`, exclusive. This allows any `ZonedDateTime` to be converted to a `PlainDateTime` regardless of its offset.
- `Temporal.PlainDate` applies the same check as `PlainDateTime` to the noon (`12:00:00`) of that date, so its valid range is `-271821-04-19` to `+275760-09-13`. This allows any `PlainDateTime` to be converted to a `PlainDate` regardless of its time, and vice versa.
- `Temporal.PlainYearMonth` has the valid range of `-271821-04` to `+275760-09`. This allows any `PlainDate` to be converted to a `PlainYearMonth` regardless of its date (except if a non-ISO month's first day falls in the ISO month `-271821-03`).

The `Temporal` objects will refuse to construct an instance representing a date/time beyond this limit. This includes:

- Using the constructor or `from()` static method.
- Using the `with()` method to update calendar fields.
- Using `add()`, `subtract()`, `round()`, or any other method to derive new instances.

Static properties

`Temporal.Duration`

Represents a difference between two time points, which can be used in date/time arithmetic. It is fundamentally represented as a combination of years, months, weeks, days, hours, minutes, seconds, milliseconds, microseconds, and nanoseconds values.

`Temporal.Instant`

Represents a unique point in time, with nanosecond precision. It is fundamentally represented as the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC), without any time zone or calendar system.

`Temporal.Now`

Provides methods for getting the current time in various formats.

`Temporal.PlainDate`

Represents a calendar date (a date without a time or time zone); for example, an event on a calendar which happens during the whole day no matter which time zone it's happening in. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system.

`Temporal.PlainDateTime`

Represents a date (calendar date) and time (wall-clock time) without a time zone. It is fundamentally represented as a combination of a date (with an associated calendar system) and a time.

`Temporal.PlainMonthDay`

Represents the month and day of a calendar date, without a year or time zone; for example, an event on a calendar that recurs every year and happens during the whole day. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The year is used to disambiguate the month-day in non-ISO calendar systems.

`Temporal.PlainTime`

Represents a time without a date or time zone; for example, a recurring event that happens at the same time every day. It is fundamentally represented as a combination of hour, minute, second, millisecond, microsecond, and nanosecond values.

`Temporal.PlainYearMonth`

Represents the year and month of a calendar date, without a day or time zone; for example, an event on a calendar that happens during the whole month. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The day is used to disambiguate the year-month in non-ISO calendar systems.

`Temporal.ZonedDateTime`

Represents a date and time with a time zone. It is fundamentally represented as a combination of an instant, a time zone, and a calendar system.

`Temporal[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal"`. This property is used in `Object.prototype.toString()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
Temporal API	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
Duration	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
Duration() constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
abs	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
blank	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
days	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
hours	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
microseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
milliseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
minutes	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
months	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
nanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
negated	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
seconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
sign	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
total	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
valueOf	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
weeks	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
with	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
years	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
Instant	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
Instant() constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
epochMilliseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
epochNanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
fromEpochMilliseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
fromEpochNanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toZonedDateTimeISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
until	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
valueOf	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
Now	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
instant	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
plainDateISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
plainDateTimeISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
plainTimeISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
timeZoneId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
zonedDateTimeISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
PlainDate	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
PlainDate() constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
calendarId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
era	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
inLeapYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
month	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthsInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`toPlainDateTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainMonthDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainYearMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toZonedDateTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`weekOfYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withCalendar`	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`year`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainDateTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainDateTime()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0
`add`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`day`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`dayOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`dayOfYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`daysInMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`daysInWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`daysInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`era`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`eraYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`hour`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`inLeapYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`microsecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`millisecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`minute`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`month`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthCode`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthsInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`nanosecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`round`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`second`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`since`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toZonedDateTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`weekOfYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withCalendar`	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`year`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainMonthDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainMonthDay()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`day`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthCode`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainTime()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`add`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`hour`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`microsecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`millisecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`minute`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`nanosecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`round`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`second`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`since`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
until	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
valueOf	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
with	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
PlainYearMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
PlainYearMonth() constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
calendarId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
era	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
inLeapYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
month	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthsInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainDate	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
until	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
valueOf	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
with	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
year	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
ZonedDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
ZonedDateTime() constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
calendarId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
epochMilliseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`epochNanoseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`era`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`eraYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`getTimeZoneTransition`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`hour`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`hoursInDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`inLeapYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`microsecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`millisecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`minute`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`month`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthCode`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthsInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`nanosecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`offset`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`offsetNanoseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`round`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`second`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`since`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`startOfDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`timeZoneId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toInstant`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainDateTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`weekOfYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withCalendar`	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withTimeZone`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`year`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

1. Partial support.
2. Calendars `islamic`, `islamic-rgsa` and `islamic-umalqura` are not available. See bug 1950425 and bug 1954138.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`
- `Intl.RelativeTimeFormat`
- `Intl.DurationFormat`
- Temporal polyfill by proposal champions
- Temporal polyfill by FullCalendar

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.Duration

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Duration`** object represents a difference between two time points, which can be used in date/time arithmetic. It is fundamentally represented as a combination of years, months, weeks, days, hours, minutes, seconds, milliseconds, microseconds, and nanoseconds values.

Description

ISO 8601 duration format

`Duration` objects can be serialized and parsed using the ISO 8601 duration format (with some extensions specified by ECMAScript). The string has the following form (spaces are only for readability and should not be present in the actual string):

```
±P nY nM nW nD T nH nM nS
```

± Optional

An optional sign character (+ or -), which represents positive or negative duration. Default is positive.

P

A literal character `P` or `p` that stands for "period".

nY, nM, nW, nD, nH, nM, nS

A number followed by a literal character, which represents the number of years (`Y`), months (`M`), weeks (`W`), days (`D`), hours (`H`), minutes (`M`), or seconds (`S`), respectively. All except the last existing component must be an integer. The last component, if it is a time component (hours, minutes, or seconds), may have a fractional part of 1 to 9 digits, led by a dot or a comma, such as `PT0.0021S` or `PT1.1H`. Any zero components may be omitted, but at least one component should be present (even if it has value zero, in which case the duration is zero).

T

A literal character `T` or `t` that separates the date part from the time part, which should be present if and only if there's at least one component after it.

Here are some examples:

ISO 8601	Meaning
`P1Y1M1DT1H1M1.1S`	1 year, 1 month, 1 day, 1 hour, 1 minute, 1 second, and 100 milliseconds
`P40D`	40 days
`P1Y1D`	1 year and 1 day
`P3DT4H59M`	3 days, 4 hours and 59 minutes
`PT2H30M`	2 hours and 30 minutes
`P1M`	1 month
`PT1M`	1 minute
`PT0.0021S`	2.1 milliseconds (2 milliseconds and 100 microseconds)
`PT0S`	Zero (canonical representation)

ISO 8601	Meaning
`P0D`	Zero

> **Note:** According to the ISO 8601-1 standard, weeks are not allowed to appear together with any other units, and durations can only be positive. As extensions to the standard, ISO 8601-2, which Temporal uses, allows a sign character at the start of the string, and allows combining weeks with other units. Therefore, if your duration is serialized to a string like `P3W1D`, `+P1M`, or `-P1M`, note that other programs may not accept it.

When serializing, the output respects the stored components as much as possible, preserving unbalanced components. However, subsecond components are serialized as a single fractional second, so their precise values, if unbalanced may be lost. The plus sign is omitted for positive durations. The zero duration is always serialized as `PT0S`.

Calendar durations

A *calendar duration* is one that contains any of the calendar units: weeks, months, and years. A non-calendar duration is portable and can participate in date/time arithmetic without any calendar information, because they unambiguously represent a fixed amount of time. However, a calendar duration is not portable because the number of days in a month or year depends on the calendar system and the reference time point. Therefore, attempting to perform any arithmetic operation on a calendar durations throws an error because durations don't keep track of a calendar themselves. For example, if we are in May of the Gregorian calendar, then "1 month" is "31 days", but if we are in April, then "1 month" becomes "30 days". To add or subtract calendar durations, you need to add them to dates instead:

```js
const dur1 = Temporal.Duration.from({ years: 1 });
const dur2 = Temporal.Duration.from({ months: 1 });

dur1.add(dur2); // RangeError: for calendar duration arithmetic, use date arithmetic
relative to a starting point

const startingPoint = Temporal.PlainDate.from("2021-01-01"); // ISO 8601 calendar
startingPoint.add(dur1).add(dur2).since(startingPoint); // "P396D"
```

Other operations, `round()`, `total()`, and `compare()`, take a `relativeTo` option to provide the necessary calendar and reference time information. This option can be a `Temporal.PlainDate`, `Temporal.PlainDateTime`, `Temporal.ZonedDateTime`, or otherwise an object or string that's convertible using `Temporal.ZonedDateTime.from()` (if the `timeZone` option is provided or the string contains time zone annotation) or `Temporal.PlainDate.from()`.

Note that `days` to `hours` conversion is also technically ambiguous because the length of a day may vary due to offset changes, such as daylight saving time. You can provide a zoned `relativeTo` to account for these changes; otherwise 24-hour days are assumed.

Duration balancing

There are many ways to represent the same duration: for example, "1 minute and 30 seconds" and "90 seconds" are equivalent. However, depending on the context, one representation may be more appropriate than the other. Therefore, generally, the `Duration` object preserves the input values as much as possible, so that when you format it, it will be displayed as you expect.

Each component of a duration has its optimal range; hours should be from 0 to 23, minutes from 0 to 59, and so on. When a component overflows its optimal range, the excess may be "carried" into the next larger component. To carry over, we need to answer the question of "how many X are in a Y?", which is a complicated question for calendar units, so in this case a calendar is needed. Also note that by default, `days` are directly carried into `months`; the weeks unit is only carried into if explicitly requested. If we carry as much as possible, the eventual result where all components are within their optimal range is called a "balanced" duration. Unbalanced durations usually come in the "top-heavy" form, where the largest unit is unbalanced (e.g., "27 hours and 30 minutes"); other forms, such as "23 hours and 270 minutes", are rarely seen.

The `round()` method always balances the duration into the "top-heavy" form, up to the `largestUnit` option. With a manual `largestUnit` option that's large enough, you can fully balance the duration. Similarly, the `add()` and `subtract()` methods balance the result duration to the largest unit of the input durations.

Note that because the ISO 8601 duration format represents subsecond components as one single fraction number, it is not possible to preserve unbalanced subsecond components during serialization using the default format. For example, "1000 milliseconds" is serialized as `"PT1S"`, and then deserialized as "1 second". If you need to preserve the subsecond components' magnitudes, you need to manually serialize it as a JSON object instead (because by default the `toJSON()` method serializes the duration in the ISO 8601 format).

Duration sign

Because a duration is a difference between two time points, it can be positive, negative, or zero. For example, if you are displaying event times in relative time, then negative durations may represent events in the past, and positive durations for the future. In our representation using a combination of time components, the sign is stored within each component: a negative duration always has all components negative (or zero), and a positive duration always has all components positive (or zero). Constructing a duration with components of mixed signs is invalid and will be rejected by the constructor or the `with()` method. The `add()` and `subtract()` methods will balance the result duration to avoid mixed signs.

Constructor

`Temporal.Duration()`

Creates a new `Temporal.Duration` object by directly supplying the underlying data.

Static methods

`Temporal.Duration.compare()`

Returns a number (-1, 0, or 1) indicating whether the first duration is shorter, equal to, or longer than the second duration.

`Temporal.Duration.from()`

Creates a new `Temporal.Duration` object from another `Temporal.Duration` object, an object with duration properties, or an ISO 8601 string.

Instance properties

These properties are defined on `Temporal.Duration.prototype` and shared by all `Temporal.Duration` instances.

`Temporal.Duration.prototype.blank`

Returns a boolean that is `true` if this duration represents a zero duration, and `false` otherwise. Equivalent to `duration.sign === 0`.

`Temporal.Duration.prototype.constructor`

The constructor function that created the instance object. For `Temporal.Duration` instances, the initial value is the `Temporal.Duration()` constructor.

`Temporal.Duration.prototype.days`

Returns an integer representing the number of days in the duration.

`Temporal.Duration.prototype.hours`

Returns an integer representing the number of hours in the duration.

`Temporal.Duration.prototype.microseconds`

Returns an integer representing the number of microseconds in the duration.

`Temporal.Duration.prototype.milliseconds`

Returns an integer representing the number of milliseconds in the duration.

`Temporal.Duration.prototype.minutes`

Returns an integer representing the number of minutes in the duration.

`Temporal.Duration.prototype.months`

Returns an integer representing the number of months in the duration.

`Temporal.Duration.prototype.nanoseconds`

Returns an integer representing the number of nanoseconds in the duration.

`Temporal.Duration.prototype.seconds`

Returns an integer representing the number of seconds in the duration.

`Temporal.Duration.prototype.sign`

Returns `1` if this duration is positive, `-1` if negative, and `0` if zero.

`Temporal.Duration.prototype.weeks`

Returns an integer representing the number of weeks in the duration.

`Temporal.Duration.prototype.years`

Returns an integer representing the number of years in the duration.

`Temporal.Duration.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.Duration"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.Duration.prototype.abs()`

Returns a new `Temporal.Duration` object with the absolute value of this duration (all fields keep the same magnitude, but sign becomes positive).

`Temporal.Duration.prototype.add()`

Returns a new `Temporal.Duration` object with the sum of this duration and a given duration (in a form convertible by `Temporal.Duration.from()`). The result is balanced.

`Temporal.Duration.prototype.negated()`

Returns a new `Temporal.Duration` object with the negated value of this duration (all fields keep the same magnitude, but sign becomes reversed).

`Temporal.Duration.prototype.round()`

Returns a new `Temporal.Duration` object with the duration rounded to the given smallest unit and/or balanced to the given largest unit.

`Temporal.Duration.prototype.subtract()`

Returns a new `Temporal.Duration` object with the difference between this duration and a given duration (in a form convertible by `Temporal.Duration.from()`). Equivalent to adding the negated value of the other duration.

`Temporal.Duration.prototype.toJSON()`

Returns a string representing this duration in the same ISO 8601 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.Duration.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this duration. In implementations with `Intl.DurationFormat` API support, this method delegates to `Intl.DurationFormat`.

`Temporal.Duration.prototype.toString()`

Returns a string representing this duration in the ISO 8601 format.

`Temporal.Duration.prototype.total()`

Returns a number representing the total duration in the given unit.

`Temporal.Duration.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.Duration` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.Duration.prototype.with()`

Returns a new `Temporal.Duration` object representing this duration with some fields replaced by new values.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-duration-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Duration`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`Duration()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`abs`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`add`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`blank`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`days`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`hours`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`microseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`milliseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`minutes`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`months`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`nanoseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`negated`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`round`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`seconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`sign`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`total`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`weeks`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`years`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/Duration · MDN page last modified 2026-09-13

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Duration()`** constructor creates `Temporal.Duration` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.Duration` objects using the `Temporal.Duration.from()` static method, which can handle a variety of input types.

Syntax

```
new Temporal.Duration()
new Temporal.Duration(years)
new Temporal.Duration(years, months)
new Temporal.Duration(years, months, weeks)
new Temporal.Duration(years, months, weeks, days)
new Temporal.Duration(years, months, weeks, days, hours)
new Temporal.Duration(years, months, weeks, days, hours, minutes)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds, milliseconds)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds, milliseconds,
microseconds)
new Temporal.Duration(years, months, weeks, days, hours, minutes, seconds, milliseconds,
microseconds, nanoseconds)
```

> **Note:** `Temporal.Duration()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`years` Optional

Number of years, or `undefined` (which is treated as `0`).

`months` Optional

Number of months, or `undefined` (which is treated as `0`).

`weeks` Optional

Number of weeks, or `undefined` (which is treated as `0`).

`days` Optional

Number of days, or `undefined` (which is treated as `0`).

`hours` Optional

Number of hours, or `undefined` (which is treated as `0`).

`minutes` Optional

Number of minutes, or `undefined` (which is treated as `0`).

`seconds` Optional

Number of seconds, or `undefined` (which is treated as `0`).

`milliseconds` Optional

Number of milliseconds, or `undefined` (which is treated as `0`).

`microseconds` Optional

Number of microseconds, or `undefined` (which is treated as `0`).

`nanoseconds` Optional

Number of nanoseconds, or `undefined` (which is treated as `0`).

Return value

A new `Temporal.Duration` object, possibly unbalanced, with the specified components.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the parameters is not an integer (including non-finite values).
- A calendar unit (years, months, weeks) has an absolute value ≥ 2^{32}.
- The non-calendar part of the duration (days and below), when expressed in seconds, has an absolute value ≥ 2^{53}.

Examples

Using Temporal.Duration()

```js
const d = new Temporal.Duration(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
console.log(d.toString()); // "P1Y2M3W4DT5H6M7.00800901S"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-duration-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Duration()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.Duration`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/compare · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Duration.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first duration is shorter, equal to, or longer than the second duration.

Syntax

```js
Temporal.Duration.compare(duration1, duration2)
Temporal.Duration.compare(duration1, duration2, options)
```

Parameters

`duration1`

A string, an object, or a `Temporal.Duration` instance representing the first duration to compare. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`duration2`

The second duration to compare, converted to a `Temporal.Duration` object using the same algorithm as `duration1`.

`options` Optional

An object containing the following property:

`relativeTo` Optional

A zoned or plain date(time) that provides the time and calendar information to resolve calendar durations (see the link for the general interpretation of this option). Required if either `duration1` or `duration2` is a calendar duration (unless they are equal component-wise, in which case `0` is returned without computations).

Return value

Returns `-1` if `duration1` is shorter than `duration2`, `0` if they are equal, and `1` if `duration1` is longer than `duration2`.

Exceptions

`RangeError`

Thrown if either `duration1` or `duration2` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), and `relativeTo` is not provided.

Description

If `relativeTo` is a zoned date-time, and either `duration1` or `duration2` is a calendar duration, the result is calculated by adding the durations to the starting point, and then comparing the resulting instants. Otherwise, the comparison is done by converting both of them to nanoseconds (assuming 24-hour days, and using the calendar of `relativeTo` if necessary) and comparing the results.

Examples

Using Temporal.Duration.compare()

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ minutes: 100 });
console.log(Temporal.Duration.compare(d1, d2)); // -1

const d3 = Temporal.Duration.from({ hours: 2 });
const d4 = Temporal.Duration.from({ minutes: 110 });
console.log(Temporal.Duration.compare(d3, d4)); // 1

const d5 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d6 = Temporal.Duration.from({ seconds: 5400 });
console.log(Temporal.Duration.compare(d5, d6)); // 0
```

Comparing calendar durations

```js
const d1 = Temporal.Duration.from({ days: 31 });
const d2 = Temporal.Duration.from({ months: 1 });

console.log(
  Temporal.Duration.compare(d1, d2, {
    relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
  }),
); // 0

console.log(
  Temporal.Duration.compare(d1, d2, {
    relativeTo: Temporal.PlainDate.from("2021-02-01"), // ISO 8601 calendar
  }),
); // 1; February has 28 days
```

Using zoned relativeTo

Using a zoned `relativeTo`, you can even take into account daylight saving time changes. On `2024-11-03`, the US shifts from daylight saving time to standard time, so that day has 25 hours because the clock is set back by 1 hour.

```js
const d1 = Temporal.Duration.from({ days: 1 });
const d2 = Temporal.Duration.from({ hours: 24 });

console.log(
  Temporal.Duration.compare(d1, d2, {
    relativeTo: Temporal.ZonedDateTime.from(
      "2024-11-03T01:00-04:00[America/New_York]",
    ),
  }),
); // 1
```

Sorting an array of durations

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```js
const durations = [
  Temporal.Duration.from({ hours: 1 }),
  Temporal.Duration.from({ hours: 2 }),
  Temporal.Duration.from({ hours: 1, minutes: 30 }),
  Temporal.Duration.from({ hours: 1, minutes: 45 }),
];

durations.sort(Temporal.Duration.compare);
console.log(durations.map((d) => d.toString()));
// [ 'PT1H', 'PT1H30M', 'PT1H45M', 'PT2H' ]
```

Pass options like this:

```js
durations.sort((a, b) =>
  Temporal.Duration.compare(a, b, {
    relativeTo: Temporal.Now.zonedDateTimeISO(),
  }),
);
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.compare

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.compare`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/from · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Duration.from()`** static method creates a new `Temporal.Duration` object from another `Temporal.Duration` object, an object with duration properties, or an ISO 8601 string.

Syntax

```js
Temporal.Duration.from(info)
```

Parameters

`info`

One of the following:

- A `Temporal.Duration` instance, which creates a copy of the instance.
- An ISO 8601 string representing a duration.
- An object containing at least one of the following properties (in the order they are retrieved and validated):
  - `days`
  - `hours`
  - `microseconds`
  - `milliseconds`
  - `minutes`
  - `months`
  - `nanoseconds`
  - `seconds`
  - `weeks`
  - `years`

  Each property should contain an integer number value. The resulting duration must not have mixed signs, so all of these properties must have the same sign (or zero). Missing properties are treated as zero.

Return value

A new `Temporal.Duration` object, possibly unbalanced, with the specified components.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the recognized properties in the `info` object is not an integer (including non-finite values).
- A calendar unit (years, months, weeks) has an absolute value ≥ 2^{32}.

- The non-calendar part of the duration (days and below), when expressed in seconds, has an absolute value ≥ 2^{53}.

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- All of the recognized properties in the `info` object are `undefined`.

Examples

Creating a duration from an object

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
console.log(d1.toString()); // "PT1H30M"

const d2 = Temporal.Duration.from({ months: 1, days: 2 });
console.log(d2.toString()); // "P1M2D"

// Uncommon because unbalanced, but valid
const unbalanced = Temporal.Duration.from({
  hours: 100,
  minutes: 100,
  seconds: 100,
});
console.log(unbalanced.toString()); // "PT100H100M100S"

const neg = Temporal.Duration.from({ hours: -1, minutes: -30 });
console.log(neg.toString()); // "-PT1H30M"
```

Creating a duration from a string

```js
const d = Temporal.Duration.from("P1Y2M3W4DT5H6M7.00800901S");
console.log(d.hours); // 5
```

Creating a duration from another duration

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from(d1);
console.log(d2.toString()); // "PT1H30M"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.duration.from`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.from`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration()`
- `Temporal.Duration.prototype.with()`

Temporal.Duration.prototype.abs()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/abs · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`abs()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the absolute value of this duration (all fields have the same magnitude, but sign becomes positive).

Syntax

```js
abs()
```

Parameters

None.

Return value

A new `Temporal.Duration` object with the absolute value of this duration, which is either the same as this duration if it is already positive, or its negation if it is negative.

Examples

Using abs()

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });

console.log(d1.abs().toString()); // "PT1H30M"
console.log(d2.abs().toString()); // "PT1H30M"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.abs

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
abs	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.abs`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.negated()`
- `Temporal.Duration.prototype.sign`

Temporal.Duration.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/add · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`add()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the sum of this duration and a given duration. The result is balanced.

Syntax

```js
add(other)
```

Parameters

`other`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this duration. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Duration` object representing the sum of this duration and `other`.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Either `this` or `other` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), because calendar durations are ambiguous without a calendar and time reference.
- The sum of `this` and `other` overflows the maximum or underflows the minimum representable duration, which is $\pm 2^{53}$ seconds.

Description

Non-calendar durations unambiguously represent a fixed amount of time. Internally, `this` and `other` are both converted to nanoseconds (assuming 24-hour days) and added together. The result is then converted back to a `Temporal.Duration` object, so the result is always balanced or top-heavy with the largest possible unit being `days`.

If you want to perform addition or subtraction with a calendar duration, you can add both durations to a starting point and then figure out the difference between the two resulting instants; that is, `dur1 + dur2` is equivalent to `(start + dur1 + dur2) - start`.

To add a duration to a date or time, use the `add()` method of the date or time object instead.

Examples

Using add()

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -20 });

const d3 = d1.add(d2);
console.log(d3.toString()); // "PT10M"
```

Adding calendar durations

```js
const d1 = Temporal.Duration.from({ days: 1 });
const d2 = Temporal.Duration.from({ months: 1 });

d1.add(d2); // RangeError: for calendar duration arithmetic, use date arithmetic relative to a starting point

const start = Temporal.PlainDateTime.from("2022-01-01T00:00"); // ISO 8601 calendar
const result = start.add(d1).add(d2).since(start);
console.log(result.toString()); // "P32D"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.add`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.negated()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/negated · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`negated()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the negated value of this duration (all fields keep the same magnitude, but sign becomes reversed).

Syntax

```
negated()
```

Parameters

None.

Return value

A new `Temporal.Duration` object, where all fields have the same magnitude as this duration, but the sign is reversed (positive fields become negative, and vice versa).

Examples

Using negated()

```
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });

console.log(d1.negated().toString()); // "-PT1H30M"
console.log(d2.negated().toString()); // "PT1H30M"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.negated`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`negated`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.negated`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.abs()`
- `Temporal.Duration.prototype.sign`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.round()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/round · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`round()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the duration rounded to the given smallest unit and/or balanced to the given largest unit.

Syntax

```js
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`largestUnit` Optional

Any of the temporal units: `"years"`, `"months"`, `"weeks"`, `"days"`, `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms, or the value `"auto"` which means the largest non-zero component of this duration or `smallestUnit`, whichever is greater. Defaults to `"auto"`. The result will not contain units larger than this; for example, if the largest unit is `"minutes"`, then "1 hour 30 minutes" will become "90 minutes".

`relativeTo` Optional

A zoned or plain date(time) that provides the time and calendar information to resolve calendar durations (see the link for the general interpretation of this option). Required if either `this` or `other` is a calendar duration, or `smallestUnit` is a calendar unit.

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. Must be in the inclusive range of 1 to 1e9. If the smallest unit is hours, minutes, seconds, milliseconds, microseconds, or nanoseconds, the increment must be a divisor of the maximum value of the unit; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12.

`roundingMode` Optional

A string representing the rounding mode specifying to round up or down in various scenarios. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit` Optional

Any of the temporal units: `"years"`, `"months"`, `"weeks"`, `"days"`, `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms. Defaults to `"nanoseconds"`. For units larger than `"nanoseconds"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings. Must be smaller or equal to `largestUnit`. At least one of `smallestUnit` and `largestUnit` must be provided.

Return value

A new `Temporal.Duration` object, where the largest unit is no larger than the `largestUnit` option, and the smallest unit is no smaller than the `smallestUnit` option. The fractional parts of the `smallestUnit` are rounded according to the `roundingIncrement` and `roundingMode` settings.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Description

The `round()` method performs two operations: rounding and balancing. It does the following:

1. It makes sure the duration is balanced. If a component is above its preferred maximum (24 hours per day, 60 minutes per hour, etc.), the excess is carried over to the next larger unit, until we reach `largestUnit`. For example, "24 hours 90 minutes" becomes "25 hours 30 minutes" if `largestUnit` is `"auto"`, and "1 day 1 hour 30 minutes" if `largestUnit` is `"days"`.
2. For all components larger than `largestUnit`, they are carried down into `largestUnit`; for example, "2 hours 30 minutes" becomes "150 minutes" if `largestUnit` is `"minutes"`.
3. For all components smaller than `smallestUnit`, they are carried up into `smallestUnit` as a fractional part, and then rounded according to the `roundingIncrement` and `roundingMode` settings.

For example, "1 hour 30 minutes" becomes "1.5 hours" if `smallestUnit` is `"hours"`, and then rounded to "2 hours" using the default settings.

Calendar units have uneven lengths. Their lengths are resolved relative to a starting point. For example, a duration of "2 years" in the Gregorian calendar may be 730 days or 731 days long, depending on whether it moves through a leap year or not. When rounding to a calendar unit, we first get the exact date-time represented by `relativeTo + duration`, then round it down and up according to `smallestUnit` and `roundingIncrement` to get two candidates. Then, we choose the candidate according to the `roundingMode` setting, and finally subtract `relativeTo` to get the final duration.

Examples

Rounding off small units

```js
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
const roundedDuration = duration.round("minutes");
console.log(roundedDuration.toString()); // "PT1H30M"
```

Avoiding larger units

```js
const duration = Temporal.Duration.from({
  days: 3,
  hours: 1,
  minutes: 41,
  seconds: 5,
});
const roundedDuration = duration.round({ largestUnit: "hours" });
console.log(
  `Time spent on this problem: ${roundedDuration.toLocaleString("en-US", { style: "digital"
})}`,
);
// Time spent on this problem: 73:41:05
```

Rounding to a whole number of hours

```js
const duration = Temporal.Duration.from({ days: 1, hours: 1, minutes: 30 });
const roundedDuration = duration.round({
  largestUnit: "hours",
  smallestUnit: "hours",
  roundingMode: "floor",
});
console.log(roundedDuration.hours); // 25
```

Rounding by 15-minute increments

```js
const duration = Temporal.Duration.from({ hours: 1, minutes: 17 });
const roundedDuration = duration.round({
  smallestUnit: "minutes",
  roundingIncrement: 15,
});
console.log(
  `The queue will take approximately ${roundedDuration.toLocaleString("en-US")}`,
);
// The queue will take approximately 1 hr, 15 min
```

Resolving calendar durations

If either the initial duration or largest/smallest unit contains a calendar unit, you must provide a `relativeTo` option to resolve the calendar durations.

```js
const duration = Temporal.Duration.from({ months: 1, days: 1, hours: 1 });
const roundedDuration = duration.round({
  largestUnit: "days",
  smallestUnit: "days",
  relativeTo: Temporal.PlainDateTime.from("2022-01-01"),
});
console.log(roundedDuration); // "P32D"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.round

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.round`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.total()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.subtract()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/subtract · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`subtract()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object with the difference between this duration and a given duration. It is equivalent to adding the negated value of the other duration.

Syntax

```js
subtract(other)
```

Parameters

`other`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this duration. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Duration` object representing the difference of this duration and `other`.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Either `this` or `other` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), because calendar durations are ambiguous without a calendar and time reference.
- The difference of `this` and `other` overflows the maximum or underflows the minimum representable duration, which is $\pm 2^{53}$ seconds.

Examples

Using subtract()

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -20 });

const d3 = d1.subtract(d2);
console.log(d3.toString()); // "PT2H50M"
```

For more examples and caveats, see the `add()` method.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.subtract

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.subtract`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.add()`
- `Temporal.Duration.prototype.negated()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of `Temporal.Duration` instances returns a string representing this duration in the same ISO 8601 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```
toJSON()
```

Parameters

None.

Return value

A string representing the given duration in the ISO 8601 format, with as much subsecond precision as necessary to represent the duration accurately.

Description

The method is automatically called by `JSON.stringify()` when a `Temporal.Duration` object is encountered during JSON serialization. The serialized value can then be deserialized using the `Temporal.Duration.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```js
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
const durationStr = duration.toJSON(); // 'PT1H30M15S'
const d2 = Temporal.Duration.from(durationStr);
```

JSON serialization and parsing

This example shows how `Temporal.Duration` can be serialized as JSON without extra effort, and how to parse it back.

```js
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
const jsonStr = JSON.stringify({ data: duration }); // '{"data":"PT1H30M15S"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "data") {
    return Temporal.Duration.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.toJSON`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Duration`
- `Temporal.Duration.from()`

- Temporal.Duration.prototype.toString()
- Temporal.Duration.prototype.toLocaleString()

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.Duration` instances returns a string with a language-sensitive representation of this duration. In implementations with `Intl.DurationFormat` API support, this method delegates to `Intl.DurationFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DurationFormat` object and use its `format()` method, because a `DurationFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DurationFormat` API, these parameters correspond exactly to the `Intl.DurationFormat()` constructor's parameters. Implementations without `Intl.DurationFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DurationFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DurationFormat()` constructor.

See the `Intl.DurationFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given duration according to language-specific conventions.

In implementations with `Intl.DurationFormat`, this is equivalent to `new Intl.DurationFormat(locales, options).format(duration)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });

console.log(duration.toLocaleString()); // 1 hr, 30 min, 15 sec
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.tolocalestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Intl.DurationFormat`
- `Temporal.Duration.prototype.toJSON()`
- `Temporal.Duration.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/toString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of `Temporal.Duration` instances returns a string representing this duration in the ISO 8601 format.

Syntax

```
toString()
toString(options)
```

Parameters

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

Return value

A string representing the given duration in the ISO 8601 format, with subsecond components formatted according to the options. The zero duration is represented as `"PT0S"`.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using toString()

```js
const duration = Temporal.Duration.from({ hours: 1, minutes: 30, seconds: 15 });
console.log(duration.toString()); // 'PT1H30M15S'

// Stringification implicitly calls toString()
console.log(`${duration}`); // 'PT1H30M15S'
```

Using options

```js
const worldRecord = Temporal.Duration.from({ seconds: 9, milliseconds: 580 });
console.log(worldRecord.toString()); // 'PT9.58S'
console.log(worldRecord.toString({ fractionalSecondDigits: 1 })); // 'PT9.5S'
console.log(worldRecord.toString({ fractionalSecondDigits: 0 })); // 'PT9S'
console.log(worldRecord.toString({ smallestUnit: "millisecond" })); // 'PT9.580S'
console.log(
  worldRecord.toString({
    fractionalSecondDigits: 1,
    roundingMode: "halfExpand",
  }),
); // 'PT9.6S'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.toJSON()`
- `Temporal.Duration.prototype.toLocaleString()`

Temporal.Duration.prototype.total()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/total · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`total()`** method of `Temporal.Duration` instances returns a number representing the total duration in the given unit.

Syntax

```
total(unit)
total(options)
```

Parameters

`unit`

A string representing the `unit` option. This is a convenience overload, so `total(unit)` is equivalent to `total({ unit })`, where `unit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`relativeTo` Optional

A zoned or plain date(time) that provides the time and calendar information to resolve calendar durations (see the link for the general interpretation of this option). Required if either `this` or `other` is a calendar duration, or `unit` is a calendar unit.

`unit`

Any of the temporal units: `"years"`, `"months"`, `"weeks"`, `"days"`, `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms.

Return value

A floating-point number representing the total duration in the given unit. May be inexact due to floating point precision limits.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `unit` is not provided or is not a valid unit.
- Either `this` or `other` is a calendar duration, or `unit` is a calendar unit, and `relativeTo` is not provided.

Description

If a `relativeTo` is provided, the result is calculated by adding the duration to the starting point, finding the difference between the result and the starting point (in nanoseconds), and then converting the difference to the requested unit by dividing by the appropriate number of nanoseconds per unit.

Providing a zoned date-time allows daylight saving time and other time zone changes to be taken into account too; otherwise, 24-hour days are assumed.

If `relativeTo` is not provided, the result is calculated by converting the duration to nanoseconds and dividing by the appropriate number of nanoseconds per unit.

Examples

Using total()

```js
const d = Temporal.Duration.from({ hours: 1, minutes: 30 });

console.log(d.total("minutes")); // 90
console.log(d.total("hours")); // 1.5
```

Total of a calendar duration

```js
const d = Temporal.Duration.from({ months: 1 });

console.log(
  d.total({ unit: "days", relativeTo: Temporal.PlainDate.from("2021-01-01") }),
); // 31
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.total`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`total`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.total`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.round()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.Duration` instances throws a `TypeError`, which prevents `Temporal.Duration` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `duration1 > duration2` would implicitly compare them as strings, which may have unexpected results such as `"PT3S" > "PT1M"`. By throwing a `TypeError`, `Temporal.Duration` instances prevent such implicit conversions. You need to explicitly convert them to numbers using `Temporal.Duration.prototype.total()`, or use the `Temporal.Duration.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.Duration

All arithmetic and comparison operations on `Temporal.Duration` instances should use the dedicated methods or convert them to primitives explicitly.

```
const duration1 = Temporal.Duration.from({ seconds: 3 });
const duration2 = Temporal.Duration.from({ minutes: 1 });
duration1 > duration2; // TypeError: can't convert Duration to primitive type
duration1.total("seconds") > duration2.total("seconds"); // false
Temporal.Duration.compare(duration1, duration2); // -1

duration1 + duration2; // TypeError: can't convert Duration to primitive type
duration1.total("seconds") + duration2.total("seconds"); // 63
duration1.add(duration2).toString(); // "PT1M3S"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.valueof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.toString()`
- `Temporal.Duration.prototype.toJSON()`
- `Temporal.Duration.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/with · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`with()`** method of `Temporal.Duration` instances returns a new `Temporal.Duration` object representing this duration with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the duration's fields.

Syntax

```js
with(info)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.Duration.from()`: `years`, `months`, `weeks`, `days`, `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, `nanoseconds`. Unspecified properties use the values from the original duration.

Return value

A new `Temporal.Duration` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original duration.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the recognized properties in the `info` object is not an integer (including non-finite values).
- A calendar unit (years, months, weeks) has an absolute value ≥ 2^{32}.
- The non-calendar part of the duration (days and below), when expressed in seconds, has an absolute value ≥ 2^{53}.

`TypeError`

Thrown in one of the following cases:

- The `info` object is not an object.
- All of the recognized properties in the `info` object are `undefined`.

Examples

Using with()

You can use `with()` to achieve fine-grained control over the fields of a `Temporal.Duration` object. For example, you can manually balance a duration only on one unit, which `round()` does not offer:

```js
function balanceMinutes(duration) {
  const { hours, minutes } = duration;
  const totalMinutes = hours * 60 + minutes;
  const balancedMinutes = totalMinutes % 60;
  const balancedHours = (totalMinutes - balancedMinutes) / 60;
  return duration.with({ hours: balancedHours, minutes: balancedMinutes });
}

const d1 = Temporal.Duration.from({ hours: 100, minutes: 100, seconds: 100 });
const d2 = balanceMinutes(d1);
console.log(d2.hours); // 101
console.log(d2.minutes); // 40
console.log(d2.seconds); // 100; remains unbalanced
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.duration.prototype.with

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.with`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.from()`
- `Temporal.Duration.prototype.add()`
- `Temporal.Duration.prototype.subtract()`
- `Temporal.Duration.prototype.round()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.blank

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/blank · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`blank`** accessor property of `Temporal.Duration` instances returns a boolean that is `true` if this duration represents a zero duration, and `false` otherwise. It is equivalent to `duration.sign === 0`.

Examples

Using blank

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ hours: 0 });

console.log(d1.blank); // false
console.log(d2.blank); // false
console.log(d3.blank); // true
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.blank`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`blank`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.blank`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.sign`

Temporal.Duration.prototype.days

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/days · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`days`** accessor property of `Temporal.Duration` instances returns an integer representing the number of days in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above days, the `days` absolute value's range depends on the calendar (how many days are in a week or month).

The set accessor of `days` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using days

```js
const d1 = Temporal.Duration.from({ weeks: 1, days: 1 });
const d2 = Temporal.Duration.from({ weeks: -1, days: -1 });
const d3 = Temporal.Duration.from({ weeks: 1 });
const d4 = Temporal.Duration.from({ days: 7 });

console.log(d1.days); // 1
console.log(d2.days); // -1
console.log(d3.days); // 0
console.log(d4.days); // 7
```

(code continues on the next page)

(code continued from the previous page)

```
// Balance d4
const d4Balanced = d4.round({
  largestUnit: "weeks",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.days); // 0
console.log(d4Balanced.weeks); // 1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.days

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
days	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.days`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.hours

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/hours · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`hours`** accessor property of `Temporal.Duration` instances returns an integer representing the number of hours in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above hours, the `hours` absolute value

will be between 0 and 23, inclusive.

The set accessor of `hours` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using hours

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ days: 1 });
const d4 = Temporal.Duration.from({ hours: 24 });

console.log(d1.hours); // 1
console.log(d2.hours); // -1
console.log(d3.hours); // 0
console.log(d4.hours); // 24

// Balance d4
const d4Balanced = d4.round({ largestUnit: "days" });
console.log(d4Balanced.hours); // 0
console.log(d4Balanced.days); // 1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.hours

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
hours	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.hours`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`

- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.microseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/microseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`microseconds`** accessor property of `Temporal.Duration` instances returns an integer representing the number of microseconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above microseconds, the `microseconds` absolute value will be between 0 and 999, inclusive.

The set accessor of `microseconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using microseconds

```js
const d1 = Temporal.Duration.from({ milliseconds: 1, microseconds: 500 });
const d2 = Temporal.Duration.from({ milliseconds: -1, microseconds: -500 });
const d3 = Temporal.Duration.from({ milliseconds: 1 });
const d4 = Temporal.Duration.from({ microseconds: 1000 });

console.log(d1.microseconds); // 500
console.log(d2.microseconds); // -500
console.log(d3.microseconds); // 0
console.log(d4.microseconds); // 1000

// Balance d4
const d4Balanced = d4.round({ largestUnit: "milliseconds" });
console.log(d4Balanced.microseconds); // 0
console.log(d4Balanced.milliseconds); // 1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.microseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`microseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.microseconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.nanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.milliseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/milliseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`milliseconds`** accessor property of `Temporal.Duration` instances returns an integer representing the number of milliseconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above milliseconds, the `milliseconds` absolute value will be between 0 and 999, inclusive.

The set accessor of `milliseconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using milliseconds

```js
const d1 = Temporal.Duration.from({ seconds: 1, milliseconds: 500 });
const d2 = Temporal.Duration.from({ seconds: -1, milliseconds: -500 });
const d3 = Temporal.Duration.from({ seconds: 1 });
const d4 = Temporal.Duration.from({ milliseconds: 1000 });

console.log(d1.milliseconds); // 500
console.log(d2.milliseconds); // -500
console.log(d3.milliseconds); // 0
console.log(d4.milliseconds); // 1000

// Balance d4
const d4Balanced = d4.round({ largestUnit: "seconds" });
console.log(d4Balanced.milliseconds); // 0
console.log(d4Balanced.seconds); // 1
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.milliseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`milliseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.milliseconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.minutes

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/minutes · MDN page last modified 2026-04-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`minutes`** accessor property of `Temporal.Duration` instances returns an integer representing the number of minutes in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above minutes, the `minutes` absolute value will be between 0 and 59, inclusive.

The set accessor of `minutes` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using minutes

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ hours: 1 });
const d4 = Temporal.Duration.from({ minutes: 60 });

console.log(d1.minutes); // 30
console.log(d2.minutes); // -30
console.log(d3.minutes); // 0
console.log(d4.minutes); // 60

// Balance d4
const d4Balanced = d4.round({ largestUnit: "hours" });
console.log(d4Balanced.minutes); // 0
console.log(d4Balanced.hours); // 1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.minutes

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`minutes`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.minutes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Temporal.Duration
- Temporal.Duration.prototype.years
- Temporal.Duration.prototype.months
- Temporal.Duration.prototype.weeks
- Temporal.Duration.prototype.days
- Temporal.Duration.prototype.hours
- Temporal.Duration.prototype.seconds
- Temporal.Duration.prototype.milliseconds
- Temporal.Duration.prototype.microseconds
- Temporal.Duration.prototype.nanoseconds

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.months

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/months · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`months`** accessor property of `Temporal.Duration` instances returns an integer representing the number of months in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above months, the `months` absolute value's range depends on the calendar (how many months are in a year).

The set accessor of `months` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using months

js

```
const d1 = Temporal.Duration.from({ years: 1, months: 1 });
const d2 = Temporal.Duration.from({ years: -1, months: -1 });
const d3 = Temporal.Duration.from({ years: 1 });
const d4 = Temporal.Duration.from({ months: 12 });

console.log(d1.months); // 1
console.log(d2.months); // -1
console.log(d3.months); // 0
console.log(d4.months); // 12

// Balance d4
const d4Balanced = d4.round({
  largestUnit: "years",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.months); // 0
console.log(d4Balanced.years); // 1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.months

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
months	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.Duration.months*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Temporal.Duration
- Temporal.Duration.prototype.years
- Temporal.Duration.prototype.weeks
- Temporal.Duration.prototype.days
- Temporal.Duration.prototype.hours
- Temporal.Duration.prototype.minutes
- Temporal.Duration.prototype.seconds
- Temporal.Duration.prototype.milliseconds
- Temporal.Duration.prototype.microseconds
- Temporal.Duration.prototype.nanoseconds

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.nanoseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/nanoseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **nanoseconds** accessor property of Temporal.Duration instances returns an integer representing the number of nanoseconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's sign property. If it is balanced to a unit above nanoseconds, the nanoseconds absolute value will be between 0 and 999, inclusive.

The set accessor of nanoseconds is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.Duration object with the desired new value.

Examples

Using nanoseconds

```js
const d1 = Temporal.Duration.from({ microseconds: 1, nanoseconds: 500 });
const d2 = Temporal.Duration.from({ microseconds: -1, nanoseconds: -500 });
const d3 = Temporal.Duration.from({ microseconds: 1 });
const d4 = Temporal.Duration.from({ nanoseconds: 1000 });

console.log(d1.nanoseconds); // 500
console.log(d2.nanoseconds); // -500
console.log(d3.nanoseconds); // 0
console.log(d4.nanoseconds); // 1000

// Balance d4
const d4Balanced = d4.round({ largestUnit: "microseconds" });
console.log(d4Balanced.nanoseconds); // 0
console.log(d4Balanced.microseconds); // 1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.nanoseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
nanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.nanoseconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.seconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/seconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **seconds** accessor property of `Temporal.Duration` instances returns an integer representing the number of seconds in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above seconds, the `seconds` absolute value will be between 0 and 59, inclusive.

The set accessor of `seconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using seconds

```js
const d1 = Temporal.Duration.from({ minutes: 1, seconds: 30 });
const d2 = Temporal.Duration.from({ minutes: -1, seconds: -30 });
const d3 = Temporal.Duration.from({ minutes: 1 });
const d4 = Temporal.Duration.from({ seconds: 60 });

console.log(d1.seconds); // 30
console.log(d2.seconds); // -30
console.log(d3.seconds); // 0
console.log(d4.seconds); // 60

// Balance d4
const d4Balanced = d4.round({ largestUnit: "minutes" });
console.log(d4Balanced.seconds); // 0
console.log(d4Balanced.minutes); // 1
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.seconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`seconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.seconds`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.Duration
- Temporal.Duration.prototype.years
- Temporal.Duration.prototype.months
- Temporal.Duration.prototype.weeks
- Temporal.Duration.prototype.days
- Temporal.Duration.prototype.hours
- Temporal.Duration.prototype.minutes
- Temporal.Duration.prototype.milliseconds
- Temporal.Duration.prototype.microseconds
- Temporal.Duration.prototype.nanoseconds

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.sign

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/sign · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **sign** accessor property of Temporal.Duration instances returns 1 if this duration is positive, -1 if negative, and 0 if zero. Because a duration never has mixed signs, the sign of a duration is determined by the sign of any of its non-zero fields.

Examples

Using sign

```js
const d1 = Temporal.Duration.from({ hours: 1, minutes: 30 });
const d2 = Temporal.Duration.from({ hours: -1, minutes: -30 });
const d3 = Temporal.Duration.from({ hours: 0 });

console.log(d1.sign); // 1
console.log(d2.sign); // -1
console.log(d3.sign); // 0

console.log(d1.abs().sign); // 1
console.log(d2.abs().sign); // 1
console.log(d3.abs().sign); // 0

console.log(d1.negated().sign); // -1
console.log(d2.negated().sign); // 1
console.log(d3.negated().sign); // 0
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.sign

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`sign`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.sign`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.abs()`
- `Temporal.Duration.prototype.negated()`
- `Temporal.Duration.prototype.blank`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.weeks

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/weeks · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`weeks`** accessor property of `Temporal.Duration` instances returns an integer representing the number of weeks in the duration.

Unless the duration is balanced, you cannot assume the range of this value, but you can know its sign by checking the duration's `sign` property. If it is balanced to a unit above weeks, the `weeks` absolute value's range depends on the calendar (how many weeks are in a month or year).

The set accessor of `weeks` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using weeks

js

```
const d1 = Temporal.Duration.from({ weeks: 1, days: 1 });
const d2 = Temporal.Duration.from({ weeks: -1, days: -1 });
const d3 = Temporal.Duration.from({ weeks: 1 });
const d4 = Temporal.Duration.from({ days: 7 });

console.log(d1.weeks); // 1
console.log(d2.weeks); // -1
console.log(d3.weeks); // 1
console.log(d4.weeks); // 0
```

(code continues on the next page)

(code continued from the previous page)

```
// Balance d4
const d4Balanced = d4.round({
  largestUnit: "weeks",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.weeks); // 1
console.log(d4Balanced.days); // 0
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.weeks

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
weeks	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.weeks`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.years`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`
- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Duration

Temporal.Duration.prototype.years

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Duration/years · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`years`** accessor property of `Temporal.Duration` instances returns an integer representing the number of years in the duration.

You can know the sign of `years` by checking the duration's `sign` property.

The set accessor of `years` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.Duration` object with the desired new value.

Examples

Using years

```js
const d1 = Temporal.Duration.from({ years: 1, months: 1 });
const d2 = Temporal.Duration.from({ years: -1, months: -1 });
const d3 = Temporal.Duration.from({ years: 1 });
const d4 = Temporal.Duration.from({ months: 12 });

console.log(d1.years); // 1
console.log(d2.years); // -1
console.log(d3.years); // 1
console.log(d4.years); // 0

// Balance d4
const d4Balanced = d4.round({
  largestUnit: "years",
  relativeTo: Temporal.PlainDate.from("2021-01-01"), // ISO 8601 calendar
});
console.log(d4Balanced.years); // 1
console.log(d4Balanced.months); // 0
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.duration.prototype.years

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
years	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Duration.years`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Duration`
- `Temporal.Duration.prototype.months`
- `Temporal.Duration.prototype.weeks`
- `Temporal.Duration.prototype.days`
- `Temporal.Duration.prototype.hours`
- `Temporal.Duration.prototype.minutes`

- `Temporal.Duration.prototype.seconds`
- `Temporal.Duration.prototype.milliseconds`
- `Temporal.Duration.prototype.microseconds`
- `Temporal.Duration.prototype.nanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.Instant

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Instant`** object represents a unique point in time, with nanosecond precision. It is fundamentally represented as the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC), without any time zone or calendar system.

Description

`Temporal.Instant` is semantically the same as `Date`. They both encapsulate a single point in time, but `Temporal.Instant` is more precise because it stores nanoseconds rather than milliseconds. `Temporal.Instant` also avoids pitfalls of `Date` because it does not assume any calendar or time zone information—if you want to read any date or time information such as year or month, you need to convert it to a `Temporal.ZonedDateTime` first, using `toZonedDateTimeISO()`.

You can convert from `Date` to `Temporal.Instant` using the `Date.prototype.toTemporalInstant()` method, which should be preferred over other methods such as `Temporal.Instant.fromEpochMilliseconds()` because the former involves less user code and may be more optimized. You can also convert from `Temporal.Instant` to `Date` using its epoch milliseconds, such as `new Date(instant.epochMilliseconds)`.

RFC 9557 format

`Instant` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss Z/±HH:mm
```

`YYYY`

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD`

A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`T`

The date-time separator, which can be `T`, `t`, or a space.

`HH`

A two-digit number from `00` to `23`.

`mm` Optional

A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

`Z/±HH:mm`

Either the UTC designator `Z` or `z`, or an offset from UTC in the form `+` or `-` followed by the same format as the time component. Note that subminute precision (`:ss.sssssssss`) may be unsupported by other systems, and is accepted but never output. If an offset is provided, the time is interpreted in the specified offset.

As an input, you may optionally include the time zone identifier and calendar, in the same format as `ZonedDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits and offset.

Constructor

`Temporal.Instant()`

Creates a new `Temporal.Instant` object by directly supplying the underlying data.

Static methods

`Temporal.Instant.compare()`

Returns a number (-1, 0, or 1) indicating whether the first instant comes before, is the same as, or comes after the second instant. Equivalent to comparing the `epochNanoseconds` of the two instants.

`Temporal.Instant.from()`

Creates a new `Temporal.Instant` object from another `Temporal.Instant` object, or an RFC 9557 string.

`Temporal.Instant.fromEpochMilliseconds()`

Creates a new `Temporal.Instant` object from the number of milliseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

`Temporal.Instant.fromEpochNanoseconds()`

Creates a new `Temporal.Instant` object from the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

Instance properties

These properties are defined on `Temporal.Instant.prototype` and shared by all `Temporal.Instant` instances.

`Temporal.Instant.prototype.constructor`

The constructor function that created the instance object. For `Temporal.Instant` instances, the initial value is the `Temporal.Instant()` constructor.

`Temporal.Instant.prototype.epochMilliseconds`

Returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. Equivalent to dividing `epochNanoseconds` by `1e6` and flooring it.

`Temporal.Instant.prototype.epochNanoseconds`

Returns a `BigInt` representing the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

`Temporal.Instant.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.Instant"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.Instant.prototype.add()`

Returns a new `Temporal.Instant` object representing this instant moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.Instant.prototype.equals()`

Returns `true` if this instant is equivalent in value to another instant (in a form convertible by `Temporal.Instant.from()`), and `false` otherwise. They are compared by their epoch nanoseconds. Equivalent to `Temporal.Instant.compare(this, other) === 0`.

`Temporal.Instant.prototype.round()`

Returns a new `Temporal.Instant` object representing this instant rounded to the given unit.

`Temporal.Instant.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another instant (in a form convertible by `Temporal.Instant.from()`) to this instant. The duration is positive if the other instant is before this instant, and negative if after.

`Temporal.Instant.prototype.subtract()`

Returns a new `Temporal.Instant` object representing this instant moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.Instant.prototype.toJSON()`

Returns a string representing this instant in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.Instant.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this instant. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

`Temporal.Instant.prototype.toString()`

Returns a string representing this instant in the RFC 9557 format using the specified time zone.

`Temporal.Instant.prototype.toZonedDateTimeISO()`

Returns a new `Temporal.ZonedDateTime` object representing this instant in the specified time zone using the ISO 8601 calendar system.

`Temporal.Instant.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this instant to another instant (in a form convertible by `Temporal.Instant.from()`). The duration is positive if the other instant is after this instant, and negative if before.

`Temporal.Instant.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.Instant` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-instant-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`Instant`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`Instant()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`add`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`epochMilliseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`epochNanoseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`fromEpochMilliseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`fromEpochNanoseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`round`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`since`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toZonedDateTimeISO`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.ZonedDateTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/Instant · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Instant()`** constructor creates `Temporal.Instant` objects.

This constructor is exactly equivalent to calling `Temporal.Instant.fromEpochNanoseconds()`.

Syntax

```
new Temporal.Instant(epochNanoseconds)
```

> **Note:** `Temporal.Instant()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`epochNanoseconds`

A BigInt representing the number of nanoseconds since the Unix epoch.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `epochNanoseconds`.

Exceptions

`RangeError`

Thrown if `epochNanoseconds` represents an instant outside the range of representable instants, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using Temporal.Instant()

```
const instant = new Temporal.Instant(0n);
console.log(instant.toString()); // 1970-01-01T00:00:00Z
const vostok1Liftoff = new Temporal.Instant(-27524838000000000n);
console.log(vostok1Liftoff.toString()); // 1961-04-12T06:07:00Z
const sts1Liftoff = new Temporal.Instant(355924804000000000n);
console.log(sts1Liftoff.toString()); // 1981-04-12T12:00:04Z
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal-instant-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Instant()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.Instant`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochNanoseconds`
- `Temporal.Instant.from()`
- `Temporal.Instant.fromEpochMilliseconds()`
- `Temporal.Instant.fromEpochNanoseconds()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/compare · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Instant.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first instant comes before, is the same as, or comes after the second instant. It is equivalent to comparing the `epochNanoseconds` of the two instants.

Syntax

```
Temporal.Instant.compare(instant1, instant2)
```

Parameters

`instant1`

A string or a `Temporal.Instant` instance representing the first instant to compare. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

`instant2`

The second instant to compare, converted to a `Temporal.Instant` object using the same algorithm as `instant1`.

Return value

Returns `-1` if `instant1` comes before `instant2`, `0` if they are the same, and `1` if `instant1` comes after `instant2`.

Examples

Using Temporal.Instant.compare()

```js
const instant1 = Temporal.Instant.from("2021-08-01T12:34:56Z");
const instant2 = Temporal.Instant.from("2021-08-01T12:34:56Z");

console.log(Temporal.Instant.compare(instant1, instant2)); // 0

const instant3 = Temporal.Instant.from("2021-08-01T13:34:56Z");
console.log(Temporal.Instant.compare(instant1, instant3)); // -1
```

Sorting an array of instants

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```js
const instants = [
  Temporal.Instant.from("2021-08-01T12:34:56Z"),
  Temporal.Instant.from("2021-08-01T12:34:56+01:00"),
  Temporal.Instant.from("2021-08-01T12:34:56-01:00"),
];

instants.sort(Temporal.Instant.compare);
console.log(instants.map((instant) => instant.toString()));
// [ '2021-08-01T11:34:56Z', '2021-08-01T12:34:56Z', '2021-08-01T13:34:56Z' ]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.compare

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.compare`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.equals()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/from · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Instant.from()`** static method creates a new `Temporal.Instant` object from another `Temporal.Instant` object, or an RFC 9557 string.

Syntax

```js
Temporal.Instant.from(info)
```

Parameters

`info`

One of the following:

- A `Temporal.Instant` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date, time, and time zone offset. The time zone name is ignored; only the offset is used.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `info`.

Exceptions

`TypeError`

Thrown if `info` is not a `Temporal.Instant` instance or a string.

`RangeError`

Thrown in one of the following cases:

- The string is not a valid RFC 9557 string.
- The info is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating an instant from a string

```js
const instant = Temporal.Instant.from("1970-01-01T00Z");
console.log(instant.toString()); // 1970-01-01T00:00:00Z

const instant2 = Temporal.Instant.from("1970-01-01T00+08:00");
console.log(instant2.toString()); // 1969-12-31T16:00:00Z

// America/New_York is UTC-5 in January 1970, not UTC+8
const instant3 = Temporal.Instant.from("1970-01-01T00+08:00[America/New_York]");
console.log(instant3.toString()); // 1969-12-31T16:00:00Z; the time zone name is ignored
```

Creating an instant from another instant

```js
const instant = Temporal.Instant.from("1970-01-01T00Z");
const instant2 = Temporal.Instant.from(instant);
console.log(instant2.toString()); // 1970-01-01T00:00:00Z
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.from`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Instant.fromEpochMilliseconds()`
- `Temporal.Instant.fromEpochNanoseconds()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.fromEpochMilliseconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/fromEpochMilliseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Instant.fromEpochMilliseconds()`** static method creates a new `Temporal.Instant` object from the number of milliseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

To convert a `Date` object to a `Temporal.Instant` object, use `Date.prototype.toTemporalInstant()` instead.

Syntax

```js
Temporal.Instant.fromEpochMilliseconds(epochMilliseconds)
```

Parameters

`epochMilliseconds`

A number representing the number of milliseconds since the Unix epoch. Internally, it is converted to a BigInt and multiplied by `1e6` to get the number of nanoseconds.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `epochMilliseconds`.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `epochMilliseconds` cannot be converted to a BigInt (e.g., not an integer).
- `epochMilliseconds` is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using Temporal.Instant.fromEpochMilliseconds()

```js
const instant = Temporal.Instant.fromEpochMilliseconds(0);
console.log(instant.toString()); // 1970-01-01T00:00:00Z
const vostok1Liftoff = Temporal.Instant.fromEpochMilliseconds(-275248380000);
console.log(vostok1Liftoff.toString()); // 1961-04-12T06:07:00Z
const sts1Liftoff = Temporal.Instant.fromEpochMilliseconds(355924804000);
console.log(sts1Liftoff.toString()); // 1981-04-12T12:00:04Z
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.instant.fromepochmilliseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fromEpochMilliseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.fromEpochMilliseconds`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.Instant
- Temporal.Instant.prototype.epochMilliseconds
- Temporal.Instant.from()
- Temporal.Instant.fromEpochNanoseconds()
- Date.prototype.toTemporalInstant()

Temporal.Instant.fromEpochNanoseconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/fromEpochNanoseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Instant.fromEpochNanoseconds()`** static method creates a new `Temporal.Instant` object from the number of nanoseconds since the Unix epoch (midnight at the beginning of January 1, 1970, UTC).

To convert a `Date` object to a `Temporal.Instant` object, use `Date.prototype.toTemporalInstant()` instead.

Syntax

```
Temporal.Instant.fromEpochNanoseconds(epochNanoseconds)
```

Parameters

`epochNanoseconds`

A BigInt representing the number of nanoseconds since the Unix epoch.

Return value

A new `Temporal.Instant` object representing the instant in time specified by `epochNanoseconds`.

Exceptions

`RangeError`

Thrown if `epochNanoseconds` is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using Temporal.Instant.fromEpochNanoseconds()

```
const instant = Temporal.Instant.fromEpochNanoseconds(0n);
console.log(instant.toString()); // 1970-01-01T00:00:00Z
const vostok1Liftoff =
  Temporal.Instant.fromEpochNanoseconds(-27524838000000000n);
console.log(vostok1Liftoff.toString()); // 1961-04-12T06:07:00Z
const sts1Liftoff = Temporal.Instant.fromEpochNanoseconds(355924804000000000n);
console.log(sts1Liftoff.toString()); // 1981-04-12T12:00:04Z
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.instant.fromepochnanoseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fromEpochNanoseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.fromEpochNanoseconds`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochNanoseconds`
- `Temporal.Instant.from()`
- `Temporal.Instant.fromEpochMilliseconds()`
- `Date.prototype.toTemporalInstant()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/add · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`add()`** method of `Temporal.Instant` instances returns a new `Temporal.Instant` object representing this instant moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```js
add(duration)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this instant. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Instant` object representing adding `duration` to this instant. If `duration` is positive, then the returned instant is later than this instant; if `duration` is negative, then the returned instant is earlier than this instant.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `duration` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), or has a non-zero `days`, because calendar durations are ambiguous without a calendar and time reference.
- The result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

In essence, the `add()` method first gets the number of nanoseconds represented by `duration`, adds it to this instant's `epochNanoseconds`, and then creates a new `Temporal.Instant` object from the result. Therefore, the duration must unambiguously represent a fixed amount of time.

If you want to add a calendar duration, the addition must be performed in the context of a calendar and a time zone to account for the variable lengths of months, years, and days (because of daylight saving time). In this case, convert the instant to a `Temporal.ZonedDateTime` object, add the duration, and then convert the result back to an instant.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a Temporal.Duration

```js
const instant = Temporal.Instant.fromEpochMilliseconds(0);
const duration = Temporal.Duration.from("PT1S");
const newInstant = instant.add(duration);
console.log(newInstant.epochMilliseconds); // 1000
```

Adding an object or a string

```js
const instant = Temporal.Instant.fromEpochMilliseconds(0);
const newInstant = instant.add({ seconds: 1 });
console.log(newInstant.epochMilliseconds); // 1000

const newInstant2 = instant.add("PT1S");
console.log(newInstant2.epochMilliseconds); // 1000
```

Adding a calendar duration

```js
const instant = Temporal.Instant.fromEpochMilliseconds(1730610000000);
const duration = Temporal.Duration.from({ days: 1 });
```

(code continues on the next page)

(code continued from the previous page)

```
// This instant is 2024-11-03T01:00:00-04:00[America/New_York],
// which is a DST transition day in the US.
const instant2 = instant
  .toZonedDateTimeISO("America/New_York")
  .add(duration)
  .toInstant();
console.log(instant2.epochMilliseconds); // 1730700000000

// The same instant is not a DST transition day in Paris.
const instant3 = instant
  .toZonedDateTimeISO("Europe/Paris")
  .add(duration)
  .toInstant();
console.log(instant3.epochMilliseconds); // 1730696400000
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.add`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Duration`
- `Temporal.Instant.prototype.subtract()`

Temporal.Instant.prototype.equals()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/equals · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`equals()`** method of `Temporal.Instant` instances returns `true` if this instant is equivalent in value to another instant (in a form convertible by `Temporal.Instant.from()`), and `false` otherwise. They are compared by their epochNanoseconds. It is equivalent to `Temporal.Instant.compare(this, other) === 0`.

Syntax

```
equals(other)
```

Parameters

`other`

A string or a `Temporal.Instant` instance representing the other instant to compare. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

Return value

`true` if this instant is equal to `other` by nanoseconds, `false` otherwise.

Examples

Using equals()

```
const instant1 = Temporal.Instant.from("2021-08-01T12:34:56Z");
const instant2 = Temporal.Instant.fromEpochMilliseconds(1627821296000);
console.log(instant1.equals(instant2)); // true
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.equals

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.equals`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Instant`
- `Temporal.Instant.compare()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.round()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/round · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`round()`** method of `Temporal.Instant` instances returns a new `Temporal.Instant` object representing this instant rounded to the given unit.

Syntax

```js
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. The increment and the `smallestUnit` must evenly divide 24 hours; for example, 45 seconds is a divisor of 86400 seconds, and 100 minutes is a divisor of 3600 minutes. This is slightly less strict than the `round()` method of the other classes, which all require the increment to be a divisor of the maximum value of the unit.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.Instant` object representing this instant rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```js
const instant = Temporal.Instant.fromEpochMilliseconds(1000);
const roundedInstant = instant.round("second");
console.log(roundedInstant.epochMilliseconds); // 1000

const instant2 = instant.round("minute");
console.log(instant2.epochMilliseconds); // 0
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.round

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.round`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Instant`

Temporal.Instant.prototype.since()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/since · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`since()`** method of `Temporal.Instant` instances returns a new `Temporal.Duration` object representing the duration from another instant (in a form convertible by `Temporal.Instant.from()`) to this instant. The duration is positive if the other instant is before this instant, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```js
since(other)
since(other, options)
```

Parameters

`other`

A string or a `Temporal.Instant` instance representing an instant to subtract from this instant. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`, `"nanoseconds"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"seconds"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"nanoseconds"`.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this instant. The duration is positive if `other` is before this instant, and negative if after.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using since()

```js
const lastUpdated = Temporal.Instant.fromEpochMilliseconds(1735235418000);
const now = Temporal.Now.instant();
const duration = now.since(lastUpdated, { smallestUnit: "minute" });
console.log(`Last updated ${duration.toLocaleString("en-US")} ago`);
```

Balancing the resulting duration

Because an instant does not carry calendar information, the resulting duration avoids calendar durations, which are ambiguous without a calendar and time reference. Therefore, the result is unbalanced because `hours` may be greater than 24. To balance the duration, round the resulting duration again with the desired `largestUnit`, passing a `relativeTo` that carries the calendar information.

```js
const lastUpdated = Temporal.Instant.fromEpochMilliseconds(1735235418000);
const now = Temporal.Now.instant();
const duration = now.since(lastUpdated, { smallestUnit: "minutes" });
const roundedDuration = duration.round({
  largestUnit: "years",
  // Use the ISO calendar; you can convert to another calendar using
  // withCalendar()
  relativeTo: now.toZonedDateTimeISO("UTC"),
});
console.log(`Last updated ${roundedDuration.toLocaleString("en-US")} ago`);
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.since

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.since`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Duration`
- `Temporal.Instant.prototype.add()`
- `Temporal.Instant.prototype.subtract()`
- `Temporal.Instant.prototype.until()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.subtract()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/subtract · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`subtract()`** method of `Temporal.Instant` instances returns a new `Temporal.Instant` object representing this instant moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two instants and get a duration, use `since()` or `until()` instead.

Syntax

```js
subtract(duration)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this instant. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.Instant` object representing subtracting `duration` from this instant. If `duration` is positive, then the returned instant is earlier than this instant; if `duration` is negative, then the returned instant is later than this instant.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `duration` is a calendar duration (it has a non-zero `years`, `months`, or `weeks`), or has a non-zero `days`, because calendar durations are ambiguous without a calendar and time reference.
- The result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a Temporal.Duration

```js
const instant = Temporal.Instant.fromEpochMilliseconds(1000);
const duration = Temporal.Duration.from("PT1S"); // One-second duration
const newInstant = instant.subtract(duration);
console.log(newInstant.epochMilliseconds); // 0
```

For more examples, see `add()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.subtract`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.subtract`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.Instant
- Temporal.Duration
- Temporal.Instant.prototype.add()
- Temporal.Instant.prototype.since()
- Temporal.Instant.prototype.until()

Temporal.Instant.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of Temporal.Instant instances returns a string representing this instant in the same RFC 9557 format as calling toString(). It is intended to be implicitly called by JSON.stringify().

Syntax

```js
toJSON()
```

Parameters

None.

Return value

A string representing the given instant in the RFC 9557 format, with as much subsecond precision as necessary to represent the duration accurately, and with the UTC time zone designator `Z`.

Description

The method is automatically called by JSON.stringify() when a Temporal.Instant object is encountered during JSON serialization. The serialized value can then be deserialized using the Temporal.Instant.from() function within the reviver of JSON.parse().

Examples

Using toJSON()

```js
const instant = Temporal.Instant.fromEpochMilliseconds(1627821296000);
const instantStr = instant.toJSON(); // '2021-08-01T12:34:56Z'
const i2 = Temporal.Instant.from(instantStr);
```

JSON serialization and parsing

This example shows how Temporal.Instant can be serialized as JSON without extra effort, and how to parse it back.

```js
const instant = Temporal.Instant.fromEpochMilliseconds(1627821296000);
const jsonStr = JSON.stringify({ time: instant }); // '{"time":"2021-08-01T12:34:56Z"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "time") {
    return Temporal.Instant.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.toJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Instant.from()`
- `Temporal.Instant.prototype.toString()`
- `Temporal.Instant.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.Instant` instances returns a string with a language-sensitive representation of this instant. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given instant according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(instant)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56Z");

console.log(instant.toLocaleString()); // 8/1/2021, 12:34:56 AM (assuming en-US locale and
device in UTC time zone)
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.tolocalestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Intl.DateTimeFormat`
- `Temporal.Instant.prototype.toJSON()`
- `Temporal.Instant.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/toString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of `Temporal.Instant` instances returns a string representing this instant in the RFC 9557 format using the specified time zone.

Syntax

```js
toString()
toString(options)
```

Parameters

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information). Defaults to `"UTC"`.

Return value

A string in the RFC 9557 format representing this instant using the specified time zone. No annotations, such as time zone names, are included.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using toString()

```js
const instant = Temporal.Instant.fromEpochMilliseconds(1627814412345);
console.log(instant.toString()); // '2021-08-01T10:40:12.345Z'

// Stringification implicitly calls toString()
console.log(`${instant}`); // '2021-08-01T10:40:12.345Z'
```

Using options

```js
const instant = Temporal.Instant.fromEpochMilliseconds(1627814412345);
console.log(instant.toString({ fractionalSecondDigits: 1 })); // '2021-08-01T10:40:12.3Z'
console.log(instant.toString({ smallestUnit: "minute" })); // '2021-08-01T10:40Z'
console.log(instant.toString({ timeZone: "America/New_York" })); // '2021-08-
01T06:40:12.345-04:00'

// The time zone name automatically resolves to the correct offset
// based on the instant; for example, America/New_York is UTC-4 in summer,
// but UTC-5 in winter.
const instant2 = Temporal.Instant.fromEpochMilliseconds(1577836800000);
console.log(instant2.toString({ timeZone: "UTC" })); // '2029-12-31T23:00:00Z'
console.log(instant2.toString({ timeZone: "America/New_York" })); // '2019-12-31T19:00:00-
05:00'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Instant.from()`
- `Temporal.Instant.prototype.toJSON()`
- `Temporal.Instant.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.toZonedDateTimeISO()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/toZonedDateTimeISO · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toZonedDateTimeISO()`** method of `Temporal.Instant` instances returns a new `Temporal.ZonedDateTime` object representing this instant in the specified time zone using the ISO 8601 calendar system.

Syntax

```
toZonedDateTimeISO(timeZone)
```

Parameters

`timeZone`

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

A new `Temporal.ZonedDateTime` object representing this instant in the specified time zone using the ISO 8601 calendar system.

Exceptions

`RangeError`

Thrown if the time zone name is invalid.

`TypeError`

Thrown if `timeZone` is not a string or a `Temporal.ZonedDateTime` instance.

Examples

Using toZonedDateTimeISO()

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56.123456789Z");
const zonedDateTime = instant.toZonedDateTimeISO("America/New_York");
console.log(zonedDateTime.toString()); // 2021-08-01T08:34:56.123456789-
04:00[America/New_York]

const localDateTime = instant.toZonedDateTimeISO(Temporal.Now.timeZoneId());
console.log(localDateTime.toString()); // This instant in your timezone
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.tozoneddatetimeiso`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toZonedDateTimeISO`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.toZonedDateTimeISO`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Instant`
- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.toInstant()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.until()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/until · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`until()`** method of `Temporal.Instant` instances returns a new `Temporal.Duration` object representing the duration from this instant to another instant (in a form convertible by `Temporal.Instant.from()`). The duration is positive if the other instant is after this instant, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```js
until(other)
until(other, options)
```

Parameters

`other`

A string or a `Temporal.Instant` instance representing an instant to subtract this instant from. It is converted to a `Temporal.Instant` object using the same algorithm as `Temporal.Instant.from()`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this instant *until* `other`. The duration is positive if `other` is after this instant, and negative if before.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using until()

```js
const launch = Temporal.Instant.fromEpochMilliseconds(2051222400000);
const now = Temporal.Now.instant();
const duration = now.until(launch, { smallestUnit: "minutes" });
console.log(`It will be ${duration.toLocaleString("en-US")} until the launch`);
```

For more examples, see `since()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.until

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
until	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.until`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.Duration`
- `Temporal.Instant.prototype.add()`
- `Temporal.Instant.prototype.subtract()`
- `Temporal.Instant.prototype.since()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.Instant` instances throws a `TypeError`, which prevents `Temporal.Instant` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `instant1 > instant2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.Instant` instances prevent such implicit conversions. You need to explicitly convert them to numbers using `Temporal.Instant.prototype.epochNanoseconds`, or use the `Temporal.Instant.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.Instant

All arithmetic and comparison operations on `Temporal.Instant` instances should use the dedicated methods or convert them to primitives explicitly.

```js
const instant1 = Temporal.Instant.fromEpochMilliseconds(0);
const instant2 = Temporal.Instant.fromEpochMilliseconds(1000);
instant1 > instant2; // TypeError: can't convert Instant to primitive type
instant1.epochNanoseconds > instant2.epochNanoseconds; // false
Temporal.Instant.compare(instant1, instant2); // -1

instant2 - instant1; // TypeError: can't convert Instant to primitive type
instant2.since(instant1).toString(); // "PT1S"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.instant.prototype.valueof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.valueOf`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Temporal.Instant
- Temporal.Instant.prototype.toString()
- Temporal.Instant.prototype.toJSON()
- Temporal.Instant.prototype.toLocaleString()

Temporal.Instant.prototype.epochMilliseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/epochMilliseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **epochMilliseconds** accessor property of Temporal.Instant instances returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. It is equivalent to dividing epochNanoseconds by 1e6 and flooring the result.

The set accessor of epochMilliseconds is undefined. You cannot change this property directly. To create a new Temporal.Instant object with the desired new epochMilliseconds value, use the Temporal.Instant.fromEpochMilliseconds() static method instead.

Examples

Using epochMilliseconds

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
console.log(instant.epochMilliseconds); // 1627821296789

const instant2 = Temporal.Instant.from("1969-08-01T12:34:56.789Z");
console.log(instant2.epochMilliseconds); // -13173903211
```

Changing epochMilliseconds

This is the method that allows you to move by any amount of time:

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = instant.add({ hours: 1 });
console.log(instant1hourLater.epochMilliseconds); // 1627824896789
```

If you already know the change in milliseconds, you can also directly construct a new Temporal.Instant object:

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = Temporal.Instant.fromEpochMilliseconds(
  instant.epochMilliseconds + 3600000,
);
console.log(instant1hourLater.epochMilliseconds); // 1627824896789
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.instant.prototype.epochmilliseconds`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`epochMilliseconds`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.epochMilliseconds`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochNanoseconds`
- `Temporal.Instant.fromEpochMilliseconds()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Instant

Temporal.Instant.prototype.epochNanoseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Instant/epochNanoseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **epochNanoseconds** accessor property of `Temporal.Instant` instances returns a `BigInt` representing the number of nanoseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

The set accessor of `epochNanoseconds` is `undefined`. You cannot change this property directly. To create a new `Temporal.Instant` object with the desired new `epochNanoseconds` value, use the `Temporal.Instant.fromEpochNanoseconds()` static method instead.

An instant can only represent $\pm 10^8$ days (about ±273,972.6 years) around the epoch, which is ±8.64e21 nanoseconds. Attempting to set `epochNanoseconds` beyond this boundary throws a `RangeError`.

Examples

Using epochNanoseconds

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
console.log(instant.epochNanoseconds); // 1627821296789000000n

const instant2 = Temporal.Instant.from("1969-08-01T12:34:56.789Z");
console.log(instant2.epochNanoseconds); // -13173903211000000n
```

Changing epochNanoseconds

This is the method that allows you to move by any amount of time:

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = instant.add({ hours: 1 });
console.log(instant1hourLater.epochNanoseconds); // 1627824896789000000n
```

If you already know the change in nanoseconds, you can also directly construct a new `Temporal.Instant` object:

```js
const instant = Temporal.Instant.from("2021-08-01T12:34:56.789Z");
const instant1hourLater = Temporal.Instant.fromEpochNanoseconds(
  instant.epochNanoseconds + 3600000000000n,
);
console.log(instant1hourLater.epochNanoseconds); // 1627824896789000000n
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.instant.prototype.epochnanoseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
epochNanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Instant.epochNanoseconds`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Instant`
- `Temporal.Instant.prototype.epochMilliseconds`
- `Temporal.Instant.fromEpochNanoseconds()`

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.Now

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Now · MDN page last modified 2026-09-01

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Now`** namespace object contains static methods for getting the current time in various formats.

Description

Unlike most global objects, `Temporal.Now` is not a constructor. You cannot use it with the new operator or invoke the `Temporal.Now` object as a function. All properties and methods of `Temporal.Now` are static (just like the `Math` object).

Most fundamentally, the system time is returned by the operating system as a time since the Unix epoch. `Temporal.Now.instant()` returns this time as a `Temporal.Instant` object. Although this object can represent nanoseconds, the resolution of the current time depends on the system clock and browser settings.

An instant can be interpreted in a time zone (which is the system time zone `Temporal.Now.timeZoneId()` by default) in the same fashion as `Temporal.Instant.prototype.toZonedDateTimeISO()`. To get a `Temporal.ZonedDateTime` object, you can use `Temporal.Now.zonedDateTimeISO()`. You can also get different parts of the date and time, using `Temporal.Now.plainDateISO()`, `Temporal.Now.plainTimeISO()`, and `Temporal.Now.plainDateTimeISO()`.

For example, if the computer is set to the time zone "America/New_York", `Temporal.Now.zonedDateTimeISO()` returns a zoned date-time like: `2021-08-01T10:40:12.345-04:00[America/New_York]`. In this case, `Temporal.Now.plainTimeISO()` would return the time part of this zoned date-time: `10:40:12.345`. However, if you call `Temporal.Now.plainTimeISO("UTC")`, it returns the time part of the zoned date-time in the UTC time zone: `14:40:12.345`. This is especially useful for cross-system communication where the other end may be expecting the time in a different time zone.

Reduced time precision

To offer protection against timing attacks and fingerprinting, the precision of the current time returned by the `Temporal.Now` methods may be reduced depending on browser settings.

In Chrome, the rounding interval is 0.1 ms, or 0.005 ms in cross-origin-isolated contexts.

In Firefox, these methods obtain the current time in the same way as `Date.now()`. They inherit the precision of that clock reading without introducing additional inaccuracy. The timestamp is always an integer number of milliseconds, so its resolution is limited to 1 ms in all contexts. Accessing `epochNanoseconds` does not increase its precision.

Static properties

`Temporal.Now[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.Now"`. This property is used in `Object.prototype.toString()`.

Static methods

`Temporal.Now.instant()`

Returns the current time as a `Temporal.Instant` object.

`Temporal.Now.plainDateISO()`

Returns the current date as a `Temporal.PlainDate` object, in the ISO 8601 calendar and the specified time zone.

`Temporal.Now.plainDateTimeISO()`

Returns the current date and time as a `Temporal.PlainDateTime` object, in the ISO 8601 calendar and the specified time zone.

`Temporal.Now.plainTimeISO()`

Returns the current time as a `Temporal.PlainTime` object, in the specified time zone.

`Temporal.Now.timeZoneId()`

Returns a time zone identifier representing the system's current time zone.

`Temporal.Now.zonedDateTimeISO()`

Returns the current date and time as a `Temporal.ZonedDateTime` object, in the ISO 8601 calendar and the specified time zone.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-now-object`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Now	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`instant`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`plainDateISO`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`plainDateTimeISO`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`plainTimeISO`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`timeZoneId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`zonedDateTimeISO`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Now`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal`
- `Temporal.Instant`
- `Temporal.PlainDate`

- Temporal.PlainDateTime
- Temporal.PlainTime
- Temporal.ZonedDateTime

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Now

Temporal.Now.instant()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Now/instant · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Now.instant()`** static method returns the current time as a Temporal.Instant object.

Syntax

```js
Temporal.Now.instant()
```

Parameters

None.

Return value

A Temporal.Instant object representing the current time, with potentially reduced precision.

Examples

Measuring time elapsed

The following example measures two instants in time and calculates the duration between them, and gets the total duration in milliseconds:

```js
const start = Temporal.Now.instant();
// Do something that takes time
const end = Temporal.Now.instant();
const duration = end.since(start);
console.log(duration.total("milliseconds"));
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.now.instant

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`instant`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Now.instant`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Now`
- `Temporal.Instant`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Now

Temporal.Now.plainDateISO()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Now/plainDateISO · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Now.plainDateISO()`** static method returns the current date as a `Temporal.PlainDate` object, in the ISO 8601 calendar and the specified time zone.

Syntax

```js
Temporal.Now.plainDateISO()
Temporal.Now.plainDateISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

The current date in the specified time zone, as a `Temporal.PlainDate` object using the ISO 8601 calendar.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.plainDateISO()

```js
// The current date in the system's time zone
const date = Temporal.Now.plainDateISO();
console.log(date); // e.g.: 2021-10-01

// The current date in the "America/New_York" time zone
const dateInNewYork = Temporal.Now.plainDateISO("America/New_York");
console.log(dateInNewYork); // e.g.: 2021-09-30
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.now.plaindateiso

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
plainDateISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Now.plainDateISO`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Now`
- `Temporal.PlainDate`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Now

Temporal.Now.plainDateTimeISO()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Now/plainDateTimeISO · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Now.plainDateTimeISO()`** static method returns the current date and time as a `Temporal.PlainDateTime` object, in the ISO 8601 calendar and the specified time zone.

Syntax

```js
Temporal.Now.plainDateTimeISO()
Temporal.Now.plainDateTimeISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

The current date and time in the specified time zone, as a `Temporal.PlainDateTime` object using the ISO 8601 calendar. Has the same precision as `Temporal.Now.instant()`.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.plainDateTimeISO()

```js
// The current date and time in the system's time zone
const dateTime = Temporal.Now.plainDateTimeISO();
console.log(dateTime); // e.g.: 2021-10-01T06:12:34.567890123

// The current date and time in the "America/New_York" time zone
const dateTimeInNewYork = Temporal.Now.plainDateTimeISO("America/New_York");
console.log(dateTimeInNewYork); // e.g.: 2021-09-30T23:12:34.567890123
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.now.plaindatetimeiso`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`plainDateTimeISO`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Now.plainDateTimeISO`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Now`
- `Temporal.PlainDateTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Now

Temporal.Now.plainTimeISO()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Now/plainTimeISO · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Now.plainTimeISO()`** static method returns the current time as a `Temporal.PlainTime` object, in the specified time zone.

Note that although the method contains "ISO" in its name, `Temporal.PlainTime` objects do not have associated calendars, as the time format is not calendar-dependent.

Syntax

```js
Temporal.Now.plainTimeISO()
Temporal.Now.plainTimeISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

The current time in the specified time zone, as a `Temporal.PlainTime` object. Has the same precision as `Temporal.Now.instant()`.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.plainTimeISO()

```js
// The current time in the system's time zone
const time = Temporal.Now.plainTimeISO();
console.log(time); // e.g.: 06:12:34.567890123

// The current time in the "America/New_York" time zone
const timeInNewYork = Temporal.Now.plainTimeISO("America/New_York");
console.log(timeInNewYork); // e.g.: 23:12:34.567890123
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.now.plaintimeiso

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
plainTimeISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Now.plainTimeISO`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Now`
- `Temporal.PlainTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Now

Temporal.Now.timeZoneId()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Now/timeZoneId · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Now.timeZoneId()`** static method returns a time zone identifier representing the system's current time zone. Most systems will return a primary time zone identifier such as `"America/New_York"`, though offset time zone identifier such as `"-04:00"` is possible too. The time zone identifier returned is the default time zone used by the other `Temporal.Now` methods.

Syntax

```js
Temporal.Now.timeZoneId()
```

Parameters

None.

Return value

A valid time zone identifier representing the system's current time zone. The returned time zone identifier is never a non-primary time zone identifier (alias). For example, it would always return `"Asia/Kolkata"` (new name) instead of `"Asia/Calcutta"` (old name). For more information, see time zones and offsets.

If the implementation does not support time zones, the method always returns `"UTC"`.

Examples

Getting the system's current time zone

```js
console.log(Temporal.Now.timeZoneId()); // e.g.: "America/New_York"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.now.timezoneid

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
timeZoneId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Now.timeZoneId`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Now`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.Now

Temporal.Now.zonedDateTimeISO()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/Now/zonedDateTimeISO · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.Now.zonedDateTimeISO()`** static method returns the current date and time as a `Temporal.ZonedDateTime` object, in the ISO 8601 calendar and the specified time zone.

Syntax

```js
Temporal.Now.zonedDateTimeISO()
Temporal.Now.zonedDateTimeISO(timeZone)
```

Parameters

`timeZone` Optional

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to interpret the system time in. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a

named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

The current date and time in the specified time zone, as a `Temporal.ZonedDateTime` object using the ISO 8601 calendar. Has the same precision as `Temporal.Now.instant()`.

Exceptions

`RangeError`

Thrown if the time zone is invalid.

Examples

Using Temporal.Now.zonedDateTimeISO()

```js
// The current date and time in the system's time zone
const dateTime = Temporal.Now.zonedDateTimeISO();
console.log(dateTime); // e.g.: 2021-10-01T06:12:34.567890123+03:00[Africa/Nairobi]

// The current date and time in the "America/New_York" time zone
const dateTimeInNewYork = Temporal.Now.zonedDateTimeISO("America/New_York");
console.log(dateTimeInNewYork); // e.g.: 2021-09-30T23:12:34.567890123-
04:00[America/New_York]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.now.zoneddatetimeiso

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
zonedDateTimeISO	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.Now.zonedDateTimeISO`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.Now`
- `Temporal.ZonedDateTime`

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.PlainDate

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDate`** object represents a calendar date (a date without a time or time zone); for example, an event on a calendar which happens during the whole day no matter which time zone it's happening in. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system.

Description

A `PlainDate` is essentially the date part of a `Temporal.PlainDateTime` object, with the time information removed. Because the date and time information don't have much interaction, all general information about date properties is documented here.

RFC 9557 format

`PlainDate` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD [u-ca=calendar_id]
```

`YYYY`

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD`

A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the time, offset, and time zone identifier, in the same format as `PlainDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure whether to display the calendar ID, and whether to add a critical flag for it.

Invalid date clamping

The `Temporal.PlainDate.from()`, `Temporal.PlainDate.prototype.with()`, `Temporal.PlainDate.prototype.add()`, `Temporal.PlainDate.prototype.subtract()` methods, and their counterparts in other `Temporal` objects, allow constructing dates using calendar-specific

properties. The date components may be out of range. In the ISO calendar, this is always an *overflow*, such as the month being greater than 12 or the day being greater than the number of days, and fixing it would only involve clamping the value to the maximum allowed value. In other calendars, the invalid case may be more complex. When using the `overflow: "constrain"` option, invalid dates are fixed to a valid one in the following way:

- If the day doesn't exist but the month does: pick the closest day in the same month. If there are two equally-close dates in that month, pick the later one.
- If the month is a leap month that doesn't exist in the year: pick another date according to the cultural conventions of that calendar's users. Usually this will result in the same day in the month before or after where that month would normally fall in a leap year.
- If the month doesn't exist in the year for other reasons: pick the closest date that is still in the same year. If there are two equally-close dates in that year, pick the later one.
- If the entire year doesn't exist: pick the closest date in a different year. If there are two equally-close dates, pick the later one.

Constructor

`Temporal.PlainDate()`

Creates a new `Temporal.PlainDate` object by directly supplying the underlying data.

Static methods

`Temporal.PlainDate.compare()`

Returns a number (-1, 0, or 1) indicating whether the first date comes before, is the same as, or comes after the second date. Equivalent to comparing the year, month, and day fields of the underlying ISO 8601 dates.

`Temporal.PlainDate.from()`

Creates a new `Temporal.PlainDate` object from another `Temporal.PlainDate` object, an object with date properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainDate.prototype` and shared by all `Temporal.PlainDate` instances.

`Temporal.PlainDate.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainDate.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainDate` instances, the initial value is the `Temporal.PlainDate()` constructor.

`Temporal.PlainDate.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

`Temporal.PlainDate.prototype.dayOfWeek`

Returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name.

Calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

`Temporal.PlainDate.prototype.dayOfYear`

Returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. Calendar-dependent.

`Temporal.PlainDate.prototype.daysInMonth`

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

`Temporal.PlainDate.prototype.daysInWeek`

Returns a positive integer representing the number of days in the week of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week.

`Temporal.PlainDate.prototype.daysInYear`

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

`Temporal.PlainDate.prototype.era`

Returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

`Temporal.PlainDate.prototype.eraYear`

Returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

`Temporal.PlainDate.prototype.inLeapYear`

Returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.PlainDate.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.PlainDate.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainDate.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.PlainDate.prototype.weekOfYear`

Returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`.

Calendar-dependent. Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year.

`Temporal.PlainDate.prototype.year`

Returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.PlainDate.prototype.yearOfWeek`

Returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. Calendar-dependent. Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1.

`Temporal.PlainDate.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainDate"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainDate.prototype.add()`

Returns a new `Temporal.PlainDate` object representing this date moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDate.prototype.equals()`

Returns `true` if this date is equivalent in value to another date (in a form convertible by `Temporal.PlainDate.from()`), and `false` otherwise. They are compared both by their date values and their calendars.

`Temporal.PlainDate.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another date (in a form convertible by `Temporal.PlainDate.from()`) to this date. The duration is positive if the other date is before this date, and negative if after.

`Temporal.PlainDate.prototype.subtract()`

Returns a new `Temporal.PlainDate` object representing this date moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDate.prototype.toJSON()`

Returns a string representing this date in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainDate.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this date.

`Temporal.PlainDate.prototype.toPlainDateTime()`

Returns a new `Temporal.PlainDateTime` object representing this date and a supplied time in the same calendar system.

`Temporal.PlainDate.prototype.toPlainMonthDay()`

Returns a new `Temporal.PlainMonthDay` object representing the `monthCode` and `day` of this date in the same calendar system.

`Temporal.PlainDate.prototype.toPlainYearMonth()`

Returns a new `Temporal.PlainYearMonth` object representing the `year` and `month` of this date in the same calendar system.

`Temporal.PlainDate.prototype.toString()`

Returns a string representing this date in the RFC 9557 format.

`Temporal.PlainDate.prototype.toZonedDateTime()`

Returns a new `Temporal.ZonedDateTime` object representing this date, a supplied time, and a supplied time zone, in the same calendar system.

`Temporal.PlainDate.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this date to another date (in a form convertible by `Temporal.PlainDate.from()`). The duration is positive if the other date is after this date, and negative if before.

`Temporal.PlainDate.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainDate` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainDate.prototype.with()`

Returns a new `Temporal.PlainDate` object representing this date with some fields replaced by new values.

`Temporal.PlainDate.prototype.withCalendar()`

Returns a new `Temporal.PlainDate` object representing this date interpreted in the new calendar system.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-plaindate-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainDate()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0
`add`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`day`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`dayOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`dayOfYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`daysInMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`daysInWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
daysInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
era	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
inLeapYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
month	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthsInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainMonthDay	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainYearMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toZonedDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
until	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
valueOf	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
weekOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
with	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
withCalendar	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
year	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
yearOfWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

1. Partial support.
2. Calendars `islamic`, `islamic-rgsa` and `islamic-umalqura` are not available. See bug 1950425 and bug 1954138.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.PlainDateTime`
- `Temporal.PlainMonthDay`
- `Temporal.PlainYearMonth`
- `Temporal.ZonedDateTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/PlainDate · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDate()`** constructor creates `Temporal.PlainDate` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainDate` objects using the `Temporal.PlainDate.from()` static method, which can handle a variety of input types.

Syntax

```js
new Temporal.PlainDate(year, month, day)
new Temporal.PlainDate(year, month, day, calendar)
```

> **Note:** `Temporal.PlainDate()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`year`

A number, truncated to an integer, representing the year in the ISO calendar system.

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`day`

A number, truncated to an integer, representing the day of the month in the ISO calendar system.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `year`, `month`, and `day` must be in the ISO 8601 calendar system.

Return value

A new `Temporal.PlainDate` object, representing the date specified by `year`, `month`, `day` (in the ISO calendar), interpreted in the calendar system specified by `calendar`.

Exceptions

`TypeError`

Thrown if `calendar` is not a string or `undefined`.

`RangeError`

Thrown in one of the following cases:

- `year`, `month`, or `day` is not a finite number.
- The `year`, `month`, and `day` combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainDate()

```js
const plainDate = new Temporal.PlainDate(2021, 7, 1);
console.log(plainDate.toString()); // 2021-07-01

// Note that the date is stored internally as ISO 8601, even when it's
// interpreted in a different calendar system. For example, even though
// 2021-07-01 ISO is 5781-10-21 in the Hebrew calendar, you still pass the
// ISO date to the constructor.
const plainDate2 = new Temporal.PlainDate(2021, 7, 1, "hebrew");
console.log(plainDate2.toString()); // 2021-07-01[u-ca=hebrew]
console.log(plainDate2.year); // 5781
console.log(plainDate2.month); // 10
console.log(plainDate2.day); // 21
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-plaindate-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainDate()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.PlainDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/compare · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDate.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first date comes before, is the same as, or comes after the second date. Equivalent to comparing the year, month, and day fields of the underlying ISO 8601 dates.

Syntax

```js
Temporal.PlainDate.compare(date1, date2)
```

Parameters

`date1`

A string, an object, or a `Temporal.PlainDate` instance representing the first date to compare. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`.

`date2`

The second date to compare, converted to a `Temporal.PlainDate` object using the same algorithm as `date1`.

Return value

Returns `-1` if `date1` comes before `date2`, `0` if they are the same, and `1` if `date1` comes after `date2`. They are compared by their underlying date values, ignoring their calendars.

Examples

Using Temporal.PlainDate.compare()

```js
const date1 = Temporal.PlainDate.from("2021-08-01");
const date2 = Temporal.PlainDate.from("2021-08-02");
console.log(Temporal.PlainDate.compare(date1, date2)); // -1

const date3 = Temporal.PlainDate.from("2021-07-31");
console.log(Temporal.PlainDate.compare(date1, date3)); // 1
```

Comparing dates in different calendars

```js
const date1 = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const date2 = Temporal.PlainDate.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "islamic-umalqura",
});
const date3 = Temporal.PlainDate.from({
```

(code continues on the next page)

(code continued from the previous page)

```
  year: 2021,
  month: 8,
  day: 1,
  calendar: "hebrew",
});
console.log(date1.toString()); // "2021-08-01"
console.log(date2.toString()); // "2582-12-17[u-ca=islamic-umalqura]"
console.log(date3.toString()); // "-001739-04-06[u-ca=hebrew]"
console.log(Temporal.PlainDate.compare(date1, date2)); // -1
console.log(Temporal.PlainDate.compare(date1, date3)); // 1
```

Sorting an array of dates

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

js

```
const dates = [
  Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 }),
  Temporal.PlainDate.from({
    year: 2021,
    month: 8,
    day: 1,
    calendar: "islamic-umalqura",
  }),
  Temporal.PlainDate.from({ year: 2021, month: 8, day: 1, calendar: "hebrew" }),
];

dates.sort(Temporal.PlainDate.compare);
console.log(dates.map((d) => d.toString()));
// [ "-001739-04-06[u-ca=hebrew]", "2021-08-01", "2582-12-17[u-ca=islamic-umalqura]" ]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.compare

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.compare`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.equals()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/from · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDate.from()`** static method creates a new `Temporal.PlainDate` object from another `Temporal.PlainDate` object, an object with date properties, or an RFC 9557 string.

Syntax

```js
Temporal.PlainDate.from(info)
Temporal.PlainDate.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainDate` instance, which creates a copy of the instance.
- A `Temporal.PlainDateTime` instance, which provides the calendar date in the same fashion as `Temporal.PlainDateTime.prototype.toPlainDate()`.
- A `Temporal.ZonedDateTime` instance, which provides the calendar date in the same fashion as `Temporal.ZonedDateTime.prototype.toPlainDate()`.
- An RFC 9557 string containing a date and optionally a calendar.
- An object containing the following properties (in the order they are retrieved and validated):

  `calendar` Optional

  A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. All other properties are interpreted in this calendar system (unlike the `Temporal.PlainDate()` constructor, which interprets the values in the ISO calendar system).

  `day`

  An integer that corresponds to the `day` property. Must be positive regardless of the `overflow` option.

  `era` and `eraYear`

  A string and an integer that correspond to the `era` and `eraYear` properties. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. At least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

  `month`

  Corresponds to the `month` property. Must be positive regardless of the `overflow` option. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

  `monthCode`

  Corresponds to the `monthCode` property. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

`year`

Corresponds to the year property. At least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

The info should explicitly specify a year (as `year` or `era` and `eraYear`), a month (as `month` or `monthCode`), and a day.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object, representing the date specified by `info` in the specified `calendar`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month` (or `monthCode`), and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainDate from an object

```js
// Year, month, and day
const d1 = Temporal.PlainDate.from({ year: 2021, month: 7, day: 1 });
console.log(d1.toString()); // "2021-07-01"

// Year, month code, and day
const d2 = Temporal.PlainDate.from({ year: 2021, monthCode: "M07", day: 1 });
console.log(d2.toString()); // "2021-07-01"

// Year, month, day in a different calendar
const d3 = Temporal.PlainDate.from({
  year: 2021,
  month: 7,
  day: 1,
  calendar: "hebrew",
});
// Note: when you construct a date with an object, the date components
// are in *that* calendar, not the ISO calendar. However, toString() always
// outputs the date in the ISO calendar. For example, the year "2021" in
// the Hebrew calendar is actually 1740 BCE in the ISO calendar.
console.log(d3.toString()); // "-001739-03-07[u-ca=hebrew]"

// Era, eraYear, month, and day
const d4 = Temporal.PlainDate.from({
  era: "meiji",
  eraYear: 4,
  month: 7,
  day: 1,
  calendar: "japanese",
});
console.log(d4.toString()); // "1871-07-01[u-ca=japanese]"
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range:

```js
const d1 = Temporal.PlainDate.from({ year: 2021, month: 13, day: 1 });
console.log(d1.toString()); // "2021-12-01"

const d2 = Temporal.PlainDate.from({ year: 2021, month: 2, day: 29 });
console.log(d2.toString()); // "2021-02-28"

const d3 = Temporal.PlainDate.from("2021-02-29");
console.log(d3.toString()); // "2021-02-28"
```

You can change this behavior to throw an error instead:

```js
const d3 = Temporal.PlainDate.from(
  { year: 2021, month: 13, day: 1 },
  { overflow: "reject" },
);
// RangeError: date value "month" not in 1..12: 13
```

Creating a PlainDate from a string

```js
const d = Temporal.PlainDate.from("2021-07-01");
console.log(d.toLocaleString("en-US", { dateStyle: "full" }));
// Thursday, July 1, 2021

// Providing a calendar
const d2 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
console.log(
  d2.toLocaleString("ja-JP", { calendar: "japanese", dateStyle: "full" }),
);
// 令和3年7月1日木曜日

// Providing a time and an offset (ignored)
const d3 = Temporal.PlainDate.from("2021-07-01T00:00+08:00");
console.log(d3.toString()); // "2021-07-01"
```

Creating a PlainDate from another Temporal instance

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:00");
const d = Temporal.PlainDate.from(dt);
console.log(d.toString()); // "2021-07-01"

const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T00:00+08:00[Asia/Shanghai]",
);
const d2 = Temporal.PlainDate.from(zdt);
console.log(d2.toString()); // "2021-07-01"

const d3 = Temporal.PlainDate.from(d);
console.log(d3.toString()); // "2021-07-01"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.from`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate()`
- `Temporal.PlainDate.prototype.with()`

Temporal.PlainDate.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/add · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`add()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this date. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object representing the date specified by the original `PlainDate`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

The `duration` is handled in this way:

- Move forward by the number of years, keeping the `monthCode` and `day` the same. If the `monthCode` is invalid in the resulting year (impossible for Gregorian and ISO 8601, but possible for calendars with leap months), we adjust based on the `overflow` option: for `constrain`, we pick another month according to the cultural conventions of that calendar's users. For example, because the leap month is usually thought of as a duplicate of another month, we may pick the month that it is a duplicate of.
- Move forward by the number of months, adjusting the year if necessary, keeping the `day` the same. If the `day` is invalid in the resulting month (e.g., February 30), we adjust based on the `overflow` option: for `constrain`, we pick the closest valid day (e.g., February 28 or 29).
- All commonly supported calendars use fixed-length weeks, so the number of weeks is just converted to the number of days. If the rule is more complex, we may take an approach similar to shifting months.
- For all non-calendar units (days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds), they are converted to the number of days. Fractional part of a day is ignored. Then, we move forward by that number of days, adjusting the month and year if necessary.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration in ISO 8601 calendar

```js
const start = Temporal.PlainDate.from("2021-01-01");
const end = start.add({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2022-03-26

const end2 = start.add({ years: -1, months: -2, weeks: -3, days: -4 });
console.log(end2.toString()); // 2019-10-07

const distance = Temporal.PlainDate.from("2020-01-01").until("2021-01-01"); // 366 days
const end3 = start.add(distance);
console.log(end3.toString()); // 2022-01-02
```

Adding a duration in a non-ISO calendar

```js
const start = Temporal.PlainDate.from("2021-01-01[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 11/18/2020
const end = start.add({ months: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 12/18/2020
```

Adding a duration with overflow

If we move a few months and the corresponding day is invalid in this month, then we adjust the day based on the `overflow` option.

```js
const start = Temporal.PlainDate.from("2021-01-31");
const end = start.add({ months: 1 });
console.log(end.toString()); // 2021-02-28

// Any further day additions are based on the clamped month-day:
const end2 = start.add({ months: 1, days: 31 });
console.log(end2.toString()); // 2021-03-31

// Compare with the same addition in a different order that results in no overflow:
const end3 = start.add({ days: 31 }).add({ months: 1 });
console.log(end3.toString()); // 2021-04-03
```

Overflow can also occur for the month, for calendars where different years have different numbers of months (usually due to leap months).

```js
const start = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 2bis/11/2023; "bis"
means leap month
const end = start.add({ years: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 3/11/2024

// Compare:
const start = Temporal.PlainDate.from("2023-04-30[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 3/11/2023
const end = start.add({ years: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 3/11/2024; same day as
above!
```

Note that the following is not an overflow because the month can just increment:

```js
const start = Temporal.PlainDate.from("2021-01-01");
const end = start.add({ days: 100 });
console.log(end.toString()); // 2021-04-11
```

You can also throw an error if the date component is out of range:

```js
const start = Temporal.PlainDate.from("2021-01-31");
const end = start.add({ months: 1 }, { overflow: "reject" }); // RangeError: date value
"day" not in 1..28: 31

const start = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
const end = start.add({ years: 1 }, { overflow: "reject" }); // RangeError: invalid
"monthCode" calendar field: M02L
```

Adding time durations

Fractional parts of a day are ignored.

```js
const start = Temporal.PlainDate.from("2021-01-01");
const end = start.add({ hours: 25 }); // Same as adding 1 day
console.log(end.toString()); // 2021-01-02
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.add`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.Duration`
- `Temporal.PlainDate.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.equals()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/equals · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`equals()`** method of `Temporal.PlainDate` instances returns `true` if this date is equivalent in value to another date (in a form convertible by `Temporal.PlainDate.from()`), and `false` otherwise. They are compared both by their date values and their calendars, so two dates from different calendars may be considered equal by `Temporal.PlainDate.compare()` but not by `equals()`.

Syntax

```js
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDate` instance representing the other date to compare. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`.

Return value

`true` if this date is equal to `other` both in their date value and their calendar, `false` otherwise.

Examples

Using equals()

```js
const date1 = Temporal.PlainDate.from("2021-08-01");
const date2 = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
console.log(date1.equals(date2)); // true

const date3 = Temporal.PlainDate.from("2021-08-01[u-ca=japanese]");
console.log(date1.equals(date3)); // false

const date4 = Temporal.PlainDate.from("2021-08-02");
console.log(date1.equals(date4)); // false
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.equals

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.equals`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.compare()`

Temporal.PlainDate.prototype.since()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/since · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`since()`** method of `Temporal.PlainDate` instances returns a new `Temporal.Duration` object representing the duration from another date (in a form convertible by `Temporal.PlainDate.from()`) to this date. The duration is positive if the other date is before this date, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```js
since(other)
since(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDate` instance representing a date to subtract from this date. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`. It must have the same calendar as `this`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"years"`, `"months"`, `"weeks"`, `"days"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"days"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"days"`. The current date is used as the `relativeTo` option. Note that using units larger than `"days"` may make the duration not portable to other calendars or dates.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this date. The duration is positive if `other` is before this date, and negative if after.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using since()

```js
const date = Temporal.PlainDate.from("2022-12-25");
const now = Temporal.Now.plainDateISO();
const duration = now.since(date);
const formatter = new Intl.DurationFormat("en-US", { style: "long" });
console.log(`It's been ${formatter.format(duration)} since that Christmas...`);
// Expected output: "It's been [number] days since that Christmas..."

const duration2 = now.since(date, { smallestUnit: "months" });
console.log(`It's been ${formatter.format(duration2)} since that Christmas...`);
// Expected output: "It's been [number] months since that Christmas..."

const duration3 = now.since(date, {
  largestUnit: "years",
  smallestUnit: "months",
});
console.log(`It's been ${formatter.format(duration3)} since that Christmas...`);
// Expected output: "It's been [number] years, [number] months since that Christmas..."
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```js
const date1 = Temporal.PlainDate.from("2022-01-01");
const date2 = Temporal.PlainDate.from("2022-01-28");
const duration = date2.since(date1, {
  smallestUnit: "days",
  roundingIncrement: 5,
  roundingMode: "ceil",
});
console.log(duration.toString()); // "P30D"
```

Comparing different calendars

By default, the two dates must have the same calendar. This is to avoid ambiguity in the meaning of months and years. If you want to compare dates from different calendars, you can convert them to the same calendar first.

```js
const date1 = Temporal.PlainDate.from("2022-01-01");
const date2 = Temporal.PlainDate.from("2022-01-28[u-ca=chinese]");
const duration = date2.withCalendar("iso8601").since(date1);
console.log(duration.toString()); // "P27D"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.since

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.since`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.Duration`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.until()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.subtract()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/subtract · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`subtract()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two dates and get a duration, use `since()` or `until()` instead.

Syntax

```
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this date. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object representing the date specified by the original `PlainDate`, minus the duration.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Subtracting a duration

```js
const start = Temporal.PlainDate.from("2022-01-01");
const end = start.subtract({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2020-10-07
```

For more examples, see `add()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.subtract`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.subtract`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainDate
- Temporal.Duration
- Temporal.PlainDate.prototype.add()
- Temporal.PlainDate.prototype.since()
- Temporal.PlainDate.prototype.until()

Temporal.PlainDate.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of Temporal.PlainDate instances returns a string representing this date in the same RFC 9557 format as calling toString(). It is intended to be implicitly called by JSON.stringify().

Syntax

```js
toJSON()
```

Parameters

None.

Return value

A string representing the given date in the RFC 9557 format, with the calendar annotation included if it is not "iso8601".

Description

The method is automatically called by JSON.stringify() when a Temporal.PlainDate object is encountered during JSON serialization. The serialized value can then be deserialized using the Temporal.PlainDate.from() function within the reviver of JSON.parse().

Examples

Using toJSON()

```js
const date = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const dateStr = date.toJSON(); // '2021-08-01'
const d2 = Temporal.PlainDate.from(dateStr);
```

JSON serialization and parsing

This example shows how Temporal.PlainDate can be serialized as JSON without extra effort, and how to parse it back.

```js
const date = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const jsonStr = JSON.stringify({ date }); // '{"date":"2021-08-01"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "date") {
    return Temporal.PlainDate.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.toJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.from()`
- `Temporal.PlainDate.prototype.toString()`
- `Temporal.PlainDate.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.PlainDate` instances returns a string with a language-sensitive representation of this date. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If this date's calendar is not `"iso8601"`, the `calendar` option must be provided with the same value; otherwise, if this date's calendar is `"iso8601"`, the `calendar` option can be any value. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `year`, `month`, and `day` will default to `"numeric"`.
- Provide `dateStyle` only: it expands to `weekday`, `era`, `year`, `month`, and `day` formats.
- Provide some date-time component options, where at least one of them is a date option (`weekday`, `year`, `month`, `day`). Only the specified date components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const date = Temporal.PlainDate.from("2021-08-01");

console.log(date.toLocaleString()); // 8/1/2021 (assuming en-US locale)
```

If the date's calendar doesn't match the locale's default calendar, and the date's calendar is not `iso8601`, an explicit `calendar` option must be provided with the same value.

```js
const date = Temporal.PlainDate.from("2021-08-01[u-ca=japanese]");
// The ja-JP locale uses the Gregorian calendar by default
date.toLocaleString("ja-JP", { calendar: "japanese" }); // R3/8/1
```

Using toLocaleString() with options

You can customize which parts of the date are included in the output by providing the `options` parameter.

```js
const date = Temporal.PlainDate.from("2021-08-01");
date.toLocaleString("en-US", { dateStyle: "full" }); // Sunday, August 1, 2021
date.toLocaleString("en-US", {
  year: "numeric",
  month: "long",
  day: "numeric",
}); // August 1, 2021
date.toLocaleString("en-US", { year: "numeric", month: "long" }); // August 2021
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.tolocalestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Intl.DateTimeFormat`
- `Temporal.PlainDate.prototype.toJSON()`
- `Temporal.PlainDate.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.toPlainDateTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/toPlainDateTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainDateTime()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDateTime` object representing this date and a supplied time in the same calendar system.

Syntax

js

```
toPlainDateTime()
toPlainDateTime(plainTime)
```

Parameters

`plainTime` Optional

A string, an object, or a `Temporal.PlainTime` instance representing the time component of the resulting `PlainDateTime`. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. Defaults to `"00:00:00"`.

Return value

A new `Temporal.PlainDateTime` object representing the date and time specified by this date and `plainTime`, interpreted in the calendar system of this date.

Examples

Using toPlainDateTime()

js

```
const date = Temporal.PlainDate.from("2021-07-01");
const dateTime = date.toPlainDateTime("12:34:56");
console.log(dateTime.toString()); // 2021-07-01T12:34:56

const midnight = date.toPlainDateTime();
console.log(midnight.toString()); // 2021-07-01T00:00:00

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
const dateTime2 = date2.toPlainDateTime("12:34:56");
console.log(dateTime2.toString()); // 2021-07-01T12:34:56[u-ca=chinese]
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.toplaindatetime`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainDateTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.toPlainDateTime`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDateTime`
- `Temporal.PlainTime`
- `Temporal.PlainDate.prototype.toPlainMonthDay()`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`
- `Temporal.PlainDateTime.prototype.toPlainDate()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.toPlainMonthDay()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/toPlainMonthDay · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainMonthDay()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainMonthDay` object representing the `monthCode` and `day` of this date in the same calendar system.

Note that `PlainMonthDay` objects do not have a `month` component, because months with the same name can have different `month` indexes in different years due to leap months.

Syntax

```js
toPlainMonthDay()
```

Parameters

None.

Return value

A new `Temporal.PlainMonthDay` object representing the `monthCode` and `day` of this date in the same calendar system.

Examples

Using toPlainMonthDay()

```js
const date = Temporal.PlainDate.from("2021-07-01");
const monthDay = date.toPlainMonthDay();
console.log(monthDay.toString()); // 07-01
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.toplainmonthday`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainMonthDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.toPlainMonthDay`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainMonthDay`
- `Temporal.PlainDate.prototype.toPlainDateTime()`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`
- `Temporal.PlainMonthDay.prototype.toPlainDate()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.toPlainYearMonth()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/toPlainYearMonth · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainYearMonth()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainYearMonth` object representing the `year` and `month` of this date in the same calendar system.

Syntax

```
toPlainYearMonth()
```

Parameters

None.

Return value

A new `Temporal.PlainYearMonth` object representing the `year` and `month` of this date in the same calendar system.

Examples

Using toPlainYearMonth()

```
const date = Temporal.PlainDate.from("2021-07-01");
const yearMonth = date.toPlainYearMonth();
console.log(yearMonth.toString()); // 2021-07
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.toplainyearmonth`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainYearMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.toPlainYearMonth`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainDate
- Temporal.PlainYearMonth
- Temporal.PlainDate.prototype.toPlainDateTime()
- Temporal.PlainDate.prototype.toPlainMonthDay()
- Temporal.PlainDate.prototype.toZonedDateTime()
- Temporal.PlainYearMonth.prototype.toPlainDate()

Temporal.PlainDate.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/toString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of Temporal.PlainDate instances returns a string representing this date in the RFC 9557 format.

Syntax

```js
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same Temporal.PlainDate instance, although the date value still remains the same.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

Return value

A string in the RFC 9557 format representing this date. The calendar annotation is included as specified.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```js
const date = Temporal.PlainDate.from("2021-08-01");
console.log(date.toString()); // '2021-08-01'
```

Using options

```js
const isoDate = Temporal.PlainDate.from({ year: 2021, month: 8, day: 1 });
const date = Temporal.PlainDate.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "islamic-umalqura",
});
console.log(isoDate.toString({ calendarName: "auto" })); // '2021-08-01'
console.log(date.toString({ calendarName: "auto" })); // '2582-12-17[u-ca=islamic-umalqura]'
console.log(isoDate.toString({ calendarName: "always" })); // '2021-08-01[u-ca=iso8601]'
console.log(date.toString({ calendarName: "always" })); // '2582-12-17[u-ca=islamic-
umalqura]'
console.log(date.toString({ calendarName: "never" })); // '2582-12-17'
console.log(isoDate.toString({ calendarName: "critical" })); // '2021-08-01[!u-ca=iso8601]'
console.log(date.toString({ calendarName: "critical" })); // '2582-12-17[!u-ca=islamic-
umalqura]'
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.tostring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.toString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainDate
- Temporal.PlainDate.from()
- Temporal.PlainDate.prototype.toJSON()
- Temporal.PlainDate.prototype.toLocaleString()

Temporal.PlainDate.prototype.toZonedDateTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/toZonedDateTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toZonedDateTime()`** method of Temporal.PlainDate instances returns a new Temporal.ZonedDateTime object representing this date, a supplied time, and a supplied time zone, in the same calendar system.

Syntax

```js
toZonedDateTime(timeZone)
toZonedDateTime(info)
```

Parameters

`timeZone`

Either a string or a Temporal.ZonedDateTime instance representing the timeZone option. This is a convenience overload, so `toZonedDateTime(timeZone)` is equivalent to `toZonedDateTime({ timeZone })`, where `timeZone` is a string or Temporal.ZonedDateTime. This overload is chosen when the first argument is not an object, or the object's `timeZone` property is `undefined` (because `ZonedDateTime` instances have a timeZoneId property instead).

`info`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`plainTime` Optional

A string, an object, or a Temporal.PlainTime instance representing the time component of the resulting `ZonedDateTime`. It is converted to a `Temporal.PlainTime` object using the same algorithm as Temporal.PlainTime.from(). Defaults to the first valid time in this time zone on this calendar date, which is usually `"00:00:00"`, but may be different if, for example, daylight saving time skips midnight.

`timeZone`

Either a string or a Temporal.ZonedDateTime instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

A new `Temporal.ZonedDateTime` object representing the date and time specified by this date, `plainTime`, and `timeZone`, interpreted in the calendar system of this date.

In the case of ambiguities, the `compatible` behavior is always used: if the time falls into a gap, we move forward by the gap length; if the time falls into an ambiguity, we choose the earlier of the two possibilities. This means the resulting `ZonedDateTime` may have a potentially different date or time than the input.

Exceptions

`TypeError`

Thrown if `timeZone` is not a string or a `Temporal.ZonedDateTime` instance.

`RangeError`

Thrown if `timeZone` is a string that is not a valid time zone identifier.

Examples

Using toZonedDateTime()

```js
const summer = Temporal.PlainDate.from("2021-07-01");
// Just time zone
const summerTime = summer.toZonedDateTime("America/New_York");
console.log(summerTime.toString()); // 2021-07-01T00:00:00-04:00[America/New_York]

const winter = Temporal.PlainDate.from("2021-01-01");
// Time zone and time
const winterTime = winter.toZonedDateTime({
  plainTime: "12:34:56",
  timeZone: "America/New_York",
});
console.log(winterTime.toString()); // 2021-01-01T12:34:56-05:00[America/New_York]

const spring = Temporal.PlainDate.from("2021-03-01");
// Time zone as object and time as object
const springTime = spring.toZonedDateTime({
  plainTime: summerTime.toPlainTime(),
  timeZone: winterTime,
});
console.log(springTime.toString()); // 2021-03-01T00:00:00-05:00[America/New_York]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.tozoneddatetime

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toZonedDateTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.toZonedDateTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.ZonedDateTime`
- `Temporal.PlainTime`
- `Temporal.PlainDate.prototype.toPlainDateTime()`
- `Temporal.PlainDate.prototype.toPlainMonthDay()`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.until()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/until · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`until()`** method of `Temporal.PlainDate` instances returns a new `Temporal.Duration` object representing the duration from this date to another date (in a form convertible by `Temporal.PlainDate.from()`). The duration is positive if the other date is after this date, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```js
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDate` instance representing a date to subtract this date from. It is converted to a `Temporal.PlainDate` object using the same algorithm as `Temporal.PlainDate.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this date *until* `other`. The duration is positive if `other` is after this date, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using until()

```js
const launch = Temporal.PlainDate.from("2035-01-01");
const now = Temporal.Now.plainDateISO();
const duration = now.until(launch);
console.log(`It will be ${duration.toLocaleString("en-US")} until the launch`);
```

For more examples, see `since()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.until`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.until`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDate`
- `Temporal.Duration`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.since()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.PlainDate` instances throws a `TypeError`, which prevents `Temporal.PlainDate` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `date1 > date2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainDate` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainDate.prototype.toString()`, or use the `Temporal.PlainDate.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainDate

All arithmetic and comparison operations on `Temporal.PlainDate` instances should use the dedicated methods or convert them to primitives explicitly.

```
const date1 = Temporal.PlainDate.from("2022-01-01");
const date2 = Temporal.PlainDate.from("2022-07-01");
date1 > date2; // TypeError: can't convert PlainDate to primitive type
Temporal.PlainDate.compare(date1, date2); // -1

date2 - date1; // TypeError: can't convert PlainDate to primitive type
date2.since(date1).toString(); // "P181D"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.valueof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.toString()`
- `Temporal.PlainDate.prototype.toJSON()`
- `Temporal.PlainDate.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/with · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`with()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date's fields.

To replace the `calendarId` property, use the `withCalendar()` method instead.

Syntax

```js
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainDate.from()` (except `calendar`): `day`, `era` and `eraYear`, `month`, `monthCode`, `year`. Unspecified properties use the values from the original date. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDate` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```js
const date = Temporal.PlainDate.from("2021-07-06");
const newDate = date.with({ day: date.daysInMonth });
console.log(newDate.toString()); // 2021-07-31
const nextDecade = date.with({ year: date.year + 10 });
console.log(nextDecade.toString()); // 2031-07-06
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.with

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.with`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.withCalendar()`
- `Temporal.PlainDate.from()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.withCalendar()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/withCalendar · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`withCalendar()`** method of `Temporal.PlainDate` instances returns a new `Temporal.PlainDate` object representing this date interpreted in the new calendar system. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date's `calendarId` property.

To replace the date component properties, use the `with()` method instead.

Syntax

```
withCalendar(calendar)
```

Parameters

`calendar`

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

Return value

A new `Temporal.PlainDate` object, representing the date specified by the original `PlainDate`, interpreted in the new calendar system.

Exceptions

`TypeError`

Thrown if `calendar` is not a string.

`RangeError`

Thrown if `calendar` is not a valid calendar identifier.

Examples

Using withCalendar()

```js
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.withCalendar("islamic-umalqura");
console.log(newDate.toLocaleString("en-US", { calendar: "islamic-umalqura" }));
// 11/21/1442 AH
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindate.prototype.withcalendar

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`withCalendar`	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

1. Partial support.
2. Calendars `islamic`, `islamic-rgsa` and `islamic-umalqura` are not available. See bug 1950425 and bug 1954138.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.withCalendar`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.from()`
- `Temporal.PlainDate.prototype.calendarId`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.calendarId

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/calendarId · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`calendarId`** accessor property of `Temporal.PlainDate` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. Use the `withCalendar()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using calendarId

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.calendarId); // "iso8601"; default

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.calendarId); // "chinese"

const date3 = date2.withCalendar("hebrew");
console.log(date3.calendarId); // "hebrew"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.calendarid

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.calendarId`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDate`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.day

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/day · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **day** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

It generally starts at 1 and is continuous, but not always. If you want to loop through all the days in a month, first use `with()` with `{ day: 1 }` (which sets to the beginning of the month, even if the actual number is not 1), then repeatedly use `add()` with `{ days: 1 }`, until the month changes.

> **Note:** Usually, the day index only changes when transitioning from one calendar system into another, such as from the Julian to the Gregorian calendar. In practice, all currently built-in calendars are proleptic, meaning the calendar system is extended indefinitely into the past and future. Assuming `day` is non-continuous guards against future introductions of non-proleptic calendars.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using day

```js
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.day); // 1

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.day); // 22; it is May 22 in the Chinese calendar
```

Looping through all days in a month

```js
const month = Temporal.PlainDate.from("2021-07-14"); // An arbitrary date in the month
for (
  let day = month.with({ day: 1 });
  day.month === month.month;
  day = day.add({ days: 1 })
) {
  console.log(day.day);
}
```

Changing day

```js
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.with({ day: 15 });
console.log(newDate.toString()); // 2021-07-15
```

You can also use `add()` or `subtract()` to move a certain number of days from the current date.

```js
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.add({ days: 14 });
console.log(newDate.toString()); // 2021-07-15
```

By default, `with()` constrains the day to the range of valid values. So you can use `{ day: 1 }` to set the day to the first day of the month, even if the first day does not have the number `1`. Similarly, the following will set the day to the last day of the month:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const lastDay = date.with({ day: Number.MAX_VALUE }); // 2021-07-31
```

> **Note:** Avoid using `daysInMonth` to set the day to the last day of the month. The last day of the month is not always the same as the number of days in the month, in the rare case where a month may have a few days skipped.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.day`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.day`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.dayOfYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.dayOfWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/dayOfWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **dayOfWeek** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from 1 to `daysInWeek`, with each number mapping to its name. It is calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

All commonly supported calendars use 7-day weeks, and you could generally expect this property to return the same value for the same date across different calendars.

The set accessor of `dayOfWeek` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDate` object with the desired new `dayOfWeek` value, use the `add()` or `subtract()` method with the appropriate number of `days`.

Examples

Using dayOfWeek

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.dayOfWeek); // 4; Thursday

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.dayOfWeek); // 4
```

Changing dayOfWeek

`PlainDate` does not support changing `dayOfWeek` directly. To change the day of the week, you need to first figure out the difference in days to your desired day of the week, then use `add` or `subtract` to adjust the date accordingly. For example, to change to the Friday of this week (whether before or after):

```js
function getDayInSameWeek(date, destDayOfWeek) {
  return date.add({ days: destDayOfWeek - date.dayOfWeek });
}

console.log(
  getDayInSameWeek(Temporal.PlainDate.from("2021-07-01"), 5).toString(),
); // 2021-07-02
console.log(
  getDayInSameWeek(Temporal.PlainDate.from("2021-07-03"), 5).toString(),
); // 2021-07-02
```

To change to the next Friday:

```js
function getNextDayInWeek(date, destDayOfWeek) {
  const distance = destDayOfWeek - date.dayOfWeek;
  return date.add({
    days: distance < 0 ? date.daysInWeek + distance : distance,
  });
}

console.log(
  getNextDayInWeek(Temporal.PlainDate.from("2021-07-01"), 5).toString(),
); // 2021-07-02
console.log(
  getNextDayInWeek(Temporal.PlainDate.from("2021-07-03"), 5).toString(),
); // 2021-07-09
```

To change to the previous Friday:

```js
function getPreviousDayInWeek(date, destDayOfWeek) {
  const distance = date.dayOfWeek - destDayOfWeek;
  return date.subtract({
    days: distance < 0 ? date.daysInWeek + distance : distance,
  });
}

console.log(
  getPreviousDayInWeek(Temporal.PlainDate.from("2021-07-01"), 5).toString(),
); // 2021-06-25
console.log(
  getPreviousDayInWeek(Temporal.PlainDate.from("2021-07-03"), 5).toString(),
); // 2021-07-02
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.dayofweek

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
dayOfWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.dayOfWeek`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`

- Temporal.PlainDate.prototype.add()
- Temporal.PlainDate.prototype.subtract()
- Temporal.PlainDate.prototype.day
- Temporal.PlainDate.prototype.dayOfYear
- Temporal.PlainDate.prototype.daysInWeek
- Temporal.PlainDate.prototype.weekOfYear
- Temporal.PlainDate.prototype.yearOfWeek

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.dayOfYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/dayOfYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **dayOfYear** accessor property of Temporal.PlainDate instances returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is 1, and the last day is the daysInYear. It is calendar-dependent.

The set accessor of dayOfYear is undefined. You cannot change this property directly. To create a new Temporal.PlainDate object with the desired new dayOfYear value, use the add() or subtract() method with the appropriate number of days.

Examples

Using dayOfYear

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.dayOfYear); // 182

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.dayOfYear); // 140

const date3 = Temporal.PlainDate.from("2020-07-01");
console.log(date3.dayOfYear); // 183; 2020 is a leap year
```

Changing dayOfYear

PlainDate does not support changing dayOfYear directly. To change the day of the year, you need to first figure out the difference in days to your desired day of the year, then use add or subtract to adjust the date accordingly. For example, to change to the 100th day of this year (whether before or after):

```js
function getDayInSameYear(date, destDayOfYear) {
  return date.add({ days: destDayOfYear - date.dayOfYear });
}

console.log(
  getDayInSameYear(Temporal.PlainDate.from("2021-07-01"), 100).toString(),
); // 2021-04-10
console.log(
  getDayInSameYear(Temporal.PlainDate.from("2021-01-01"), 100).toString(),
); // 2021-04-10
console.log(
  getDayInSameYear(Temporal.PlainDate.from("2020-01-01"), 100).toString(),
); // 2020-04-09
```

By default, `with()` constrains the day to the range of valid values. So you can always use `{ month: 1, day: 1 }` to set the day to the first day of the year, even if the first day does not have the number `1`. Similarly, the following will set the day to the last day of the year, regardless of how many days are in the last month or year:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const lastDay = date.with({ month: Number.MAX_VALUE, day: Number.MAX_VALUE }); // 2021-12-31
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.dayofyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
dayOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.dayOfYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.day`
- `Temporal.PlainDate.prototype.dayOfWeek`

- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDate.prototype.daysInMonth

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/daysInMonth · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInMonth`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

Note that the days in month is not always equal to the `day` of the last day of the month, in the rare case where a month may have a few days skipped.

The set accessor of `daysInMonth` is `undefined`. You cannot change this property directly.

Examples

Using daysInMonth

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.daysInMonth); // 31

const date2 = Temporal.PlainDate.from("2021-02-01");
console.log(date2.daysInMonth); // 28; 2021 is not a leap year

const date3 = Temporal.PlainDate.from("2020-02-01");
console.log(date3.daysInMonth); // 29; 2020 is a leap year

const date4 = Temporal.PlainDate.from("2021-04-01[u-ca=chinese]");
console.log(date4.month); // 2
console.log(date4.daysInMonth); // 30; the Chinese 2nd month has 30 days
```

Changing to the second last day of the month

You can use `daysInMonth` to change to the second last day of the month:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const secondLastDay = date.with({ day: date.daysInMonth - 1 });
console.log(secondLastDay.toString()); // 2021-07-30
```

This is not totally safe, though, because `daysInMonth` is not guaranteed to have any connection with the day index. Here's a safer way to get the second last day:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const secondLastDay = date
  .with({ day: Number.MAX_SAFE_INTEGER })
  .subtract({ days: 1 });
console.log(secondLastDay.toString()); // 2021-07-30
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.daysinmonth`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`daysInMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.daysInMonth`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.day`
- `Temporal.PlainDate.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.daysInWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/daysInWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInWeek`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of days in the week of this date. It is calendar-dependent.

For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week. All commonly supported calendars use 7-day weeks.

The set accessor of `daysInWeek` is `undefined`. You cannot change this property directly.

Examples

Using daysInWeek

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.daysInWeek); // 7

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.daysInWeek); // 7
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.daysinweek

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
daysInWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.daysInWeek`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.yearOfWeek`
- `Temporal.PlainDate.prototype.weekOfYear`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.daysInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.daysInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/daysInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInYear`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

For the ISO 8601 calendar, this is 365, or 366 in a leap year. In other calendar systems, it likely differs, especially in non-solar calendars.

The set accessor of `daysInWeek` is `undefined`. You cannot change this property directly.

Examples

Using daysInYear

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.daysInYear); // 365

const date2 = Temporal.PlainDate.from("2020-07-01");
console.log(date2.daysInYear); // 366; 2020 is a leap year

const date3 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date3.daysInYear); // 354

const date4 = Temporal.PlainDate.from("2023-07-01[u-ca=chinese]");
console.log(date4.daysInYear); // 384; 2023 is a Chinese leap year
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.daysinyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`daysInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.daysInYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainDate
- Temporal.PlainDate.prototype.with()
- Temporal.PlainDate.prototype.add()
- Temporal.PlainDate.prototype.subtract()
- Temporal.PlainDate.prototype.year
- Temporal.PlainDate.prototype.dayOfYear
- Temporal.PlainDate.prototype.daysInMonth
- Temporal.PlainDate.prototype.daysInWeek

Temporal.PlainDate.prototype.era

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/era · MDN page last modified 2026-02-19

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **era** accessor property of Temporal.PlainDate instances returns a calendar-specific lowercase string representing the era of this date, or undefined if the calendar does not use eras (e.g., ISO 8601). era and eraYear together uniquely identify a year in a calendar, in the same way that year does. It is calendar-dependent. For Gregorian, it is either "ce" or "bce".

Value

All specified calendars have eras fully defined by the spec.

- The following calendars have a single era:
  - buddhist: "be"
  - coptic: "am"
  - ethioaa: "aa"
  - hebrew: "am"
  - indian: "shaka"
  - persian: "ap"
- The following calendars have two eras. One is the *epoch era*, in which eraYear starts at 1 and is the same as year. The other is the inverse era, in which eraYear also starts at 1 and is equal to 1 - year (so eraYear: 1 corresponds to year 0, eraYear: 2 to year -1, etc.):
  - gregory: epoch era "ce", inverse era "bce"
  - islamic-civil, islamic-tbla, islamic-umalqura: epoch era "ah", inverse era "bh"
  - roc: epoch era "roc", inverse era "broc"
- The ethiopic calendar has an "am" era which is the epoch era. Years before 1 belong to the "aa" era, whose eraYear is equal to year - 5500 (so eraYear: -1000 corresponds to year -6500, eraYear: 1 corresponds to year -5499, up to eraYear: 5500 as year 0).
- The japanese calendar adds an era for each new emperor, so the output year and era for a future date may not match the input year and era when your code runs on a future engine version, and we won't enumerate them here. Each era's year starts at 1. It is also the only calendar known to have

eras starting in the middle of a year, which means that the same `year` may correspond to different `(era, eraYear)` pairs depending on the month and day.

> **Warning:** As of October 2025, in the `japanese` calendar, dates prior to 1868-10-23 ISO (the start date of the year 1 Meiji) don't work as expected in browsers in two ways. First, CLDR had the wrong start date for the Meiji era, which causes calendar implementations to extend the Meiji era further to the past than it actually did. Second, the upcoming Intl era and monthCode Proposal specifies that dates prior to 1873-01-01 ISO should use Gregorian eras, but browsers have traditionally used approximations of prior Japanese eras instead. The `japanese` calendar was taken into use on January 1, 6 Meiji / 1873-01-01 ISO, so these problems only affect proleptic dates.

- Other specified calendars: `chinese`, `dangi`, `iso8601`, don't use eras and return `undefined`.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value. When setting eras, the `"ad"` and `"bc"` aliases are also accepted for the `"ce"` and `"bce"` eras of the `gregory` or `japanese` calendars.

> **Note:** This string is not intended for display to users. Use `toLocaleString()` with the appropriate options to get a localized string.

Examples

Using era

```js
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.era); // undefined

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
console.log(date2.era); // ce

const date3 = Temporal.PlainDate.from("-002021-07-01[u-ca=gregory]");
console.log(date3.era); // bce

const date4 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
console.log(date4.era); // reiwa
```

Changing era

You can only set `era` for calendars that support them. For example, the ISO 8601 calendar does not have eras. Note that you must provide `era` and `eraYear` together.

```js
const date = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
const newDate = date.with({ era: "bc", eraYear: 100 });
console.log(newDate.toString()); // -000099-07-01[u-ca=gregory]

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
const newDate2 = date2.with({ era: "meiji", eraYear: 1 });
console.log(newDate2.toString()); // 1868-07-01[u-ca=japanese]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.era

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
era	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.era`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.eraYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.eraYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/eraYear · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`eraYear`** accessor property of `Temporal.PlainDate` instances returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

Unlike `year`, `era` and `eraYear` may change in the middle of a calendar year. For example, Japan started the Reiwa era on May 1, 2019, so dates from 2019-01-01 to 2019-04-30 have `{ era: "heisei", eraYear: 31 }`, and dates from 2019-05-01 onwards have `{ era: "reiwa", eraYear: 1 }`, but the `year` is always 2019 (because the Japanese calendar uses the ISO 8601 year as the arithmetic year).

For more information about the values of `era` and `eraYear` for different calendars, see the `era` property.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using eraYear

```js
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.eraYear); // undefined

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
console.log(date2.eraYear); // 2021

const date3 = Temporal.PlainDate.from("-002021-07-01[u-ca=gregory]");
console.log(date3.eraYear); // 2022; 0000 is used for the year 1 BC

const date4 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
console.log(date4.eraYear); // 3
```

Changing eraYear

You can only set `eraYear` for calendars that support them. For example, the ISO 8601 calendar does not have eras. Note that you must provide `era` and `eraYear` together.

```js
const date = Temporal.PlainDate.from("2021-07-01[u-ca=gregory]");
const newDate = date.with({ era: "bc", eraYear: 100 });
console.log(newDate.toString()); // -000099-07-01[u-ca=gregory]

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
const newDate2 = date2.with({ era: "meiji", eraYear: 1 });
console.log(newDate2.toString()); // 1868-07-01[u-ca=japanese]
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.erayear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`eraYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.eraYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`

- Temporal.PlainDate.prototype.add()
- Temporal.PlainDate.prototype.subtract()
- Temporal.PlainDate.prototype.year
- Temporal.PlainDate.prototype.era

Temporal.PlainDate.prototype.inLeapYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/inLeapYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`inLeapYear`** accessor property of `Temporal.PlainDate` instances returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

For the ISO 8601 calendar, a leap year is a year that is evenly divisible by 4, except for years that are evenly divisible by 100, unless the year is also evenly divisible by 400. For the ISO 8601 calendar, leap years have 366 days, while common years have 365 days. For other calendar systems, the rules likely differ, and leap years may have more days added (such as a leap month).

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

Examples

Using inLeapYear

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.inLeapYear); // false
console.log(date.daysInYear); // 365
console.log(date.monthsInYear); // 12

const date2 = Temporal.PlainDate.from("2020-07-01");
console.log(date2.inLeapYear); // true
console.log(date2.daysInYear); // 366
console.log(date2.monthsInYear); // 12

const date3 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date3.inLeapYear); // false
console.log(date3.daysInYear); // 354
console.log(date3.monthsInYear); // 12

const date4 = Temporal.PlainDate.from("2023-07-01[u-ca=chinese]");
console.log(date4.inLeapYear); // true
console.log(date4.daysInYear); // 384
console.log(date4.monthsInYear); // 13
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.inleapyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`inLeapYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.inLeapYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.daysInYear`
- `Temporal.PlainDate.prototype.monthsInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.month

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/month · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`month`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. It is calendar-dependent.

Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

> **Note:** Do not use this property to identify the actual month, including its name. Use `monthCode` for that purpose. Use `month` only for identifying months within the context of a year, or to figure out their order.

The set accessor of `month` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using month

```js
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7

const date2 = Temporal.PlainDate.from("2021-05-01[u-ca=chinese]");
console.log(date2.monthCode); // "M03"
console.log(date2.month); // 3; it is March 20 in the Chinese calendar

const date3 = Temporal.PlainDate.from("2023-05-01[u-ca=chinese]");
console.log(date3.monthCode); // "M03"
console.log(date3.month); // 4, although it is also March (M03)!

const date4 = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
console.log(date4.monthCode); // "M02L"
console.log(date4.month); // 3, this month is a leap month, i.e. a duplicate February
```

Looping through all months in a year

```js
const year = Temporal.PlainDate.from("2021-07-14"); // An arbitrary date in the year
for (
  let month = year.with({ month: 1 });
  month.year === year.year;
  month = month.add({ months: 1 })
) {
  console.log(month.month);
}
```

Alternatively, this is also a safe way (unlike the day example):

```js
for (let month = 1; month <= year.monthsInYear; month++) {
  const monthDate = year.with({ month });
}
```

Changing month

```js
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.with({ month: 2 });
console.log(newDate.toString()); // 2021-02-01
```

You can also use `add()` or `subtract()` to move a certain number of months from the current date.

```js
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.add({ months: 3 });
console.log(newDate.toString()); // 2021-10-01
```

By default, `with()` constrains the day to the range of valid values. Both of the following will set the month to the last month of the year:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const lastMonth = date.with({ month: date.monthsInYear }); // 2021-12-01
const lastMonth2 = date.with({ month: Number.MAX_VALUE }); // 2021-12-01
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.month`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`month`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.month`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.day`
- `Temporal.PlainDate.prototype.monthCode`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.monthCode

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/monthCode · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`monthCode`** accessor property of `Temporal.PlainDate` instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Value

The basic format of `monthCode` is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`).

All calendars have at least 12 months, with codes from `"M01"` to `"M12"`.

All specified calendars have month codes fully defined by the spec. Most don't have month rules distinct from `iso8601`. The `coptic`, `ethioaa`, and `ethiopic` calendars have an additional `M13` month. The `chinese`

and `dangi` calendars have 12 additional leap months possible, with codes from `"M01L"` to `"M12L"`. The `hebrew` calendar has one leap month, `"M05L"` (Adar I).

> **Note:** Don't assume that `monthCode` is a user-friendly string; use `toLocaleString()` to format your date instead. Generally, don't cache the name of months in an array or object. Even though `monthCode` usually maps to the month's name within one calendar, we recommend always computing the month's name using, for example, `date.toLocaleString("en-US", { calendar: date.calendarId, month: "long" })`.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

When setting the date to a different year, the `monthCode` remains the same, but the `month` may change if the target year has a different leap month structure. If the current `monthCode` does not exist in the target year and the method is not configured to reject, then for the `chinese` and `dangi` calendars, the previous month is used instead (e.g., from `"M03L"` to `"M03"`, which is from 闰三月 to 三月). For `hebrew`, the *next* month is used instead (from `"M05L"` to `"M06"`, which is from Adar I to Adar II).

Examples

Using monthCode

```js
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7

const date2 = Temporal.PlainDate.from("2021-05-01[u-ca=chinese]");
console.log(date2.monthCode); // "M03"
console.log(date2.month); // 3; it is March 20 in the Chinese calendar

const date3 = Temporal.PlainDate.from("2023-05-01[u-ca=chinese]");
console.log(date3.monthCode); // "M03"
console.log(date3.month); // 4, although it is also March (M03)!

const date4 = Temporal.PlainDate.from("2023-04-01[u-ca=chinese]");
console.log(date4.monthCode); // "M02L"
console.log(date4.month); // 3, this month is a leap month, i.e. a duplicate February
```

Changing monthCode

```js
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.with({ month: 2 });
console.log(newDate.toString()); // 2021-02-01
```

You can also use `add()` or `subtract()` to move a certain number of months from the current date.

```js
const date = Temporal.PlainDate.from("2021-07-01");
const newDate = date.add({ months: 3 });
console.log(newDate.toString()); // 2021-10-01
```

By default, `with()` constrains the day to the range of valid values. Both of the following will set the month to the last month of the year:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const lastMonth = date.with({ month: date.monthsInYear }); // 2021-12-01
const lastMonth2 = date.with({ month: Number.MAX_VALUE }); // 2021-12-01
```

Formatting month names

Don't do this:

```js · ✗ bad example
const names = [
  "January", "February", "March", "April", "May", "June",
  "July", "August", "September", "October", "November", "December",
];

const date = Temporal.PlainDate.from("2021-07-01");
console.log(names[date.month - 1]); // July
```

Also don't do this:

```js · ✗ bad example
const names = {
  "M01": "January", "M02": "February", "M03": "March", "M04": "April",
  "M05": "May", "M06": "June", "M07": "July", "M08": "August",
  "M09": "September", "M10": "October", "M11": "November", "M12": "December",
};

const date = Temporal.PlainDate.from("2021-07-01");
console.log(names[date.monthCode]); // July
```

Instead, always do this, which is more user-friendly and less error-prone, and easily generalizes to other calendars:

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(
  date.toLocaleString("en-US", { calendar: date.calendarId, month: "long" }),
); // July
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.monthcode

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.monthCode`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.monthsInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/monthsInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`monthsInYear`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ. For example, in calendars using leap months, leap years will have one more month than common years.

The set accessor of `monthsInYear` is `undefined`. You cannot change this property directly.

Examples

Using monthsInYear

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.monthsInYear); // 12

const date2 = Temporal.PlainDate.from("2021-07-01[u-ca=chinese]");
console.log(date2.monthsInYear); // 12

const date3 = Temporal.PlainDate.from("2023-07-01[u-ca=chinese]");
console.log(date3.monthsInYear); // 13; 2023 is a Chinese leap year
```

Changing to the second last month of the year

You can use `monthsInYear` to change to the second last month of the year:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const secondLastMonth = date.with({ month: date.monthsInYear - 1 });
console.log(secondLastMonth.toString()); // 2021-11-01
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.monthsinyear`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`monthsInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.monthsInYear`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.monthCode`
- `Temporal.PlainDate.prototype.daysInMonth`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.weekOfYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/weekOfYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`weekOfYear`** accessor property of `Temporal.PlainDate` instances returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. It is calendar-dependent.

Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year. Namely, if a week crosses two years, then it belongs to the year that has the majority of its days. To get the year that the `weekOfYear` belongs to, use the `yearOfWeek` property, not the `year` property.

The set accessor of `weekOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDate` object with the desired new `weekOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `weeks`.

Examples

Using weekOfYear

```js
const date = Temporal.PlainDate.from("2021-07-01");
console.log(date.weekOfYear); // 26

// If 01-01 is a Friday/Saturday/Sunday, it belongs to the last week of the previous year
const date2 = Temporal.PlainDate.from("2021-01-01");
console.log(date2.dayOfWeek); // 5
console.log(date2.weekOfYear); // 53; 2020 has 53 weeks
console.log(date2.yearOfWeek); // 2020

// Otherwise, it belongs to the first week of the year
const date3 = Temporal.PlainDate.from("2020-01-01");
console.log(date3.dayOfWeek); // 3
console.log(date3.weekOfYear); // 1
console.log(date3.yearOfWeek); // 2020

// Similarly, if 12-31 is a Monday/Tuesday/Wednesday, it belongs to the first week of the
next year
const date4 = Temporal.PlainDate.from("2019-12-31");
console.log(date4.dayOfWeek); // 2
console.log(date4.weekOfYear); // 1
console.log(date4.yearOfWeek); // 2020
```

Changing weekOfYear

`PlainDate` does not support changing `weekOfYear` directly. To change the week, you need to first figure out the difference in weeks to your desired week, then use `add` or `subtract` to adjust the date accordingly. For example, to change to the previous week:

```js
const date = Temporal.PlainDate.from("2021-07-01");
const previousWeek = date.subtract({ weeks: 1 });
console.log(previousWeek.toString()); // 2021-06-24
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.weekofyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`weekOfYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.weekOfYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.yearOfWeek`
- `Temporal.PlainDate.prototype.dayOfWeek`
- `Temporal.PlainDate.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.year

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/year · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **year** accessor property of `Temporal.PlainDate` instances returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. This property has the same function as the `era`/`eraYear` pair as a unique identifier of a year in a calendar. It is calendar-dependent.

Value

Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. Because `year` is relative to the start of the epoch year, not the epoch date, if the epoch is in the middle of the year (only known to happen for the `japanese` calendar), that year will have the same `year` value before and after the start date of the era (for the `japanese` calendar, `year` is the same as the ISO 8601 year).

All specified calendars have arithmetic years fully defined by the spec.

- The following calendars have the same epoch year as ISO 8601: `chinese`, `dangi`, `gregory`, `japanese`, in which `year: 1` corresponds to the ISO year `1`.
- The `buddhist` calendar uses the Buddhist epoch of 543 BCE, so `year: 1` corresponds to the ISO year `-542`.
- The `coptic` calendar uses the Coptic epoch of 284 CE, so `year: 1` corresponds to the ISO year `284`.
- The `ethioaa` calendar uses the Anno Mundi epoch of 5493 BCE, so `year: 1` corresponds to the ISO year `-5492`.
- The `ethiopic` calendar uses the Ethiopic epoch of 8 CE, so `year: 1` corresponds to the ISO year `8`.
- The `hebrew` calendar uses the Anno Mundi epoch of 3761 BCE, so `year: 1` corresponds to the ISO year `-3760`.
- The `indian` calendar uses the Śaka epoch of 79 CE, so `year: 1` corresponds to the ISO year `79`.
- The following calendars use the Hijri epoch of 622 CE: `islamic-civil`, `islamic-tbla`, `islamic-umalqura`, `persian`, in which `year: 1` corresponds to the ISO year `622`.
- The `roc` calendar uses the Minguo epoch of 1912 CE, so `year: 1` corresponds to the ISO year `1912`.

> **Note:** For the `chinese` and `dangi` calendars, the CLDR data uses the Huangdi epoch of 2637 BCE by default, but Temporal defined it to use the ISO 8601 epoch for simplicity.

The set accessor of `year` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDate` object with the desired new value.

Examples

Using year

```js
const date = Temporal.PlainDate.from("2021-07-01"); // ISO 8601 calendar
console.log(date.year); // 2021

const date2 = Temporal.PlainDate.from("-002021-07-01");
console.log(date2.year); // -2021

const date3 = Temporal.PlainDate.from("2021-07-01[u-ca=japanese]");
console.log(date3.year); // 2021; although the Japanese calendar uses eras,
// there's no obvious "default era", so the year is the same as the ISO year

const date4 = Temporal.PlainDate.from("2021-07-01[u-ca=hebrew]");
console.log(date4.year); // 5781; the Hebrew calendar uses the Anno Mundi epoch, which
starts in 3761 BC
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.year

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
year	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.year`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.era`
- `Temporal.PlainDate.prototype.eraYear`

- `Temporal.PlainDate.prototype.yearOfWeek`
- `Temporal.PlainDate.prototype.month`
- `Temporal.PlainDate.prototype.day`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDate

Temporal.PlainDate.prototype.yearOfWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDate/yearOfWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`yearOfWeek`** accessor property of `Temporal.PlainDate` instances returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. It is calendar-dependent.

Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1. See `weekOfYear` for more details.

The set accessor of `yearOfWeek` is `undefined`. You cannot change this property directly.

Examples

See the examples in the `weekOfYear` page.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindate.prototype.yearofweek`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDate.yearOfWeek`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.with()`
- `Temporal.PlainDate.prototype.add()`
- `Temporal.PlainDate.prototype.subtract()`
- `Temporal.PlainDate.prototype.year`
- `Temporal.PlainDate.prototype.weekOfYear`

- Temporal.PlainDate.prototype.dayOfWeek
- Temporal.PlainDate.prototype.daysInWeek
- Temporal.PlainDate.prototype.daysInYear

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.PlainDateTime

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDateTime`** object represents a date (calendar date) and time (wall-clock time) without a time zone. It is fundamentally represented as a combination of a date (with an associated calendar system) and a time.

Description

A `PlainDateTime` is essentially the combination of a `Temporal.PlainDate` and a `Temporal.PlainTime`. Because the date and time information don't have much interaction, all general information about date properties is documented in the `PlainDate` object, and all general information about time properties is documented in the `PlainTime` object.

If the date-time represents a specific instant that should remain invariant across time zones, you should use the `Temporal.ZonedDateTime` object instead. Use `PlainDateTime` when you need to represent an event happening at a specific wall-clock time that may be a different instant in different time zones.

RFC 9557 format

`PlainDateTime` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss [u-ca=calendar_id]
```

`YYYY`

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD`

A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`T` Optional

The date-time separator, which can be `T`, `t`, or a space. Present if and only if `HH` is present.

`HH` Optional

A two-digit number from `00` to `23`. Defaults to `00`.

`mm` Optional

A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by

prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the offset and time zone identifier, in the same format as `ZonedDateTime`, but they will be ignored. Note that the offset must *not* be `Z`. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits, whether to display the calendar ID, and whether to add a critical flag for it.

Constructor

`Temporal.PlainDateTime()`

Creates a new `Temporal.PlainDateTime` object by directly supplying the underlying data.

Static methods

`Temporal.PlainDateTime.compare()`

Returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. Equivalent to first comparing their dates, then comparing their times if the dates are the same.

`Temporal.PlainDateTime.from()`

Creates a new `Temporal.PlainDateTime` object from another `Temporal.PlainDateTime` object, an object with date and time properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainDateTime.prototype` and shared by all `Temporal.PlainDateTime` instances.

`Temporal.PlainDateTime.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainDateTime.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainDateTime` instances, the initial value is the `Temporal.PlainDateTime()` constructor.

`Temporal.PlainDateTime.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

`Temporal.PlainDateTime.prototype.dayOfWeek`

Returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. Calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

`Temporal.PlainDateTime.prototype.dayOfYear`

Returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. Calendar-dependent.

`Temporal.PlainDateTime.prototype.daysInMonth`

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

`Temporal.PlainDateTime.prototype.daysInWeek`

Returns a positive integer representing the number of days in the week of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week.

`Temporal.PlainDateTime.prototype.daysInYear`

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

`Temporal.PlainDateTime.prototype.era`

Returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

`Temporal.PlainDateTime.prototype.eraYear`

Returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

`Temporal.PlainDateTime.prototype.hour`

Returns an integer from 0 to 23 representing the hour component of this time.

`Temporal.PlainDateTime.prototype.inLeapYear`

Returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.PlainDateTime.prototype.microsecond`

Returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

`Temporal.PlainDateTime.prototype.millisecond`

Returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

`Temporal.PlainDateTime.prototype.minute`

Returns an integer from 0 to 59 representing the minute component of this time.

`Temporal.PlainDateTime.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.PlainDateTime.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`.

If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainDateTime.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.PlainDateTime.prototype.nanosecond`

Returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

`Temporal.PlainDateTime.prototype.second`

Returns an integer from 0 to 59 representing the second component of this time.

`Temporal.PlainDateTime.prototype.weekOfYear`

Returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. Calendar-dependent. Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year.

`Temporal.PlainDateTime.prototype.year`

Returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.PlainDateTime.prototype.yearOfWeek`

Returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. Calendar-dependent. Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1.

`Temporal.PlainDateTime.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainDateTime"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainDateTime.prototype.add()`

Returns a new `Temporal.PlainDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDateTime.prototype.equals()`

Returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`), and `false` otherwise. They are compared both by their date and time values and their calendars, so two date-times from different calendars may be considered equal by `Temporal.PlainDateTime.compare()` but not by `equals()`.

`Temporal.PlainDateTime.prototype.round()`

Returns a new `Temporal.PlainDateTime` object representing this date-time rounded to the given unit.

`Temporal.PlainDateTime.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.PlainDateTime.from()`) to this date-time. The duration is positive if

the other date-time is before this date-time, and negative if after.

`Temporal.PlainDateTime.prototype.subtract()`

Returns a new `Temporal.PlainDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainDateTime.prototype.toJSON()`

Returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainDateTime.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this date-time.

`Temporal.PlainDateTime.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing the date part (year, month, day) of this date-time in the same calendar system.

`Temporal.PlainDateTime.prototype.toPlainTime()`

Returns a new `Temporal.PlainTime` object representing the time part (hour, minute, second, and subsecond components) of this date-time.

`Temporal.PlainDateTime.prototype.toString()`

Returns a string representing this date-time in the RFC 9557 format.

`Temporal.PlainDateTime.prototype.toZonedDateTime()`

Returns a new `Temporal.ZonedDateTime` instance representing the same date-time as this plain date-time, but in the specified time zone.

`Temporal.PlainDateTime.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

`Temporal.PlainDateTime.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainDateTime.prototype.with()`

Returns a new `Temporal.PlainDateTime` object representing this date-time with some fields replaced by new values.

`Temporal.PlainDateTime.prototype.withCalendar()`

Returns a new `Temporal.PlainDateTime` object representing this date-time interpreted in the new calendar system.

`Temporal.PlainDateTime.prototype.withPlainTime()`

Returns a new `Temporal.PlainDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`).

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-plaindatetime-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
PlainDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
PlainDateTime() constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
calendarId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
era	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
hour	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
inLeapYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
microsecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
millisecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
minute	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
month	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthsInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
nanosecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
second	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainDate	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toZonedDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
until	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
valueOf	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
weekOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
with	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
withCalendar	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
withPlainTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`year`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

1. Partial support.
2. Calendars `islamic`, `islamic-rgsa` and `islamic-umalqura` are not available. See bug 1950425 and bug 1954138.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.PlainDate`
- `Temporal.PlainTime`
- `Temporal.ZonedDateTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/PlainDateTime · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDateTime()`** constructor creates `Temporal.PlainDateTime` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainDateTime` objects using the `Temporal.PlainDateTime.from()` static method, which can handle a variety of input types.

Syntax

```js
new Temporal.PlainDateTime(year, month, day)
new Temporal.PlainDateTime(year, month, day, hour)
new Temporal.PlainDateTime(year, month, day, hour, minute)
new Temporal.PlainDateTime(year, month, day, hour, minute, second)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond, microsecond)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond, microsecond,
nanosecond)
new Temporal.PlainDateTime(year, month, day, hour, minute, second, millisecond, microsecond,
nanosecond, calendar)
```

> **Note:** `Temporal.PlainDateTime()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`year`

A number, truncated to an integer, representing the year in the ISO calendar system.

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`day`

A number, truncated to an integer, representing the day of the month in the ISO calendar system.

`hour` Optional

A number, truncated to an integer, representing the hour component.

`minute` Optional

A number, truncated to an integer, representing the minute component.

`second` Optional

A number, truncated to an integer, representing the second component.

`millisecond` Optional

A number, truncated to an integer, representing the millisecond component.

`microsecond` Optional

A number, truncated to an integer, representing the microsecond component.

`nanosecond` Optional

A number, truncated to an integer, representing the nanosecond component.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `year`, `month`, and `day` must be in the ISO 8601 calendar system.

Return value

A new `Temporal.PlainDateTime` object, representing the date-time specified by the parameters.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any date-time component is not a finite number.
- The date-time component combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainDateTime()

```js
const dt = new Temporal.PlainDateTime(2021, 7, 1);
console.log(dt.toString()); // 2021-07-01T00:00:00

const dt2 = new Temporal.PlainDateTime(2021, 7, 1, 0, 0, 0, 0, 0, 0, "hebrew");
console.log(dt2.toString()); // 2021-07-01T00:00:00[u-ca=hebrew]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal-plaindatetime-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainDateTime()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.PlainDateTime`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/compare · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDateTime.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. Equivalent to first comparing their dates, then comparing their times if the dates are the same.

Syntax

```js
Temporal.PlainDateTime.compare(dateTime1, dateTime2)
```

Parameters

`dateTime1`

A string, an object, or a `Temporal.PlainDateTime` instance representing the first date-time to compare. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`.

`dateTime2`

The second date-time to compare, converted to a `Temporal.PlainDateTime` object using the same algorithm as `dateTime1`.

Return value

Returns `-1` if `dateTime1` comes before `dateTime2`, `0` if they are the same, and `1` if `dateTime1` comes after `dateTime2`. They are compared by their underlying date and time values, ignoring their calendars.

Examples

Using Temporal.PlainDateTime.compare()

```js
const dt1 = Temporal.PlainDateTime.from("2021-08-01T01:00:00");
const dt2 = Temporal.PlainDateTime.from("2021-08-02T00:00:00");
console.log(Temporal.PlainDateTime.compare(dt1, dt2)); // -1

const dt3 = Temporal.PlainDateTime.from("2021-08-01T00:00:00");
console.log(Temporal.PlainDateTime.compare(dt1, dt3)); // 1
```

Comparing dates in different calendars

```js
const dt1 = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
const dt2 = Temporal.PlainDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "islamic-umalqura",
});
const dt3 = Temporal.PlainDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  calendar: "hebrew",
});
console.log(dt1.toString()); // "2021-08-01T00:00:00"
console.log(dt2.toString()); // "2582-12-17T00:00:00[u-ca=islamic-umalqura]"
console.log(dt3.toString()); // "-001739-04-06T00:00:00[u-ca=hebrew]"
console.log(Temporal.PlainDateTime.compare(dt1, dt2)); // -1
console.log(Temporal.PlainDateTime.compare(dt1, dt3)); // 1
```

Sorting an array of date-times

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```js
const dateTimes = [
  Temporal.PlainDateTime.from("2021-08-01"),
  Temporal.PlainDateTime.from("2021-08-02"),
  Temporal.PlainDateTime.from("2021-08-01T01:00:00"),
];

dateTimes.sort(Temporal.PlainDateTime.compare);
console.log(dateTimes.map((d) => d.toString()));
// [ "2021-08-01T00:00:00", "2021-08-01T01:00:00", "2021-08-02T00:00:00" ]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.compare

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.compare`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.equals()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/from · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainDateTime.from()`** static method creates a new `Temporal.PlainDateTime` object from another `Temporal.PlainDateTime` object, an object with date and time properties, or an RFC 9557 string.

Syntax

```js
Temporal.PlainDateTime.from(info)
Temporal.PlainDateTime.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainDateTime` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date, optionally a time, and optionally a calendar.
- An object containing properties that are recognized by either `Temporal.PlainDate.from()` (`calendar`, `era`, `eraYear`, `year`, `month`, `monthCode`, `day`) or `Temporal.PlainTime.from()` (`hour`, `minute`, `second`, `millisecond`, `microsecond`, `nanosecond`). The info should explicitly specify a year (as `year` or `era` and `eraYear`), a month (as `month` or `monthCode`), and a day; others are optional and will be set to their default values.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object, representing the date and time specified by `info` in the specified `calendar`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month` (or `monthCode`), and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainDateTime from an object

```js
// Year + month + day + hour + minute + second
const dt = Temporal.PlainDateTime.from({
  year: 2021,
  month: 7,
  day: 1,
  hour: 12,
  minute: 34,
  second: 56,
});
console.log(dt.toString()); // "2021-07-01T12:34:56"
```

Creating a PlainDateTime from a string

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
console.log(dt.toLocaleString()); // "7/1/2021, 12:34:56 PM" (assuming en-US locale)
```

For more examples, especially regarding different calendars and overflow settings, see `Temporal.PlainDate.from()` and `Temporal.PlainTime.from()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.from`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.from`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime()`
- `Temporal.PlainDateTime.prototype.with()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/add · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`add()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```js
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object representing the date-time specified by the original `PlainDateTime`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

For how calendar durations are added, see `Temporal.PlainDate.prototype.add()`.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration

```js
const start = Temporal.PlainDateTime.from("2021-01-01T12:34:56");
const end = start.add({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2022-03-26T17:41:03.008
```

For more examples, especially with how different calendars and the `overflow` option interact with calendar durations, see `Temporal.PlainDate.prototype.add()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.add`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.subtract()`

Temporal.PlainDateTime.prototype.equals()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/equals · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`equals()`** method of `Temporal.PlainDateTime` instances returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`), and `false` otherwise. They are compared both by their date and time values and their calendars, so two date-times from different calendars may be considered equal by `Temporal.PlainDateTime.compare()` but not by `equals()`.

Syntax

```
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDateTime` instance representing the other date-time to compare. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`.

Return value

`true` if this date-time is equal to `other` both in their date/time value and their calendar, `false` otherwise.

Examples

Using equals()

```
const dt1 = Temporal.PlainDateTime.from("2021-08-01");
const dt2 = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
console.log(dt1.equals(dt2)); // true

const dt3 = Temporal.PlainDateTime.from("2021-08-01[u-ca=japanese]");
console.log(dt1.equals(dt3)); // false

const dt4 = Temporal.PlainDateTime.from("2021-08-01T01:00:00");
console.log(dt1.equals(dt4)); // false
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.equals

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.equals`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.compare()`

Temporal.PlainDateTime.prototype.round()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/round · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`round()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time rounded to the given unit.

Syntax

```
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. For all values of `smallestUnit` except `"day"`, the increment must be a divisor of the maximum value of the unit; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12. For `"day"`, the increment must be 1.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"day"`, `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.PlainDateTime` object representing this date-time rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
const nearestMillisecond = dt.round("millisecond");
console.log(nearestMillisecond.toString()); // 2021-07-01T12:34:56.123

const nearestHalfHour = dt.round({
  smallestUnit: "minute",
  roundingIncrement: 30,
});
console.log(nearestHalfHour.toString()); // 2021-07-01T12:30:00

const nextDay = dt.round({ smallestUnit: "day", roundingMode: "ceil" });
console.log(nextDay.toString()); // 2021-07-02T00:00:00
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.round`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.round`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`

Temporal.PlainDateTime.prototype.since()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/since · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`since()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.PlainDateTime.from()`) to this date-time. The duration is positive if the other date-time is before this date-time, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```js
since(other)
since(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDateTime` instance representing a date-time to subtract from this date-time. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` accept all possible units. For `largestUnit`, the default value `"auto"` means `"days"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"nanoseconds"`. The current date is used as the `relativeTo` option. Note that using units larger than `"days"` may make the duration not portable to other calendars or dates.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this date-time. The duration is positive if `other` is before this date-time, and negative if after.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using since()

```js
let lastBilling = Temporal.PlainDateTime.from({
  year: Temporal.Now.plainDateISO().year,
  month: 4,
  day: 1,
});
const now = Temporal.Now.plainDateTimeISO().round("second");
if (Temporal.PlainDateTime.compare(lastBilling, now) > 0) {
  lastBilling = lastBilling.subtract({ years: 1 });
}
const duration = now.since(lastBilling);
console.log(`${duration.toLocaleString("en-US")} since last billing`);
// Expected output: "[number] days, [number] hr, [number] min, [number] sec since last
billing"

const duration2 = now.since(lastBilling, { smallestUnit: "days" });
console.log(`${duration2.toLocaleString("en-US")} since last billing`);
// Expected output: "[number] days since last billing"

const duration3 = now.since(lastBilling, {
  largestUnit: "years",
  smallestUnit: "days",
});
console.log(`${duration3.toLocaleString("en-US")} since last billing`);
// Expected output: "[number] months, [number] days since last billing"
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```js
const dt1 = Temporal.PlainDateTime.from("2022-01-01T00:00:00");
const dt2 = Temporal.PlainDateTime.from("2022-01-28T12:34:56");
const duration = dt2.since(dt1, {
  smallestUnit: "days",
  roundingIncrement: 5,
  roundingMode: "ceil",
});
console.log(duration.toString()); // "P30D"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.since

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.since`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.until()`

Temporal.PlainDateTime.prototype.subtract()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/subtract · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`subtract()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two date-times and get a duration, use `since()` or `until()` instead.

Syntax

```js
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object representing the date-time specified by the original `PlainDateTime`, minus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a duration

```js
const start = Temporal.PlainDateTime.from("2022-01-01T12:34:56");
const end = start.subtract({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2020-10-07T07:28:48.992
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.subtract`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.subtract`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.since()`
- `Temporal.PlainDateTime.prototype.until()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of `Temporal.PlainDateTime` instances returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```js
toJSON()
```

Parameters

None.

Return value

A string representing the given date-time in the RFC 9557 format, with the calendar annotation included if it is not `"iso8601"`.

Description

The method is automatically called by `JSON.stringify()` when a `Temporal.PlainDateTime` object is encountered during JSON serialization. The serialized value can then be deserialized using the `Temporal.PlainDateTime.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```js
const dt = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
const dtStr = dt.toJSON(); // '2021-08-01T00:00:00'
const dt2 = Temporal.PlainDateTime.from(dtStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainDateTime` can be serialized as JSON without extra effort, and how to parse it back.

```js
const dt = Temporal.PlainDateTime.from({ year: 2021, month: 8, day: 1 });
const jsonStr = JSON.stringify({ nextBilling: dt }); // '{"nextBilling":"2021-08-
01T00:00:00"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "nextBilling") {
    return Temporal.PlainDateTime.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.toJSON`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.toString()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.PlainDateTime` instances returns a string with a language-sensitive representation of this date-time. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If this date-time's calendar is not `"iso8601"`, the `calendar` option must be provided with the same value; otherwise, if this date-time's calendar is `"iso8601"`, the `calendar` option can be any value. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `year`, `month`, `day`, `hour`, `minute`, and `second` will default to `"numeric"`.
- Provide at least one of `dateStyle` or `timeStyle`: the date-time components will be set according to the specified style and the locale.
- Provide some date-time component options. Only the specified date-time components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given date-time according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(dateTime)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");

console.log(dt.toLocaleString()); // 8/1/2021, 12:34:56 PM (assuming en-US locale)
```

If the date's calendar doesn't match the locale's default calendar, and the date's calendar is not `iso8601`, an explicit `calendar` option must be provided with the same value.

```js
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56[u-ca=japanese]");
// The ja-JP locale uses the Gregorian calendar by default
dt.toLocaleString("ja-JP", { calendar: "japanese" }); // R3/8/1 12:34:56
```

Using toLocaleString() with options

You can customize which parts of the date are included in the output by providing the `options` parameter.

```js
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");
dt.toLocaleString("en-US", { dateStyle: "full", timeStyle: "full" }); // Sunday, August 1,
2021 at 12:34:56 PM
dt.toLocaleString("en-US", {
  year: "numeric",
  month: "long",
  hour: "numeric",
}); // August 2021 at 12 PM
dt.toLocaleString("en-US", {
  year: "numeric",
  hour: "numeric",
  minute: "numeric",
}); // 2021, 12:34 PM
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.tolocalestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.toLocaleString`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Intl.DateTimeFormat`
- `Temporal.PlainDateTime.prototype.toJSON()`
- `Temporal.PlainDateTime.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.toPlainDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/toPlainDate · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainDate()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDate` object representing the date part (year, month, day) of this date-time in the same calendar system.

Syntax

```js
toPlainDate()
```

Parameters

None.

Return value

A new `Temporal.PlainDate` object representing the date part (year, month, day) of this date-time in the same calendar system.

Examples

Using toPlainDate()

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const date = dt.toPlainDate();
console.log(date.toString()); // '2021-07-01'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.toplaindate

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toPlainDate	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.toPlainDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDate`
- `Temporal.PlainDateTime.prototype.toPlainTime()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`
- `Temporal.PlainDate.prototype.toPlainDateTime()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.toPlainTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/toPlainTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainTime()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainTime` object representing the time part (hour, minute, second, and subsecond components) of this date-time.

Syntax

```js
toPlainTime()
```

Parameters

None.

Return value

A new `Temporal.PlainTime` object representing the time part (hour, minute, second, and subsecond components) of this date-time.

Examples

Using toPlainTime()

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const time = dt.toPlainTime();
console.log(time.toString()); // '12:34:56'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.toplaintime

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.toPlainTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainTime`
- `Temporal.PlainDateTime.prototype.toPlainDate()`
- `Temporal.PlainDateTime.prototype.toZonedDateTime()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/toString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of `Temporal.PlainDateTime` instances returns a string representing this date-time in the RFC 9557 format.

Syntax

```js
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.PlainDateTime` instance, although the date value still remains the same.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

Return value

A string in the RFC 9557 format representing this date-time. The calendar annotation is included as specified.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```js
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");
console.log(dt.toString()); // '2021-08-01T12:34:56'
```

For examples with rounding times, see `Temporal.PlainTime.prototype.toString()`. For examples with displaying calendars, see `Temporal.PlainDate.prototype.toString()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.toString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.toJSON()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.toZonedDateTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/toZonedDateTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toZonedDateTime()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.ZonedDateTime` instance representing the same date-time as this plain date-time, but in the specified time zone.

Syntax

```js
toZonedDateTime(timeZone)
toZonedDateTime(timeZone, options)
```

Parameters

`timeZone`

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

`options` Optional

An object containing the following property:

`disambiguation` Optional

A string specifying what to do when this plain time corresponds to zero or more than one instants in the time zone, usually because of daylight saving time shifts. Possible values are `"compatible"`, `"earlier"`, `"later"`, and `"reject"`. Defaults to `"compatible"`. For more information about these values, see ambiguity and gaps from local time to UTC time.

Return value

A new `Temporal.ZonedDateTime` instance representing the same date-time as this plain date-time, but in the specified time zone.

Exceptions

`RangeError`

Thrown in one of the following cases:

- Any of the options is invalid.
- `timeZone` is not a valid time zone identifier.
- The wall-clock time is ambiguous in the time zone, and `options.disambiguation` is set to `"reject"`.

`TypeError`

Thrown if any of the arguments are not of the expected type.

Examples

Using toZonedDateTime()

```js
const dt = Temporal.PlainDateTime.from("2021-08-01T12:34:56");
const zdt = dt.toZonedDateTime("America/New_York");
console.log(zdt.toString()); // '2021-08-01T12:34:56-04:00[America/New_York]'

const dt2 = Temporal.PlainDateTime.from("2021-01-01T12:34:56");
const zdt2 = dt2.toZonedDateTime("America/New_York");
console.log(zdt2.toString()); // '2021-01-01T12:34:56-05:00[America/New_York]'
```

Handling ambiguous times

Below, we have two wall-clock times that we want to interpret in the `America/New_York` time zone. The first one, `dtNotExist`, never existed because of a forward daylight saving time shift, so we need to choose from the times `01:05:00-05:00` or `03:05:00-04:00`. The second one, `dtAmbiguous`, appeared twice because of a backward daylight saving time shift, so we need to choose from the times `01:05:00-04:00` or `01:05:00-05:00`. For a more detailed explanation of this situation, see ambiguity and gaps from local time to UTC time.

```js
const dtNotExist = Temporal.PlainDateTime.from("2024-03-10T02:05:00");
const dtAmbiguous = Temporal.PlainDateTime.from("2024-11-03T01:05:00");

// Default: compatible
console.log(dtNotExist.toZonedDateTime("America/New_York").toString());
// '2024-03-10T03:05:00-04:00[America/New_York]'
console.log(dtAmbiguous.toZonedDateTime("America/New_York").toString());
// '2024-11-03T01:05:00-04:00[America/New_York]'

// Use the earlier time for ambiguous times
console.log(
  dtNotExist
    .toZonedDateTime("America/New_York", { disambiguation: "earlier" })
    .toString(),
);
// '2024-03-10T01:05:00-05:00[America/New_York]'
console.log(
  dtAmbiguous
    .toZonedDateTime("America/New_York", { disambiguation: "earlier" })
    .toString(),
);
// '2024-11-03T01:05:00-04:00[America/New_York]'

// Use the later time for ambiguous times
console.log(
  dtNotExist
    .toZonedDateTime("America/New_York", { disambiguation: "later" })
    .toString(),
);
// '2024-03-10T03:05:00-04:00[America/New_York]'
console.log(
  dtAmbiguous
    .toZonedDateTime("America/New_York", { disambiguation: "later" })
    .toString(),
```

(code continues on the next page)

(code continued from the previous page)

```
);
// '2024-11-03T01:05:00-05:00[America/New_York]'

// Throw an error for ambiguous times
dtNotExist.toZonedDateTime("America/New_York", { disambiguation: "reject" });
// RangeError: instant is ambiguous
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.tozoneddatetime

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toZonedDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.toZonedDateTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.ZonedDateTime`
- `Temporal.PlainDateTime.prototype.toPlainDate()`
- `Temporal.PlainDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.until()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/until · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`until()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.PlainDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainDateTime` instance representing a date-time to subtract this date-time from. It is converted to a `Temporal.PlainDateTime` object using the same algorithm as `Temporal.PlainDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this date-time *until* `other`. The duration is positive if `other` is after this date-time, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using until()

```js
let nextBilling = Temporal.PlainDateTime.from({
  year: Temporal.Now.plainDateISO().year,
  month: 4,
  day: 1,
});
const now = Temporal.Now.plainDateTimeISO().round("second");
if (Temporal.PlainDateTime.compare(nextBilling, now) < 0) {
  nextBilling = nextBilling.add({ years: 1 });
}
const duration = now.until(nextBilling);
console.log(`${duration.toLocaleString("en-US")} until next billing`);
```

For more examples, see `since()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.until

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.until`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.Duration`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.since()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.PlainDateTime` instances throws a `TypeError`, which prevents `Temporal.PlainDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```js
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `dateTime1 > dateTime2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainDateTime` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainDateTime.prototype.toString()`, or use the `Temporal.PlainDateTime.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainDateTime

All arithmetic and comparison operations on `Temporal.PlainDateTime` instances should use the dedicated methods or convert them to primitives explicitly.

```js
const dt1 = Temporal.PlainDateTime.from("2022-01-01T00:00:00");
const dt2 = Temporal.PlainDateTime.from("2022-07-01T00:00:00");
dt1 > dt2; // TypeError: can't convert PlainDateTime to primitive type
Temporal.PlainDateTime.compare(dt1, dt2); // -1

dt2 - dt1; // TypeError: can't convert PlainDateTime to primitive type
dt2.since(dt1).toString(); // "P181D"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.valueof`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.toString()`
- `Temporal.PlainDateTime.prototype.toJSON()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`

Temporal.PlainDateTime.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/with · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`with()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's fields.

To replace the `calendarId` property, use the `withCalendar()` method instead.

Syntax

```js
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainDateTime.from()` (except `calendar`): `day`, `era` and `eraYear`, `hour`, `microsecond`, `millisecond`, `minute`, `month`, `monthCode`, `nanosecond`, `second`, `year`. Unspecified properties use the values from the original date-time. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainDateTime` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date-time.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const newDT = dt.with({ hour: 13 });
console.log(newDT.toString()); // "2021-07-01T13:34:56"
const newDT2 = dt.with({ month: 2, day: 22, millisecond: 222 });
console.log(newDT2.toString()); // "2021-02-22T13:34:56.222"
const nextDecade = dt.with({ year: dt.year + 10 });
console.log(nextDecade.toString()); // "2031-07-01T13:34:56"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.with`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.with`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.withCalendar()`
- `Temporal.PlainDateTime.prototype.withPlainTime()`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`

Temporal.PlainDateTime.prototype.withCalendar()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/withCalendar · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`withCalendar()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time interpreted in the new calendar system. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's `calendarId` property.

To replace the date-time component properties, use the `with()` method instead.

Syntax

```
withCalendar(calendar)
```

Parameters

`calendar`

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

Return value

A new `Temporal.PlainDateTime` object, representing the date-time specified by the original `PlainDateTime`, interpreted in the new calendar system.

Exceptions

`TypeError`

Thrown if `calendar` is not a string.

`RangeError`

Thrown if `calendar` is not a valid calendar identifier.

Examples

Using withCalendar()

```
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");
const newDT = dt.withCalendar("islamic-umalqura");
console.log(newDT.toLocaleString("en-US", { calendar: "islamic-umalqura" }));
// 11/21/1442 AH, 12:34:56 PM
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.withcalendar

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`withCalendar`	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

1. Partial support.
2. Calendars `islamic`, `islamic-rgsa` and `islamic-umalqura` are not available. See bug 1950425 and bug 1954138.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.withCalendar`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.withPlainTime()`
- `Temporal.PlainDateTime.from()`
- `Temporal.PlainDateTime.prototype.calendarId`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.withPlainTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/withPlainTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`withPlainTime()`** method of `Temporal.PlainDateTime` instances returns a new `Temporal.PlainDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`)

This method will replace all time properties, defaulting to `0` where properties are unspecified. If you only want to replace some of the time properties, use the `with()` method instead.

Syntax

```js
withPlainTime()
withPlainTime(plainTime)
```

Parameters

`plainTime` Optional

A string, an object, or a `Temporal.PlainTime` instance representing the new time. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. If not specified, the time part is set to `00:00:00`.

Return value

A new `Temporal.PlainDateTime` object, with the date part copied from the original date-time and the time part replaced by the new time.

Examples

Using withPlainTime()

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56");

// You can pass a string
const newDT = dt.withPlainTime("13:45:00");
console.log(newDT.toString()); // "2021-07-01T13:45:00"

// You can only specify some time properties, and the rest default to 0;
// for the with() method, they would be copied from the original date-time
const newDT2 = dt.withPlainTime({ hour: 13 });
console.log(newDT2.toString()); // "2021-07-01T13:00:00"

// You can pass nothing to set the time to midnight
const newDT3 = dt.withPlainTime();
console.log(newDT3.toString()); // "2021-07-01T00:00:00"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaindatetime.prototype.withplaintime

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
withPlainTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.withPlainTime`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.withCalendar()`
- `Temporal.PlainDateTime.prototype.toPlainTime()`
- `Temporal.PlainDateTime.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.calendarId

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/calendarId · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`calendarId`** accessor property of `Temporal.PlainDateTime` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. Use the `withCalendar()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

Examples

Using calendarId

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T08:00:00");
console.log(dt.calendarId); // "iso8601"; default

const dt2 = Temporal.PlainDateTime.from("2021-07-01T08:00:00[u-ca=chinese]");
console.log(dt2.calendarId); // "chinese"

const dt3 = dt2.withCalendar("hebrew");
console.log(dt3.calendarId); // "hebrew"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.calendarid`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.calendarId`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.day

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/day · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **day** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.day`.

Examples

Using day

```js
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.day); // 1
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.day`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.day`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.daysInMonth`

- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.dayOfYear`
- `Temporal.PlainDate.prototype.day`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.dayOfWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/dayOfWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`dayOfWeek`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. It is calendar-dependent.

The set accessor of `dayOfWeek` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDateTime` object with the desired new `dayOfWeek` value, use the `add()` or `subtract()` method with the appropriate number of `days`.

For general information and more examples, see `Temporal.PlainDate.prototype.dayOfWeek`.

Examples

Using dayOfWeek

```js
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.dayOfWeek); // 4; Thursday
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.dayofweek`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`dayOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.dayOfWeek`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`

- Temporal.PlainDateTime.prototype.subtract()
- Temporal.PlainDateTime.prototype.day
- Temporal.PlainDateTime.prototype.dayOfYear
- Temporal.PlainDateTime.prototype.daysInWeek
- Temporal.PlainDateTime.prototype.weekOfYear
- Temporal.PlainDateTime.prototype.yearOfWeek
- Temporal.PlainDate.prototype.dayOfWeek

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.dayOfYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/dayOfYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **dayOfYear** accessor property of Temporal.PlainDateTime instances returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is 1, and the last day is the daysInYear. It is calendar-dependent.

The set accessor of dayOfYear is undefined. You cannot change this property directly. To create a new Temporal.PlainDateTime object with the desired new dayOfYear value, use the add() or subtract() method with the appropriate number of days.

For general information and more examples, see Temporal.PlainDate.prototype.dayOfYear.

Examples

Using dayOfYear

```js
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.dayOfYear); // 182
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.dayofyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
dayOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.PlainDateTime.dayOfYear. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.day`
- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.dayOfYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.daysInMonth

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/daysInMonth · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInMonth`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

The set accessor of `daysInMonth` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInMonth`.

Examples

Using daysInMonth

```js
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.daysInMonth); // 31
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.daysinmonth`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`daysInMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.daysInMonth`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainDateTime
- Temporal.PlainDateTime.prototype.with()
- Temporal.PlainDateTime.prototype.add()
- Temporal.PlainDateTime.prototype.subtract()
- Temporal.PlainDateTime.prototype.year
- Temporal.PlainDateTime.prototype.month
- Temporal.PlainDateTime.prototype.day
- Temporal.PlainDateTime.prototype.daysInWeek
- Temporal.PlainDateTime.prototype.daysInYear
- Temporal.PlainDate.prototype.daysInMonth

Temporal.PlainDateTime.prototype.daysInWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/daysInWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **daysInWeek** accessor property of Temporal.PlainDateTime instances returns a positive integer representing the number of days in the week of this date. It is calendar-dependent.

The set accessor of daysInWeek is undefined. You cannot change this property directly.

For general information and more examples, see Temporal.PlainDate.prototype.daysInWeek.

Examples

Using daysInWeek

```js
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.daysInWeek); // 7
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.daysinweek

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
daysInWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.daysInWeek`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.yearOfWeek`
- `Temporal.PlainDateTime.prototype.weekOfYear`
- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInWeek`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.daysInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/daysInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInYear`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

The set accessor of `daysInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInYear`.

Examples

Using daysInYear

js

```
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.daysInYear); // 365
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.daysinyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
daysInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.daysInYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.dayOfYear`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDateTime.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

Temporal.PlainDateTime.prototype.era

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/era · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **era** accessor property of `Temporal.PlainDateTime` instances returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.era`.

Examples

Using era

js

```
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.era); // undefined

const dt2 = Temporal.PlainDateTime.from("2021-07-01[u-ca=gregory]");
console.log(dt2.era); // ce
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.era`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`era`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.era`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.eraYear`
- `Temporal.PlainDate.prototype.era`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.eraYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/eraYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`eraYear`** accessor property of `Temporal.PlainDateTime` instances returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.eraYear`.

Examples

Using eraYear

```js
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.eraYear); // undefined

const dt2 = Temporal.PlainDateTime.from("2021-07-01[u-ca=gregory]");
console.log(dt2.eraYear); // 2021

const dt3 = Temporal.PlainDateTime.from("-002021-07-01[u-ca=gregory]");
console.log(dt3.eraYear); // 2022; 0000 is used for the year 1 BC

const dt4 = Temporal.PlainDateTime.from("2021-07-01[u-ca=japanese]");
console.log(dt4.eraYear); // 3
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.erayear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.eraYear`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.era`
- `Temporal.PlainDate.prototype.eraYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.hour

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/hour · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **hour** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 23 representing the hour component of this time.

The set accessor of `hour` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.hour`.

Examples

Using hour

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.hour); // 12
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.hour

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
hour	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.hour`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.hour`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.inLeapYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/inLeapYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`inLeapYear`** accessor property of `Temporal.PlainDateTime` instances returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.inLeapYear`.

Examples

Using inLeapYear

```js
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.inLeapYear); // false
console.log(dt.daysInYear); // 365
console.log(dt.monthsInYear); // 12
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.inleapyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`inLeapYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.inLeapYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.daysInYear`

- `Temporal.PlainDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.inLeapYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.microsecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/microsecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`microsecond`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

The set accessor of `microsecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.microsecond`.

Examples

Using microsecond

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.microsecond); // 456
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.microsecond`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`microsecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.microsecond`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.second`

- `Temporal.PlainDateTime.prototype.millisecond`
- `Temporal.PlainDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.microsecond`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.millisecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/millisecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`millisecond`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

The set accessor of `millisecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.millisecond`.

Examples

Using millisecond

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.millisecond); // 123
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.millisecond`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`millisecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.millisecond`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`

- Temporal.PlainDateTime.prototype.second
- Temporal.PlainDateTime.prototype.microsecond
- Temporal.PlainDateTime.prototype.nanosecond
- Temporal.PlainTime.prototype.millisecond

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.minute

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/minute · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`minute`** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 59 representing the minute component of this time.

The set accessor of `minute` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.minute`.

Examples

Using minute

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.minute); // 34
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.minute

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`minute`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.minute`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainDateTime
- Temporal.PlainDateTime.prototype.with()
- Temporal.PlainDateTime.prototype.add()

- Temporal.PlainDateTime.prototype.subtract()
- Temporal.PlainDateTime.prototype.minute

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.month

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/month · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **month** accessor property of Temporal.PlainDateTime instances returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is 1, and the last month is the monthsInYear. It is calendar-dependent.

The set accessor of month is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.PlainDateTime object with the desired new value.

For general information and more examples, see Temporal.PlainDate.prototype.month.

Examples

Using month

js

```
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.monthCode); // "M07"
console.log(dt.month); // 7
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.month

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
month	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.PlainDateTime.month*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Temporal.PlainDateTime
- Temporal.PlainDateTime.prototype.with()
- Temporal.PlainDateTime.prototype.add()
- Temporal.PlainDateTime.prototype.subtract()

- Temporal.PlainDateTime.prototype.year
- Temporal.PlainDateTime.prototype.day
- Temporal.PlainDateTime.prototype.monthCode
- Temporal.PlainDateTime.prototype.daysInMonth
- Temporal.PlainDateTime.prototype.monthsInYear
- Temporal.PlainDate.prototype.month

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.monthCode

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/monthCode · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **monthCode** accessor property of Temporal.PlainDateTime instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Usually it is M plus a two-digit month number. For leap months, it is the previous month's code followed by L (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code M05L but Adar II has code M06). If the leap month is the first month of the year, the code is M00L.

The set accessor of monthCode is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.PlainDateTime object with the desired new value.

For general information and more examples, see Temporal.PlainDate.prototype.monthCode.

Examples

Using monthCode

```js
const date = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.monthcode

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.monthCode`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.monthCode`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.monthsInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/monthsInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`monthsInYear`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

The set accessor of `monthsInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.monthsInYear`.

Examples

Using monthsInYear

```js
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.monthsInYear); // 12
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.monthsinyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`monthsInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.monthsInYear`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.monthCode`
- `Temporal.PlainDateTime.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.monthsInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.nanosecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/nanosecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **nanosecond** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

The set accessor of `nanosecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.nanosecond`.

Examples

Using nanosecond

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.nanosecond); // 789
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.nanosecond

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
nanosecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.nanosecond`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.second`
- `Temporal.PlainDateTime.prototype.millisecond`
- `Temporal.PlainDateTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.second

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/second · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **second** accessor property of `Temporal.PlainDateTime` instances returns an integer from 0 to 59 representing the second component of this time.

The set accessor of `second` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.second`.

Examples

Using second

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:34:56.123456789");
console.log(dt.second); // 56
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.second`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`second`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.second`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.millisecond`
- `Temporal.PlainDateTime.prototype.microsecond`
- `Temporal.PlainDateTime.prototype.nanosecond`
- `Temporal.PlainTime.prototype.second`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.weekOfYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/weekOfYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`weekOfYear`** accessor property of `Temporal.PlainDateTime` instances returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. It is calendar-dependent.

The set accessor of `weekOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.PlainDateTime` object with the desired new `weekOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `weeks`.

For general information and more examples, see `Temporal.PlainDate.prototype.weekOfYear`.

Examples

Using weekOfYear

```js
const dt = Temporal.PlainDateTime.from("2021-07-01");
console.log(dt.weekOfYear); // 26
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.weekofyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
weekOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.weekOfYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.yearOfWeek`
- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.daysInWeek`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.weekOfYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.year

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/year · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **year** accessor property of `Temporal.PlainDateTime` instances returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. It is calendar-dependent.

The set accessor of `year` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.year`.

Examples

Using year

```js
const dt = Temporal.PlainDateTime.from("2021-07-01"); // ISO 8601 calendar
console.log(dt.year); // 2021
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.year

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
year	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.year`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.era`
- `Temporal.PlainDateTime.prototype.eraYear`
- `Temporal.PlainDateTime.prototype.yearOfWeek`
- `Temporal.PlainDateTime.prototype.month`
- `Temporal.PlainDateTime.prototype.day`
- `Temporal.PlainDate.prototype.year`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainDateTime

Temporal.PlainDateTime.prototype.yearOfWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainDateTime/yearOfWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`yearOfWeek`** accessor property of `Temporal.PlainDateTime` instances returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. It is calendar-dependent.

The set accessor of `yearOfWeek` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.yearOfWeek`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaindatetime.prototype.yearofweek`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainDateTime.yearOfWeek`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainDateTime`
- `Temporal.PlainDateTime.prototype.with()`
- `Temporal.PlainDateTime.prototype.add()`
- `Temporal.PlainDateTime.prototype.subtract()`
- `Temporal.PlainDateTime.prototype.year`
- `Temporal.PlainDateTime.prototype.weekOfYear`
- `Temporal.PlainDateTime.prototype.dayOfWeek`
- `Temporal.PlainDateTime.prototype.daysInWeek`
- `Temporal.PlainDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.yearOfWeek`

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.PlainMonthDay

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainMonthDay`** object represents the month and day of a calendar date, without a year or time zone; for example, an event on a calendar that recurs every year and happens during the whole day. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The year is used to disambiguate the month-day in non-ISO calendar systems.

Description

A `PlainMonthDay` is essentially the month-day part of a `Temporal.PlainDate` object, without the year. Because the meaning of a month-day can change from year to year (for example, whether it exists, or what the month-day of the next day is), this object doesn't provide much functionality on its own, such as comparison, addition, or subtraction. It doesn't even have a `month` property, because the month index is not meaningful without a year (for example, two months from two years with the same index can have different names in the case of leap months).

RFC 9557 format

`PlainMonthDay` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD [u-ca=calendar_id]
```

YYYY Optional

Either a four-digit number, or a six-digit number with a `+` or `-` sign. It is required for non-ISO calendars, and optional otherwise. If omitted, you can either replace `YYYY-` with `--` (so the string looks like `--MM-DD` or `--MMDD`), or omit the `YYYY-` part entirely (so the string looks like `MM-DD` or `MMDD`). Note that the reference year actually stored may be different from the one you provide, but the represented month-day is the same. See `Temporal.PlainMonthDay.from()` for more information.

MM

A two-digit number from `01` to `12`.

DD

A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the time, offset, and time zone identifier, in the same format as `PlainDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored,

and they must not have the critical flag.

When serializing, you can configure whether to display the calendar ID, and whether to add a critical flag for it.

Constructor

`Temporal.PlainMonthDay()`

Creates a new `Temporal.PlainMonthDay` object by directly supplying the underlying data.

Static methods

`Temporal.PlainMonthDay.from()`

Creates a new `Temporal.PlainMonthDay` object from another `Temporal.PlainMonthDay` object, an object with month and day properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainMonthDay.prototype` and shared by all `Temporal.PlainMonthDay` instances.

`Temporal.PlainMonthDay.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainMonthDay.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainMonthDay` instances, the initial value is the `Temporal.PlainMonthDay()` constructor.

`Temporal.PlainMonthDay.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

`Temporal.PlainMonthDay.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainMonthDay.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainMonthDay"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainMonthDay.prototype.equals()`

Returns `true` if this month-day is equivalent in value to another month-day (in a form convertible by `Temporal.PlainMonthDay.from()`), and `false` otherwise. They are compared both by their date values and their calendars.

`Temporal.PlainMonthDay.prototype.toJSON()`

Returns a string representing this month-day in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainMonthDay.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this month-day.

`Temporal.PlainMonthDay.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing this month-day and a supplied year in the same calendar system.

`Temporal.PlainMonthDay.prototype.toString()`

Returns a string representing this month-day in the RFC 9557 format.

`Temporal.PlainMonthDay.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainMonthDay` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainMonthDay.prototype.with()`

Returns a new `Temporal.PlainMonthDay` object representing this month-day with some fields replaced by new values.

Examples

Getting the next occurrence of a festival

```js
// Chinese New Years are on 1/1 in the Chinese calendar
const chineseNewYear = Temporal.PlainMonthDay.from({
  monthCode: "M01",
  day: 1,
  calendar: "chinese",
});
const currentYear = Temporal.Now.plainDateISO().withCalendar("chinese").year;
let nextCNY = chineseNewYear.toPlainDate({ year: currentYear });
if (Temporal.PlainDate.compare(nextCNY, Temporal.Now.plainDateISO()) <= 0) {
  nextCNY = nextCNY.add({ years: 1 });
}
console.log(
  `The next Chinese New Year is on ${nextCNY.withCalendar("iso8601").toLocaleString()}`,
);
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal-plainmonthday-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainMonthDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainMonthDay()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`day`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthCode`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`
- `Temporal.PlainDate`
- `Temporal.PlainYearMonth`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/PlainMonthDay · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainMonthDay()`** constructor creates `Temporal.PlainMonthDay` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainMonthDay` objects using the `Temporal.PlainMonthDay.from()` static method, which can handle a variety of input types.

Syntax

```js
new Temporal.PlainMonthDay(month, day)
new Temporal.PlainMonthDay(month, day, calendar)
new Temporal.PlainMonthDay(month, day, calendar, referenceYear)
```

> **Note:** `Temporal.PlainMonthDay()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

> **Warning:** Avoid using the `calendar` and `referenceYear` parameters, because `equals()` will consider the reference year for equality, causing two equivalent month-days to be considered different if they have different reference years. To create a `Temporal.PlainMonthDay` object with a non-ISO calendar, use the `Temporal.PlainMonthDay.from()` static method.

Parameters

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`day`

A number, truncated to an integer, representing the day of the month in the ISO calendar system.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `referenceYear`, `month`, and `day` must be in the ISO 8601 calendar system.

`referenceYear` Optional

A number, truncated to an integer, representing the year in the ISO calendar system. Defaults to `1972`. The same ISO month-day can represent different dates in different years with non-ISO calendars. For example, the days 2021-07-01 and 1972-07-01 may fall on different month-days in a non-Gregorian calendar, and just specifying "07-01" is insufficient to unambiguously determine a month-day in the target calendar. Therefore, you virtually always want to specify a `referenceYear` when using a non-ISO calendar.

Return value

A new `Temporal.PlainMonthDay` object, representing the month-day of the date specified by `referenceYear`, `month`, `day` (in the ISO calendar), interpreted in the calendar system specified by `calendar`.

Exceptions

`TypeError`

Thrown if `calendar` is not a string or `undefined`.

`RangeError`

Thrown in one of the following cases:

- `referenceYear`, `month`, or `day` is not a finite number.
- The `referenceYear`, `month`, and `day` combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainMonthDay()

```js
const md = new Temporal.PlainMonthDay(7, 1);
console.log(md.toString()); // 07-01

const md2 = new Temporal.PlainMonthDay(7, 1, "chinese");
console.log(md2.toString()); // 1972-07-01[u-ca=chinese]

const md3 = new Temporal.PlainMonthDay(7, 1, "chinese", 2021);
console.log(md3.toString()); // 2021-07-01[u-ca=chinese]
```

Improper usage

You should avoid using the `calendar` and `referenceYear` parameters, unless you know that the `referenceYear` is the canonical reference year that would be selected by `Temporal.PlainMonthDay.from()` for the same month-day.

```js
const md = new Temporal.PlainMonthDay(7, 1, "chinese", 2021);
const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=chinese]");
console.log(md.equals(md2)); // false
console.log(md.toString()); // 2021-07-01[u-ca=chinese]
console.log(md2.toString()); // 1972-07-02[u-ca=chinese]
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-plainmonthday-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainMonthDay()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.PlainMonthDay`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/from · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainMonthDay.from()`** static method creates a new `Temporal.PlainMonthDay` object from another `Temporal.PlainMonthDay` object, an object with month and day properties, or an RFC 9557 string.

Syntax

```js
Temporal.PlainMonthDay.from(info)
Temporal.PlainMonthDay.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainMonthDay` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date and optionally a calendar. If the calendar is not `iso8601`, a year is required.
- An object containing the following properties (in the order they are retrieved and validated):

  `calendar` Optional

  A string that corresponds to the `calendarId` property. Defaults to `"iso8601"`. All other properties are interpreted in this calendar system (unlike the `Temporal.PlainMonthDay()` constructor, which interprets the values in the ISO calendar system). See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

  `day`

  An integer that corresponds to the `day` property. Must be positive regardless of the `overflow` option.

  `era` and `eraYear`

  A string and an integer that can be used instead of `year`. See `era` and `eraYear` of `PlainDate`. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. If `month` is specified, at least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

  `month`

  A positive integer that can be used instead of `monthCode`. See `month` of `PlainDate`. Must be positive regardless of the `overflow` option. If `month` is provided, and the calendar is not `iso8601`, then `year` (or `eraYear` together with `era` as a substitution) must be provided too, because the same `month` may map to multiple possible `monthCode` values in different years. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

`monthCode`

Corresponds to the `monthCode` property. At least one of `month` or `monthCode` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

`year`

An integer used to disambiguate `month` if provided, because for some calendars, the same `month` can mean different `monthCode` in different years. See `year` of `PlainDate`. If a year is provided, then the `overflow` option validates the month-day in the given year, not just any year. If `month` is specified, at least one of `eraYear` (together with `era`) or `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainMonthDay` object, representing the month and day specified by `info` in the specified `calendar`.

Each `PlainMonthDay` stores a whole ISO 8601 date internally, which has the same month-day in the target calendar as what's exposed. The reference year is visible when stringifying with `toString()`, which outputs an ISO date. The reference year is chosen arbitrarily but consistently (that is, every `(monthCode, day)` pair always maps to the same ISO reference year). It does not use the year provided in the input. Instead, the reference year is chosen by finding the latest date before December 31, 1972 that has the same month-day in the target calendar, or the earliest date after December 31, 1972 if no such date exists.

For example, for Gregorian-derived calendars, the reference year is 1972. For the Hebrew calendar, the reference year is 1972 in the Gregorian calendar, but if the month is Adar I (`M05L`), which is a leap month, the reference year is 1970 (5730 in Hebrew calendar) instead, because the next leap year is 1973 (5733 in Hebrew calendar), which is after 1972.

This reference year canonicalization ensures that `equals()` can directly compare the underlying ISO dates without extra computation.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month`, and a `day`, or a `monthCode` and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainMonthDay from an object

```js
// Month code + day
const md = Temporal.PlainMonthDay.from({ monthCode: "M05", day: 2 });
console.log(md.toString()); // 05-02

// Month + day (only for ISO calendar)
const md2 = Temporal.PlainMonthDay.from({ month: 7, day: 1 });
console.log(md2.toString()); // 07-01

// Year + month + day
const md3 = Temporal.PlainMonthDay.from({ year: 2021, month: 7, day: 1 });
console.log(md3.toString()); // 07-01

// Year + month + day in a different calendar (where year is required)
const md4 = Temporal.PlainMonthDay.from({
  year: 2021,
  month: 7,
  day: 1,
  calendar: "hebrew",
});
console.log(md4.toString()); // 1972-03-16[u-ca=hebrew]

// Month code + day in a different calendar
const md5 = Temporal.PlainMonthDay.from({
  monthCode: "M05L",
  day: 1,
  calendar: "hebrew",
});
console.log(md5.toString()); // 1970-02-07[u-ca=hebrew]
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range. A month-day without an explicit reference year is valid as long as there exists one year in which it is valid, even if it doesn't appear every year. If year, month, and day are all given, then the rules for mapping to a valid month-day could be complex and specific to each calendar, but here's the usual behavior:

- If the `year`/`month` combination is invalid, the `month` is clamped to obtain a valid `monthCode` in the year.
- If the `year`/`monthCode` combination is invalid, a different year is chosen to keep the `monthCode` as-is.
- The `day` is clamped in the given year-month to obtain a valid month-day.

This is slightly different from usual date clamping, which favors the year over the month code.

```js
// Month always out of range
const md1 = Temporal.PlainMonthDay.from({ month: 13, day: 1 });
console.log(md1.toString()); // 12-01

// Month out of range for the specific year: 5732 is not a Hebrew leap year,
// so month is clamped to 12 to resolve to a valid monthCode
const md2 = Temporal.PlainMonthDay.from({
  year: 5732,
  month: 13,
  day: 1,
  calendar: "hebrew",
});
console.log(md2.toLocaleString("en-US", { calendar: "hebrew" })); // 1 Elul
const underlyingDate = Temporal.PlainDate.from(md2.toString());
console.log(underlyingDate.year, underlyingDate.month); // 5732 12

// Month code exists but not for the specific year: 5731 is not a Hebrew leap year,
// so a different year is chosen to keep the monthCode as M05L
const md3 = Temporal.PlainMonthDay.from({
  year: 5731,
  monthCode: "M05L",
  day: 1,
  calendar: "hebrew",
});
console.log(md3.toLocaleString("en-US", { calendar: "hebrew" })); // 1 Adar I
const underlyingDate2 = Temporal.PlainDate.from(md3.toString());
console.log(underlyingDate2.year, underlyingDate2.monthCode); // 5730 M05L

// Day always out of range
const md4 = Temporal.PlainMonthDay.from({ month: 2, day: 30 });
console.log(md4.toString()); // 02-29

// Day out of range for the specific year-month
const md5 = Temporal.PlainMonthDay.from({ year: 2021, month: 2, day: 29 });
console.log(md5.toString()); // 02-28
```

You can change this behavior to throw an error instead:

```js
Temporal.PlainMonthDay.from(
  { year: 2021, month: 13, day: 1 },
  { overflow: "reject" },
);
// RangeError: date value "month" not in 1..12: 13
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.from`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay()`
- `Temporal.PlainMonthDay.prototype.with()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.prototype.equals()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/equals · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`equals()`** method of `Temporal.PlainMonthDay` instances returns `true` if this month-day is equivalent in value to another month-day (in a form convertible by `Temporal.PlainMonthDay.from()`), and `false` otherwise. They are compared both by their underlying ISO date values and their calendars.

> **Note:** `PlainMonthDay` objects keep track of a reference ISO year, which is also used in the comparison. This year is automatically set when using the `Temporal.PlainMonthDay.from()` method, but can be set manually using the `Temporal.PlainMonthDay()` constructor, causing two equivalent month-days to be considered different if they have different reference years. For this reason, you should avoid using the constructor directly and prefer the `from()` method.

Syntax

```
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainMonthDay` instance representing the other month-day to compare. It is converted to a `Temporal.PlainMonthDay` object using the same algorithm as `Temporal.PlainMonthDay.from()`.

Return value

`true` if this month-day is equal to `other` both in their date value and their calendar, `false` otherwise.

Examples

Using equals()

```js
const md1 = Temporal.PlainMonthDay.from("2021-08-01");
const md2 = Temporal.PlainMonthDay.from({ year: 2020, month: 8, day: 1 }); // Year doesn't matter
console.log(md1.equals(md2)); // true

const md3 = Temporal.PlainMonthDay.from("2021-08-01[u-ca=japanese]");
console.log(md1.equals(md3)); // false

const md4 = Temporal.PlainMonthDay.from("2021-08-02");
console.log(md1.equals(md4)); // false
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.prototype.equals

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.equals`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainMonthDay`

Temporal.PlainMonthDay.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of `Temporal.PlainMonthDay` instances returns a string representing this month-day in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```js
toJSON()
```

Parameters

None.

Return value

A string representing the given month-day in the RFC 9557 format, with the year and calendar annotation included if it is not `"iso8601"`.

Description

The method is automatically called by `JSON.stringify()` when a `Temporal.PlainMonthDay` object is encountered during JSON serialization. The serialized value can then be deserialized using the `Temporal.PlainMonthDay.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```js
const md = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
const mdStr = md.toJSON(); // '08-01'
const md2 = Temporal.PlainMonthDay.from(mdStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainMonthDay` can be serialized as JSON without extra effort, and how to parse it back.

```js
const md = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
const jsonStr = JSON.stringify({ birthday: md }); // '{"birthday":"08-01"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "birthday") {
    return Temporal.PlainMonthDay.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.toJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.from()`
- `Temporal.PlainMonthDay.prototype.toString()`
- `Temporal.PlainMonthDay.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.PlainMonthDay` instances returns a string with a language-sensitive representation of this month-day. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. The `calendar` option must be provided with the same value as this month-day's calendar. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `month` and `day` will default to `"numeric"`.
- Provide `dateStyle` only: it expands to `month` and `day` formats.
- Provide some date-time component options, where at least one of them is `month` or `day`. Only the specified date components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given month-day according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(monthDay)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
// Note that just specifying "08-01" defaults to the ISO 8601 calendar,
// which throws an error if the locale's default calendar is not ISO 8601.
const md = Temporal.PlainMonthDay.from("2021-08-01[u-ca=gregory]");

console.log(md.toLocaleString()); // 8/1 (assuming en-US locale and Gregorian calendar)
```

If the month-day's calendar doesn't match the locale's default calendar, even when its calendar is `iso8601`, an explicit `calendar` option must be provided with the same value.

```js
const md = Temporal.PlainMonthDay.from("08-01");
md.toLocaleString("en-US", { calendar: "iso8601" }); // 08-01
```

Using toLocaleString() with options

You can customize which parts of the month-day are included in the output by providing the `options` parameter.

```js
const md = Temporal.PlainMonthDay.from("2021-08-01[u-ca=gregory]");
md.toLocaleString("en-US", { dateStyle: "full" }); // August 1
md.toLocaleString("en-US", { month: "long" }); // August
md.toLocaleString("en-US", { day: "numeric" }); // 1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.prototype.tolocalestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainMonthDay`
- `Intl.DateTimeFormat`
- `Temporal.PlainMonthDay.prototype.toJSON()`
- `Temporal.PlainMonthDay.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.prototype.toPlainDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/toPlainDate · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainDate()`** method of `Temporal.PlainMonthDay` instances returns a new `Temporal.PlainDate` object representing this month-day and a supplied year in the same calendar system.

Syntax

```js
toPlainDate(yearInfo)
```

Parameters

`yearInfo`

An object representing the year component of the resulting `PlainDate`, containing the following properties (in the order they are retrieved and validated):

`era` and `eraYear`

A string and an integer that correspond to the `era` and `eraYear` properties. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. If they are not provided, then `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

`year`

Corresponds to the `year` property.

Return value

A new `Temporal.PlainDate` object representing the date specified by this month-day and the year in `yearInfo`, interpreted in the calendar system of this month-day.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `yearInfo` is not an object.

Examples

Using toPlainDate()

```js
const md = Temporal.PlainMonthDay.from("07-01");
const date = md.toPlainDate({ year: 2021 });
console.log(date.toString()); // 2021-07-01

const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=japanese]");
const date2 = md2.toPlainDate({ era: "reiwa", eraYear: 1 });
console.log(date2.toString()); // 2019-07-01[u-ca=japanese]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.prototype.toplaindate

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.toPlainDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.toPlainMonthDay()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/toString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of `Temporal.PlainMonthDay` instances returns a string representing this month-day in the RFC 9557 format.

Syntax

```js
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`. The reference year is included if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation. The reference year is always included too.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.PlainMonthDay` instance, although the month-day value still remains the same.

The reference year is included if the calendar is not `"iso8601"`.

`"critical"`
Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself. The reference year is always included too.

Return value

A string in the RFC 9557 format representing this month-day. The calendar annotation is included as specified. The reference year is included if a calendar annotation is included or if the calendar is not `"iso8601"`.

Exceptions

`RangeError`
Thrown if any of the options is invalid.

`TypeError`
Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```js
const md = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
console.log(md.toString()); // '08-01'

const md2 = Temporal.PlainMonthDay.from({
  monthCode: "M08",
  day: 1,
  calendar: "chinese",
});
console.log(md2.toString()); // '1972-09-08[u-ca=chinese]'
```

Using options

```js
const isoMD = Temporal.PlainMonthDay.from({ month: 8, day: 1 });
const md = Temporal.PlainMonthDay.from({
  monthCode: "M08",
  day: 1,
  calendar: "chinese",
});
console.log(isoMD.toString({ calendarName: "auto" })); // '08-01'
console.log(md.toString({ calendarName: "auto" })); // '1972-09-08[u-ca=chinese]'
console.log(isoMD.toString({ calendarName: "always" })); // '1972-08-01[u-ca=iso8601]'
console.log(md.toString({ calendarName: "always" })); // '1972-09-08[u-ca=chinese]'
console.log(isoMD.toString({ calendarName: "never" })); // '08-01'
console.log(md.toString({ calendarName: "never" })); // '1972-09-08'
console.log(isoMD.toString({ calendarName: "critical" })); // '1972-08-01[!u-ca=iso8601]'
console.log(md.toString({ calendarName: "critical" })); // '1972-09-08[!u-ca=chinese]'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.toString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.from()`
- `Temporal.PlainMonthDay.prototype.toJSON()`
- `Temporal.PlainMonthDay.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.PlainMonthDay` instances throws a `TypeError`, which prevents `Temporal.PlainMonthDay` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `monthDay1 > monthDay2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainMonthDay` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainMonthDay.prototype.toString()`.

Examples

Arithmetic and comparison operations on Temporal.PlainMonthDay

All arithmetic and comparison operations on `Temporal.PlainMonthDay` instances should use the dedicated methods or convert them to primitives explicitly.

```js
const md1 = Temporal.PlainMonthDay.from("01-01");
const md2 = Temporal.PlainMonthDay.from("07-01");
md1 > md2; // TypeError: can't convert PlainMonthDay to primitive type
Temporal.PlainDate.compare(
  md1.toPlainDate({ year: 2021 }),
  md2.toPlainDate({ year: 2021 }),
); // -1

md2 - md1; // TypeError: can't convert PlainMonthDay to primitive type
md2
  .toPlainDate({ year: 2021 })
  .since(md1.toPlainDate({ year: 2021 }))
  .toString(); // "P181D"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.prototype.valueof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Temporal.PlainMonthDay
- Temporal.PlainMonthDay.prototype.toString()
- Temporal.PlainMonthDay.prototype.toJSON()
- Temporal.PlainMonthDay.prototype.toLocaleString()

Temporal.PlainMonthDay.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/with · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`with()`** method of Temporal.PlainMonthDay instances returns a new `Temporal.PlainMonthDay` object representing this month-day with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the month-day's fields.

There's no obvious way to create a new `Temporal.PlainMonthDay` object that represents the same month-day in a different calendar, so to replace its `calendarId` property, you need to convert it to a Temporal.PlainDate object using toPlainDate(), change the calendar, and then convert it back.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by Temporal.PlainMonthDay.from() (except `calendar`): `day`, `era` and `eraYear`, `month`, `monthCode`, `year`. Unspecified properties use the values from the original month-day. You need to provide the year if and only if you provide `month` and the calendar is not `iso8601`. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A RangeError is thrown if the date component is out of range.

Return value

A new `Temporal.PlainMonthDay` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- If the year is provided, the calendar is not `iso8601`, and the year is not in the representable range or years, which is from `-271821` to `275760`.

Examples

Using with()

```js
const md = Temporal.PlainMonthDay.from("07-01");
const newMd = md.with({ day: 2 });
console.log(newMd.toString()); // "07-02"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plainmonthday.prototype.with`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.with`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainMonthDay
- Temporal.PlainMonthDay.from()

Temporal.PlainMonthDay.prototype.calendarId

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/calendarId · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`calendarId`** accessor property of Temporal.PlainMonthDay instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See Intl.supportedValuesOf() for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. There's no obvious way to create a new `Temporal.PlainMonthDay` object with a different calendar that represents the same month-day, so you need to convert it to a Temporal.PlainDate object first using toPlainDate(), change the calendar, and then convert it back.

Examples

Using calendarId

js

```
const md = Temporal.PlainMonthDay.from("07-01");
console.log(md.calendarId); // "iso8601"; default

const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=chinese]");
console.log(md2.calendarId); // "chinese"
```

Changing calendarId

js

```
const md = Temporal.PlainMonthDay.from("07-01");
const newMD = md
  .toPlainDate({ year: 2021 })
  .withCalendar("chinese")
  .toPlainMonthDay();
console.log(newMD.monthCode, newMD.day); // "M05" 22

const newMD2 = md
  .toPlainDate({ year: 2022 })
  .withCalendar("chinese")
  .toPlainMonthDay();
console.log(newMD2.monthCode, newMD2.day); // "M06" 3
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plainmonthday.prototype.calendarid

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.calendarId`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainMonthDay`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.prototype.day

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/day · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **day** accessor property of `Temporal.PlainMonthDay` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainMonthDay` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.day`.

Examples

Using day

```js
const md = Temporal.PlainMonthDay.from("07-01"); // ISO 8601 calendar
console.log(md.day); // 1

const md2 = Temporal.PlainMonthDay.from("2021-07-01[u-ca=chinese]");
console.log(md2.day); // 22; it is May 22 in the Chinese calendar
```

Changing day

```js
const md = Temporal.PlainMonthDay.from("07-01");
const newMD = md.with({ day: 15 });
console.log(newMD.toString()); // 07-15
```

By default, `with()` constrains the day to the range of valid values. So you can use `{ day: 1 }` to set the day to the first day of the month, even if the first day does not have the number `1`. Similarly, the following will set the day to the last day of the month:

```js
const md = Temporal.PlainMonthDay.from("07-01");
const lastMD = md.with({ day: Number.MAX_VALUE }); // 07-31
```

For the purpose of `PlainMonthDay`, February is always considered to have 29 days.

```js
const md = Temporal.PlainMonthDay.from("02-01");
const lastMD = md.with({ day: Number.MAX_VALUE }); // 02-29
console.log(lastMD.day); // 29
```

For other calendars, as long as there exists a year in which the month-day is valid, the month-day is considered valid, and the underlying reference year may therefore change. For example:

```js
const md = Temporal.PlainMonthDay.from({
  monthCode: "M02",
  day: 29,
  calendar: "hebrew",
});
console.log(md.toString()); // 1972-11-06[u-ca=hebrew]
console.log(md.toLocaleString("en-US", { calendar: "hebrew" })); // 29 Heshvan
const lastMD = md.with({ day: Number.MAX_VALUE });
// 30 Heshvan does not exist in 1972, so the reference year changes to 1971
console.log(lastMD.toString()); // 1971-11-18[u-ca=hebrew]
console.log(lastMD.toLocaleString("en-US", { calendar: "hebrew" })); // 30 Heshvan
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plainmonthday.prototype.day

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.day`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.prototype.with()`
- `Temporal.PlainMonthDay.prototype.monthCode`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainMonthDay

Temporal.PlainMonthDay.prototype.monthCode

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainMonthDay/monthCode · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`monthCode`** accessor property of `Temporal.PlainMonthDay` instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`). If the leap month is the first month of the year, the code is `M00L`.

Because `month` is an index within a year, but `PlainMonthDay` doesn't have a year, there's no `month` property for `PlainMonthDay`. Therefore, `monthCode` is used to represent the month in a way that is independent of the year.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainMonthDay` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.monthCode`.

Examples

Using monthCode

```js
const md = Temporal.PlainMonthDay.from("07-01"); // ISO 8601 calendar
console.log(md.monthCode); // "M07"

const md2 = Temporal.PlainMonthDay.from("2021-05-01[u-ca=chinese]");
console.log(md2.monthCode); // "M03"

const md3 = Temporal.PlainMonthDay.from("2023-04-01[u-ca=chinese]");
console.log(md3.monthCode); // "M02L"
```

Changing monthCode

```js
const md = Temporal.PlainMonthDay.from("07-01");
const newMD = md.with({ monthCode: "M03" });
console.log(newMD.toString()); // 03-01
```

For other calendars, as long as there exists a year in which the month-day is valid, the month-day is considered valid, and the underlying reference year may therefore change. For example:

```js
const md = Temporal.PlainMonthDay.from({
  monthCode: "M02",
  day: 30,
  calendar: "hebrew",
});
console.log(md.toString()); // 1971-11-18[u-ca=hebrew]
console.log(md.toLocaleString("en-US", { calendar: "hebrew" })); // 30 Heshvan
// 30 Heshvan only exists in 1971, but this year is not a leap year
const newMD = md.with({ monthCode: "M05L" });
console.log(newMD.toString()); // 1970-03-08[u-ca=hebrew]
console.log(newMD.toLocaleString("en-US", { calendar: "hebrew" })); // 30 Adar I
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plainmonthday.prototype.monthcode

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainMonthDay.monthCode`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainMonthDay`
- `Temporal.PlainMonthDay.prototype.with()`
- `Temporal.PlainMonthDay.prototype.day`

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.PlainTime

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainTime`** object represents a time without a date or time zone; for example, a recurring event that happens at the same time every day. It is fundamentally represented as a combination of hour, minute, second, millisecond, microsecond, and nanosecond values.

Description

A `PlainTime` is essentially the time part of a `Temporal.PlainDateTime` object, with the date information removed. Because the date and time information don't have much interaction, all general information about time properties is documented here.

RFC 9557 format

`PlainTime` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form:

```
HH:mm:ss.sssssssss
```

`HH`

A two-digit number from `00` to `23`. It may be prefixed by the time designator `T` or `t`.

`mm` Optional

A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

As an input, you may optionally include the date, offset, time zone identifier, and calendar, in the same format as `PlainDateTime`, but they will be ignored. A date-only string will be rejected. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits.

Constructor

`Temporal.PlainTime()`

Creates a new `Temporal.PlainTime` object by directly supplying the underlying data.

Static methods

`Temporal.PlainTime.compare()`

Returns a number (-1, 0, or 1) indicating whether the first time comes before, is the same as, or comes after the second time. Equivalent to comparing the hour, minute, second, millisecond, microsecond, and nanosecond fields one by one.

`Temporal.PlainTime.from()`

Creates a new `Temporal.PlainTime` object from another `Temporal.PlainTime` object, an object with time properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainTime.prototype` and shared by all `Temporal.PlainTime` instances.

`Temporal.PlainTime.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainTime` instances, the initial value is the `Temporal.PlainTime()` constructor.

`Temporal.PlainTime.prototype.hour`

Returns an integer from 0 to 23 representing the hour component of this time.

`Temporal.PlainTime.prototype.microsecond`

Returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

`Temporal.PlainTime.prototype.millisecond`

Returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

`Temporal.PlainTime.prototype.minute`

Returns an integer from 0 to 59 representing the minute component of this time.

`Temporal.PlainTime.prototype.nanosecond`

Returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

`Temporal.PlainTime.prototype.second`

Returns an integer from 0 to 59 representing the second component of this time.

`Temporal.PlainTime.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainTime"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainTime.prototype.add()`

Returns a new `Temporal.PlainTime` object representing this time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

`Temporal.PlainTime.prototype.equals()`

Returns `true` if this time is equivalent in value to another time (in a form convertible by `Temporal.PlainTime.from()`), and `false` otherwise. They are compared by their time values. Equivalent to `Temporal.PlainTime.compare(this, other) === 0`.

`Temporal.PlainTime.prototype.round()`

Returns a new `Temporal.PlainTime` object representing this time rounded to the given unit.

`Temporal.PlainTime.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another time (in a form convertible by `Temporal.PlainTime.from()`) to this time. The duration is positive if the other time is

before this time, and negative if after.

`Temporal.PlainTime.prototype.subtract()`

Returns a new `Temporal.PlainTime` object representing this time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

`Temporal.PlainTime.prototype.toJSON()`

Returns a string representing this time in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainTime.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this time.

`Temporal.PlainTime.prototype.toString()`

Returns a string representing this time in the RFC 9557 format.

`Temporal.PlainTime.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this time to another time (in a form convertible by `Temporal.PlainTime.from()`). The duration is positive if the other time is after this time, and negative if before.

`Temporal.PlainTime.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainTime.prototype.with()`

Returns a new `Temporal.PlainTime` object representing this time with some fields replaced by new values.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-plaintime-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainTime()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`add`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`hour`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`microsecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`millisecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`minute`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`nanosecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`round`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`second`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`since`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.PlainDateTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/PlainTime · MDN page last modified 2026-09-13

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainTime()`** constructor creates `Temporal.PlainTime` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainTime` objects using the `Temporal.PlainTime.from()` static method, which can handle a variety of input types.

Syntax

```js
new Temporal.PlainTime()
new Temporal.PlainTime(hour)
new Temporal.PlainTime(hour, minute)
new Temporal.PlainTime(hour, minute, second)
new Temporal.PlainTime(hour, minute, second, millisecond)
new Temporal.PlainTime(hour, minute, second, millisecond, microsecond)
new Temporal.PlainTime(hour, minute, second, millisecond, microsecond, nanosecond)
```

> **Note:** `Temporal.PlainTime()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`hour` Optional

A number, truncated to an integer, representing the hour component.

`minute` Optional

A number, truncated to an integer, representing the minute component.

`second` Optional

A number, truncated to an integer, representing the second component.

`millisecond` Optional

A number, truncated to an integer, representing the millisecond component.

`microsecond` Optional

A number, truncated to an integer, representing the microsecond component.

`nanosecond` Optional

A number, truncated to an integer, representing the nanosecond component.

Return value

A new `Temporal.PlainTime` object, representing the time specified by the parameters.

Exceptions

`RangeError`

Thrown if any of the components is not a finite number, or they don't represent a valid time.

Examples

Using Temporal.PlainTime()

```js
const time = new Temporal.PlainTime(12, 34, 56, 123, 456, 789);
console.log(time.toString()); // 12:34:56.123456789
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-properties-of-the-temporal-plaintime-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainTime()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.PlainTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/compare · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainTime.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first time comes before, is the same as, or comes after the second time. It is equivalent to comparing the hour, minute, second, millisecond, microsecond, and nanosecond fields one by one.

Syntax

```js
Temporal.PlainTime.compare(time1, time2)
```

Parameters

`time1`

A string, an object, or a `Temporal.PlainTime` instance representing the first time to compare. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`.

`time2`

The second time to compare, converted to a `Temporal.PlainTime` object using the same algorithm as `time1`.

Return value

Returns `-1` if `time1` comes before `time2`, `0` if they are the same, and `1` if `time1` comes after `time2`.

Examples

Using Temporal.PlainTime.compare()

```js
const time1 = Temporal.PlainTime.from("12:34:56");
const time2 = Temporal.PlainTime.from("12:34:57");
console.log(Temporal.PlainTime.compare(time1, time2)); // -1

const time3 = Temporal.PlainTime.from("11:34:56");
console.log(Temporal.PlainTime.compare(time1, time3)); // 1
```

Sorting an array of times

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```js
const times = ["12:34:56", "11:34:56", "12:34:57"];

times.sort(Temporal.PlainTime.compare);
console.log(times);
// [ "11:34:56", "12:34:56", "12:34:57" ]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.compare

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.compare`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.equals()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/from · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainTime.from()`** static method creates a new `Temporal.PlainTime` object from another `Temporal.PlainTime` object, an object with time properties, or an RFC 9557 string.

Syntax

```js
Temporal.PlainTime.from(info)
Temporal.PlainTime.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainTime` instance, which creates a copy of the instance.
- A `Temporal.PlainDateTime` instance, which provides the time in the same fashion as `Temporal.PlainDateTime.prototype.toPlainTime()`.
- A `Temporal.ZonedDateTime` instance, which provides the time in the same fashion as `Temporal.ZonedDateTime.prototype.toPlainTime()`.
- An RFC 9557 string containing a time.
- An object containing at least one of the following properties (in the order they are retrieved and validated):
  - `hour`
  - `microsecond`
  - `millisecond`
  - `minute`
  - `nanosecond`
  - `second`

  They are truncated to be integers. Out-of-range values are handled by the `overflow` option.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a time component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The time component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the time component is out of range.

Return value

A new `Temporal.PlainTime` object, representing the time specified by `info`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object with at least one recognized property or a string.
- `options` is not an object or `undefined`.

`RangeError`

Thrown if the provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.

Examples

Creating a PlainTime from an object

```js
const t1 = Temporal.PlainTime.from({ hour: 0 });
console.log(t1.toString()); // "00:00:00"

const t2 = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
console.log(t2.toString()); // "12:34:56"

const t3 = Temporal.PlainTime.from({
  hour: 12,
  minute: 34,
  second: 56,
  millisecond: 123,
  microsecond: 456,
  nanosecond: 789,
});
console.log(t3.toString()); // "12:34:56.123456789"
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range:

```js
const t1 = Temporal.PlainTime.from({ hour: 25 });
console.log(t1.toString()); // "23:00:00"

const t2 = Temporal.PlainTime.from({ hour: 25, minute: 60 });
console.log(t2.toString()); // "23:59:00"
```

You can change this behavior to throw an error instead:

```js
Temporal.PlainTime.from({ hour: 25 }, { overflow: "reject" });
// RangeError: time value "hour" not in 0..23: 25
```

Creating a PlainTime from a string

```js
const t1 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(t1.toLocaleString("en-US", { timeStyle: "full" }));
// 12:34:56 PM
```

Creating a PlainTime from another Temporal instance

```js
const dt = Temporal.PlainDateTime.from("2021-07-01T12:00");
const t = Temporal.PlainTime.from(dt);
console.log(t.toString()); // "12:00:00"

const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T00:00+08:00[Asia/Shanghai]",
);
const t2 = Temporal.PlainTime.from(zdt);
console.log(t2.toString()); // "00:00:00"

const t3 = Temporal.PlainTime.from(t);
console.log(t3.toString()); // "12:00:00"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.from`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime()`
- `Temporal.PlainTime.prototype.with()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/add · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`add()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

Syntax

```js
add(duration)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.PlainTime` object representing the time specified by the original `PlainTime`, plus the duration. Any units above `hours` are ignored, and if the time goes past midnight, it wraps around to the next day.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration

```js
const start = Temporal.PlainTime.from("12:34:56");
const end = start.add({ hours: 1, minutes: 30 });
console.log(end.toString()); // 14:04:56

const end2 = start.add({ hours: -1, minutes: -30 });
console.log(end2.toString()); // 11:04:56

const distance = Temporal.PlainTime.from("00:00:00").until("01:23:45"); // 1h 23m 45s
const end3 = start.add(distance);
console.log(end3.toString()); // 13:58:41
```

Time wrapping

If the time goes past midnight, it wraps around to the next day:

```js
const start = Temporal.PlainTime.from("12:34:56");
const end = start.add({ hours: 12 });
console.log(end.toString()); // 00:34:56
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.add`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.equals()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/equals · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`equals()`** method of `Temporal.PlainTime` instances returns `true` if this time is equivalent in value to another time (in a form convertible by `Temporal.PlainTime.from()`), and `false` otherwise. They are compared by their time values. It is equivalent to `Temporal.PlainTime.compare(this, other) === 0`.

Syntax

```js
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainTime` instance representing the other time to compare. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`.

Return value

`true` if this time is equal to `other` both in their time value and their calendar, `false` otherwise.

Examples

Using equals()

```js
const time1 = Temporal.PlainTime.from("12:34:56");
const time2 = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
console.log(time1.equals(time2)); // true

const time3 = Temporal.PlainTime.from("00:34:56");
console.log(time1.equals(time3)); // false
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.equals

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.equals`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.compare()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.round()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/round · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`round()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time rounded to the given unit.

Syntax

```js
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. The increment must be a divisor of the maximum value of `smallestUnit`; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.PlainTime` object representing this time rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```js
const time = Temporal.PlainTime.from("12:34:56.123456789");
const nearestMillisecond = time.round("millisecond");
console.log(nearestMillisecond.toString()); // 12:34:56.123

const nearestHalfHour = time.round({
  smallestUnit: "minute",
  roundingIncrement: 30,
});
console.log(nearestHalfHour.toString()); // 12:30:00

const nextHour = time.round({ smallestUnit: "hour", roundingMode: "ceil" });
console.log(nextHour.toString()); // 13:00:00
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.round`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`round`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.round`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.since()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/since · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`since()`** method of `Temporal.PlainTime` instances returns a new `Temporal.Duration` object representing the duration from another time (in a form convertible by `Temporal.PlainTime.from()`) to this time. The duration is positive if the other time is before this time, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```js
since(other)
since(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainTime` instance representing a time to subtract from this time. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"hours"`, `"minutes"`, `"seconds"`, `"milliseconds"`, `"microseconds"`,

`"nanoseconds"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"hours"`. For `smallestUnit`, the default value is `"nanoseconds"`.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this time. The duration is positive if `other` is before this time, and negative if after.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using since()

```js
const lunchTime = Temporal.PlainTime.from("12:30:00");
const now = Temporal.Now.plainTimeISO();
const duration = now.since(lunchTime);
console.log(`You had lunch ${duration.toLocaleString("en-US")} ago`);
// Example output: "You had lunch 3 hr, 42 min, 21 sec, 343 ms, 131 µs, 718 ns ago"

const duration2 = now.since(lunchTime, { smallestUnit: "minutes" });
console.log(`You had lunch ${duration2.toLocaleString("en-US")} ago`);
// Example output: "You had lunch 3 hr, 42 min ago"

const duration3 = now.since(lunchTime, {
  largestUnit: "minutes",
  smallestUnit: "minutes",
});
console.log(`You had lunch ${duration3.toLocaleString("en-US")} ago`);
// Example output: "You had lunch 222 min ago"
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```js
const time1 = Temporal.PlainTime.from("12:30:00");
const time2 = Temporal.PlainTime.from("12:30:01");
const duration = time2.since(time1, {
  smallestUnit: "seconds",
  roundingIncrement: 15,
  roundingMode: "ceil",
});
console.log(duration.toString()); // "PT15S"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.since

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.since`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.until()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.subtract()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/subtract · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`subtract()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`), wrapping around the clock if necessary.

If you want to subtract two times and get a duration, use `since()` or `until()` instead.

Syntax

```js
subtract(duration)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

Return value

A new `Temporal.PlainTime` object representing the time specified by the original `PlainTime`, minus the duration.

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a duration

```js
const start = Temporal.PlainTime.from("12:34:56");
const end = start.subtract({ hours: 1, minutes: 30 });
console.log(end.toString()); // 11:04:56
```

For more examples, see `add()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.subtract

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.subtract`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.since()`
- `Temporal.PlainTime.prototype.until()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of `Temporal.PlainTime` instances returns a string representing this time in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```js
toJSON()
```

Parameters

None.

Return value

A string representing the given time in the RFC 9557 format.

Description

The method is automatically called by `JSON.stringify()` when a `Temporal.PlainTime` object is encountered during JSON serialization. The serialized value can then be deserialized using the `Temporal.PlainTime.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```js
const time = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
const timeStr = time.toJSON(); // '12:34:56'
const t2 = Temporal.PlainTime.from(timeStr);
```

JSON serialization and parsing

This example shows how `Temporal.PlainTime` can be serialized as JSON without extra effort, and how to parse it back.

```js
const time = Temporal.PlainTime.from({ hour: 12, minute: 34, second: 56 });
const jsonStr = JSON.stringify({ time }); // '{"time":"12:34:56"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "time") {
    return Temporal.PlainTime.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.toJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`
- `Temporal.PlainTime.prototype.toString()`
- `Temporal.PlainTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.PlainTime` instances returns a string with a language-sensitive representation of this time. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `hour`, `minute`, and `second` will default to `"numeric"`.
- Provide `timeStyle` only: it expands to `dayPeriod`, `hour`, `minute`, `second`, and `fractionalSecondDigits` formats.
- Provide some date-time component options, where at least one of them is a time option (`dayPeriod`, `hour`, `minute`, `second`, `fractionalSecondDigits`). Only the specified time components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given time according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(time)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const time = Temporal.PlainTime.from("12:34:56");

console.log(time.toLocaleString()); // 12:34:56 PM (assuming en-US locale)
```

Using toLocaleString() with options

You can customize which parts of the time are included in the output by providing the `options` parameter.

```js
const time = Temporal.PlainTime.from("12:34:56");
time.toLocaleString("en-US", { timeStyle: "short" }); // 12:34 PM
time.toLocaleString("en-US", { hour: "2-digit" }); // 12 PM
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.tolocalestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.toLocaleString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainTime`
- `Intl.DateTimeFormat`
- `Temporal.PlainTime.prototype.toJSON()`
- `Temporal.PlainTime.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/toString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of `Temporal.PlainTime` instances returns a string representing this time in the RFC 9557 format.

Syntax

```js
toString()
toString(options)
```

Parameters

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

Return value

A string in the RFC 9557 format representing this time.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```js
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.toString()); // '12:34:56'
```

Using options

```js
const time1 = Temporal.PlainTime.from("12:00:00");
console.log(time1.toString()); // '12:00:00'
console.log(time1.toString({ fractionalSecondDigits: 1 })); // '12:00:00.0'
console.log(time1.toString({ smallestUnit: "minute" })); // '12:00'
console.log(time1.toString({ smallestUnit: "nanosecond" })); // '12:00:00.000000000'

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.toString({ fractionalSecondDigits: 4 })); // '12:34:56.1234'
console.log(
  time2.toString({ fractionalSecondDigits: 4, roundingMode: "halfExpand" }),
); // '12:34:56.1235'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`
- `Temporal.PlainTime.prototype.toJSON()`
- `Temporal.PlainTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.until()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/until · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`until()`** method of `Temporal.PlainTime` instances returns a new `Temporal.Duration` object representing the duration from this time to another time (in a form convertible by `Temporal.PlainTime.from()`). The duration is positive if the other time is after this time, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```js
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainTime` instance representing a time to subtract this time from. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this time *until* `other`. The duration is positive if `other` is after this time, and negative if before.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Using until()

```js
const lunchTime = Temporal.PlainTime.from("12:30:00");
const now = Temporal.Now.plainTimeISO();
const duration = now.until(lunchTime);
console.log(`It will be ${duration.toLocaleString("en-US")} until lunch`);
// Example output: "It will be 3 hr, 42 min, 21 sec, 343 ms, 131 µs, 718 ns until lunch"
```

For more examples, see `since()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.until`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.until`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.Duration`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.since()`

Temporal.PlainTime.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.PlainTime` instances throws a `TypeError`, which prevents `Temporal.PlainTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `time1 > time2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainTime` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainTime.prototype.toString()`, or use the `Temporal.PlainTime.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainTime

All arithmetic and comparison operations on `Temporal.PlainTime` instances should use the dedicated methods or convert them to primitives explicitly.

```
const time1 = Temporal.PlainTime.from("00:00:00");
const time2 = Temporal.PlainTime.from("12:00:00");
time1 > time2; // TypeError: can't convert PlainTime to primitive type
Temporal.PlainTime.compare(time1, time2); // -1

time2 - time1; // TypeError: can't convert PlainTime to primitive type
time2.since(time1).toString(); // "PT12H"
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.valueof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.toString()`
- `Temporal.PlainTime.prototype.toJSON()`
- `Temporal.PlainTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/with · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`with()`** method of `Temporal.PlainTime` instances returns a new `Temporal.PlainTime` object representing this time with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the time's fields.

Syntax

```
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainTime.from()`: `hour`, `microsecond`, `millisecond`, `minute`, `nanosecond`, `second`. Unspecified properties use the values from the original time.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a time component is out of range. Possible values are:

`"constrain"` (default)

The time component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the time component is out of range.

Return value

A new `Temporal.PlainTime` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original time.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object with at least one recognized property or a string.
- `options` is not an object or `undefined`.

`RangeError`

Thrown if the provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.

Examples

Using with()

```js
const time = Temporal.PlainTime.from("12:34:56.123456789");
const newTime = time.with({ hour: 23 });
console.log(newTime.toString()); // '23:34:56.123456789'
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plaintime.prototype.with

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.with`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.from()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`

Temporal.PlainTime.prototype.hour

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/hour · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **hour** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 23 representing the hour component of this time.

The set accessor of `hour` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using hour

```js
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.hour); // 12
```

Changing hour

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ hour: 15 });
console.log(newTime.toString()); // 15:34:56
```

You can also use `add()` or `subtract()` to move a certain number of hours from the current time.

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ hours: 3 });
console.log(newTime.toString()); // 15:34:56
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaintime.prototype.hour

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
hour	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.hour`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.microsecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/microsecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`microsecond`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

The set accessor of `microsecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using microsecond

```js
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.microsecond); // 0

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.microsecond); // 456
```

Changing microsecond

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ microsecond: 100 });
console.log(newTime.toString()); // 12:34:56.0001
```

You can also use `add()` or `subtract()` to move a certain number of microseconds from the current time.

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ microseconds: 100 });
console.log(newTime.toString()); // 12:34:56.0001
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaintime.prototype.microsecond

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
microsecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.microsecond`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.second`
- `Temporal.PlainTime.prototype.millisecond`
- `Temporal.PlainTime.prototype.nanosecond`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.millisecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/millisecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`millisecond`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

The set accessor of `millisecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using millisecond

```js
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.millisecond); // 0

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.millisecond); // 123
```

Changing millisecond

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ millisecond: 100 });
console.log(newTime.toString()); // 12:34:56.1
```

You can also use `add()` or `subtract()` to move a certain number of milliseconds from the current time.

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ milliseconds: 100 });
console.log(newTime.toString()); // 12:34:56.1
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaintime.prototype.millisecond

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
millisecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.millisecond`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.second`
- `Temporal.PlainTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.minute

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/minute · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`minute`** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 59 representing the minute component of this time.

The set accessor of `minute` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using minute

```js
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.minute); // 34
```

Changing minute

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ minute: 58 });
console.log(newTime.toString()); // 12:58:56
```

You can also use `add()` or `subtract()` to move a certain number of minutes from the current time.

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ minutes: 24 });
console.log(newTime.toString()); // 12:58:56
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plaintime.prototype.minute`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`minute`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.minute`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainTime
- Temporal.PlainTime.prototype.with()
- Temporal.PlainTime.prototype.add()
- Temporal.PlainTime.prototype.subtract()

Temporal.PlainTime.prototype.nanosecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/nanosecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **nanosecond** accessor property of Temporal.PlainTime instances returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

The set accessor of nanosecond is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.PlainTime object with the desired new value.

Examples

Using nanosecond

```js
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.nanosecond); // 0

const time2 = Temporal.PlainTime.from("12:34:56.123456789");
console.log(time2.nanosecond); // 789
```

Changing nanosecond

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ nanosecond: 100 });
console.log(newTime.toString()); // 12:34:56.0000001
```

You can also use add() or subtract() to move a certain number of nanoseconds from the current time.

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.add({ nanoseconds: 100 });
console.log(newTime.toString()); // 12:34:56.0000001
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaintime.prototype.nanosecond

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
nanosecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.nanosecond`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.second`
- `Temporal.PlainTime.prototype.millisecond`
- `Temporal.PlainTime.prototype.microsecond`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainTime

Temporal.PlainTime.prototype.second

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainTime/second · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **second** accessor property of `Temporal.PlainTime` instances returns an integer from 0 to 59 representing the second component of this time.

The set accessor of `second` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainTime` object with the desired new value.

Examples

Using second

```js
const time = Temporal.PlainTime.from("12:34:56");
console.log(time.second); // 56
```

Changing second

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.with({ second: 15 });
console.log(newTime.toString()); // 12:34:15
```

You can also use `add()` or `subtract()` to move a certain number of seconds from the current time.

```js
const time = Temporal.PlainTime.from("12:34:56");
const newTime = time.subtract({ seconds: 41 });
console.log(newTime.toString()); // 12:34:15
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plaintime.prototype.second

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
second	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainTime.second`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainTime`
- `Temporal.PlainTime.prototype.with()`
- `Temporal.PlainTime.prototype.add()`
- `Temporal.PlainTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.millisecond`
- `Temporal.PlainTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.PlainYearMonth

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainYearMonth`** object represents the year and month of a calendar date, without a day or time zone; for example, an event on a calendar that happens during the whole month. It is fundamentally represented as an ISO 8601 calendar date, with year, month, and day fields, and an associated calendar system. The day is used to disambiguate the year-month in non-ISO calendar systems.

Description

A `PlainYearMonth` is essentially the year-month part of a `Temporal.PlainDate` object, without the day.

RFC 9557 format

`PlainYearMonth` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD [u-ca=calendar_id]
```

`YYYY`

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD` Optional

A two-digit number from `01` to `31`. It is required for non-ISO calendars, and optional otherwise. If omitted, the string looks like `YYYY-MM` or `YYYYMM`. Note that the reference day actually stored may be different from the one you provide, but the represented year-month is the same. See `Temporal.PlainYearMonth.from()` for more information. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, you may optionally include the time, offset, and time zone identifier, in the same format as `PlainDateTime`, but they will be ignored. Other annotations in the `[key=value]` format are also ignored, and they must not have the critical flag.

When serializing, you can configure whether to display the calendar ID, and whether to add a critical flag for it.

Constructor

`Temporal.PlainYearMonth()`

Creates a new `Temporal.PlainYearMonth` object by directly supplying the underlying data.

Static methods

`Temporal.PlainYearMonth.compare()`

Returns a number (-1, 0, or 1) indicating whether the first year-month comes before, is the same as, or comes after the second year-month. Equivalent to comparing their underlying ISO 8601 dates. Two year-months from different calendars may be considered equal if they start on the same ISO date.

`Temporal.PlainYearMonth.from()`

Creates a new `Temporal.PlainYearMonth` object from another `Temporal.PlainYearMonth` object, an object with year and month properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.PlainYearMonth.prototype` and shared by all `Temporal.PlainYearMonth` instances.

`Temporal.PlainYearMonth.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.PlainYearMonth.prototype.constructor`

The constructor function that created the instance object. For `Temporal.PlainYearMonth` instances, the initial value is the `Temporal.PlainYearMonth()` constructor.

`Temporal.PlainYearMonth.prototype.daysInMonth`

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

`Temporal.PlainYearMonth.prototype.daysInYear`

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

`Temporal.PlainYearMonth.prototype.era`

Returns a calendar-specific lowercase string representing the era of this year-month, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

`Temporal.PlainYearMonth.prototype.eraYear`

Returns a non-negative integer representing the year of this year-month within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

`Temporal.PlainYearMonth.prototype.inLeapYear`

Returns a boolean indicating whether this year-month is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.PlainYearMonth.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this year-month. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.PlainYearMonth.prototype.monthCode`

Returns a calendar-specific string representing the month of this year-month. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.PlainYearMonth.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.PlainYearMonth.prototype.year`

Returns an integer representing the number of years of this year-month relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.PlainYearMonth.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.PlainYearMonth"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.PlainYearMonth.prototype.add()`

Returns a new `Temporal.PlainYearMonth` object representing this year-month moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainYearMonth.prototype.equals()`

Returns `true` if this year-month is equivalent in value to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`), and `false` otherwise. They are compared both by their underlying ISO date values and their calendars, so two year-months from different calendars may be considered equal by `Temporal.PlainYearMonth.compare()` but not by `equals()`.

`Temporal.PlainYearMonth.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`) to this year-month. The duration is positive if the other month is before this month, and negative if after.

`Temporal.PlainYearMonth.prototype.subtract()`

Returns a new `Temporal.PlainYearMonth` object representing this year-month moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.PlainYearMonth.prototype.toJSON()`

Returns a string representing this year-month in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.PlainYearMonth.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this year-month.

`Temporal.PlainYearMonth.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing this year-month and a supplied day in the same calendar system.

`Temporal.PlainYearMonth.prototype.toString()`

Returns a string representing this year-month in the RFC 9557 format.

`Temporal.PlainYearMonth.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this year-month to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`). The duration is positive if the other month is after this month, and negative if before.

`Temporal.PlainYearMonth.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.PlainYearMonth` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.PlainYearMonth.prototype.with()`

Returns a new `Temporal.PlainYearMonth` object representing this year-month with some fields replaced by new values.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-plainyearmonth-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainYearMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`PlainYearMonth()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0
`add`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`daysInMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`daysInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`era`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`eraYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`inLeapYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`month`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthCode`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`monthsInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`since`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toPlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`year`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`
- `Temporal.PlainDate`
- `Temporal.PlainMonthDay`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/PlainYearMonth · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainYearMonth()`** constructor creates `Temporal.PlainYearMonth` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.PlainYearMonth` objects using the `Temporal.PlainYearMonth.from()` static method, which can handle a variety of input types.

Syntax

```js
new Temporal.PlainYearMonth(year, month)
new Temporal.PlainYearMonth(year, month, calendar)
new Temporal.PlainYearMonth(year, month, calendar, referenceDay)
```

> **Note:** `Temporal.PlainYearMonth()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

> **Warning:** Avoid using the `calendar` and `referenceDay` parameters, because `equals()` and `compare()` will consider the reference day for comparison, causing two equivalent year-months to be considered different if they have different reference days. To create a `Temporal.PlainYearMonth` object with a non-ISO calendar, use the `Temporal.PlainYearMonth.from()` static method.

Parameters

`year` Optional

A number, truncated to an integer, representing the year in the ISO calendar system.

`month`

A number, truncated to an integer, representing the month in the ISO calendar system.

`calendar` Optional

A string representing the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. Note that irrespective of the `calendar`, the `year`, `month`, and `referenceDay` must be in the ISO 8601 calendar system.

`referenceDay`

A number, truncated to an integer, representing the day of the month in the ISO calendar system. Defaults to `1`. The same ISO year-month can represent different months on different days with non-ISO calendars. For example, the days 2021-07-01 and 2021-07-31 may fall in different months in a non-Gregorian calendar, and just specifying "2021-07" is insufficient to unambiguously determine a month in the target calendar. Therefore, you virtually always want to specify a `referenceDay` when using a non-ISO calendar.

Return value

A new `Temporal.PlainYearMonth` object, representing the year-month of the date specified by `year`, `month`, and `referenceDay` (in the ISO calendar), interpreted in the calendar system specified by `calendar`.

Exceptions

`TypeError`

Thrown if `calendar` is not a string or `undefined`.

`RangeError`

Thrown in one of the following cases:

- `year`, `month`, or `referenceDay` is not a finite number.
- The `year`, `month`, and `referenceDay` combination does not represent a valid date in the ISO calendar system, or is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.PlainYearMonth()

```js
const ym = new Temporal.PlainYearMonth(2021, 7);
console.log(ym.toString()); // 2021-07

const ym2 = new Temporal.PlainYearMonth(2021, 7, "chinese");
console.log(ym2.toString()); // 2021-07-01[u-ca=chinese]

const ym3 = new Temporal.PlainYearMonth(2021, 7, "chinese", 31);
console.log(ym3.toString()); // 2021-07-31[u-ca=chinese]
```

Improper usage

You should avoid using the `calendar` and `referenceDay` parameters, unless you know that the `referenceDay` is the canonical reference day that would be selected by `Temporal.PlainYearMonth.from()` for the same year-month.

```js
const ym = new Temporal.PlainYearMonth(2021, 7, "chinese", 1);
const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=chinese]");
console.log(ym.equals(ym2)); // false
console.log(ym.toString()); // 2021-07-01[u-ca=chinese]
console.log(ym2.toString()); // 2021-06-10[u-ca=chinese]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal-plainyearmonth-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PlainYearMonth()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	?	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.PlainYearMonth`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/compare · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainYearMonth.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first year-month comes before, is the same as, or comes after the second year-month. It is equivalent to comparing their underlying ISO 8601 dates. Two year-months from different calendars may be considered equal if they start on the same ISO date.

> **Note:** `PlainYearMonth` objects keep track of a reference ISO day, which is also used in the comparison. This day is automatically set when using the `Temporal.PlainYearMonth.from()` method, but can be set manually using the `Temporal.PlainYearMonth()` constructor, causing two equivalent year-months to be considered different if they have different reference days. For this reason, you should avoid using the constructor directly and prefer the `from()` method.

Syntax

```js
Temporal.PlainYearMonth.compare(yearMonth1, yearMonth2)
```

Parameters

`yearMonth1`

A string, an object, or a `Temporal.PlainYearMonth` instance representing the first year-month to compare. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`.

`yearMonth2`

The second year-month to compare, converted to a `Temporal.PlainYearMonth` object using the same algorithm as `yearMonth1`.

Return value

Returns `-1` if `yearMonth1` comes before `yearMonth2`, `0` if they are the same, and `1` if `yearMonth1` comes after `yearMonth2`. They are compared by their underlying date values (usually the first day of the month), ignoring their calendars.

Examples

Using Temporal.PlainYearMonth.compare()

```js
const ym1 = Temporal.PlainYearMonth.from("2021-08");
const ym2 = Temporal.PlainYearMonth.from("2021-09");
console.log(Temporal.PlainYearMonth.compare(ym1, ym2)); // -1

const ym3 = Temporal.PlainYearMonth.from("2021-07");
console.log(Temporal.PlainYearMonth.compare(ym1, ym3)); // 1
```

Comparing year-months in different calendars

```js
const ym1 = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ym2 = Temporal.PlainYearMonth.from({
  year: 2021,
  month: 8,
  calendar: "islamic-umalqura",
});
const ym3 = Temporal.PlainYearMonth.from({
  year: 2021,
  month: 8,
  calendar: "hebrew",
});
console.log(ym1.toString()); // "2021-08"
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(ym2.toString()); // "2582-12-17[u-ca=islamic-umalqura]"
console.log(ym3.toString()); // "-001739-04-06[u-ca=hebrew]"
console.log(Temporal.PlainYearMonth.compare(ym1, ym2)); // -1
console.log(Temporal.PlainYearMonth.compare(ym1, ym3)); // 1
```

Sorting an array of year-months

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

js

```
const months = [
  Temporal.PlainYearMonth.from({ year: 2021, month: 8 }),
  Temporal.PlainYearMonth.from({
    year: 2021,
    month: 8,
    calendar: "islamic-umalqura",
  }),
  Temporal.PlainYearMonth.from({ year: 2021, month: 8, calendar: "hebrew" }),
];

months.sort(Temporal.PlainYearMonth.compare);
console.log(months.map((d) => d.toString()));
// [ "-001739-04-06[u-ca=hebrew]", "2021-08", "2582-12-17[u-ca=islamic-umalqura]" ]
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.compare`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.compare`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.equals()`

Temporal.PlainYearMonth.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/from · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.PlainYearMonth.from()`** static method creates a new `Temporal.PlainYearMonth` object from another `Temporal.PlainYearMonth` object, an object with year and month properties, or an RFC 9557 string.

Syntax

```js
Temporal.PlainYearMonth.from(info)
Temporal.PlainYearMonth.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.PlainYearMonth` instance, which creates a copy of the instance.
- An RFC 9557 string containing a date and optionally a calendar. If the calendar is not `iso8601`, a day is required.
- An object containing the following properties (in the order they are retrieved and validated):

  `calendar` Optional

  A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`. All other properties are interpreted in this calendar system (unlike the `Temporal.PlainYearMonth()` constructor, which interprets the values in the ISO calendar system).

  `era` and `eraYear`

  A string and an integer that correspond to the `era` and `eraYear` properties. Are only used if the calendar system has eras. `era` and `eraYear` must be provided simultaneously. If they are not provided, then `year` must be provided. If all of `era`, `eraYear`, and `year` are provided, they must be consistent.

  `month`

  Corresponds to the `month` property. Must be positive regardless of the `overflow` option.

  `monthCode`

  Corresponds to the `monthCode` property. If it is not provided, then `month` must be provided. If both `month` and `monthCode` are provided, they must be consistent.

  `year`

  Corresponds to the `year` property.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object, representing the year and month specified by `info` in the specified `calendar`.

Each `PlainYearMonth` stores a whole ISO 8601 date internally, which has the same year-month in the target calendar as what's exposed. The reference day is visible when stringifying with `toString()`, which outputs an ISO date. The reference day is chosen arbitrarily but consistently; that is, every `(year, month)` pair always maps to the same ISO reference day. It does not use the day provided in the input. Instead, the reference day is always chosen to be the first valid day of the month.

This reference day canonicalization ensures that `equals()` can directly compare the underlying ISO dates without extra computation.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`) and a `month` (or a `monthCode`).

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The info is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a PlainYearMonth from an object

```js
// Year + month code
const ym = Temporal.PlainYearMonth.from({ year: 2021, monthCode: "M05" });
console.log(ym.toString()); // 2021-05

// Year + month
const ym2 = Temporal.PlainYearMonth.from({ year: 2021, month: 7 });
console.log(ym2.toString()); // 2021-07

// Year + month in a different calendar
const ym3 = Temporal.PlainYearMonth.from({
  year: 5730,
  month: 6,
  calendar: "hebrew",
});
console.log(ym3.toString()); // 1970-02-07[u-ca=hebrew]

// Year + month code in a different calendar
const ym4 = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
console.log(ym4.toString()); // 1970-02-07[u-ca=hebrew]
```

Controlling overflow behavior

By default, out-of-range values are clamped to the valid range.

```js
const ym1 = Temporal.PlainYearMonth.from({ year: 2021, month: 13 });
console.log(ym1.toString()); // 2021-12

// 5732 is not a Hebrew leap year, so a different monthCode is chosen
const ym2 = Temporal.PlainYearMonth.from({
  year: 5732,
  monthCode: "M05L",
  calendar: "hebrew",
});
console.log(ym2.toLocaleString("en-US", { calendar: "hebrew" })); // Adar 5732
const underlyingDate = Temporal.PlainDate.from(ym2.toString());
console.log(underlyingDate.year, underlyingDate.monthCode); // 5732 M06
```

You can change this behavior to throw an error instead:

```js
Temporal.PlainYearMonth.from({ year: 2021, month: 13 }, { overflow: "reject" });
// RangeError: date value "month" not in 1..12: 13
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.from`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth()`
- `Temporal.PlainYearMonth.prototype.with()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/add · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`add()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.PlainYearMonth` object representing this year-month moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```js
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this year-month. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object representing the year-month specified by the original `PlainYearMonth`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

The `duration` is handled in this way:

- Move forward by the number of years, keeping the `monthCode` the same. If the `monthCode` is invalid in the resulting year (impossible for Gregorian and ISO 8601, but possible for calendars with leap months), we adjust based on the `overflow` option: for `constrain`, we pick another month according to the cultural conventions of that calendar's users. For example, because the leap month is usually thought of as a duplicate of another month, we may pick the month that it is a duplicate of.
- Move forward by the number of months, adjusting the year if necessary.
- For all units smaller than `months` (weeks, days, hours, minutes, seconds, milliseconds, microseconds, nanoseconds), they are converted to the number of days. All commonly supported calendars use fixed-length weeks, so the number of weeks is just converted to the number of days. If the rule is more complex, we may take an approach similar to shifting months. Then, we move forward by that number of days, starting on the first day of the month, adjusting the month and year if necessary. Durations smaller than the current month's length therefore have no effect.

The internal reference day is then chosen to be the first valid day of the month, irrespective of the original reference day or the number of days in the duration. For the Gregorian calendar, overflow cannot happen because every year always has 12 months, and any increment less than a month is just ignored.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration in ISO 8601 calendar

```js
const start = Temporal.PlainYearMonth.from("2021-01");
const end = start.add({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2022-03

const end2 = start.add({ years: -1, months: -2, weeks: -3, days: -4 });
console.log(end2.toString()); // 2019-11

const distance = Temporal.PlainYearMonth.from("2020-01").until("2021-01"); // 366 days
const end3 = start.add(distance);
console.log(end3.toString()); // 2022-01
```

Adding a duration in a non-ISO calendar

```js
const start = Temporal.PlainYearMonth.from("2021-02-01[u-ca=chinese]");
console.log(start.toLocaleString("en-US", { calendar: "chinese" })); // 12/2020
console.log(start.toString()); // 2021-01-13[u-ca=chinese]
const end = start.add({ months: 1 });
console.log(end.toLocaleString("en-US", { calendar: "chinese" })); // 1/2021
console.log(end.toString()); // 2021-02-12[u-ca=chinese]

// Adding an extra day has no effect at all
const end2 = start.add({ months: 1, days: 1 });
console.log(end2.toLocaleString("en-US", { calendar: "chinese" })); // 1/2021
// The reference day doesn't change, because it's always the first day of the Chinese month
console.log(end2.toString()); // 2021-02-12[u-ca=chinese]

// Start in a leap month
const start2 = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
console.log(start2.toLocaleString("en-US", { calendar: "hebrew" })); // Adar I 5730
// End in another leap month
const end3 = start2.add({ years: 3 });
console.log(end3.toLocaleString("en-US", { calendar: "hebrew" })); // Adar I 5733
```

Adding a duration with overflow

If we move a few years and the corresponding month is invalid in this year, then we adjust the month based on the `overflow` option.

```js
// Start in a leap month
const start = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
// Hebrew leap years occur every 2 or 3 years, and 5731 is not a leap year
const end = start.add({ years: 1 });
console.log(end.toLocaleString("en-US", { calendar: "hebrew" })); // Adar 5731
console.log(end.monthCode); // M06
console.log(end.toString()); // 1971-02-26[u-ca=hebrew]

// Any further month additions are based on the clamped year-month
const end2 = start.add({ years: 1, months: 2 });
console.log(end2.monthCode); // M08
console.log(end2.toString()); // 1971-04-26[u-ca=hebrew]

// Compare with the same addition in a different order that results in no overflow:
const end3 = start.add({ months: 2 }).add({ years: 1 });
console.log(end3.monthCode); // M07
console.log(end3.toString()); // 1971-03-27[u-ca=hebrew]
```

Note that the following is not an overflow because the year can just increment:

```js
const start = Temporal.PlainYearMonth.from("2021-01");
const end = start.add({ months: 100 });
console.log(end.toString()); // 2029-05
```

You can also throw an error if the date component is out of range:

```js
const start = Temporal.PlainYearMonth.from({
  year: 5730,
  monthCode: "M05L",
  calendar: "hebrew",
});
const end = start.add({ years: 1 }, { overflow: "reject" }); // RangeError: invalid
"monthCode" calendar field: M05L
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.add`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.Duration`
- `Temporal.PlainYearMonth.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.equals()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/equals · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`equals()`** method of `Temporal.PlainYearMonth` instances returns `true` if this year-month is equivalent in value to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`), and `false` otherwise. They are compared both by their underlying ISO date values and their calendars, so two year-months from different calendars may be considered equal by `Temporal.PlainYearMonth.compare()` but not by `equals()`.

> **Note:** `PlainYearMonth` objects keep track of a reference ISO day, which is also used in the comparison. This day is automatically set when using the `Temporal.PlainYearMonth.from()` method, but can be set manually using the `Temporal.PlainYearMonth()` constructor, causing two equivalent year-months to be considered different if they have different reference days. For this reason, you should avoid using the constructor directly and prefer the `from()` method.

Syntax

```js
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainYearMonth` instance representing the other year-month to compare. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`.

Return value

`true` if this year-month is equal to `other` both in their date value and their calendar, `false` otherwise.

Examples

Using equals()

```js
const ym1 = Temporal.PlainYearMonth.from("2021-08");
const ym2 = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
console.log(ym1.equals(ym2)); // true

const ym3 = Temporal.PlainYearMonth.from("2021-08-01[u-ca=japanese]");
console.log(ym1.equals(ym3)); // false

const ym4 = Temporal.PlainYearMonth.from("2021-09");
console.log(ym1.equals(ym4)); // false
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.equals

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`equals`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.equals`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.compare()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.since()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/since · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`since()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.Duration` object representing the duration from another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`) to this year-month. The duration is positive if the other month is before this month, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```js
since(other)
since(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainYearMonth` instance representing a year-month to subtract from this year-month. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`. It must have the same calendar as `this`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` only accept the units: `"years"`, `"months"`, or their singular forms. For `largestUnit`, the default value `"auto"` means `"years"`. For `smallestUnit`, the default value is `"months"`. The current date is used as the `relativeTo` option.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this year-month. The duration is positive if `other` is before this year-month, and negative if after.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using since()

```js
const lastUpdated = Temporal.PlainYearMonth.from("2022-01");
const now = Temporal.Now.plainDateISO().toPlainYearMonth();
const duration = now.since(lastUpdated);
console.log(`Last updated ${duration.toLocaleString("en-US")} ago`);
// Expected output: "Last updated [number] years, [number] months ago"

const duration2 = now.since(lastUpdated, { largestUnit: "months" });
console.log(`Last updated ${duration2.toLocaleString("en-US")} ago`);
// Expected output: "Last updated [number] months ago"

const duration3 = now.since(lastUpdated, { smallestUnit: "years" });
console.log(`Last updated ${duration3.toLocaleString("en-US")} ago`);
// Expected output: "Last updated [number] years ago"
```

Rounding the result

By default the fractional part of the `smallestUnit` is truncated. You can round it up using the `roundingIncrement` and `roundingMode` options.

```js
const ym1 = Temporal.PlainYearMonth.from("2022-01");
const ym2 = Temporal.PlainYearMonth.from("2022-11");
const duration = ym2.since(ym1, {
  smallestUnit: "years",
  roundingMode: "ceil",
});
console.log(duration.toString()); // "P1Y"
```

Getting the result in days

By default, the resulting duration never contains days, because `PlainYearMonth` does not offer day-level precision. You can get the result in days by converting it to a `Temporal.PlainDate` first with an unambiguous day.

```js
const ym1 = Temporal.PlainYearMonth.from("2022-01");
const ym2 = Temporal.PlainYearMonth.from("2022-11");
const duration = ym2.toPlainDate({ day: 1 }).since(ym1.toPlainDate({ day: 1 }));
console.log(duration.toString()); // "P304D"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.since

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.since`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.Duration`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.until()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.subtract()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/subtract · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`subtract()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.PlainYearMonth` object representing this year-month moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two year-months and get a duration, use `since()` or `until()` instead.

Syntax

```js
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this year-month. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object representing the year-month specified by the original `PlainYearMonth`, minus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply. Subtracting a positive duration starts from the end of the year-month and moves backward, so any increment smaller than the month's length is ignored.

Examples

Subtracting a duration

```js
const start = Temporal.PlainYearMonth.from("2022-01");
const end = start.subtract({ years: 1, months: 2, weeks: 3, days: 4 });
console.log(end.toString()); // 2020-11
```

For more examples, see `add()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.subtract`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.subtract`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainYearMonth
- Temporal.Duration
- Temporal.PlainYearMonth.prototype.add()
- Temporal.PlainYearMonth.prototype.since()
- Temporal.PlainYearMonth.prototype.until()

Temporal.PlainYearMonth.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of Temporal.PlainYearMonth instances returns a string representing this year-month in the same RFC 9557 format as calling toString(). It is intended to be implicitly called by JSON.stringify().

Syntax

```js
toJSON()
```

Parameters

None.

Return value

A string representing the given date in the RFC 9557 format, with the calendar annotation included if it is not "iso8601".

Description

The method is automatically called by JSON.stringify() when a Temporal.PlainYearMonth object is encountered during JSON serialization. The serialized value can then be deserialized using the Temporal.PlainYearMonth.from() function within the reviver of JSON.parse().

Examples

Using toJSON()

```js
const ym = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ymStr = ym.toJSON(); // '2021-08'
const ym2 = Temporal.PlainYearMonth.from(ymStr);
```

JSON serialization and parsing

This example shows how Temporal.PlainYearMonth can be serialized as JSON without extra effort, and how to parse it back.

```js
const ym = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ymStr = JSON.stringify({ event: ym }); // '{"event":"2021-08"}'
const obj = JSON.parse(ymStr, (key, value) => {
  if (key === "event") {
    return Temporal.PlainYearMonth.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.toJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`
- `Temporal.PlainYearMonth.prototype.toString()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.PlainYearMonth` instances returns a string with a language-sensitive representation of this year-month. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Syntax

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support return the exact same string as `toString()`, ignoring both parameters.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. The `calendar` option must be provided with the same value as this year-month's calendar. Regarding the date-time component options and the style shortcuts (`dateStyle` and `timeStyle`), the options should follow one of these forms:

- Provide none of them: `year` and `month` will default to `"numeric"`.
- Provide `dateStyle` only: it expands to `era`, `year`, and `month` formats.
- Provide some date-time component options, where at least one of them is `year` or `month`. Only the specified date components will be included in the output.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given year-month according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(yearMonth)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
// Note that just specifying "2021-08" defaults to the ISO 8601 calendar,
// which throws an error if the locale's default calendar is not ISO 8601.
const ym = Temporal.PlainYearMonth.from("2021-08-01[u-ca=gregory]");

console.log(ym.toLocaleString()); // 8/2021 (assuming en-US locale and Gregorian calendar)
```

If the year-month's calendar doesn't match the locale's default calendar, even when its calendar is `iso8601`, an explicit `calendar` option must be provided with the same value.

```js
const ym = Temporal.PlainYearMonth.from("2021-08");
ym.toLocaleString("en-US", { calendar: "iso8601" }); // 2021-08
```

Using toLocaleString() with options

You can customize which parts of the year-month are included in the output by providing the `options` parameter.

```js
const ym = Temporal.PlainYearMonth.from("2021-08-01[u-ca=gregory]");
ym.toLocaleString("en-US", { dateStyle: "full" }); // August 2021
ym.toLocaleString("en-US", { year: "2-digit" }); // 21
ym.toLocaleString("en-US", { month: "long" }); // August
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.tolocalestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.toLocaleString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainYearMonth
- Intl.DateTimeFormat
- Temporal.PlainYearMonth.prototype.toJSON()
- Temporal.PlainYearMonth.prototype.toString()

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.toPlainDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/toPlainDate · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainDate()`** method of Temporal.PlainYearMonth instances returns a new Temporal.PlainDate object representing this year-month and a supplied day in the same calendar system.

Syntax

```js
toPlainDate(dayInfo)
```

Parameters

`dayInfo` Optional

An object representing the day component of the resulting `PlainDate`, containing the following property:

`day`

Corresponds to the day property.

Return value

A new `Temporal.PlainDate` object representing the date specified by this year-month and the day in `dayInfo`, interpreted in the calendar system of this year-month.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `dayInfo` is not an object.

Examples

Using toPlainDate()

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
const date = ym.toPlainDate({ day: 1 });
console.log(date.toString()); // 2021-07-01

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=chinese]");
const date2 = ym2.toPlainDate({ day: 15 });
console.log(date2.toString()); // 2021-06-24[u-ca=chinese]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.toplaindate

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toPlainDate	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.toPlainDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainDate`
- `Temporal.PlainDate.prototype.toPlainYearMonth()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/toString · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of `Temporal.PlainYearMonth` instances returns a string representing this year-month in the RFC 9557 format.

Syntax

```js
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`. The reference day is included if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation. The reference day is always included too.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.PlainYearMonth` instance, although the year-month value still remains the same. The reference day is included if the calendar is not `"iso8601"`.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself. The reference day is always included too.

Return value

A string in the RFC 9557 format representing this year-month. The calendar annotation is included as specified. The reference day is included if a calendar annotation is included or if the calendar is not `"iso8601"`.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```js
const ym = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
console.log(ym.toString()); // '2021-08'

const ym2 = Temporal.PlainYearMonth.from({
  year: 5781,
  monthCode: "M08",
  calendar: "hebrew",
});
console.log(ym2.toString()); // '2021-04-13[u-ca=hebrew]'
```

Using options

```js
const isoYM = Temporal.PlainYearMonth.from({ year: 2021, month: 8 });
const ym = Temporal.PlainYearMonth.from({
  year: 5781,
  monthCode: "M08",
  calendar: "hebrew",
});
console.log(isoYM.toString({ calendarName: "auto" })); // '2021-08'
console.log(ym.toString({ calendarName: "auto" })); // '2021-04-13[u-ca=hebrew]'
console.log(isoYM.toString({ calendarName: "always" })); // '2021-08-01[u-ca=iso8601]'
console.log(ym.toString({ calendarName: "always" })); // '2021-04-13[u-ca=hebrew]'
console.log(isoYM.toString({ calendarName: "never" })); // '2021-08'
console.log(ym.toString({ calendarName: "never" })); // '2021-04-13'
console.log(isoYM.toString({ calendarName: "critical" })); // '2021-08-01[!u-ca=iso8601]'
console.log(ym.toString({ calendarName: "critical" })); // '2021-04-13[!u-ca=hebrew]'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`
- `Temporal.PlainYearMonth.prototype.toJSON()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`

Temporal.PlainYearMonth.prototype.until()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/until · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`until()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.Duration` object representing the duration from this year-month to another year-month (in a form convertible by `Temporal.PlainYearMonth.from()`). The duration is positive if the other month is after this month, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.PlainYearMonth` instance representing a year-month to subtract this year-month from. It is converted to a `Temporal.PlainYearMonth` object using the same algorithm as `Temporal.PlainYearMonth.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this year-month *until* `other`. The duration is positive if `other` is after this year-month, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.

Examples

Using until()

```
const launch = Temporal.PlainYearMonth.from("2035-01");
const now = Temporal.Now.plainDateISO().toPlainYearMonth();
const duration = now.until(launch);
console.log(`It will be ${duration.toLocaleString("en-US")} until the launch`);
```

For more examples, see `since()`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.until`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.until`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.Duration`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.since()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.PlainYearMonth` instances throws a `TypeError`, which prevents `Temporal.PlainYearMonth` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```js
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `yearMonth1 > yearMonth2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.PlainYearMonth` instances prevent such implicit conversions. You need to explicitly convert them to strings using `Temporal.PlainYearMonth.prototype.toString()`, or use the `Temporal.PlainYearMonth.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.PlainYearMonth

All arithmetic and comparison operations on `Temporal.PlainYearMonth` instances should use the dedicated methods or convert them to primitives explicitly.

```js
const ym1 = Temporal.PlainYearMonth.from("2021-01");
const ym2 = Temporal.PlainYearMonth.from("2021-07");
ym1 > ym2; // TypeError: can't convert PlainYearMonth to primitive type
Temporal.PlainYearMonth.compare(ym1, ym2); // -1

ym2 - ym1; // TypeError: can't convert PlainYearMonth to primitive type
ym2.since(ym1).toString(); // "P6M"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.valueof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.valueOf`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.toString()`
- `Temporal.PlainYearMonth.prototype.toJSON()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/with · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`with()`** method of `Temporal.PlainYearMonth` instances returns a new `Temporal.PlainYearMonth` object representing this year-month with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the year-month's fields.

There's no obvious way to create a new `Temporal.PlainYearMonth` object that represents the same year-month in a different calendar, so to replace its `calendarId` property, you need to convert it to a `Temporal.PlainDate` object first using `toPlainDate()`, change the calendar, and then convert it back.

Syntax

```js
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.PlainYearMonth.from()` (except `calendar`): `era` and `eraYear`, `month`, `monthCode`, `year`. Unspecified properties use the values from the original year-month. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.PlainYearMonth` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The result is not in the representable range, which is $\pm(10^8 + 1)$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
const newYM = ym.with({ year: 2024 });
console.log(newYM.toString()); // "2024-07"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.plainyearmonth.prototype.with

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.with`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.from()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.calendarId

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/calendarId · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`calendarId`** accessor property of `Temporal.PlainYearMonth` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. There's no obvious way to create a new `Temporal.PlainYearMonth` object with a different calendar that represents the same year-month, so you need to convert it to a `Temporal.PlainDate` object first using `toPlainDate()`, change the calendar, and then convert it back.

Examples

Using calendarId

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.calendarId); // "iso8601"; default

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=chinese]");
console.log(ym2.calendarId); // "chinese"
```

Changing calendarId

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
const newYM = ym
  .toPlainDate({ day: 1 })
  .withCalendar("chinese")
  .toPlainYearMonth();
console.log(newYM.year, newYM.monthCode); // 2021 "M05"

const newYM2 = ym
  .toPlainDate({ day: 31 })
  .withCalendar("chinese")
  .toPlainYearMonth();
console.log(newYM2.year, newYM2.monthCode); // 2021 "M06"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.calendarid

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.calendarId`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.daysInMonth

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/daysInMonth · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInMonth`** accessor property of `Temporal.PlainYearMonth` instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

The set accessor of `daysInMonth` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInMonth`.

Examples

Using daysInMonth

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.daysInMonth); // 31
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.daysinmonth`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`daysInMonth`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.daysInMonth`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.month`
- `Temporal.PlainYearMonth.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInMonth`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.daysInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/daysInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInYear`** accessor property of `Temporal.PlainYearMonth` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

The set accessor of `daysInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInYear`.

Examples

Using daysInYear

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.daysInYear); // 365
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.daysinyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`daysInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.daysInYear`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.daysInMonth`
- `Temporal.PlainDate.prototype.daysInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.era

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/era · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`era`** accessor property of `Temporal.PlainYearMonth` instances returns a calendar-specific lowercase string representing the era of this year-month, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.era`.

Examples

Using era

```js
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.era); // undefined

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=gregory]");
console.log(ym2.era); // ce
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.era`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`era`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.era`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.eraYear`
- `Temporal.PlainDate.prototype.era`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.eraYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/eraYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`eraYear`** accessor property of `Temporal.PlainYearMonth` instances returns a non-negative integer representing the year of this year-month within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.eraYear`.

Examples

Using eraYear

```js
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.eraYear); // undefined

const ym2 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=gregory]");
console.log(ym2.eraYear); // 2021

const ym3 = Temporal.PlainYearMonth.from("-002021-07-01[u-ca=gregory]");
console.log(ym3.eraYear); // 2022; 0000 is used for the year 1 BC

const ym4 = Temporal.PlainYearMonth.from("2021-07-01[u-ca=japanese]");
console.log(ym4.eraYear); // 3
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.erayear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.eraYear`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.era`
- `Temporal.PlainDate.prototype.eraYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.inLeapYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/inLeapYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`inLeapYear`** accessor property of `Temporal.PlainYearMonth` instances returns a boolean indicating whether this year-month is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.inLeapYear`.

Examples

Using inLeapYear

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.inLeapYear); // false
console.log(ym.daysInYear); // 365
console.log(ym.monthsInYear); // 12
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.inleapyear`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`inLeapYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.inLeapYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.daysInYear`

- `Temporal.PlainYearMonth.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.inLeapYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.month

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/month · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`month`** accessor property of `Temporal.PlainYearMonth` instances returns a positive integer representing the 1-based month index in the year of this year-month. The first month of this year is `1`, and the last month is the `monthsInYear`. It is calendar-dependent.

The set accessor of `month` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.month`.

Examples

Using month

```js
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.monthCode); // "M07"
console.log(ym.month); // 7
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.month`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`month`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.month`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`

- `Temporal.PlainYearMonth.prototype.year`
- `Temporal.PlainYearMonth.prototype.monthCode`
- `Temporal.PlainYearMonth.prototype.daysInMonth`
- `Temporal.PlainYearMonth.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.month`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.monthCode

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/monthCode · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`monthCode`** accessor property of `Temporal.PlainYearMonth` instances returns a calendar-specific string representing the month of this year-month. It is calendar-dependent.

Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L` (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code `M05L` but Adar II has code `M06`). If the leap month is the first month of the year, the code is `M00L`.

The set accessor of `monthCode` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.PlainYearMonth` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.monthCode`.

Examples

Using monthCode

```js
const date = Temporal.PlainYearMonth.from("2021-07-01"); // ISO 8601 calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.monthcode`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`monthCode`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.monthCode`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainYearMonth
- Temporal.PlainYearMonth.prototype.with()
- Temporal.PlainYearMonth.prototype.add()
- Temporal.PlainYearMonth.prototype.subtract()
- Temporal.PlainYearMonth.prototype.year
- Temporal.PlainYearMonth.prototype.month
- Temporal.PlainYearMonth.prototype.daysInMonth
- Temporal.PlainYearMonth.prototype.monthsInYear
- Temporal.PlainDate.prototype.monthCode

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.monthsInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/monthsInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **monthsInYear** accessor property of Temporal.PlainYearMonth instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

The set accessor of monthsInYear is undefined. You cannot change this property directly.

For general information and more examples, see Temporal.PlainDate.prototype.monthsInYear.

Examples

Using monthsInYear

```js
const ym = Temporal.PlainYearMonth.from("2021-07");
console.log(ym.monthsInYear); // 12
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.monthsinyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
monthsInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.PlainYearMonth.monthsInYear. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.PlainYearMonth
- Temporal.PlainYearMonth.prototype.with()
- Temporal.PlainYearMonth.prototype.add()
- Temporal.PlainYearMonth.prototype.subtract()
- Temporal.PlainYearMonth.prototype.year
- Temporal.PlainYearMonth.prototype.month
- Temporal.PlainYearMonth.prototype.monthCode
- Temporal.PlainYearMonth.prototype.daysInMonth
- Temporal.PlainDate.prototype.monthsInYear

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.PlainYearMonth

Temporal.PlainYearMonth.prototype.year

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/PlainYearMonth/year · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **year** accessor property of Temporal.PlainYearMonth instances returns an integer representing the number of years of this year-month relative to the start of a calendar-specific epoch year. It is calendar-dependent.

The set accessor of year is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.PlainYearMonth object with the desired new value.

For general information and more examples, see Temporal.PlainDate.prototype.year.

Examples

Using year

```js
const ym = Temporal.PlainYearMonth.from("2021-07"); // ISO 8601 calendar
console.log(ym.year); // 2021
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.plainyearmonth.prototype.year

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
year	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.PlainYearMonth.year`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.PlainYearMonth`
- `Temporal.PlainYearMonth.prototype.with()`
- `Temporal.PlainYearMonth.prototype.add()`
- `Temporal.PlainYearMonth.prototype.subtract()`
- `Temporal.PlainYearMonth.prototype.era`
- `Temporal.PlainYearMonth.prototype.eraYear`
- `Temporal.PlainYearMonth.prototype.month`
- `Temporal.PlainDate.prototype.year`

JavaScript › Reference › Standard built-in objects › Temporal

Temporal.ZonedDateTime

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.ZonedDateTime`** object represents a date and time with a time zone. It is fundamentally represented as a combination of an instant, a time zone, and a calendar system.

Description

A `ZonedDateTime` functions as a bridge between an exact time and a wall-clock time: it simultaneously represents an instant in history (like a `Temporal.Instant`) and a local, wall-clock time (like a `Temporal.PlainDateTime`). It does so by storing the instant, the time zone, and the calendar system. The time zone is used to convert between the instant and the local time, and the calendar system is used to interpret the local time.

`ZonedDateTime` is the only `Temporal` class that is time zone-aware. The addition of a time zone makes `ZonedDateTime` objects have important behavior differences from `Temporal.PlainDateTime` objects. Namely, you can no longer assume that "the time 1 minute afterwards" is the same every day, or that a day has 24 hours. In the worst case, an entire day may be missing from the local calendar. Below, we offer a quick overview of time zones and offsets and how they affect conversion between UTC time and local time.

Time zones and offsets

All times in JavaScript have one golden standard: the UTC time, which increments continuously and uniformly as physical time progresses. By contrast, users are more interested in their local time, which is the time they read on their calendars and clocks. The process of converting between UTC time and local time involves a time zone *offset*, which is calculated as:

```
local time = UTC time + offset
```

For example, if the UTC time is 1970-01-01T00:00:00, and the offset is "-05:00", then the local time is:

```
1970-01-01T00:00:00 + -05:00 = 1969-12-31T19:00:00
```

By suffixing this local time with the offset, thus expressing this time as "1969-12-31T19:00:00-05:00", it can now be unambiguously understood as an instant in history.

To know the offset, we need two pieces of information, the *time zone*, and the *instant*. The time zone is a region on Earth where the same offset is used at all times. Two clocks in the same time zone will always show the same time simultaneously, but the offset is not necessarily constant: that is, these clocks' times may change abruptly. This commonly happens during daylight saving time transitions, where the offset changes by an hour, which happens twice a year. Offsets can also change permanently due to political changes, e.g., a country switching time zones.

The time zones are stored in the IANA Time Zone Database. Each IANA time zone has:

- A *primary time zone identifier* that uniquely identifies the time zone. It usually refers to a geographic area anchored by a city (e.g., `Europe/Paris` or `Africa/Kampala`), but can also denote single-offset time zones like `UTC` (a consistent `+00:00` offset) or `Etc/GMT+5` (which for historical reasons is a

negative offset `-05:00`). For historical reasons, the primary name for the UTC time zone is `UTC`, though in IANA it is `Etc/UTC`.

- A *time zone definition* in the form of a table that maps UTC date/time ranges (including future ranges) to specific offsets.
- Zero or more *non-primary time zone identifiers* that are aliases to the primary time zone identifier. These are usually historical names that are no longer in use, but are kept for compatibility reasons. See below for more information.

As input, named identifiers are matched case-insensitively. Internally, they are stored in their preferred case, and non-primary identifiers will *not* be converted to their primary identifier.

> **Note:** When setting the time zone name, you rarely want to set it to `"UTC"`. `ZonedDateTime` is intended to be displayed to users, but no human lives in the "UTC" time zone. If you don't know the time zone at construction time but know the wall-clock time, use a `Temporal.PlainDateTime`. If you know the exact instant, use a `Temporal.Instant`.

When a `Temporal` API accepts a *time zone identifier*, in addition to primary time zone identifiers and non-primary time zone identifiers, it also accepts an *offset time zone identifier*, which is in the same form as the offset, except subminute precision is not allowed. For example, `+05:30`, `-08`, `+0600` are all valid offsets identifiers. Internally, offset identifiers are stored in the `±HH:mm` form.

> **Note:** Avoid using offset identifiers if there is a named time zone you can use instead. Even if a region has always used a single offset, it is better to use the named identifier to guard against future political changes to the offset.
>
> If a region uses (or has used) multiple offsets, then using its named time zone is even more important. This is because `Temporal.ZonedDateTime` can use methods like `add` or `with` to create new instances at a different instant. If those derived instances correspond to an instant that uses a different offset (for example, after a Daylight Saving Time transition) then your calculations will have an incorrect local time. Using a named time zone ensures that local dates and times are always adjusted for the correct offset for that instant.

For convenience, when providing a time zone identifier to `Temporal` APIs such as `Temporal.ZonedDateTime.prototype.withTimeZone()` and the `timeZoneId` option of `Temporal.ZonedDateTime.from()`, you can provide it in a few other forms:

- As another `ZonedDateTime` instance, whose `timeZoneId` will be used.
- As an RFC 9557 string with a time zone annotation, whose time zone identifier will be used.
- As an ISO 8601 / RFC 3339 string containing an offset, whose offset will be used as an offset identifier; or, if using `Z`, then the `"UTC"` time zone is used. This usage is generally not recommended, because as discussed above, offset identifiers lack the ability to safely derive other `Temporal.ZonedDateTime` instances across an offset transition like when daylight saving time starts or ends. Instead, consider just using `Temporal.Instant` or fetching the user's actual named time zone.

The IANA time zone database changes from time to time, usually to add new time zones in response to political changes. However, on rare occasions, IANA time zone identifiers are renamed to match updated English translation of a city name or to update outdated naming conventions. For example, here are a few notable name changes:

Current IANA primary identifier	Old, now non-primary identifier
`America/Argentina/Buenos_Aires`	`America/Buenos_Aires`
`Asia/Kolkata`	`Asia/Calcutta`
`Asia/Ho_Chi_Minh`	`Asia/Saigon`
`Europe/Kyiv`	`Europe/Kiev`

Historically, these renames caused problems for programmers because the Unicode CLDR database (a library used by browsers to supply time zone identifiers and data) did not follow IANA's renaming for stability reasons. As a result, some browsers like Chrome and Safari reported CLDR's outdated identifiers, while other browsers like Firefox overrode CLDR's defaults and reported the up-to-date primary identifiers.

With the introduction of Temporal, this behavior is now more standardized:

- CLDR data now includes an `"_iana"` attribute that indicates the most up-to-date identifier if the older, stable identifier has been renamed. Browsers can use this new attribute to provide up-to-date identifiers to callers.
- Time zone identifiers provided by the programmer will never be replaced with an alias. For example, if the caller provides `Asia/Calcutta` or `Asia/Kolkata` as the identifier input to `Temporal.ZonedDateTime.from()`, then the same identifier is returned in the resulting instance's `timeZoneId`. Note that the letter case of outputs are normalized to match IANA, so that `ASIA/calCuTTa` as input generates a `timeZoneId` of `Asia/Calcutta` as output.
- When a time zone identifier is not provided by a caller but is instead sourced from the system itself (for example, when using `Temporal.Now.timeZoneId()`), modern identifiers are returned in all browsers. For city renames, there is a two-year lag before these system-provided-identifier APIs expose the new name, thereby giving other components (like a Node server) time to update their copies of the IANA database to recognize the new name.

Note that the attribution of primary identifiers preserves the country code: for example, the IANA database records `Atlantic/Reykjavik` as an alias for `Africa/Abidjan`, but because they correspond to different countries (Iceland and Côte d'Ivoire, respectively), they are treated as distinct primary identifiers.

This standardization applies outside of `Temporal` as well. For example, the `timeZone` option returned by `Intl.DateTimeFormat.prototype.resolvedOptions()` is also never replaced with an alias, though browsers have traditionally canonicalized these identifiers prior to standardization by Temporal. On the other hand, `Intl.Locale.prototype.getTimeZones()` and `Intl.supportedValuesOf()` (`timeZone` option) will return the most up-to-date identifier, while some browsers used to return the old, non-primary identifier.

RFC 9557 format

`ZonedDateTime` objects can be serialized and parsed using the RFC 9557 format, an extension to the ISO 8601 / RFC 3339 format. The string has the following form (spaces are only for readability and should not be present in the actual string):

```
YYYY-MM-DD T HH:mm:ss.sssssssss Z/±HH:mm [time_zone_id] [u-ca=calendar_id]
```

YYYY

Either a four-digit number, or a six-digit number with a `+` or `-` sign.

`MM`

A two-digit number from `01` to `12`.

`DD`

A two-digit number from `01` to `31`. The `YYYY`, `MM`, and `DD` components can be separated by `-` or nothing.

`T` Optional

The date-time separator, which can be `T`, `t`, or a space. Present if and only if `HH` is present.

`HH` Optional

A two-digit number from `00` to `23`. Defaults to `00`.

`mm` Optional

A two-digit number from `00` to `59`. Defaults to `00`.

`ss.sssssssss` Optional

A two-digit number from `00` to `59`. May optionally be followed by a `.` or `,` and one to nine digits. Defaults to `00`. The `HH`, `mm`, and `ss` components can be separated by `:` or nothing. You can omit either just `ss` or both `ss` and `mm`, so the time can be one of three forms: `HH`, `HH:mm`, or `HH:mm:ss.sssssssss`.

`Z/±HH:mm` Optional

Either the UTC designator `Z` or `z`, or an offset from UTC in the form `+` or `-` followed by the same format as the time component. Note that subminute precision (`:ss.sssssssss`) may be unsupported by other systems, and is accepted but never output. If omitted, the offset is derived from the time zone identifier. If present, then the time must be provided too. `Z` is not the same as `+00:00`: the former means that the time is given in UTC form regardless of the time zone identifier, while the latter means that the time is given in local time which happens to be UTC+0, and will be validated against the time zone identifier via the `offset` option.

`[time_zone_id]`

Replace `time_zone_id` with the time zone identifier (named or offset) as described above. May have a *critical flag* by prefixing the identifier with `!`: e.g., `[!America/New_York]`. This flag generally tells other systems that it cannot be ignored if they don't support it. Note that it is required for `Temporal.ZonedDateTime.from()`: omitting it causes a `RangeError`. If you want to parse ISO 8601 / RFC 3339 strings without time zone identifier annotations, use `Temporal.Instant.from()` instead.

`[u-ca=calendar_id]` Optional

Replace `calendar_id` with the calendar to use. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `[u-ca=iso8601]`. May have a *critical flag* by prefixing the key with `!`: e.g., `[!u-ca=iso8601]`. This flag generally tells other systems that it cannot be ignored if they don't support it. The `Temporal` parser will throw an error if the annotations contain two or more calendar annotations and one of them is critical. Note that the `YYYY-MM-DD` is always interpreted as an ISO 8601 calendar date and then converted to the specified calendar.

As an input, other annotations in the `[key=value]` format are ignored, and they must not have the critical flag.

When serializing, you can configure the fractional second digits, whether to display the offset/time zone ID/calendar ID, and whether to add a critical flag for the annotations.

Ambiguity and gaps from local time to UTC time

Given a time zone, conversion from UTC to local time is straightforward: you first get the offset using the time zone name and the instant, then add the offset to the instant. The reverse is not true:

conversion from local time to UTC time, without an explicit offset, is ambiguous, because one local time can correspond to zero, one, or many UTC times. Consider the most common cause: daylight saving time transitions. Take New York as an example. Its standard offset is UTC-5, but during DST, all clocks are set forward by an hour, so the offset becomes UTC-4. In the US, transitions happen at 2:00 AM local time, so consider these two transition days:

UTC time	New York time
2024-03-10T06:58:00Z	2024-03-10T01:58:00-05:00
2024-03-10T06:59:00Z	2024-03-10T01:59:00-05:00
2024-03-10T07:00:00Z	2024-03-10T03:00:00-04:00
---	---
2024-11-03T05:58:00Z	2024-11-03T01:58:00-04:00
2024-11-03T05:59:00Z	2024-11-03T01:59:00-04:00
2024-11-03T06:00:00Z	2024-11-03T01:00:00-05:00

As you can see, in March, one hour disappeared from the local time, and in November, we have two hours that have the same wall-clock time. Suppose that we stored a `PlainDateTime` that says "2024-03-10T02:05:00", and we want to interpret it in the `America/New_York` time zone, there will be no time that corresponds to it, while a `PlainDateTime` that says "2024-11-03T01:05:00" can correspond to two different instants.

When constructing a `ZonedDateTime` from a local time (using `Temporal.ZonedDateTime.from()`, `Temporal.ZonedDateTime.prototype.with()`, `Temporal.PlainDateTime.prototype.toZonedDateTime()`), the behavior for ambiguity and gaps is configurable via the `disambiguation` option:

`earlier`
: If there are two possible instants, choose the earlier one. If there is a gap, go back by the gap duration.

`later`
: If there are two possible instants, choose the later one. If there is a gap, go forward by the gap duration.

`compatible` (default)
: Same behavior as `Date`: use `later` for gaps and `earlier` for ambiguities.

`reject`
: Throw a `RangeError` whenever there is an ambiguity or a gap.

There are several cases where there's no ambiguity when constructing a `ZonedDateTime`:

- If the time is specified in UTC via the `Z` offset.
- If the offset is explicitly provided and used (see below).

Offset ambiguity

We already demonstrated how ambiguity may arise from interpreting a local time in a time zone, without providing an explicit offset. However, if you provide an explicit offset, then another conflict arises: between the offset as specified, and the offset as calculated from the time zone and the local time. This is an unavoidable real-world issue: if you store a time in the future, with an anticipated offset, then before that time comes, the time zone definition may have changed due to political reasons. For

example, suppose in 2018, we set a reminder at the time `2019-12-23T12:00:00-02:00[America/Sao_Paulo]` (which is a daylight saving time; Brazil is in the southern hemisphere, so it enters DST in October and exits in February). But before that time comes, in early 2019, Brazil decides to stop observing DST, so the real offset becomes `-03:00`. Should the reminder now still fire at noon (keeping the local time), or should it fire at 11:00 AM (keeping the exact time)?

For offset ambiguity to exist, a timestamp string must be parsed using different IANA Time Zone Database rules than the rules used when the timestamp was originally generated. This will never happen when timestamps are generated during the same execution of a JavaScript program, because the ECMAScript specification requires that IANA Time Zone Database rules must be consistent for the lifetime of a JavaScript program.

However, offset ambiguity may exist when a JavaScript program is parsing timestamps that were stored earlier, as in the `America/Sao_Paulo` example above, and the IANA Time Zone Database has been updated since the timestamp was originally generated. It can also happen when communicating timestamps between computers (or, rarely, between different software on the same computer!) that use different versions of the IANA Time Zone Database. The IANA Time Zone Database also has build options (for example, using or not using deprecated rules in `backzone`) that can cause offset ambiguity when communicating timestamps between computers using different software, even when the IANA Time Zone Database version is the same.

Offset ambiguity is rarely encountered and will almost always apply only to timestamps before 1970 or for timestamps that are months or years in the future. But when this problem happens, a `RangeError` will be thrown by default. When constructing a `ZonedDateTime` with `Temporal.ZonedDateTime.from()` or when updating it using the `with()` method, you can prevent this exception by using the `offset` option to decide whether the offset or the time zone identifier "wins":

`use`
: Use the offset to calculate the exact time. This option "uses" the offset to decide the instant intended by the timestamp string, even if the offset at that instant has changed. The time zone identifier is still used to then infer the (possibly updated) offset and use that offset to convert the exact time to local time. In the `2019-12-23T12:00:00-02:00[America/Sao_Paulo]` example above, this option would cause the reminder to fire at 11:00 AM local time.

`ignore`
: Use the time zone identifier to re-calculate the offset, ignoring the offset that's specified in the string. This option keeps the same local time as originally calculated when we stored the time, but may result in a different instant. Note that by ignoring the offset, the same local time interpretation ambiguity may arise, which is resolved using the `disambiguation` option. In the `2019-12-23T12:00:00-02:00[America/Sao_Paulo]` example above, this option would cause the reminder to fire at 12:00 PM local time.

`reject`
: Throw a `RangeError` whenever there is a conflict between the offset and the time zone identifier. This is the default for `Temporal.ZonedDateTime.from()`.

`prefer`
: Use the offset if it's valid, otherwise calculate the offset from the time zone identifier. This is the default for `Temporal.ZonedDateTime.prototype.with()` (see that method for more details). This is different from `ignore` because in the case of local time ambiguity, the offset is used to resolve it rather than the `disambiguation` option.

If you know in advance how you want to handle offset ambiguity, consider using the `offset` option to avoid exceptions that are thrown by default. For example, a calendar application likely wants the time zone identifier to "win" so that recurring meetings are shown in the most up-to-date local time for that time zone, so `offset: "ignore"` is appropriate. On the other hand, a task scheduler application that executes a task exactly 3 hours from now should probably pick `offset: "use"` because changes to time zone rules shouldn't change the meaning of "3 hours from now".

In some cases, you may not know which `offset` option is the correct one without getting input from the user. In these cases, you may want to catch the `RangeError` and then ask your user which local time is the correct one, and then retry parsing using a different `offset` option corresponding to the user's choice.

Note that the `Z` offset is not equivalent to `+00:00`. The `Z` offset means "the time in UTC is known, but the offset to local time is unknown", as per RFC 9557. When the time string uses the `Z` offset, the `offset` option is ignored, and the offset is derived from the time zone ID. On the other hand, the `+00:00` offset is interpreted as a local time offset that happens to match UTC and is validated against the time zone ID.

> **Note:** Although `Temporal.Instant.from()` also takes an RFC 9557 string in the same form, there is no ambiguity because it always ignores the time zone identifier and reads the offset only.

Constructor

`Temporal.ZonedDateTime()`

Creates a new `Temporal.ZonedDateTime` object by directly supplying the underlying data.

Static methods

`Temporal.ZonedDateTime.compare()`

Returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. Equivalent to comparing the `epochNanoseconds` of the two date-times.

`Temporal.ZonedDateTime.from()`

Creates a new `Temporal.ZonedDateTime` object from another `Temporal.ZonedDateTime` object, an object with date, time, and time zone properties, or an RFC 9557 string.

Instance properties

These properties are defined on `Temporal.ZonedDateTime.prototype` and shared by all `Temporal.ZonedDateTime` instances.

`Temporal.ZonedDateTime.prototype.calendarId`

Returns a string representing the calendar used to interpret the internal ISO 8601 date.

`Temporal.ZonedDateTime.prototype.constructor`

The constructor function that created the instance object. For `Temporal.ZonedDateTime` instances, the initial value is the `Temporal.ZonedDateTime()` constructor.

`Temporal.ZonedDateTime.prototype.day`

Returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. Calendar-dependent. Generally starts at 1 and is continuous, but not always.

Temporal.ZonedDateTime.prototype.dayOfWeek

Returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. Calendar-dependent. 1 usually represents Monday in the calendar, even when locales using the calendar may consider a different day as the first day of the week (see `Intl.Locale.prototype.getWeekInfo()`).

Temporal.ZonedDateTime.prototype.dayOfYear

Returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is `1`, and the last day is the `daysInYear`. Calendar-dependent.

Temporal.ZonedDateTime.prototype.daysInMonth

Returns a positive integer representing the number of days in the month of this date. Calendar-dependent.

Temporal.ZonedDateTime.prototype.daysInWeek

Returns a positive integer representing the number of days in the week of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 7, but in other calendar systems it may differ from week to week.

Temporal.ZonedDateTime.prototype.daysInYear

Returns a positive integer representing the number of days in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is 365, or 366 in a leap year.

Temporal.ZonedDateTime.prototype.epochMilliseconds

Returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. Equivalent to dividing `epochNanoseconds` by `1e6` and flooring the result.

Temporal.ZonedDateTime.prototype.epochNanoseconds

Returns a `BigInt` representing the number of nanoseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

Temporal.ZonedDateTime.prototype.era

Returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent. For Gregorian, it is either `"ce"` or `"bce"`.

Temporal.ZonedDateTime.prototype.eraYear

Returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. Calendar-dependent.

Temporal.ZonedDateTime.prototype.hour

Returns an integer from 0 to 23 representing the hour component of this time.

Temporal.ZonedDateTime.prototype.hoursInDay

Returns a positive integer representing the number of hours in the day of this date in the time zone. It may be more or less than 24 in the case of offset changes such as daylight saving time.

Temporal.ZonedDateTime.prototype.inLeapYear

Returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. Calendar-dependent.

`Temporal.ZonedDateTime.prototype.microsecond`

Returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

`Temporal.ZonedDateTime.prototype.millisecond`

Returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

`Temporal.ZonedDateTime.prototype.minute`

Returns an integer from 0 to 59 representing the minute component of this time.

`Temporal.ZonedDateTime.prototype.month`

Returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. Calendar-dependent. Note that unlike `Date.prototype.getMonth()`, the index is 1-based. If the calendar has leap months, then the month with the same `monthCode` may have different `month` indexes for different years.

`Temporal.ZonedDateTime.prototype.monthCode`

Returns a calendar-specific string representing the month of this date. Calendar-dependent. Usually it is `M` plus a two-digit month number. For leap months, it is the previous month's code followed by `L`. If the leap month is the first month of the year, the code is `M00L`.

`Temporal.ZonedDateTime.prototype.monthsInYear`

Returns a positive integer representing the number of months in the year of this date. Calendar-dependent. For the ISO 8601 calendar, this is always 12, but in other calendar systems it may differ.

`Temporal.ZonedDateTime.prototype.nanosecond`

Returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

`Temporal.ZonedDateTime.prototype.offset`

Returns a string representing the offset used to interpret the internal instant, in the form `±HH:mm` (or `±HH:mm:ss.sssssssss` with as much subminute precision as necessary).

`Temporal.ZonedDateTime.prototype.offsetNanoseconds`

Returns an integer representing the offset used to interpret the internal instant, as a number of nanoseconds (positive or negative).

`Temporal.ZonedDateTime.prototype.second`

Returns an integer from 0 to 59 representing the second component of this time.

`Temporal.ZonedDateTime.prototype.timeZoneId`

Returns a string representing the time zone identifier used to interpret the internal instant. It uses the same string used when constructing the `Temporal.ZonedDateTime` object, which is either an IANA time zone name or a fixed offset.

`Temporal.ZonedDateTime.prototype.weekOfYear`

Returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. Calendar-dependent. Note that for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year.

`Temporal.ZonedDateTime.prototype.year`

Returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. Calendar-dependent. Usually year 1 is either the first year of the latest era or the

ISO 8601 year `0001`. If the epoch is in the middle of the year, that year will have the same value before and after the start date of the era.

`Temporal.ZonedDateTime.prototype.yearOfWeek`

Returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. Calendar-dependent. Usually this is the year of the date, but for ISO 8601, the first and last few days of the year may be attributed to the last week of the previous year or the first week of the next year, causing the `yearOfWeek` to differ by 1.

`Temporal.ZonedDateTime.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Temporal.ZonedDateTime"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Temporal.ZonedDateTime.prototype.add()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.ZonedDateTime.prototype.equals()`

Returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`), and `false` otherwise. They are compared both by their instant values, time zones, and their calendars, so two date-times from different calendars or time zones may be considered equal by `Temporal.ZonedDateTime.compare()` but not by `equals()`.

`Temporal.ZonedDateTime.prototype.getTimeZoneTransition()`

Returns a `Temporal.ZonedDateTime` object representing the first instant after or before this instant at which the time zone's UTC offset changes, or `null` if there is no such transition. This is useful for finding out the offset rules of a time zone, such as its daylight saving time pattern.

`Temporal.ZonedDateTime.prototype.round()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time rounded to the given unit.

`Temporal.ZonedDateTime.prototype.since()`

Returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`) to this date-time. The duration is positive if the other date-time is before this date-time, and negative if after.

`Temporal.ZonedDateTime.prototype.startOfDay()`

Returns a `Temporal.ZonedDateTime` object representing the first instant of this date in the time zone. It usually has a time of `00:00:00`, but may be different if the midnight doesn't exist due to offset changes, in which case the first time that exists is returned.

`Temporal.ZonedDateTime.prototype.subtract()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

`Temporal.ZonedDateTime.prototype.toInstant()`

Returns a new `Temporal.Instant` object representing the instant of this date-time.

`Temporal.ZonedDateTime.prototype.toJSON()`

Returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. Intended to be implicitly called by `JSON.stringify()`.

`Temporal.ZonedDateTime.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this date-time.

`Temporal.ZonedDateTime.prototype.toPlainDate()`

Returns a new `Temporal.PlainDate` object representing the date portion of this date-time.

`Temporal.ZonedDateTime.prototype.toPlainDateTime()`

Returns a new `Temporal.PlainDateTime` object representing the date and time portions of this date-time. Only the time zone information is removed.

`Temporal.ZonedDateTime.prototype.toPlainTime()`

Returns a new `Temporal.PlainTime` object representing the time portion of this date-time.

`Temporal.ZonedDateTime.prototype.toString()`

Returns a string representing this date-time in the RFC 9557 format.

`Temporal.ZonedDateTime.prototype.until()`

Returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

`Temporal.ZonedDateTime.prototype.valueOf()`

Throws a `TypeError`, which prevents `Temporal.ZonedDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

`Temporal.ZonedDateTime.prototype.with()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time with some fields replaced by new values.

`Temporal.ZonedDateTime.prototype.withCalendar()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time interpreted in the new calendar system.

`Temporal.ZonedDateTime.prototype.withPlainTime()`

Returns a new `Temporal.ZonedDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`)

`Temporal.ZonedDateTime.prototype.withTimeZone()`

Returns a new `Temporal.ZonedDateTime` object representing the same instant as this date-time but in the new time zone.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal-zoneddatetime-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
ZonedDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
ZonedDateTime() constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
calendarId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
compare	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
dayOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
daysInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
epochMilliseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
epochNanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
era	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
from	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
getTimeZoneTransition	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
hour	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
hoursInDay	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
inLeapYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
microsecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
millisecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
minute	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
month	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
monthsInYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
nanosecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
offset	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
offsetNanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
second	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
startOfDay	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
subtract	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
timeZoneId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toInstant	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toJSON	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toLocaleString	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainDate	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
toPlainDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`toPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`valueOf`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`weekOfYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`with`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withCalendar`	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`withTimeZone`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`year`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

1. Partial support.
2. Calendars `islamic`, `islamic-rgsa` and `islamic-umalqura` are not available. See bug 1950425 and bug 1954138.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal`
- `Temporal.Duration`
- `Temporal.Instant`
- `Temporal.PlainDateTime`
- `Temporal.PlainDate`
- `Temporal.PlainTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/ZonedDateTime · MDN page last modified 2026-09-18

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.ZonedDateTime()`** constructor creates `Temporal.ZonedDateTime` objects.

This constructor allows you to create instances by directly supplying the underlying data. Like all other `Temporal` classes, you should usually construct `Temporal.ZonedDateTime` objects using the `Temporal.ZonedDateTime.from()` static method, which can handle a variety of input types.

Syntax

```js
new Temporal.ZonedDateTime(epochNanoseconds, timeZone)
new Temporal.ZonedDateTime(epochNanoseconds, timeZone, calendar)
```

> **Note:** `Temporal.ZonedDateTime()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`epochNanoseconds`

A BigInt that corresponds to the `epochNanoseconds` property.

`timeZone`

A string that corresponds to the `timeZoneId` property. Unlike all other `Temporal` APIs, it must be a valid time zone identifier (either named or offset) as-is, and cannot be a `Temporal.ZonedDateTime` instance or a date-time string.

`calendar` Optional

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types. Defaults to `"iso8601"`.

Return value

A new `Temporal.ZonedDateTime` object, representing the specific instant specified by the parameters.

Exceptions

`TypeError`

Thrown if `timeZone` or `calendar` is not a string.

`RangeError`

Thrown in one of the following cases:

- `epochNanoseconds` is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.
- `timeZone` is not a valid time zone identifier.
- `calendar` is not a valid calendar identifier.

Examples

Using Temporal.ZonedDateTime()

```js
const zdt = new Temporal.ZonedDateTime(0n, "America/New_York");
console.log(zdt.toString()); // '1969-12-31T19:00:00-05:00[America/New_York]'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal-zoneddatetime-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ZonedDateTime()` constructor	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.ZonedDateTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.from()`

Temporal.ZonedDateTime.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/compare · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.ZonedDateTime.compare()`** static method returns a number (-1, 0, or 1) indicating whether the first date-time comes before, is the same as, or comes after the second date-time. It is equivalent to comparing the `epochNanoseconds` of the two date-times.

Syntax

```js
Temporal.ZonedDateTime.compare(dateTime1, dateTime2)
```

Parameters

`dateTime1`

A string, an object, or a `Temporal.ZonedDateTime` instance representing the first date-time to compare. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`.

`dateTime2`

The second date-time to compare, converted to a `Temporal.ZonedDateTime` object using the same algorithm as `dateTime1`.

Return value

Returns `-1` if `dateTime1` comes before `dateTime2`, `0` if they are the same, and `1` if `dateTime1` comes after `dateTime2`. They are compared by their underlying instant values, ignoring their calendars or time zones.

Examples

Using Temporal.ZonedDateTime.compare()

```js
const dt1 = Temporal.ZonedDateTime.from("2021-08-01T01:00:00[Europe/London]");
const dt2 = Temporal.ZonedDateTime.from("2021-08-02T00:00:00[Europe/London]");
console.log(Temporal.ZonedDateTime.compare(dt1, dt2)); // -1

const dt3 = Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Europe/London]");
console.log(Temporal.ZonedDateTime.compare(dt1, dt3)); // 1
```

Sorting an array of date-times

The purpose of this `compare()` function is to act as a comparator to be passed to `Array.prototype.sort()` and related functions.

```js
const dateTimes = [
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[America/New_York]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Asia/Hong_Kong]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Europe/London]"),
];

dateTimes.sort(Temporal.ZonedDateTime.compare);
console.log(dateTimes.map((d) => d.toString()));
// [ "2021-08-01T00:00:00+08:00[Asia/Hong_Kong]", "2021-08-
01T00:00:00+01:00[Europe/London]", "2021-08-01T00:00:00-04:00[America/New_York]" ]
```

Note that they are compared by their instant values. In the very rare case where you want to compare them by their wall-clock times, convert them to `PlainDateTime` first.

```js
const dateTimes = [
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[America/New_York]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Asia/Hong_Kong]"),
  Temporal.ZonedDateTime.from("2021-08-01T00:00:00[Europe/London]"),
];

dateTimes.sort((a, b) =>
  Temporal.PlainDateTime.compare(a.toPlainDateTime(), b.toPlainDateTime()),
);
console.log(dateTimes.map((d) => d.toString()));
// [ "2021-08-01T00:00:00-04:00[America/New_York]", "2021-08-
01T00:00:00+08:00[Asia/Hong_Kong]", "2021-08-01T00:00:00+01:00[Europe/London]" ]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.compare

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`compare`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.compare`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.equals()`

Temporal.ZonedDateTime.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/from · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Temporal.ZonedDateTime.from()`** static method creates a new `Temporal.ZonedDateTime` object from another `Temporal.ZonedDateTime` object, an object with date, time, and time zone properties, or an RFC 9557 string.

Syntax

```
Temporal.ZonedDateTime.from(info)
Temporal.ZonedDateTime.from(info, options)
```

Parameters

`info`

One of the following:

- A `Temporal.ZonedDateTime` instance, which creates a copy of the instance.
- An RFC 9557 format string containing a date, optionally a time, optionally an offset, a time zone annotation, and optionally a calendar.
- An object containing properties that are accepted by either `Temporal.PlainDate.from()` (`calendar`, `era`, `eraYear`, `year`, `month`, `monthCode`, `day`) or `Temporal.PlainTime.from()` (`hour`, `minute`, `second`, `millisecond`, `microsecond`, `nanosecond`). The info should explicitly specify a year (as `year` or as `era` and `eraYear`), a month (as `month` or `monthCode`), and a day; others are optional and will be set to their default values. The following properties should be provided too:

  `timeZone`

  Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone

identifier or an offset (see time zones and offsets for more information). The time properties are interpreted in this time zone.

`offset` Optional

An offset string, in the same format as the RFC 9557 offset but with optional seconds and subsecond components (`±HH:mm:ss.sssssssss`), representing the offset from UTC. If omitted, it will be calculated from the time zone and the date-time. `"Z"` is not allowed.

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`disambiguation` Optional

What to do if the local date-time is ambiguous in the given time zone (there are more than one instants with such local time, or the local time does not exist). Possible values are `"compatible"`, `"earlier"`, `"later"`, and `"reject"`. Defaults to `"compatible"`. For more information about these values, see ambiguity and gaps from local time to UTC time.

`offset` Optional

What to do if the offset is explicitly provided in `info` but the offset is invalid for the given time zone in the given local time. Possible values are `"use"`, `"ignore"`, `"reject"`, and `"prefer"`. Defaults to `"reject"`. For more information about these values, see offset ambiguity.

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object, representing the date and time specified by `info` in the specified `calendar` and `timeZone`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object or a string.
- `options` is not an object or `undefined`.
- The provided properties are insufficient to unambiguously determine a date. You usually need to provide a `year` (or `era` and `eraYear`), a `month` (or `monthCode`), and a `day`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.

- The wall-clock time is ambiguous in the time zone, and `options.disambiguation` is set to `"reject"`.
- The info is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Creating a ZonedDateTime from an object

```js
// Year + month + day + hour + minute + second
const zdt = Temporal.ZonedDateTime.from({
  timeZone: "America/New_York",
  year: 2021,
  month: 7,
  day: 1,
  hour: 12,
  minute: 34,
  second: 56,
});
console.log(zdt.toString()); // "2021-07-01T12:34:56-04:00[America/New_York]"
```

Creating a ZonedDateTime from a string

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56-04:00[America/New_York]",
);
console.log(zdt.toLocaleString()); // "7/1/2021, 12:34:56 PM EDT" (assuming en-US locale)

// Time given as UTC, and converted to local
const zdt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56Z[America/New_York]",
);
console.log(zdt2.toString()); // "2021-07-01T08:34:56-04:00[America/New_York]"
```

Creating a ZonedDateTime from an ISO 8601 / RFC 3339 string

Note that `Temporal.ZonedDateTime.from()` rejects ISO 8601 strings, which do not include a time zone identifier. This is to ensure that the time zone is always known and can be used to derive different offsets as the local time changes.

If you want to parse an ISO 8601 string, first construct a `Temporal.Instant` object and then convert it to a `Temporal.ZonedDateTime` object. You can provide any time zone, even if it doesn't match the offset originally given in the string, and the local time will be adjusted accordingly.

```js
const isoString = "2021-07-01T12:34:56+02:00";
const instant = Temporal.Instant.from(isoString);
const zdt = instant.toZonedDateTimeISO("America/New_York");
console.log(zdt.toString()); // "2021-07-01T06:34:56-04:00[America/New_York]"
```

Local time disambiguation

See ambiguity and gaps from local time to UTC time for an introduction to this situation.

```js
const localTimeNotExist = "2024-03-10T02:05:00[America/New_York]";
// For non-existent times, "compatible" is equivalent to "later"
const zdt = Temporal.ZonedDateTime.from(localTimeNotExist);
console.log(zdt.toString()); // "2024-03-10T03:05:00-04:00[America/New_York]"

const zdt2 = Temporal.ZonedDateTime.from(localTimeNotExist, {
  disambiguation: "earlier",
});
console.log(zdt2.toString()); // "2024-03-10T01:05:00-05:00[America/New_York]"

const localTimeAmbiguous = "2024-11-03T01:05:00[America/New_York]";
// For ambiguous times, "compatible" is equivalent to "earlier"
const zdt3 = Temporal.ZonedDateTime.from(localTimeAmbiguous);
console.log(zdt3.toString()); // "2024-11-03T01:05:00-04:00[America/New_York]"

const zdt4 = Temporal.ZonedDateTime.from(localTimeAmbiguous, {
  disambiguation: "later",
});
console.log(zdt4.toString()); // "2024-11-03T01:05:00-05:00[America/New_York]"
```

Resolving offset ambiguity

See offset ambiguity for an introduction to this situation.

```js
const offsetAmbiguous = "2019-12-23T12:00:00-02:00[America/Sao_Paulo]";

Temporal.ZonedDateTime.from(offsetAmbiguous);
// RangeError: date-time can't be represented in the given time zone
Temporal.ZonedDateTime.from(offsetAmbiguous, { offset: "use" }).toString();
// "2019-12-23T11:00:00-03:00[America/Sao_Paulo]"
Temporal.ZonedDateTime.from(offsetAmbiguous, { offset: "ignore" }).toString();
// "2019-12-23T12:00:00-03:00[America/Sao_Paulo]"
```

For more examples, especially regarding different calendars and overflow settings, see `Temporal.PlainDate.from()` and `Temporal.PlainTime.from()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.from`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime()`
- `Temporal.ZonedDateTime.prototype.with()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/add · MDN page last modified 2026-08-11

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`add()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time moved forward by a given duration (in a form convertible by `Temporal.Duration.from()`).

Syntax

```js
add(duration)
add(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to add to this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object representing the date-time specified by the original `ZonedDateTime`, plus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

For how calendar durations are added, see `Temporal.PlainDate.prototype.add()`.

Addition and subtraction are performed according to rules defined in RFC 5545 (iCalendar):

- Add/subtract the date portion of a duration using calendar arithmetic; in other words, add the date portion to its `PlainDateTime` using `Temporal.PlainDateTime.prototype.add()`, and then interpret the result in the same time zone. The result will automatically adjust for daylight saving time using the rules of this instance's `timeZone` field. For example, `2024-11-03T01:00:00-04:00[America/New_York]` plus one day is `2024-11-04T01:00:00-05:00[America/New_York]`, as if the day has 25 hours.
  - If the date-time is ambiguous or invalid due to a time zone offset transition, it is resolved using the `disambiguation: "compatible"` behavior: the later of the two possible instants will be used for time-skipped transitions and the earlier of the two possible instants will be used for time-repeated transitions. For example, `2024-03-09T02:05:00-05:00[America/New_York]` plus one day is supposedly `2024-03-10T02:05:00-05:00[America/New_York]`, but this time doesn't exist, so the wall-clock time one hour after, `2024-03-10T03:05:00-04:00[America/New_York]`, is returned. Similarly, both `2024-11-02T01:00:00-04:00[America/New_York]` plus one day and `2024-11-04T01:00:00-05:00[America/New_York]` minus one day produce `2024-11-03T01:00:00-04:00[America/New_York]`, the earlier of the two possible instants.
  - If the resulting date-time's components are out of bounds, they are resolved using the `overflow` option. For example, `2024-08-31` plus one month is `2024-09-31` which doesn't exist, so it is clamped to `2024-09-30` by default.
- Add/subtract the time portion of a duration using real-world time; in other words, add the time portion to its `Instant` using `Temporal.Instant.prototype.add()`, and then interpret the result in the same time zone. For example, `2024-11-03T01:00:00-04:00[America/New_York]` plus one hour is `2024-11-03T01:00:00-05:00[America/New_York]`.

These rules make arithmetic with `Temporal.ZonedDateTime` "DST-safe", which means that the results most closely match the expectations of both real-world users and implementers of other standards-compliant calendar applications. These expectations include:

- Adding or subtracting days should keep clock time consistent across DST transitions. For example, if you have an appointment on Saturday at 1:00PM and you ask to reschedule it 1 day later, you would expect the reschedule appointment to still be at 1:00PM, even if there was a DST transition overnight.
- Adding or subtracting the time portion of a duration should ignore DST transitions. For example, a friend you've asked to meet in 2 hours will be annoyed if you show up 1 hour or 3 hours later. There should be a consistent and relatively-unsurprising order of operations.
- If results are at or near a DST transition, ambiguities should be handled automatically (no crashing) and deterministically.

Adding a duration is equivalent to subtracting its negation.

Examples

Adding a duration

```js
const start = Temporal.ZonedDateTime.from(
  "2021-11-01T12:34:56-04:00[America/New_York]",
);
const end = start.add({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2023-01-26T17:41:03.008-05:00[America/New_York]
```

For more examples, especially with how different calendars and the `overflow` option interact with calendar durations, see `Temporal.PlainDate.prototype.add()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.add`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.subtract()`

Temporal.ZonedDateTime.prototype.equals()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/equals · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`equals()`** method of `Temporal.ZonedDateTime` instances returns `true` if this date-time is equivalent in value to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`), and `false` otherwise. They are compared both by their instant values, time zones, and their calendars, so two date-times from different calendars or time zones may be considered equal by `Temporal.ZonedDateTime.compare()` but not by `equals()`.

Syntax

```
equals(other)
```

Parameters

`other`

A string, an object, or a `Temporal.ZonedDateTime` instance representing the other date-time to compare. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`.

Return value

`true` if this date-time is equal to `other` both in their instant value, time zone, and their calendar, `false` otherwise.

Note that the time zones are canonicalized before comparison, so if their time zone IDs are both named and identify the same time zone, then they would be considered the same even when the exact names may be aliases of each other. Offset identifiers are compared by the offset values they represent. Offset identifiers never compare equal to named identifiers even when the named identifier's time zone always uses that offset.

Examples

Using equals()

```
// Asia/Kolkata and Asia/Calcutta are aliases of each other
const dt1 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56+05:30[Asia/Kolkata]",
);
const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56+05:30[Asia/Calcutta]",
);
console.log(dt1.equals(dt2)); // true
```

(code continues on the next page)

(code continued from the previous page)

```
const dt3 = Temporal.ZonedDateTime.from("2021-07-01T12:34:56+05:30[+05:30]");
console.log(dt1.equals(dt3)); // false

const dt4 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56+05:30[Asia/Kolkata][u-ca=buddhist]",
);
console.log(dt1.equals(dt4)); // false
```

Testing if two time zone identifiers are equivalent

js

```
function sameTimeZone(timeZone1, timeZone2) {
  const dt1 = Temporal.ZonedDateTime.from({
    year: 2021,
    month: 7,
    day: 1,
    timeZone: timeZone1,
  });
  const dt2 = Temporal.ZonedDateTime.from({
    year: 2021,
    month: 7,
    day: 1,
    timeZone: timeZone2,
  });
  return dt1.equals(dt2);
}

console.log(sameTimeZone("Asia/Kolkata", "Asia/Calcutta")); // true
console.log(sameTimeZone("Asia/Shanghai", "Asia/Taipei")); // false
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.equals

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
equals	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.equals`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.compare()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.getTimeZoneTransition()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/getTimeZoneTransition · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`getTimeZoneTransition()`** method of `Temporal.ZonedDateTime` instances returns a `Temporal.ZonedDateTime` object representing the closest instant after or before this instant at which the time zone's UTC offset changes (the returned instant is the first instant after the change), or `null` if there is no such transition. This is useful for finding out the offset rules of a time zone, such as its daylight saving time pattern.

Note that instants returned about the future may be unreliable, for example due to changes to the time zone definitions.

Syntax

```js
getTimeZoneTransition(direction)
getTimeZoneTransition(options)
```

Parameters

`direction`

A string representing the `direction` option. This is a convenience overload, so `getTimeZoneTransition(direction)` is equivalent to `getTimeZoneTransition({ direction })`, where `direction` is a string.

`options`

An object containing the following property:

`direction`

Whether to search before or after the current instant. Must be one of `"next"` or `"previous"`.

Return value

A `Temporal.ZonedDateTime` object with instant `t`, such that:

- The time zone offset at `t` is different from the offset one nanosecond before `t`.
- `t < this.epochNanoseconds` if `direction` is `"previous"`, or `t > this.epochNanoseconds` if `direction` is `"next"`.
- For all instants between `this.epochNanoseconds` and `t`, exclusive, the offset is constant.

If there is no such transition, `null` is returned.

Examples

Finding the next time zone transition

```js
const dt = Temporal.ZonedDateTime.from("2024-01-01T00-05:00[America/New_York]");
const transition = dt.getTimeZoneTransition("next");
console.log(transition.toString()); // "2024-03-10T03:00:00-04:00[America/New_York]"

const transition2 = transition.getTimeZoneTransition("next");
console.log(transition2.toString()); // "2024-11-03T01:00:00-05:00[America/New_York]"

const transition3 = dt.getTimeZoneTransition("previous");
console.log(transition3.toString()); // "2023-11-05T01:00:00-05:00[America/New_York]"

const dt2 = Temporal.ZonedDateTime.from("2024-01-01T00Z[UTC]");
console.log(dt2.getTimeZoneTransition("next")); // null
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.gettimezonetransition

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getTimeZoneTransition	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.getTimeZoneTransition`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.timeZoneId`
- `Temporal.ZonedDateTime.prototype.offset`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.round()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/round · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`round()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time rounded to the given unit.

Syntax

```js
round(smallestUnit)
round(options)
```

Parameters

`smallestUnit`

A string representing the `smallestUnit` option. This is a convenience overload, so `round(smallestUnit)` is equivalent to `round({ smallestUnit })`, where `smallestUnit` is a string.

`options`

An object containing some or all of the following properties (in the order they are retrieved and validated):

`roundingIncrement` Optional

A number (truncated to an integer) representing the rounding increment in the given `smallestUnit`. Defaults to `1`. For all values of `smallestUnit` except `"day"`, the increment must be a divisor of the maximum value of the unit; for example, if the unit is hours, the increment must be a divisor of 24 and must not be 24 itself, which means it can be 1, 2, 3, 4, 6, 8, or 12. For `"day"`, the increment must be 1.

`roundingMode` Optional

A string specifying how to round off the fractional part of `smallestUnit`. See `Intl.NumberFormat()`. Defaults to `"halfExpand"`.

`smallestUnit`

A string representing the smallest unit to include in the output. The value must be one of the following: `"day"`, `"hour"`, `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, `"nanosecond"`, or their plural forms. For units larger than `"nanosecond"`, fractional parts of the `smallestUnit` will be rounded according to the `roundingIncrement` and `roundingMode` settings.

Return value

A new `Temporal.ZonedDateTime` object representing this date-time rounded to the given unit, where all units smaller than `smallestUnit` are zeroed out.

If `smallestUnit` is `"day"`, the returned date-time will be the start of day of this date or the next day, depending on the `roundingMode` and the distance to these two instants. Otherwise, the rounding is first performed on its `PlainDateTime` (same as `Temporal.PlainDateTime.prototype.round()`), and then re-interpreted in the same time zone, with `disambiguation: "compatible", offset: "prefer"`. See ambiguity and gaps from local time to UTC time and offset ambiguity.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

Examples

Rounding off small units

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789[America/New_York]",
);
const nearestMillisecond = zdt.round("millisecond");
console.log(nearestMillisecond.toString()); // 2021-07-01T12:34:56.123-
04:00[America/New_York]

const nearestHalfHour = zdt.round({
  smallestUnit: "minute",
  roundingIncrement: 30,
});
console.log(nearestHalfHour.toString()); // 2021-07-01T12:30:00-04:00[America/New_York]

const nextDay = zdt.round({ smallestUnit: "day", roundingMode: "ceil" });
console.log(nextDay.toString()); // 2021-07-02T00:00:00-04:00[America/New_York]
```

Ambiguity after rounding

It's possible that the rounded date-time is ambiguous in the given time zone. The ambiguity is always resolved using `disambiguation: "compatible", offset: "prefer"`. Here's a quick example:

```js
const zdt = Temporal.ZonedDateTime.from(
  "2024-03-10T01:00:00-05:00[America/New_York]",
);
const rounded = zdt.round({ smallestUnit: "hour", roundingIncrement: 2 });
// The result is supposed to be 2024-03-10T02:00:00-05:00[America/New_York],
// but this time does not exist. `disambiguation: "compatible"` tells us to move
// forward by 1 hour.
console.log(rounded.toString()); // 2024-03-10T03:00:00-04:00[America/New_York]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.round

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
round	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.round`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`

Temporal.ZonedDateTime.prototype.since()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/since · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`since()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.Duration` object representing the duration from another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`) to this date-time. The duration is positive if the other date-time is before this date-time, and negative if after.

This method does `this - other`. To do `other - this`, use the `until()` method.

Syntax

```js
since(other)
since(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.ZonedDateTime` instance representing a date-time to subtract from this date-time. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

An object containing the options for `Temporal.Duration.prototype.round()`, which includes `largestUnit`, `roundingIncrement`, `roundingMode`, and `smallestUnit`. `largestUnit` and `smallestUnit` accept all possible units. For `largestUnit`, the default value `"auto"` means `"hours"` or `smallestUnit`, whichever is greater. For `smallestUnit`, the default value is `"nanoseconds"`. The current date is used as the `relativeTo` option. Note that using units larger than `"hours"` may make the duration not portable to other calendars, dates, or time zones.

Return value

A new `Temporal.Duration` object representing the duration *since* `other` to this date-time. The duration is positive if `other` is before this date-time, and negative if after.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.
- `other` has a different time zone than `this`, and `largestUnit` is `"days"` or above.

Description

The duration returned is a "hybrid" duration. This means that the duration's date portion represents full calendar days like `Temporal.PlainDateTime.prototype.since()` would return, while its time portion represents real-world elapsed time like `Temporal.Instant.prototype.since()` would return. This "hybrid duration" approach automatically adjusts for DST and matches widely-adopted industry standards like RFC 5545 (iCalendar). See below for examples.

Examples

Offset transitions

When transitions happen, a day may not have exactly 24 hours.

```js
const start = Temporal.ZonedDateTime.from(
  "2024-11-03T01:00:00-04:00[America/New_York]",
);
const end = Temporal.ZonedDateTime.from(
  "2024-11-04T01:00:00-05:00[America/New_York]",
);
console.log(end.since(start).toString()); // PT25H
console.log(end.since(start, { largestUnit: "days" }).toString()); // PT1D

const start2 = Temporal.ZonedDateTime.from(
  "2024-03-10T01:00:00-05:00[America/New_York]",
);
const end2 = Temporal.ZonedDateTime.from(
  "2024-03-11T01:00:00-04:00[America/New_York]",
);
console.log(end2.since(start2).toString()); // PT23H
console.log(end2.since(start2, { largestUnit: "days" }).toString()); // PT1D
```

For this reason, the returned duration is purely time-based with no date portion by default, so that it stays unambiguous.

Different time zones

The time portion of the returned duration is purely based on instants and is not affected by time zones. However, if you want to include any date units like `day`, then the start and end must be in the same time zone.

```js
const start = Temporal.ZonedDateTime.from(
  "2024-11-03T01:00:00-04:00[America/New_York]",
);
// Peru does not use DST so its offset remains -05:00 year-round
const end = Temporal.ZonedDateTime.from(
  "2024-11-04T01:00:00-05:00[America/Lima]",
);

end.since(start); // PT25H
end.since(start, { largestUnit: "days" }); // RangeError: time zones "America/Lima" and
"America/New_York" aren't compatible

end.withTimeZone("America/New_York").since(start, { largestUnit: "days" }); // P1D
end.since(start.withTimeZone("America/Lima"), { largestUnit: "days" }); // P1D1H
```

For more examples about how to use `since()`, especially with rounding, see `Temporal.PlainDate.prototype.since()` and `Temporal.PlainTime.prototype.since()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.since

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
since	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.since`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.until()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.startOfDay()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/startOfDay · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`startOfDay()`** method of `Temporal.ZonedDateTime` instances returns a `Temporal.ZonedDateTime` object representing the first instant of this date in the time zone. It usually has a time of `00:00:00`, but may be different if the midnight doesn't exist due to offset changes, in which case the first time that exists is returned.

It is equivalent to calling `withPlainTime()` with no arguments.

Syntax

```js
startOfDay()
```

Parameters

None.

Return value

A `Temporal.ZonedDateTime` object with instant `t`, such that:

- The date at `t` is different from the date one nanosecond before `t`.
- The date at `t` is the same as the date of `this`.

Examples

Using startOfDay()

```js
// In the US, DST transitions happen at 2am, so the midnight exists
const dt = Temporal.ZonedDateTime.from(
  "2024-03-10T12:00:00-04:00[America/New_York]",
);
console.log(dt.startOfDay().toString()); // "2024-03-10T00:00:00-05:00[America/New_York]"

// In Brazil, DST transitions happened at midnight, so the midnight didn't exist
const dt2 = Temporal.ZonedDateTime.from(
  "2015-10-18T12:00-02:00[America/Sao_Paulo]",
);
console.log(dt2.startOfDay().toString()); // "2015-10-18T01:00:00-02:00[America/Sao_Paulo]"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.startofday

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`startOfDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.startOfDay`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.hour`
- `Temporal.ZonedDateTime.prototype.hoursInDay`
- `Temporal.ZonedDateTime.prototype.timeZoneId`
- `Temporal.ZonedDateTime.prototype.getTimeZoneTransition()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.subtract()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/subtract · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`subtract()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time moved backward by a given duration (in a form convertible by `Temporal.Duration.from()`).

If you want to subtract two date-times and get a duration, use `since()` or `until()` instead.

Syntax

```js
subtract(duration)
subtract(duration, options)
```

Parameters

`duration`

A string, an object, or a `Temporal.Duration` instance representing a duration to subtract from this date-time. It is converted to a `Temporal.Duration` object using the same algorithm as `Temporal.Duration.from()`.

`options` Optional

An object containing the following property:

`overflow` Optional

A string specifying the behavior when a date component is out of range. Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object representing the date-time specified by the original `ZonedDateTime`, minus the duration.

Exceptions

`RangeError`

Thrown if the result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Description

Subtracting a duration is equivalent to adding its negation, so all the same considerations apply.

Examples

Subtracting a duration

```js
const start = Temporal.ZonedDateTime.from(
  "2021-01-01T12:34:56-05:00[America/New_York]",
);
const end = start.subtract({
  years: 1,
  months: 2,
  weeks: 3,
  days: 4,
  hours: 5,
  minutes: 6,
  seconds: 7,
  milliseconds: 8,
});
console.log(end.toString()); // 2019-10-07T07:28:48.992-04:00[America/New_York]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.subtract

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`subtract`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.subtract`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.since()`
- `Temporal.ZonedDateTime.prototype.until()`

Temporal.ZonedDateTime.prototype.toInstant()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/toInstant · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toInstant()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.Instant` object representing the instant of this date-time.

Syntax

```
toInstant()
```

Parameters

None.

Return value

A new `Temporal.Instant` object representing the instant of this date-time.

Examples

Using toInstant()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
const instant = zdt.toInstant();
console.log(instant.toString()); // 2021-07-01T16:34:56.987654321Z
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.toinstant`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toInstant`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.toInstant`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.Instant`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`
- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`
- `Temporal.Instant.prototype.toZonedDateTimeISO()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/toJSON · MDN page last modified 2026-10-30

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toJSON()`** method of `Temporal.ZonedDateTime` instances returns a string representing this date-time in the same RFC 9557 format as calling `toString()`. It is intended to be implicitly called by `JSON.stringify()`.

Syntax

```js
toJSON()
```

Parameters

None.

Return value

A string representing the given date-time in the RFC 9557 format, with the calendar annotation included if it is not `"iso8601"`, and the offset and time zone annotation always included.

Description

The method is automatically called by `JSON.stringify()` when a `Temporal.ZonedDateTime` object is encountered during JSON serialization. The serialized value can then be deserialized using the `Temporal.ZonedDateTime.from()` function within the reviver of `JSON.parse()`.

Examples

Using toJSON()

```js
const zdt = Temporal.ZonedDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  timeZone: "America/New_York",
});
const zdtStr = zdt.toJSON(); // '2021-08-01T00:00:00-04:00[America/New_York]'
const zdt2 = Temporal.ZonedDateTime.from(zdtStr);
```

JSON serialization and parsing

This example shows how `Temporal.ZonedDateTime` can be serialized as JSON without extra effort, and how to parse it back.

```js
const zdt = Temporal.ZonedDateTime.from({
  year: 2021,
  month: 8,
  day: 1,
  timeZone: "America/New_York",
});
const jsonStr = JSON.stringify({ meeting: zdt }); // '{"meeting":"2021-08-01T00:00:00-
04:00[America/New_York]"}'
const obj = JSON.parse(jsonStr, (key, value) => {
  if (key === "meeting") {
    return Temporal.ZonedDateTime.from(value);
  }
  return value;
});
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.tojson

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toJSON`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.toJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.toString()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/toLocaleString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toLocaleString()`** method of `Temporal.ZonedDateTime` instances returns a string with a language-sensitive representation of this date-time. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat` and passes this date-time converted to a `Temporal.Instant` (because `Intl.DateTimeFormat` cannot directly format a `Temporal.ZonedDateTime`).

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context. However, currently `Intl.DateTimeFormat` does not support formatting `Temporal.ZonedDateTime` objects, so you must convert them to `Temporal.Instant` objects first before passing them to `format()`.

Syntax

```js
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the Intl.DateTimeFormat API, these parameters correspond exactly to the Intl.DateTimeFormat() constructor's parameters. Implementations without Intl.DateTimeFormat support return the exact same string as toString(), ignoring both parameters.

locales Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the locales parameter of the Intl.DateTimeFormat() constructor.

options Optional

An object adjusting the output format. Corresponds to the options parameter of the Intl.DateTimeFormat() constructor. If this date-time's calendar is not "iso8601", the calendar option must be provided with the same value; otherwise, if this date-time's calendar is "iso8601", the calendar option can be any value. The timeZone option must not be provided, as it is automatically set to be the date-time's timeZoneId. Regarding the date-time component options and the style shortcuts (dateStyle and timeStyle), the options should follow one of these forms:

- Provide none of them: year, month, day, hour, minute, and second will default to "numeric".
- Provide at least one of dateStyle or timeStyle: the date-time components will be set according to the specified style and the locale.
- Provide some date-time component options. Only the specified date-time components will be included in the output.

See the Intl.DateTimeFormat() constructor for details on these parameters and how to use them.

Return value

A string representing the given date-time according to language-specific conventions.

In implementations with Intl.DateTimeFormat, this is equivalent to new Intl.DateTimeFormat(locales, { ...options, timeZone: dateTime.timeZoneId }).format(dateTime.toInstant()), where options has been normalized as described above.

> **Note:** Most of the time, the formatting returned by toLocaleString() is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of toLocaleString() to hardcoded constants.

Exceptions

RangeError

Thrown if any of the options is invalid.

TypeError

Thrown if any of the options is not of the expected type.

Examples

Using toLocaleString()

Basic use of this method without specifying a locale returns a formatted string in the default locale and with default options.

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56-04:00[America/New_York]",
);

console.log(zdt.toLocaleString()); // 8/1/2021, 12:34:56 PM EDT (assuming en-US locale)
```

If the date's calendar doesn't match the locale's default calendar, and the date's calendar is not `iso8601`, an explicit `calendar` option must be provided with the same value.

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56+09:00[Asia/Tokyo][u-ca=japanese]",
);
// The ja-JP locale uses the Gregorian calendar by default
zdt.toLocaleString("ja-JP", { calendar: "japanese" }); // R3/8/1 12:34:56 JST
```

Using toLocaleString() with options

You can customize which parts of the date are included in the output by providing the `options` parameter.

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56+09:00[Asia/Tokyo][u-ca=japanese]",
);
zdt.toLocaleString("ja-JP", {
  calendar: "japanese",
  dateStyle: "full",
  timeStyle: "full",
}); // 令和3年8月1日日曜日 12時34分56秒 日本標準時
zdt.toLocaleString("ja-JP", {
  calendar: "japanese",
  year: "numeric",
  month: "long",
  hour: "numeric",
  timeZoneName: "shortGeneric",
}); // 令和3年8月 12時 JST
zdt.toLocaleString("ja-JP", {
  calendar: "japanese",
  year: "numeric",
  hour: "numeric",
  minute: "numeric",
}); // 令和3年 12:34
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.tolocalestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Intl.DateTimeFormat`
- `Temporal.ZonedDateTime.prototype.toJSON()`
- `Temporal.ZonedDateTime.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.toPlainDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/toPlainDate · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainDate()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.PlainDate` object representing the date portion of this date-time.

Syntax

```
toPlainDate()
```

Parameters

None.

Return value

A new `Temporal.PlainDate` object representing the date portion of this date-time.

Examples

Using toPlainDate()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
const plainDate = zdt.toPlainDate();
console.log(plainDate.toString()); // 2021-07-01
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.toplaindate`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainDate`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.toPlainDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.PlainDate`
- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`
- `Temporal.ZonedDateTime.prototype.toInstant()`
- `Temporal.PlainDate.prototype.toZonedDateTime()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.toPlainDateTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/toPlainDateTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainDateTime()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.PlainDateTime` object representing the date and time portions of this date-time. Only the time zone information is removed.

> **Warning:** After a `Temporal.ZonedDateTime` is converted to `Temporal.PlainDateTime`, it's no longer time-zone-aware. Subsequent operations like arithmetic or `with()` operations will not adjust for DST and may not yield the same results as equivalent operations with the original `Temporal.ZonedDateTime`. However, unless you perform those operations across a time zone offset transition, it's impossible to notice the difference. Therefore, be very careful when performing this conversion because subsequent results may be correct most of the time, but only turn out incorrect when moving across offset transitions like when DST starts or ends.

Syntax

```
toPlainDateTime()
```

Parameters

None.

Return value

A new `Temporal.PlainDateTime` object representing the date and time portions of this date-time.

Examples

Using toPlainDateTime()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
const plainDateTime = zdt.toPlainDateTime();
console.log(plainDateTime.toString()); // 2021-07-01T12:34:56.987654321
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.toplaindatetime

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toPlainDateTime	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.toPlainDateTime`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.PlainDateTime`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`
- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.prototype.toInstant()`
- `Temporal.PlainDateTime.prototype.toZonedDateTime()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.toPlainTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/toPlainTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toPlainTime()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.PlainTime` object representing the time portion of this date-time.

> **Warning:** After a `Temporal.ZonedDateTime` is converted to `Temporal.PlainTime`, it's no longer time-zone-aware. Subsequent operations like arithmetic or `with()` operations will not adjust for DST and may not yield the same results as equivalent operations with the original `Temporal.ZonedDateTime`. However, unless you perform those operations across a time zone offset transition, it's impossible to notice the difference. Therefore, be very careful when performing this conversion because subsequent results may be correct most of the time, but only turn out incorrect when moving across offset transitions like when DST starts or ends.

Syntax

```js
toPlainTime()
```

Parameters

None.

Return value

A new `Temporal.PlainTime` object representing the time portion of this date-time.

Examples

Using toPlainTime()

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.987654321-04:00[America/New_York]",
);
const plainTime = zdt.toPlainTime();
console.log(plainTime.toString()); // 12:34:56.987654321
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.toplaintime

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.toPlainTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.PlainTime`
- `Temporal.ZonedDateTime.prototype.toPlainDate()`
- `Temporal.ZonedDateTime.prototype.toPlainDateTime()`
- `Temporal.ZonedDateTime.prototype.toInstant()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/toString · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toString()`** method of `Temporal.ZonedDateTime` instances returns a string representing this date-time in the RFC 9557 format.

Syntax

```js
toString()
toString(options)
```

Parameters

`options` Optional

An object containing the following property:

`calendarName` Optional

Whether to show the calendar annotation (`[u-ca=calendar_id]`) in the return value. Possible values are:

`"auto"` (default)

Include the calendar annotation if the calendar is not `"iso8601"`.

`"always"`

Always include the calendar annotation.

`"never"`

Never include the calendar annotation. This makes the returned string not recoverable to the same `Temporal.ZonedDateTime` instance, although the date value still remains the same.

`"critical"`

Always include the calendar annotation, and add a critical flag: `[!u-ca=calendar_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

`fractionalSecondDigits` Optional

Either an integer from 0 to 9, or the string `"auto"`. The default is `"auto"`. If `"auto"`, then trailing zeros are removed from the fractional seconds. Otherwise, the fractional part of the second component contains this many digits, padded with zeros or rounded as necessary.

`roundingMode` Optional

A string specifying how to round off fractional second digits beyond `fractionalSecondDigits`. See `Intl.NumberFormat()`. Defaults to `"trunc"`.

`smallestUnit` Optional

A string specifying the smallest unit to include in the output. Possible values are `"minute"`, `"second"`, `"millisecond"`, `"microsecond"`, and `"nanosecond"`, or their plural forms, which (except `"minute"`) are equivalent to `fractionalSecondDigits` values of `0`, `3`, `6`, `9`, respectively. If specified, then `fractionalSecondDigits` is ignored.

`timeZoneName` Optional

Whether to show the time zone name (`[time_zone_id]`) in the return value. Possible values are:

`"auto"` (default)

Always include the time zone name.

`"never"`

Never include the time zone name. This makes the returned string not recoverable to the same `Temporal.ZonedDateTime` instance.

`"critical"`

Always include the time zone name, and add a critical flag: `[!time)zone_id]`. Useful when sending the string to certain systems, but not useful for Temporal itself.

`offset` Optional

Whether to show the offset (`±HH:mm`) in the return value. Possible values are:

`"auto"` (default)

Always include the offset.

`"never"`

Never include the offset. This makes the returned string not recoverable to the same `Temporal.ZonedDateTime` instance, if the time zone is included but the time is ambiguous, or if the time zone is also not included.

Return value

A string in the RFC 9557 format representing this date-time. The offset and calendar/time zone annotations are included as specified.

Exceptions

`RangeError`

Thrown if any of the options is invalid.

`TypeError`

Thrown if `options` is not an object or `undefined`.

Examples

Using toString()

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56[America/New_York]",
);
console.log(zdt.toString()); // '2021-08-01T12:34:56-04:00[America/New_York]'
```

Even for the `UTC` time zone, the offset is `+00:00`, not `Z`:

```js
const zdt = Temporal.ZonedDateTime.from("2021-08-01T12:34:56[UTC]");
console.log(zdt.toString()); // '2021-08-01T12:34:56+00:00[UTC]'
```

Using options

For examples with rounding times, see `Temporal.PlainTime.prototype.toString()`. For examples with displaying calendars, see `Temporal.PlainDate.prototype.toString()`. Here we show controlling the display of time zone and offset:

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-08-01T12:34:56[America/New_York]",
);
console.log(zdt.toString({ timeZoneName: "auto", offset: "never" })); // '2021-08-
01T12:34:56[America/New_York]'
console.log(zdt.toString({ timeZoneName: "never", offset: "auto" })); // '2021-08-
01T12:34:56-04:00'
console.log(zdt.toString({ timeZoneName: "never", offset: "never" })); // '2021-08-
01T12:34:56'
console.log(zdt.toString({ timeZoneName: "critical", offset: "never" })); // '2021-08-
01T12:34:56[!America/New_York]'
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.toJSON()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.until()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/until · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`until()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.Duration` object representing the duration from this date-time to another date-time (in a form convertible by `Temporal.ZonedDateTime.from()`). The duration is positive if the other date-time is after this date-time, and negative if before.

This method does `other - this`. To do `this - other`, use the `since()` method.

Syntax

```js
until(other)
until(other, options)
```

Parameters

`other`

A string, an object, or a `Temporal.ZonedDateTime` instance representing a date-time to subtract this date-time from. It is converted to a `Temporal.ZonedDateTime` object using the same algorithm as `Temporal.ZonedDateTime.from()`. It must have the same calendar as `this`.

`options` Optional

The same options as `since()`.

Return value

A new `Temporal.Duration` object representing the duration from this date-time *until* `other`. The duration is positive if `other` is after this date-time, and negative if before.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `other` has a different calendar than `this`.
- Any of the options is invalid.
- `other` has a different time zone than `this`, and `largestUnit` is `"days"` or above.

Examples

Using until()

```js
const flight = Temporal.ZonedDateTime.from(
  "2024-12-21T13:31:00-05:00[America/New_York]",
);
const now = Temporal.Now.zonedDateTimeISO("America/New_York").round("second");
if (Temporal.ZonedDateTime.compare(flight, now) < 0) {
  console.error(
    "The flight is already in the past. The result may not make sense.",
  );
}
const duration = now.until(flight, { largestUnit: "days" });
console.log(`The flight is in ${duration.toLocaleString("en-US")}`);
```

For more examples, see `since()`.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.until

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`until`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.until`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.Duration`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`

- `Temporal.ZonedDateTime.prototype.since()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/valueOf · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`valueOf()`** method of `Temporal.ZonedDateTime` instances throws a `TypeError`, which prevents `Temporal.ZonedDateTime` instances from being implicitly converted to primitives when used in arithmetic or comparison operations.

Syntax

```js
valueOf()
```

Parameters

None.

Return value

None.

Exceptions

`TypeError`

Always thrown.

Description

Because both primitive conversion and number conversion call `valueOf()` before `toString()`, if `valueOf()` is absent, then an expression like `yearMonth1 > yearMonth2` would implicitly compare them as strings, which may have unexpected results. By throwing a `TypeError`, `Temporal.ZonedDateTime` instances prevent such implicit conversions. You need to explicitly convert them to numbers using `Temporal.ZonedDateTime.prototype.epochNanoseconds`, or use the `Temporal.ZonedDateTime.compare()` static method to compare them.

Examples

Arithmetic and comparison operations on Temporal.ZonedDateTime

All arithmetic and comparison operations on `Temporal.ZonedDateTime` instances should use the dedicated methods or convert them to primitives explicitly.

```js
const zdt1 = Temporal.ZonedDateTime.from(
  "2022-01-01T00:00:00[America/New_York]",
);
const zdt2 = Temporal.ZonedDateTime.from(
  "2022-07-01T00:00:00[America/New_York]",
);
zdt1 > zdt2; // TypeError: can't convert ZonedDateTime to primitive type
Temporal.ZonedDateTime.compare(zdt1, zdt2); // -1

zdt2 - zdt1; // TypeError: can't convert ZonedDateTime to primitive type
zdt2.since(zdt1).toString(); // "PT4343H"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.valueof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
valueOf	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.toString()`
- `Temporal.ZonedDateTime.prototype.toJSON()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/with · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`with()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time with some fields replaced by new values. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's fields.

To replace the `calendarId` property, use the `withCalendar()` method. To replace the `timeZoneId` property, use the `withTimeZone()` method.

Syntax

```js
with(info)
with(info, options)
```

Parameters

`info`

An object containing at least one of the properties recognized by `Temporal.ZonedDateTime.from()` (except `calendar` and `timeZone`): `day`, `era` and `eraYear`, `hour`, `microsecond`, `millisecond`, `minute`, `month`, `monthCode`, `nanosecond`, `offset`, `second`, `year`. Unspecified properties use the values from the original date-time. You only need to provide one of `month` or `monthCode`, and one of `era` and `eraYear` or `year`, and the other will be updated accordingly.

`options` Optional

An object containing some or all of the following properties (in the order they are retrieved and validated):

`disambiguation` Optional

What to do if the local date-time is ambiguous in the given time zone (there are more than one instants with such local time, or the local time does not exist). Possible values are `"compatible"`, `"earlier"`, `"later"`, and `"reject"`. Defaults to `"compatible"`. For more information about these values, see ambiguity and gaps from local time to UTC time.

`offset` Optional

What to do if the offset is explicitly provided in `info` but the offset is invalid for the given time zone in the given local time. Possible values are `"use"`, `"ignore"`, `"reject"`, and `"prefer"`. Defaults to `"prefer"`. For more information about these values, see offset ambiguity.

`overflow` Optional

A string specifying the behavior when a date component is out of range (when using the object `info`). Possible values are:

`"constrain"` (default)

The date component is clamped to the valid range.

`"reject"`

A `RangeError` is thrown if the date component is out of range.

Return value

A new `Temporal.ZonedDateTime` object, where the fields specified in `info` that are not `undefined` are replaced by the corresponding values, and the rest of the fields are copied from the original date-time.

Exceptions

`TypeError`

Thrown in one of the following cases:

- `info` is not an object.
- `options` is not an object or `undefined`.

`RangeError`

Thrown in one of the following cases:

- The provided properties that specify the same component are inconsistent.
- The provided non-numerical properties are not valid; for example, if `monthCode` is never a valid month code in this calendar.
- The provided numerical properties are out of range, and `options.overflow` is set to `"reject"`.
- The wall-clock time represented by the provided properties is ambiguous in the time zone, and `options.disambiguation` is set to `"reject"`.
- The result is not in the representable range, which is $\pm 10^8$ days, or about ±273,972.6 years, from the Unix epoch.

Examples

Using with()

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56[America/New_York]",
);
const newZDT = zdt.with({ hour: 13 });
console.log(newZDT.toString()); // "2021-07-01T13:34:56-04:00[America/New_York]"
```

For more examples, see the documentation for the individual properties that can be set using `with()`.

Offset during date changes

By default, the `offset` option is set to `"prefer"`, which means we use the original offset (or that provided in `info`) if it's valid, and recalculate otherwise. This means if you set to another date that has a different offset due to a DST transition, the offset will be recalculated:

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ month: 12 });
// The offset is recalculated to -05:00
console.log(newZDT.toString()); // "2021-12-01T12:00:00-05:00[America/New_York]"
```

And if you set the time to within the DST transition, the offset is used to resolve the ambiguity:

```js
const zdt = Temporal.ZonedDateTime.from(
  "2024-11-02T01:05:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ day: 3 });
console.log(newZDT.toString()); // "2024-11-03T01:05:00-04:00[America/New_York]"

const zdt2 = Temporal.ZonedDateTime.from(
  "2024-11-04T01:05:00-05:00[America/New_York]",
);
const newZDT2 = zdt2.with({ day: 3 });
console.log(newZDT2.toString()); // "2024-11-03T01:05:00-05:00[America/New_York]"
```

If you use `offset: "use"`, then the offset will be used as-is to obtain the exact time first, and *then* recalculate the offset:

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ month: 12 }, { offset: "use" });
// The offset is recalculated to -05:00, but the wall-clock time changes
console.log(newZDT.toString()); // "2021-12-01T11:00:00-05:00[America/New_York]"
```

You can also set `offset: "reject"` to throw an error if the original offset is invalid, forcing an explicit new offset to be specified:

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-04:00[America/New_York]",
);
zdt.with({ month: 12 }, { offset: "reject" });
// RangeError: date-time can't be represented in the given time zone
zdt.with({ month: 12, offset: "-05:00" }, { offset: "reject" }).toString();
// "2021-12-01T12:00:00-05:00[America/New_York]"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.with

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
with	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.with`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.withCalendar()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.withPlainTime()`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.withCalendar()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/withCalendar · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`withCalendar()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time interpreted in the new calendar system. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's `calendarId` property.

To replace the date-time component properties, use the `with()` method. To replace its time zone, use the `withTimeZone()` method.

Syntax

```
withCalendar(calendar)
```

Parameters

`calendar`

A string that corresponds to the `calendarId` property. See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

Return value

A new `Temporal.ZonedDateTime` object, representing the date-time specified by the original `ZonedDateTime`, interpreted in the new calendar system.

Exceptions

`TypeError`

Thrown if `calendar` is not a string.

`RangeError`

Thrown if `calendar` is not a valid calendar identifier.

Examples

Using withCalendar()

```
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56[America/New_York]",
);
const newZDT = zdt.withCalendar("islamic-umalqura");
console.log(newZDT.toLocaleString("en-US", { calendar: "islamic-umalqura" }));
// 11/21/1442 AH, 12:34:56 PM EDT
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.withcalendar`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`withCalendar`	144	144	139[1,2]	128	Prev.	144	139[1,2]	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

1. Partial support.
2. Calendars `islamic`, `islamic-rgsa` and `islamic-umalqura` are not available. See bug 1950425 and bug 1954138.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.withCalendar`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.withPlainTime()`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.calendarId`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.withPlainTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/withPlainTime · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`withPlainTime()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing this date-time with the time part entirely replaced by the new time (in a form convertible by `Temporal.PlainTime.from()`)

This method will replace all time properties, defaulting to `0` where properties are unspecified. If you only want to replace some of the time properties, use the `with()` method instead.

Syntax

```js
withPlainTime()
withPlainTime(plainTime)
```

Parameters

plainTime Optional

A string, an object, or a `Temporal.PlainTime` instance representing the new time. It is converted to a `Temporal.PlainTime` object using the same algorithm as `Temporal.PlainTime.from()`. If not specified, the time part is set to the start of the day (which is usually `00:00:00` unless it doesn't exist due to offset transitions). Disambiguation always happens in the `"compatible"` mode; if you want to use a different mode, use the `with()` method instead.

Return value

A new `Temporal.ZonedDateTime` object, with the date part and the time zone copied from the original date-time and the time part replaced by the new time.

Examples

Using withPlainTime()

```js
const zdt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56[America/New_York]",
);

// You can pass a string
const newZDT = zdt.withPlainTime("13:45:00");
console.log(newZDT.toString()); // "2021-07-01T13:45:00-04:00[America/New_York]"

// You can only specify some time properties, and the rest default to 0;
// for the with() method, they would be copied from the original date-time
const newZDT2 = zdt.withPlainTime({ hour: 13 });
console.log(newZDT2.toString()); // "2021-07-01T13:00:00-04:00[America/New_York]"

// You can pass nothing to set the time to midnight
const newZDT3 = zdt.withPlainTime();
console.log(newZDT3.toString()); // "2021-07-01T00:00:00-04:00[America/New_York]"

// But, if midnight doesn't exist, it may be a different time
const zdt2 = Temporal.ZonedDateTime.from(
  "2015-10-18T12:00-02:00[America/Sao_Paulo]",
);
console.log(zdt2.withPlainTime().toString()); // "2015-10-18T01:00:00-
02:00[America/Sao_Paulo]"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.withplaintime

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`withPlainTime`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.withPlainTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withCalendar()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.toPlainTime()`
- `Temporal.ZonedDateTime.from()`

Temporal.ZonedDateTime.prototype.withTimeZone()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/withTimeZone · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`withTimeZone()`** method of `Temporal.ZonedDateTime` instances returns a new `Temporal.ZonedDateTime` object representing the same instant as this date-time but in the new time zone. Because all `Temporal` objects are designed to be immutable, this method essentially functions as the setter for the date-time's `timeZoneId` property.

To replace the date-time component properties, use the `with()` method. To replace its calendar, use the `withCalendar()` method.

Syntax

```
withTimeZone(timeZone)
```

Parameters

`timeZone`

Either a string or a `Temporal.ZonedDateTime` instance representing the time zone to use. If a `Temporal.ZonedDateTime` instance, its time zone is used. If a string, it can be a named time zone identifier, an offset time zone identifier, or a date-time string containing a time zone identifier or an offset (see time zones and offsets for more information).

Return value

A new `Temporal.ZonedDateTime` object representing the same instant as this date-time but in the new time zone.

Exceptions

`TypeError`

Thrown if `timeZone` is not a string or a `Temporal.ZonedDateTime` instance.

`RangeError`

Thrown if the time zone name is invalid.

Examples

Using withTimeZone()

```js
const meetingTime = Temporal.ZonedDateTime.from(
  "2021-08-01T12:00[America/New_York]",
);
const meetingTimeInParis = meetingTime.withTimeZone("Europe/Paris");
console.log(meetingTimeInParis.toString()); // 2021-08-01T18:00:00+02:00[Europe/Paris]
```

Replacing the time zone while keeping the same wall-clock time

In the rare case where you want to keep the wall-clock time the same but change the time zone (and result in a different instant), convert it to a `Temporal.PlainDateTime` first:

```js
const meetingTime = Temporal.ZonedDateTime.from(
  "2021-08-01T12:00[America/New_York]",
);
const meetingTimeInParis = meetingTime
  .toPlainDateTime()
  .toZonedDateTime("Europe/Paris");
console.log(meetingTimeInParis.toString()); // 2021-08-01T12:00:00+02:00[Europe/Paris]
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-temporal.zoneddatetime.prototype.withtimezone`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`withTimeZone`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.withTimeZone`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withCalendar()`
- `Temporal.ZonedDateTime.prototype.withPlainTime()`
- `Temporal.ZonedDateTime.from()`
- `Temporal.ZonedDateTime.prototype.timeZoneId`

Temporal.ZonedDateTime.prototype.calendarId

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/calendarId · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`calendarId`** accessor property of `Temporal.ZonedDateTime` instances returns a string representing the calendar used to interpret the internal ISO 8601 date.

See `Intl.supportedValuesOf()` for a list of commonly supported calendar types.

The set accessor of `calendarId` is `undefined`. You cannot change this property directly. Use the `withCalendar()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

Examples

Using calendarId

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T08:00:00-04:00[America/New_York]",
);
console.log(dt.calendarId); // "iso8601"; default

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T08:00:00+08:00[Asia/Shanghai][u-ca=chinese]",
);
console.log(dt2.calendarId); // "chinese"

const dt3 = dt2.withCalendar("hebrew");
console.log(dt3.calendarId); // "hebrew"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.calendarid

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`calendarId`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.calendarId`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.day

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/day · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **day** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based day index in the month of this date, which is the same day number you would see on a calendar. It is calendar-dependent.

The set accessor of `day` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.day`.

For `PlainDate`, `day` can only be non-continuous if the calendar skips days. For `ZonedDateTime`, `day` can also be non-continuous if the time zone changes its offset by 24 hours; this actually happened. See the example below.

Examples

Using day

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.day); // 1
```

Non-continuous day

To better align times with its trading partners in Asia, the country of Samoa changed its time zone to the other side of the International Date Line, shifting its offset from -10:00 to +14:00 (daylight saving time). This resulted in a 24-hour abrupt change in the local time, therefore skipping the day December 30, 2011 entirely. `2011-12-29T23:59:59-10:00[Pacific/Apia]` is immediately followed by `2011-12-31T00:00:00+14:00[Pacific/Apia]`.

```js
const dt = Temporal.ZonedDateTime.from(
  "2011-12-29T23:59:59-10:00[Pacific/Apia]",
);
console.log(dt.day); // 29
const nextDay = dt.add({ seconds: 1 });
console.log(nextDay.day); // 31
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `day` property.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.day`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
day	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.day`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.dayOfYear`
- `Temporal.PlainDate.prototype.day`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.dayOfWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/dayOfWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **dayOfWeek** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based day index in the week of this date. Days in a week are numbered sequentially from `1` to `daysInWeek`, with each number mapping to its name. It is calendar-dependent.

The set accessor of `dayOfWeek` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `dayOfWeek` value, use the `add()` or `subtract()` method with the appropriate number of `days`.

For general information and more examples, see `Temporal.PlainDate.prototype.dayOfWeek`.

Examples

Using dayOfWeek

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.dayOfWeek); // 4; Thursday
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.dayofweek

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
dayOfWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.dayOfWeek`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.day`
- `Temporal.ZonedDateTime.prototype.dayOfYear`

- Temporal.ZonedDateTime.prototype.daysInWeek
- Temporal.ZonedDateTime.prototype.weekOfYear
- Temporal.ZonedDateTime.prototype.yearOfWeek
- Temporal.PlainDate.prototype.dayOfWeek

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.dayOfYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/dayOfYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **dayOfYear** accessor property of Temporal.ZonedDateTime instances returns a positive integer representing the 1-based day index in the year of this date. The first day of this year is 1, and the last day is the daysInYear. It is calendar-dependent.

The set accessor of dayOfYear is undefined. You cannot change this property directly. To create a new Temporal.ZonedDateTime object with the desired new dayOfYear value, use the add() or subtract() method with the appropriate number of days.

For general information and more examples, see Temporal.PlainDate.prototype.dayOfYear.

Examples

Using dayOfYear

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.dayOfYear); // 182
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.dayofyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
dayOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.ZonedDateTime.dayOfYear. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.ZonedDateTime
- Temporal.ZonedDateTime.prototype.with()
- Temporal.ZonedDateTime.prototype.add()
- Temporal.ZonedDateTime.prototype.subtract()
- Temporal.ZonedDateTime.prototype.year
- Temporal.ZonedDateTime.prototype.day
- Temporal.ZonedDateTime.prototype.dayOfWeek
- Temporal.ZonedDateTime.prototype.daysInYear
- Temporal.PlainDate.prototype.dayOfYear

Temporal.ZonedDateTime.prototype.daysInMonth

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/daysInMonth · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **daysInMonth** accessor property of Temporal.ZonedDateTime instances returns a positive integer representing the number of days in the month of this date. It is calendar-dependent.

The set accessor of daysInMonth is undefined. You cannot change this property directly.

For general information and more examples, see Temporal.PlainDate.prototype.daysInMonth.

Examples

Using daysInMonth

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.daysInMonth); // 31
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.daysinmonth

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
daysInMonth	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.ZonedDateTime.daysInMonth. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Temporal.ZonedDateTime
- Temporal.ZonedDateTime.prototype.with()
- Temporal.ZonedDateTime.prototype.add()
- Temporal.ZonedDateTime.prototype.subtract()
- Temporal.ZonedDateTime.prototype.year
- Temporal.ZonedDateTime.prototype.month
- Temporal.ZonedDateTime.prototype.day
- Temporal.ZonedDateTime.prototype.daysInWeek
- Temporal.ZonedDateTime.prototype.daysInYear
- Temporal.PlainDate.prototype.daysInMonth

Temporal.ZonedDateTime.prototype.daysInWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/daysInWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **daysInWeek** accessor property of Temporal.ZonedDateTime instances returns a positive integer representing the number of days in the week of this date. It is calendar-dependent.

The set accessor of daysInWeek is undefined. You cannot change this property directly.

For general information and more examples, see Temporal.PlainDate.prototype.daysInWeek.

Examples

Using daysInWeek

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.daysInWeek); // 7
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.daysinweek

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
daysInWeek	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.daysInWeek`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.yearOfWeek`
- `Temporal.ZonedDateTime.prototype.weekOfYear`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.daysInWeek`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.daysInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/daysInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`daysInYear`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of days in the year of this date. It is calendar-dependent.

The set accessor of `daysInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.daysInYear`.

Examples

Using daysInYear

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.daysInYear); // 365
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.daysinyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`daysInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.daysInYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.dayOfYear`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.PlainDate.prototype.daysInYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.epochMilliseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/epochMilliseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`epochMilliseconds`** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the number of milliseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant. It is equivalent to dividing `epochNanoseconds` by `1e6` and flooring the result.

The set accessor of `epochMilliseconds` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `epochMilliseconds` value, see below.

Examples

Using epochMilliseconds

```js
const zdt = Temporal.ZonedDateTime.from("2021-08-01T12:34:56.789Z[UTC]");
console.log(zdt.epochMilliseconds); // 1627821296789

const zdt2 = Temporal.ZonedDateTime.from("1969-08-01T12:34:56.789Z[UTC]");
console.log(zdt2.epochMilliseconds); // -13173903211
```

Creating a ZonedDateTime object from an epochMilliseconds value

You can create a `Temporal.ZonedDateTime` object from an `epochMilliseconds` value by first constructing a `Temporal.Instant` object using `Temporal.Instant.fromEpochMilliseconds()`, and then converting it to a `Temporal.ZonedDateTime` object using `Temporal.Instant.prototype.toZonedDateTimeISO()`:

```js
const epochMilliseconds = 1627821296789;
const instant = Temporal.Instant.fromEpochMilliseconds(epochMilliseconds);
const zdt = instant.toZonedDateTimeISO("UTC");
console.log(zdt.toString()); // 2021-08-01T12:34:56.789+00:00[UTC]
```

Alternatively, use the `Temporal.ZonedDateTime()` constructor, but convert the milliseconds to nanoseconds first:

```js
const epochMilliseconds = 1627821296789;
const epochNanoseconds = BigInt(epochMilliseconds) * 1_000_000n;
const zdt = new Temporal.ZonedDateTime(epochNanoseconds, "UTC");
console.log(zdt.toString()); // 2021-08-01T12:34:56.789+00:00[UTC]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.epochmilliseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
epochMilliseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.epochMilliseconds`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.epochNanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.epochNanoseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/epochNanoseconds · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **epochNanoseconds** accessor property of `Temporal.ZonedDateTime` instances returns a `BigInt` representing the number of nanoseconds elapsed since the Unix epoch (midnight at the beginning of January 1, 1970, UTC) to this instant.

The set accessor of `epochNanoseconds` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `epochNanoseconds` value, use the `new Temporal.ZonedDateTime()` constructor instead.

An instant can only represent $\pm 10^8$ days (about ±273,972.6 years) around the epoch, which is ±8.64e21 nanoseconds. Attempting to set `epochNanoseconds` beyond this boundary throws a `RangeError`.

Examples

Using epochNanoseconds

```js
const zdt = Temporal.ZonedDateTime.from("2021-08-01T12:34:56.789Z[UTC]");
console.log(zdt.epochNanoseconds); // 1627821296789000000n

const zdt2 = Temporal.ZonedDateTime.from("1969-08-01T12:34:56.789Z[UTC]");
console.log(zdt2.epochNanoseconds); // -13173903211000000n
```

Creating a ZonedDateTime object from an epochNanoseconds value

You can create a `Temporal.ZonedDateTime` object from an `epochNanoseconds` value using the `Temporal.ZonedDateTime()` constructor.

```js
const epochNanoseconds = 1627821296789000000n;
const zdt = new Temporal.ZonedDateTime(epochNanoseconds, "UTC");
console.log(zdt.toString()); // 2021-08-01T12:34:56.789+00:00[UTC]
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.epochnanoseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
epochNanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.epochNanoseconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.epochMilliseconds`

Temporal.ZonedDateTime.prototype.era

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/era · MDN page last modified 2026-02-17

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **era** accessor property of `Temporal.ZonedDateTime` instances returns a calendar-specific lowercase string representing the era of this date, or `undefined` if the calendar does not use eras (e.g., ISO 8601). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `era` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.era`.

Examples

Using era

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.era); // undefined

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01[America/New_York][u-ca=gregory]",
);
console.log(dt2.era); // ce
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.era

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
era	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.era`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.eraYear`
- `Temporal.PlainDate.prototype.era`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.eraYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/eraYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`eraYear`** accessor property of `Temporal.ZonedDateTime` instances returns a non-negative integer representing the year of this date within the era, or `undefined` if the calendar does not use eras (e.g., ISO 8601). The year index usually starts from 1 (more common) or 0, and years in an era can decrease with time (e.g., Gregorian BCE). `era` and `eraYear` together uniquely identify a year in a calendar, in the same way that `year` does. It is calendar-dependent.

The set accessor of `eraYear` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.eraYear`.

Examples

Using eraYear

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.eraYear); // undefined

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01[America/New_York][u-ca=gregory]",
);
console.log(dt2.eraYear); // 2021

const dt3 = Temporal.ZonedDateTime.from(
  "-002021-07-01[America/New_York][u-ca=gregory]",
);
console.log(dt3.eraYear); // 2022; 0000 is used for the year 1 BC

const dt4 = Temporal.ZonedDateTime.from(
  "2021-07-01[America/New_York][u-ca=japanese]",
);
console.log(dt4.eraYear); // 3
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.erayear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
eraYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.eraYear`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.era`
- `Temporal.PlainDate.prototype.eraYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.hour

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/hour · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **hour** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 23 representing the hour component of this time.

The set accessor of `hour` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.hour`.

For `ZonedDateTime`, `hour` can be non-continuous due to offset changes such as daylight saving time transitions. In this case, the hour may be repeated or skipped.

Examples

Using hour

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.hour); // 12
```

Non-continuous hour

Non-continuous hour is very common due to daylight saving time transitions, which is explained more in Ambiguity and gaps from local time to UTC time.

```js
const dt = Temporal.ZonedDateTime.from(
  "2024-11-03T01:59:00-04:00[America/New_York]",
);
console.log(dt.hour); // 1
const dt2 = dt.add({ minutes: 1 });
console.log(dt2.hour); // 1
console.log(dt2.toString()); // 2024-11-03T01:00:00-05:00[America/New_York]

const dt3 = Temporal.ZonedDateTime.from(
  "2024-03-10T01:59:00-05:00[America/New_York]",
);
console.log(dt3.hour); // 1
const dt4 = dt3.add({ minutes: 1 });
console.log(dt4.hour); // 3
console.log(dt4.toString()); // 2024-03-10T03:00:00-04:00[America/New_York]
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `hour` property.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.hour

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
hour	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.hour`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.PlainTime.prototype.hour`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.hoursInDay

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/hoursInDay · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`hoursInDay`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of hours in the day of this date in the time zone. It may be more or less than 24 in the case of offset changes such as daylight saving time.

Because `ZonedDateTime` is the only class that's time zone-aware, and hours in a day can only change by offset changes, all other classes assume 24-hour days.

The set accessor of `hoursInDay` is `undefined`. You cannot change this property directly.

Examples

Using hoursInDay

```js
const dt = Temporal.ZonedDateTime.from(
  "2024-03-10T01:58:00-05:00[America/New_York]",
);
console.log(dt.hoursInDay); // 23; this is the day of transition into DST

const dt2 = Temporal.ZonedDateTime.from(
  "2024-11-03T01:58:00-04:00[America/New_York]",
);
console.log(dt2.hoursInDay); // 25; this is the day of transition out of DST

const dt3 = Temporal.ZonedDateTime.from(
  "2024-11-04T01:58:00-05:00[America/New_York]",
);
console.log(dt3.hoursInDay); // 24
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.hoursinday`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`hoursInDay`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.hoursInDay`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.hour`
- `Temporal.ZonedDateTime.prototype.dayOfYear`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.daysInWeek`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.inLeapYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/inLeapYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`inLeapYear`** accessor property of `Temporal.ZonedDateTime` instances returns a boolean indicating whether this date is in a leap year. A leap year is a year that has more days (due to a leap day or leap month) than a common year. It is calendar-dependent.

The set accessor of `inLeapYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.inLeapYear`.

Examples

Using inLeapYear

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.inLeapYear); // false
console.log(dt.daysInYear); // 365
console.log(dt.monthsInYear); // 12
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.inleapyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`inLeapYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.inLeapYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.daysInYear`

- `Temporal.ZonedDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.inLeapYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.microsecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/microsecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`microsecond`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 999 representing the microsecond (10^{-6} second) component of this time.

The set accessor of `microsecond` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.microsecond`.

Examples

Using microsecond

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.microsecond); // 456
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.microsecond

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`microsecond`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.microsecond`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`

- Temporal.ZonedDateTime.prototype.second
- Temporal.ZonedDateTime.prototype.millisecond
- Temporal.ZonedDateTime.prototype.nanosecond
- Temporal.PlainTime.prototype.microsecond

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.millisecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/millisecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **millisecond** accessor property of Temporal.ZonedDateTime instances returns an integer from 0 to 999 representing the millisecond (10^{-3} second) component of this time.

The set accessor of millisecond is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.ZonedDateTime object with the desired new value.

For general information and more examples, see Temporal.PlainTime.prototype.millisecond.

Examples

Using millisecond

js

```
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.millisecond); // 123
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.millisecond

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
millisecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.ZonedDateTime.millisecond*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Temporal.ZonedDateTime
- Temporal.ZonedDateTime.prototype.with()
- Temporal.ZonedDateTime.prototype.add()
- Temporal.ZonedDateTime.prototype.subtract()
- Temporal.ZonedDateTime.prototype.second
- Temporal.ZonedDateTime.prototype.microsecond
- Temporal.ZonedDateTime.prototype.nanosecond
- Temporal.PlainTime.prototype.millisecond

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.minute

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/minute · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`minute`** accessor property of Temporal.ZonedDateTime instances returns an integer from 0 to 59 representing the minute component of this time.

The set accessor of `minute` is `undefined`. You cannot change this property directly. Use the with() method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see Temporal.PlainTime.prototype.minute.

For `ZonedDateTime`, `minute` can be non-continuous due to offset changes. While much rarer than `hour` changes (because daylight saving time shifts are usually by whole hours), it can still happen.

Examples

Using minute

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.minute); // 34
```

Non-continuous minute

Typically, `minute` always goes from 0 to 59 and then back to 0, even when passing through a daylight saving time transition. There's one particular case where DST has a 30-minute offset: Lord Howe Island in Australia, which shifts between +10:30 and +11:00. In this case, the minute can be non-continuous.

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-10-03T01:59:00+10:30[Australia/Lord_Howe]",
);
console.log(dt.minute); // 59
const dt2 = dt.add({ minutes: 1 });
console.log(dt2.minute); // 30
console.log(dt2.toString()); // 2021-10-03T02:30:00+11:00[Australia/Lord_Howe]
```

There's a second peculiar case where the minute can be non-continuous: the standardization of hourly time zones. In the early 20th century, most countries were using their own time zones which were often not a whole hour offset from UTC. For example, Paris used to have an offset of UTC+0:09:21, which was changed to UTC+0 on March 11, 1911.

```js
const dt = Temporal.ZonedDateTime.from(
  "1911-03-10T23:59:00+00:09:21[Europe/Paris]",
);
console.log(dt.minute); // 59
const dt2 = dt.add({ minutes: 1 });
console.log(dt2.minute); // 50
console.log(dt2.toString()); // 1911-03-10T23:50:39+00:00[Europe/Paris]
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `minute` property.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.minute

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`minute`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.minute`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.minute`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.month

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/month · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`month`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based month index in the year of this date. The first month of this year is `1`, and the last month is the `monthsInYear`. It is calendar-dependent.

The set accessor of `month` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.month`.

Examples

Using month

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.monthCode); // "M07"
console.log(dt.month); // 7
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.month`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`month`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.month`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.day`

- Temporal.ZonedDateTime.prototype.monthCode
- Temporal.ZonedDateTime.prototype.daysInMonth
- Temporal.ZonedDateTime.prototype.monthsInYear
- Temporal.PlainDate.prototype.month

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.monthCode

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/monthCode · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **monthCode** accessor property of Temporal.ZonedDateTime instances returns a calendar-specific string representing the month of this date. It is calendar-dependent.

Usually it is M plus a two-digit month number. For leap months, it is the previous month's code followed by L (even if it's conceptually a derivative of the following month; for example, in the Hebrew calendar, Adar I has code M05L but Adar II has code M06). If the leap month is the first month of the year, the code is M00L.

The set accessor of monthCode is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.ZonedDateTime object with the desired new value.

For general information and more examples, see Temporal.PlainDate.prototype.monthCode.

Examples

Using monthCode

```js
const date = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601
calendar
console.log(date.monthCode); // "M07"
console.log(date.month); // 7
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.monthcode

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
monthCode	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Temporal.ZonedDateTime.monthCode. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.daysInMonth`
- `Temporal.ZonedDateTime.prototype.monthsInYear`
- `Temporal.PlainDate.prototype.monthCode`

Temporal.ZonedDateTime.prototype.monthsInYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/monthsInYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`monthsInYear`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the number of months in the year of this date. It is calendar-dependent.

The set accessor of `monthsInYear` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.monthsInYear`.

Examples

Using monthsInYear

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.monthsInYear); // 12
```

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.monthsinyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`monthsInYear`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.monthsInYear`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Temporal.ZonedDateTime
- Temporal.ZonedDateTime.prototype.with()
- Temporal.ZonedDateTime.prototype.add()
- Temporal.ZonedDateTime.prototype.subtract()
- Temporal.ZonedDateTime.prototype.year
- Temporal.ZonedDateTime.prototype.month
- Temporal.ZonedDateTime.prototype.monthCode
- Temporal.ZonedDateTime.prototype.daysInMonth
- Temporal.PlainDate.prototype.monthsInYear

Temporal.ZonedDateTime.prototype.nanosecond

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/nanosecond · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **nanosecond** accessor property of Temporal.ZonedDateTime instances returns an integer from 0 to 999 representing the nanosecond (10^{-9} second) component of this time.

The set accessor of nanosecond is undefined. You cannot change this property directly. Use the with() method to create a new Temporal.ZonedDateTime object with the desired new value.

For general information and more examples, see Temporal.PlainTime.prototype.nanosecond.

Examples

Using nanosecond

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.nanosecond); // 789
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.nanosecond

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
nanosecond	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.nanosecond`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.second`
- `Temporal.ZonedDateTime.prototype.millisecond`
- `Temporal.ZonedDateTime.prototype.microsecond`
- `Temporal.PlainTime.prototype.nanosecond`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.offset

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/offset · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`offset`** accessor property of `Temporal.ZonedDateTime` instances returns a string representing the offset used to interpret the internal instant, in the form `±HH:mm` (or `±HH:mm:ss.sssssssss` with as much subminute precision as necessary). This offset is guaranteed to be valid for the given instant and time zone at construction time.

The set accessor of `offset` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value (usually also changing the date/time), or use the `withTimeZone()` method to create a new `Temporal.ZonedDateTime` object in another time zone.

Examples

Using offset

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
console.log(dt.offset); // "-07:00"

const dt2 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00-07[-07]");
console.log(dt2.offset); // "-07:00"

const dt3 = Temporal.ZonedDateTime.from(
  "1900-01-01T00:00:00+00:09:21[Europe/Paris]",
);
console.log(dt3.offset); // "+00:09:21"

const dt4 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00Z[Asia/Shanghai]");
console.log(dt4.offset); // "+08:00"
```

Changing offset

If the local time happens to have two valid offsets, such as within a DST transition, you can change the offset without changing anything else:

```js
const zdt = Temporal.ZonedDateTime.from(
  "2024-11-03T01:05:00-04:00[America/New_York]",
);
const newZDT = zdt.with({ offset: "-05:00" });
console.log(newZDT.toString()); // "2024-11-03T01:05:00-05:00[America/New_York]"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.offset

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
offset	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.offset`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`

- Temporal.ZonedDateTime.prototype.timeZoneId
- Temporal.ZonedDateTime.prototype.offsetNanoseconds

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.offsetNanoseconds

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/offsetNanoseconds · MDN page last modified 2026-07-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`offsetNanoseconds`** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the offset used to interpret the internal instant, as a number of nanoseconds (positive or negative). The value is a safe integer because it's less than a day, which is 8.64e15 nanoseconds.

The set accessor of `offsetNanoseconds` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new `offset` value (which will change this property too), or use the `withTimeZone()` method to create a new `Temporal.ZonedDateTime` object in another time zone.

Examples

Using offsetNanoseconds

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
console.log(dt.offsetNanoseconds); // -25200000000000

const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+08:00[Asia/Shanghai]",
);
console.log(dt2.offsetNanoseconds); // 28800000000000

const dt3 = Temporal.ZonedDateTime.from(
  "1900-01-01T00:00:00+00:09:21[Europe/Paris]",
);
console.log(dt3.offsetNanoseconds); // 561000000000
```

Here's one way to get a `ZonedDateTime` representing the same wall-clock time in UTC:

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
const dtInUTC = dt.add({ nanoseconds: dt.offsetNanoseconds });
console.log(dtInUTC.withTimeZone("UTC").toString()); // "2021-07-01T12:00:00+00:00[UTC]"
```

Here's a better way to get the same result:

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
const dtInUTC = dt.toPlainDateTime().toZonedDateTime("UTC");
console.log(dtInUTC.toString()); // "2021-07-01T12:00:00+00:00[UTC]"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.offsetnanoseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
offsetNanoseconds	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.offsetNanoseconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.offset`
- `Temporal.ZonedDateTime.prototype.timeZoneId`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.second

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/second · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`second`** accessor property of `Temporal.ZonedDateTime` instances returns an integer from 0 to 59 representing the second component of this time.

The set accessor of `second` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainTime.prototype.second`.

For `ZonedDateTime`, `second` can be non-continuous due to offset changes. While much rarer than `hour` or `minute` changes (because daylight saving time shifts are usually by whole hours), it can still happen.

Examples

Using second

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:34:56.123456789-04:00[America/New_York]",
);
console.log(dt.second); // 56
```

Non-continuous second

Typically, `second` always goes from 0 to 59 and then back to 0, even when passing through a daylight saving time transition. There's a peculiar case where the second can be non-continuous: the standardization of hourly time zones. In the early 20th century, most countries were using their own time zones which were often not a whole hour offset from UTC. For example, Paris used to have an offset of UTC+0:09:21, which was changed to UTC+0 on March 11, 1911.

```js
const dt = Temporal.ZonedDateTime.from(
  "1911-03-10T23:59:59+00:09:21[Europe/Paris]",
);
console.log(dt.second); // 59
const dt2 = dt.add({ seconds: 1 });
console.log(dt2.second); // 39
console.log(dt2.toString()); // 1911-03-10T23:50:39+00:00[Europe/Paris]
```

For this reason, you should always prefer `add()` and `subtract()` to manipulate dates and times, rather than directly changing the `second` property.

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.second

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
second	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.second`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.millisecond`

- Temporal.ZonedDateTime.prototype.microsecond
- Temporal.ZonedDateTime.prototype.nanosecond
- Temporal.PlainTime.prototype.second

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.timeZoneId

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/timeZoneId · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`timeZoneId`** accessor property of `Temporal.ZonedDateTime` instances returns a string representing the time zone identifier used to interpret the internal instant. The string is either a named identifier in the preferred case (such as `"America/New_York"`), or an offset in the form `"±hh:mm"`. If the time zone has aliases, the `timeZoneId` is the identifier used to create the `ZonedDateTime`, without canonicalization to the primary identifier.

The set accessor of `timeZoneId` is `undefined`. You cannot change this property directly. Use the `withTimeZone()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

> **Note:** This string is not intended for display to users. Use `toLocaleString()` with the appropriate options to get a localized string.

Examples

Using timeZoneId

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00-07:00[America/Los_Angeles]",
);
console.log(dt.timeZoneId); // "America/Los_Angeles"

const dt2 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00-07:00[-07:00]");
console.log(dt2.timeZoneId); // "-07:00"

const dt3 = dt2.withTimeZone("Asia/Shanghai");
console.log(dt3.timeZoneId); // "Asia/Shanghai"
```

The `timeZoneId` is never canonicalized to the primary identifier; it is the same as the one used to create the `ZonedDateTime`.

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+07:00[Asia/Ho_Chi_Minh]",
);
const dt2 = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+07:00[Asia/Saigon]",
);
console.log(dt.timeZoneId); // "Asia/Ho_Chi_Minh"
console.log(dt2.timeZoneId); // "Asia/Saigon"
```

However, presentational differences will get canonicalized.

```js
const dt = Temporal.ZonedDateTime.from(
  "2021-07-01T12:00:00+07:00[asia/ho_chi_minh]",
);
console.log(dt.timeZoneId); // "Asia/Ho_Chi_Minh"

const dt2 = Temporal.ZonedDateTime.from("2021-07-01T12:00:00+07:00[+07]");
console.log(dt2.timeZoneId); // "+07:00"
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.timezoneid

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
timeZoneId	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.timeZoneId`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.withTimeZone()`
- `Temporal.ZonedDateTime.prototype.offset`
- `Temporal.ZonedDateTime.prototype.offsetNanoseconds`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.weekOfYear

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/weekOfYear · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`weekOfYear`** accessor property of `Temporal.ZonedDateTime` instances returns a positive integer representing the 1-based week index in the `yearOfWeek` of this date, or `undefined` if the calendar does not have a well-defined week system. The first week of the year is `1`. It is calendar-dependent.

The set accessor of `weekOfYear` is `undefined`. You cannot change this property directly. To create a new `Temporal.ZonedDateTime` object with the desired new `weekOfYear` value, use the `add()` or `subtract()` method with the appropriate number of `weeks`.

For general information and more examples, see `Temporal.PlainDate.prototype.weekOfYear`.

Examples

Using weekOfYear

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]");
console.log(dt.weekOfYear); // 26
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.weekofyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
weekOfYear	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.weekOfYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.yearOfWeek`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.weekOfYear`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.year

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/year · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **year** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the number of years of this date relative to the start of a calendar-specific epoch year. It is calendar-dependent.

The set accessor of `year` is `undefined`. You cannot change this property directly. Use the `with()` method to create a new `Temporal.ZonedDateTime` object with the desired new value.

For general information and more examples, see `Temporal.PlainDate.prototype.year`.

Examples

Using year

```js
const dt = Temporal.ZonedDateTime.from("2021-07-01[America/New_York]"); // ISO 8601 calendar
console.log(dt.year); // 2021
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.year

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
year	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.year`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.era`
- `Temporal.ZonedDateTime.prototype.eraYear`
- `Temporal.ZonedDateTime.prototype.yearOfWeek`
- `Temporal.ZonedDateTime.prototype.month`
- `Temporal.ZonedDateTime.prototype.day`
- `Temporal.PlainDate.prototype.year`

JavaScript › Reference › Standard built-in objects › Temporal › Temporal.ZonedDateTime

Temporal.ZonedDateTime.prototype.yearOfWeek

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Temporal/ZonedDateTime/yearOfWeek · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`yearOfWeek`** accessor property of `Temporal.ZonedDateTime` instances returns an integer representing the year to be paired with the `weekOfYear` of this date, or `undefined` if the calendar does not have a well-defined week system. It is calendar-dependent.

The set accessor of `yearOfWeek` is `undefined`. You cannot change this property directly.

For general information and more examples, see `Temporal.PlainDate.prototype.yearOfWeek`.

Specifications

Specification
Temporal `https://tc39.es/proposal-temporal/#sec-get-temporal.zoneddatetime.prototype.yearofweek`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`yearOfWeek`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Temporal.ZonedDateTime.yearOfWeek`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Temporal.ZonedDateTime`
- `Temporal.ZonedDateTime.prototype.with()`
- `Temporal.ZonedDateTime.prototype.add()`
- `Temporal.ZonedDateTime.prototype.subtract()`
- `Temporal.ZonedDateTime.prototype.year`
- `Temporal.ZonedDateTime.prototype.weekOfYear`
- `Temporal.ZonedDateTime.prototype.dayOfWeek`
- `Temporal.ZonedDateTime.prototype.daysInWeek`
- `Temporal.ZonedDateTime.prototype.daysInYear`
- `Temporal.PlainDate.prototype.yearOfWeek`